

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I

GIÁO TRÌNH KỸ THUẬT ĐỒ HỌA



CHO NGÀNH ĐA PHƯƠNG TIỆN

HÀ NỘI - 2018

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I

GIÁO TRÌNH
KỸ THUẬT ĐỒ HOẠ

CHO NGÀNH ĐA PHƯƠNG TIỆN

NGƯỜI VIẾT: TRỊNH THỊ VÂN ANH

HÀ NỘI - 2018

LỜI NÓI ĐẦU

Đồ hoạ máy tính (Computer Graphics) là một lĩnh vực lý thú và có nhiều ứng dụng trong thực tế, nó góp phần làm cho giao tiếp giữa con người và máy tính trở nên thân thiện hơn. Giao diện kiểu văn bản (text) đã được thay thế hoàn toàn bằng giao diện đồ hoạ. Tuy nhiên, việc dạy và học kỹ thuật đồ hoạ thì không đơn giản do chủ đề này có nhiều phức tạp. Kỹ thuật đồ hoạ liên quan đến tin học và toán học bởi vì hầu hết các giải thuật vẽ, tô cùng các phép biến hình đều được xây dựng dựa trên nền tảng của hình học không gian hai chiều và ba chiều. Hiện nay, Kỹ thuật đồ hoạ là một môn học được giảng dạy cho sinh viên chuyên ngành Công Nghệ Thông Tin.

Trong cuốn giáo trình này, tôi muốn mang lại cho bạn đọc các cơ sở lý thuyết về đồ hoạ máy tính từ đơn giản nhất như các thuật toán vẽ đường thẳng, đường tròn, đa giác, ký tự... Tiếp đến các kỹ thuật xén tỉa, các phép biến đổi đồ hoạ trong không gian 2D và 3D...Chúng ta lần lượt làm quen với thế giới màu sắc thông qua các hệ màu: RGB, CMYK, HSV.... Phức tạp hơn nữa là các phép chiếu. Cuối chúng ta nghiên cứu cài đặt thư viện đồ hoạ mở OpenGL, với các nội dung: vẽ hình, các phép biến đổi 3D, ánh sáng, tạo cảnh 3D. Kết quả cuối cùng là xây dựng được một game dùng thư viện OpenGL viết trên nền visual studio C++.

Giáo trình gồm bảy chương, trong đó chương một giúp bạn đọc có cái nhìn tổng quan về kỹ thuật đồ hoạ từ trước đến giờ cùng định hướng tương lai cho lĩnh vực này. Các chương tiếp theo, mỗi chương sẽ là một vấn đề từ đơn giản đến phức tạp về cơ sở nền tảng cho ngành kỹ thuật đồ hoạ. Cuối mỗi chương đều có phần bài tập để kiểm tra lại kiến thức vừa đọc được. Bài tập gồm hai dạng: dạng tính toán và dạng lập trình, đối với dạng lập trình bạn có thể viết bằng C/C++ hay BC thậm chí bằng VB đều được. Cuối cùng là phần phụ lục gồm các hướng dẫn làm bài tập lập trình, ngôn ngữ hay dùng ở đây là C/C++ hay BC.

Bố cục rõ ràng, hình ảnh phong phú, đa dạng. Tôi hy vọng rằng giáo trình là một bộ tham khảo đầy đủ các thông tin hữu ích và có tính thực tiễn cao cho môn kỹ thuật đồ hoạ.

Trong quá trình biên soạn mặc dù đã cố gắng hết sức nhưng vẫn không tránh khỏi những sai sót, rất mong nhận được sự đóng góp chân thành từ quý bạn đọc.

Xin chân thành cảm ơn.

Tác giả

MỤC LỤC

Contents

LỜI NÓI ĐẦU.....	1
MỤC LỤC	2
CHƯƠNG 1: TỔNG QUAN VỀ KỸ THUẬT ĐỒ HOẠ	7
1.1. CÁC KHÁI NIỆM TỔNG QUAN CỦA KỸ THUẬT ĐỒ HOẠ MÁY TÍNH (COMPUTER GRAPHICS).....	7
1.1.1. Lịch sử phát triển.....	7
1.1.2. Kỹ thuật đồ họa vi tính.	8
1.2. CÁC KỸ THUẬT ĐỒ HOẠ	8
1.2.1. Kỹ thuật đồ họa điểm (Sample based-Graphics).....	8
1.2.2. Kỹ thuật đồ họa vector.....	9
1.2.3. Phân loại của đồ họa máy tính.....	10
1.2.4. Các ứng dụng tiêu biểu của kỹ thuật đồ họa.....	11
1.2.5. Các chuẩn giao diện của hệ đồ họa.....	12
1.3. PHẦN CỨNG ĐỒ HOẠ (GRAPHICS HARDWARE).....	13
1.3.1. Các thành phần phần cứng của hệ đồ họa tương tác.....	13
1.3.2. Máy in.....	13
1.3.3. Màn hình CRT	14
1.3.4. Màn hình tinh thể lỏng (Liquid Crystal Display – LCD)	16
Tóm tắt chương:	17
Bài tập:.....	17
CHƯƠNG 2: CÁC GIẢI THUẬT SINH THỰC THỂ CƠ SỞ.....	19
2.1. CÁC HỆ THỐNG TOẠ ĐỘ TRONG ĐỒ HOẠ.....	19
2.1.1. Hệ toạ độ thực (WCS – World Coordinate System).....	19
2.1.2. Hệ toạ độ thiết bị (DCS – Device Coordinate System)	20
2.1.3. toạ độ thiết bị chuẩn (NDCS – Normalized Device Coordinate System).....	20
2.2. ĐIỂM VÀ ĐOẠN THẲNG.....	20
2.2.1. Điểm	20
2.2.2. Đoạn thẳng.....	20
2.3. CÁC GIẢI THUẬT XÂY DỰNG THỰC THỂ CƠ SỞ	21
2.3.1. Giải thuật vẽ đoạn thẳng thông thường	21
2.3.2. Giải thuật Bresenham	21
2.3.3. Giải thuật trung điểm-Midpoint.....	23
2.3.4. Giải thuật sinh đường tròn (Scan Converting Circles)(Bresenham).....	25
2.3.5. Giải thuật sinh đường tròn Midpoint	28
2.3.6. Giải thuật sinh đường ellipse	30

2.3.7. Giải thuật sinh ký tự	33
2.3.8. Giải thuật sinh đa giác (Polygon)	34
Tóm tắt chương:	40
Bài tập:.....	40
CHƯƠNG 3: CÁC PHÉP BIẾN ĐỔI ĐỒ HOẠ	40
3.1. CÁC PHÉP BIẾN ĐỔI HÌNH HỌC HAI CHIỀU	40
3.1.1. Phép biến đổi Affine (Affine Transformations)	40
3.1.2. Các phép biến đổi đối tượng.....	40
3.2. TỌA ĐỘ ĐỒNG NHẤT VÀ CÁC PHÉP BIẾN ĐỔI	44
3.2.1. Tọa độ đồng nhất.....	44
3.2.2. Phép biến đổi với tọa độ đồng nhất	45
3.2.3. Cài đặt c/c++ cho phép quay tam giác quanh 1 điểm (xq,yq):	46
3.3. CÁC PHÉP BIẾN ĐỔI HÌNH HỌC BA CHIỀU.....	47
3.3.1. Biểu diễn điểm trong không gian 3 chiều.....	47
3.3.2. Phép tịnh tiến.....	47
3.3.3. Phép tỉ lệ	47
3.3.4. Phép biến dạng.....	48
3.3.5. Phép lấy đối xứng	48
3.3.6. Phép quay 3 chiều.....	49
3.3.7. Cài đặt bằng c/c++ như sau:	52
Tóm tắt:.....	53
Bài tập:.....	54
CHƯƠNG 4: CÁC GIẢI THUẬT ĐỒ HOẠ CƠ SỞ.....	57
4.1. MÔ HÌNH CHUYỂN ĐỔI GIỮA BA HỆ THỐNG TỌA ĐỘ	57
4.1.1. Mô hình chuyển đổi.....	57
4.1.2. Phép ánh xạ từ cửa sổ vào khung nhìn	57
4.2. CÁC GIẢI THUẬT XÉN TỈA (CLIPPING)	59
4.2.1. Khái niệm	59
4.2.2. Clipping điểm	59
4.2.3. Xén tỉa đoạn thẳng.....	59
4.2.4. Giải thuật xén tỉa đa giác (Sutherland Hodgman)	66
Tóm tắt chương:	70
Bài tập:.....	70
CHƯƠNG 5: PHÉP CHIẾU –PROJECTION.....	71
5.1. KHÁI NIỆM CHUNG.....	71
5.1.1. Nguyên lý về 3D (three-Dimension)	71
5.1.2. Đặc điểm của kỹ thuật đồ hoạ 3D.....	71
5.1.3. Các phương pháp hiển thị 3D	71

5.2. PHÉP CHIẾU	72
5.3. PHÉP CHIẾU SONG SONG (Parallel Projections).....	74
5.3.1. Phép chiếu trực giao (Orthographic projection)	74
5.3.2. Phép chiếu trục lượng (Axonometric)	75
5.3.3. Phép chiếu xiên - Oblique	78
5.4. PHÉP CHIẾU PHỐI CẢNH (Perspective Projection)	79
5.4.1. Phép chiếu phối cảnh một tâm chiếu	80
5.4.2. Phép chiếu phối cảnh hai tâm chiếu	81
5.4.3. Phép chiếu phối cảnh ba tâm chiếu	82
Tóm tắt chương:	83
Bài tập:.....	83
CHƯƠNG 6: MÀU SẮC TRONG ĐỒ HOẠ	85
6.1. ÁNH SÁNG VÀ MÀU SẮC (light and color).....	85
6.1.1. Quan niệm về ánh sáng.....	85
6.1.2. Yếu tố vật lý	85
6.1.3. Cảm nhận màu sắc của con người (Physiology - Sinh lý - Human Vision)	87
6.1.4. Các đặc trưng cơ bản của ánh sáng.....	89
6.2. ÁNH SÁNG ĐƠN SẮC	89
6.2.1. Cường độ sáng và cách tính.....	90
6.2.2. Phép hiệu chỉnh gama.....	90
6.2.3. Xấp xỉ bán tông - halftone	91
6.2.4. Ma trận Dither và phép lấy xấp xỉ bán tông	92
6.3. CÁC HỆ MÀU TRONG MÀN HÌNH ĐỒ HOẠ.....	93
6.3.1. Mô hình màu RGB (Red, Green, Blue - đỏ, lục, lam).....	94
6.3.2. Mô hình màu CMY (Cyan, Magenta, Yellow - xanh tím, Đỏ tươi, vàng)	94
6.3.3. Mô hình màu YIQ.....	95
3.4. Mô hình màu HSV (Hue, Saturation, Value) - Mỹ thuật.....	96
6.3.5. Biểu đồ màu CIE (1931 – Commission Internationale de l'Eclairage)	97
6.4. CHUYỂN ĐỔI GIỮA CÁC HỆ MÀU	100
6.4.1. Chuyển đổi HSV - RGB	100
6.4.2. Chuyển đổi RGB sang XYZ.....	101
Tóm tắt:.....	102
Bài tập:.....	102
CHƯƠNG 7: ĐƯỜNG CONG VÀ MẶT CONG TRONG 3D	104
7.1. ĐƯỜNG CONG - CURVE	104
7.1.1. Điểm biểu diễn đường cong (curve represents points)	104
7.1.2. Đường cong đa thức bậc ba tham biến	104
7.1.3. Đường cong Hermite	105

7.1.4. Đường cong Bezier	106
7.1.5. Đường cong B-spline.....	108
7.2. MÔ HÌNH BỀ MẶT (Surface) VÀ CÁC PHƯƠNG PHÁP XÂY DỰNG	114
7.2.1. Các khái niệm cơ bản	114
7.2.2. Biểu diễn mảnh tứ giác	115
7.2.3. Mô hình hoá các mặt cong (Surface Patches).....	117
7.2.3.1. Mặt kẻ (Ruled Surface).....	117
7.2.4. Mặt từ các đường cong	120
Tóm tắt:.....	124
Bài tập:.....	125
CHƯƠNG 8: ÁNH SÁNG	127
8.1. GIỚI THIỆU.....	127
8.1.1. Mục tiêu chính trong đồ họa máy tính.....	127
8.1.2. Các giải pháp trong đồ họa máy tính	127
8.2. CÁC KỸ THUẬT CHIẾU SÁNG TRONG ĐỒ HỌA MÁY TÍNH.....	129
8.2.1. Đánh giá về cường độ ánh sáng.....	129
8.2.2. Cường độ ánh sáng	130
8.2.3. Những thuộc tính bao quanh của vật chất	131
8.2.4. Thuộc tính khuếch tán của vật chất	132
8.2.5. Sự tương tác bề mặt/ánh sáng.....	133
8.2.6. Sự khúc xạ và sự truyền sáng	133
8.3. CÁC CÔNG NGHỆ.....	134
8.3.1. Raytracing.....	134
8.3.2. Radiosity	138
8.3.3. Photon Mapping	143
8.4. SỰ SO SÁNH GIỮA CÁC KỸ THUẬT (COMPARISON OF TECHNIQUES) ..	147
8.4.1. Raytracing.....	148
8.4.2. Radiosity	148
8.4.3. Photon mapping.....	148
Tóm tắt:.....	149
Chương 9: Thư viện đồ họa OpenGL	165
9.1. Khái niệm về thư viện OpenGL.....	165
9.1.1. Giới thiệu OpenGL	165
9.1.2. Cài đặt OpenGL trong Visual C++.....	165
9.2. Vẽ hình	166
9.2.1. Cấu trúc lệnh OpenGL và các kiểu dữ liệu trong OpenGL	166
9.2.2. Chương trình OpenGL tối thiểu.....	168
9.2.3. Vẽ hình	169

9.2.4. Màu sắc.....	178
9.3. Các phép biến đổi 3D	181
9.3.1. Quá trình chuyển đổi tọa từ không gian 3D đến pixel trên màn hình.....	181
9.3.2. Thao tác trên ModelView	182
9.3.3. Thao tác trên Projection.....	183
9.3.4. Phép biến đổi cổng nhìn (viewport transformation)	184
9.4. Ánh sáng	185
9.4.1. Các loại nguồn sáng.....	185
9.4.2. Thiết lập các mảng giá trị ánh sáng	185
9.4.3. Chuyển mảng cho OpenGL	185
9.4.4. Kích hoạt nguồn sáng	186
9.4.5. Định nghĩa tính chất vật liệu.....	187
9.5. Tạo cảnh 3D.....	189
9.5.1. Sử dụng các phép biến hình OpenGL để tạo cảnh 3D.....	189
9.5.2. Sử dụng các Stack ma trận.....	190
9.5.4. Ảnh và các hiệu ứng chỉ ảnh.....	192
9.5.5. Định nghĩa các vertex và cấu trúc của chúng	193
9.6. Cài đặt game đơn giản	194
9.6.1. Phát triển game 2D đơn giản	194
9.6.2. Ví dụ game quả bóng.....	197
9.7. Bài toán game	201
9.7.1. Giới thiệu bài toán game.....	201
9.7.2. Khái quát về ứng dụng.....	201
9.7.3. Phân tích bài toán game.....	201
9.8. Thiết kế và cài đặt game	203
9.8.1. Nội dung và các chức năng chính của trò chơi.....	203
9.8.2. Đồ họa trong game	205
9.8.3. Va chạm và xử lý va chạm	206
9.8.4. Âm thanh và hình ảnh sử dụng trong game	207
9.8.5. Xây dựng các class	208
9.8.6. Quan hệ giữa các lớp	210
9.8.7. Một số giao diện cho chương trình game	210
Tóm tắt chương.....	213
PHỤ LỤC	214
1. Yêu cầu	214
2. Khởi tạo và đóng chế độ đồ họa	214
3. Các hàm cơ bản	215
3.1. Bảng màu của màn hình đồ họa.....	215

3.2. Điểm	216
3.3. Đường	216
3.4. Hình chữ nhật	216
3.5. Hình tròn.....	216
3.6. Đa giác.....	217
3.7. Văn bản.....	217
3.8. Cửa sổ (viewport)	218
3.9. Tạo hình ảnh chuyển động.....	219
Các code chương trình ví dụ cho bài tập lập trình	220
Bài 1: quay đổi tượng	220
Bài 2: xén tia.....	228
Bài 3: Phép chiếu.....	230
TÀI LIỆU THAM KHẢO	185

CHƯƠNG 1: TỔNG QUAN VỀ KỸ THUẬT ĐỒ HOẠ

1.1. CÁC KHÁI NIỆM TỔNG QUAN CỦA KỸ THUẬT ĐỒ HOẠ MÁY TÍNH (COMPUTER GRAPHICS)

1.1.1. Lịch sử phát triển

Lịch sử của đồ họa máy tính là vào thập niên 1960 được đánh dấu bởi dự án SketchPad được phát triển tại Học viện Công nghệ Massachusetts (MIT) bởi Ivan Sutherland. Các thành tựu thu được đã được báo cáo tại hội nghị Fall Joint Computer và đây cũng chính là sự kiện lần đầu tiên người ta có thể tạo mới, hiển thị và thay đổi được dữ liệu hình ảnh trực tiếp trên màn hình máy tính trong thời gian thực. Hệ thống Sketchpad này được dùng để thiết kế hệ thống mạch điện và bao gồm những thành phần sau:

- CRT màn hình
- Bút sáng và một bàn phím bao gồm các phím chức năng
- Máy tính chứa chương trình xử lý các thông tin

Với hệ thống này, người sử dụng có thể vẽ trực tiếp các sơ đồ mạch điện lên màn hình thông qua bút sáng, chương trình sẽ phân tích và tính toán các thông số cần thiết của mạch điện do người dùng vẽ nên.

Cũng trong năm 1960 này, William Fetter nhà khoa học người Mỹ. Ông đang nghiên cứu xây dựng mô hình buồng lái máy bay cho hãng Boeing của Mỹ. Ông dựa trên hình ảnh ba chiều của mô hình người phi công trong buồng lái của máy bay để xây dựng nên một mô hình tối ưu cho buồng lái máy bay. Phương pháp này cho phép các nhà thiết kế quan sát một cách trực quan vị trí của người lái trong khoang. Ông đặt tên cho phương pháp này là đồ họa máy tính (Computer Graphics) Màn hình là thiết bị thông dụng nhất trong hệ đồ họa, các thao tác của hầu hết các màn hình đều dựa trên thiết kế ống tia âm cực CRT (Cathode ray tube).

Kỹ thuật đồ họa được liên tục hoàn thiện vào thập niên 1970 với sự xuất hiện của các chuẩn đồ họa làm tăng cường khả năng giao tiếp và tái sử dụng của phần mềm cũng như các thư viện đồ họa.

Sự phát triển vượt bậc của công nghệ vi điện tử và phần cứng máy tính vào thập niên 1980 làm xuất hiện hàng loạt các vi mạch hỗ trợ cho việc truy xuất đồ họa đi cùng với sự giảm giá đáng kể của máy tính cá nhân làm đồ họa ngày càng đi sâu vào cuộc sống thực tế.

Những năm 1980 có raster graphics (đồ họa điểm). Bắt đầu chuẩn đồ họa ví dụ như: GKS(Graphics Kernel System): European effort (kết quả của châu âu), Becomes ISO 2D standard.

Thập niên 90 phát triển đặc biệt về phần cứng, thiết bị hình học đồ họa Silicon. Xuất hiện các chuẩn công nghiệp: PHIGS (Programmers Hierarchical Interactive Graphics Standard) xác định các phương pháp chuẩn cho các mô hình thời gian thực và lập trình hướng đối tượng. Giao diện người máy Human-Computer Interface (HCI).

Ngày nay xuất hiện ảnh hiện thực, các đồ họa cho máy tính (Graphics cards for PCs), game boxes và game players. Công nghiệp phim ảnh nhờ vào đồ họa máy tính (Computer graphics becoming routine in movie industry), Maya (thế giới vật chất tri giác được)....

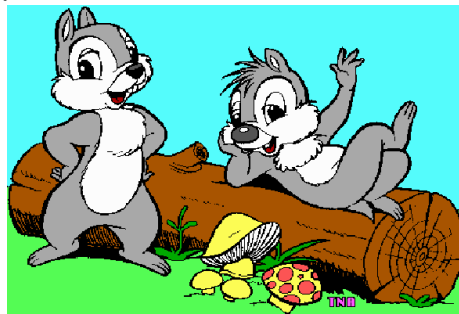
1.1.2. Kỹ thuật đồ họa vi tính. Đồ họa máy tính là một lĩnh vực của khoa học máy tính nghiên cứu về cơ sở toán học, các thuật toán cũng như các kỹ thuật để cho phép tạo, hiển thị và điều khiển hình ảnh trên màn hình máy tính. Đồ họa máy tính có liên quan ít nhiều đến một số lĩnh vực như đại số, hình học giải tích, hình học họa hình, quang học,... và kỹ thuật máy tính, đặc biệt là chế tạo phần cứng (các loại màn hình, các thiết bị xuất, nhập, các vi mạch đồ họa...).

Theo nghĩa rộng hơn, đồ họa máy tính là phương pháp và công nghệ dùng trong việc chuyển đổi qua lại giữa dữ liệu và hình ảnh trên màn hình bằng máy tính. Đồ họa máy tính hay kỹ thuật đồ họa máy tính còn được hiểu dưới dạng phương pháp và kỹ thuật tạo hình ảnh từ các mô hình toán học mô tả các đối tượng hay dữ liệu lấy được từ các đối tượng trong thực tế.

1.2. CÁC KỸ THUẬT ĐỒ HỌA

1.2.1. Kỹ thuật đồ họa điểm (Sample based-Graphics)

- Các mô hình, hình ảnh của các đối tượng được hiển thị thông qua từng pixel (từng mẫu rời rạc)
- Đặc điểm: Có thể thay đổi thuộc tính của từng điểm ảnh rời rạc
 - Xoá đi từng pixel của mô hình và hình ảnh các đối tượng.
 - Các mô hình hình ảnh được hiển thị như một lưới điểm (grid) các pixel rời rạc,
 - Từng pixel đều có vị trí xác định, được hiển thị với một giá trị rời rạc (số nguyên) các thông số hiển thị (màu sắc hoặc độ sáng)
- Tập hợp tất cả các pixel của grid cho chúng ta mô hình, hình ảnh đối tượng mà chúng ta muốn hiển thị.



Hình 1.1 Ảnh đồ họa điểm

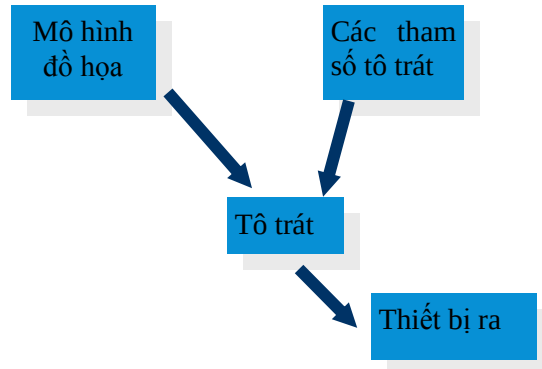
Phương pháp để tạo ra các pixel Phương pháp dùng phần mềm để vẽ trực tiếp từng pixel một.

- Dựa trên các lý thuyết mô phỏng (lý thuyết Fractal, v.v) để xây dựng nên hình ảnh mô phỏng của sự vật.

Chương 1: Tổng quan về kỹ thuật đồ hoạ

- Phương pháp rời rạc hoá (số hoá) hình ảnh thực của đối tượng.
- Có thể sửa đổi (image editing) hoặc xử lý (image processing) mảng các pixel thu được theo những phương pháp khác nhau để thu được hình ảnh đặc trưng của đối tượng.

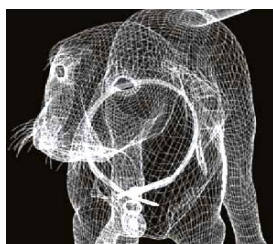
1.2.2. Kỹ thuật đồ hoạ vector



Hình 1.2 Mô hình đồ hoạ vector

- Mô hình hình học (geometrical model) của đối tượng
- Xác định các thuộc tính của mô hình hình học này,
- Quá trình tô trát (rendering) để hiển thị từng điểm của mô hình, hình ảnh thực của đối tượng

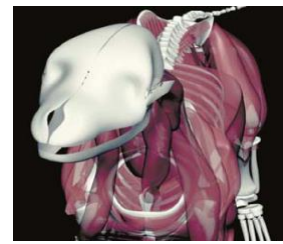
Ví dụ về hình ảnh đồ hoạ Vector



Wireframe Model



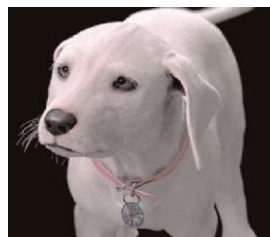
Skeletal Model



Muscle Model



Skin Model



Hair Model



Render and Touch up

Hình 1.3 Ví dụ về đồ hoạ vector

Có thể định nghĩa đồ hoạ vector: **Đồ hoạ vector = geometrical model + rendering**

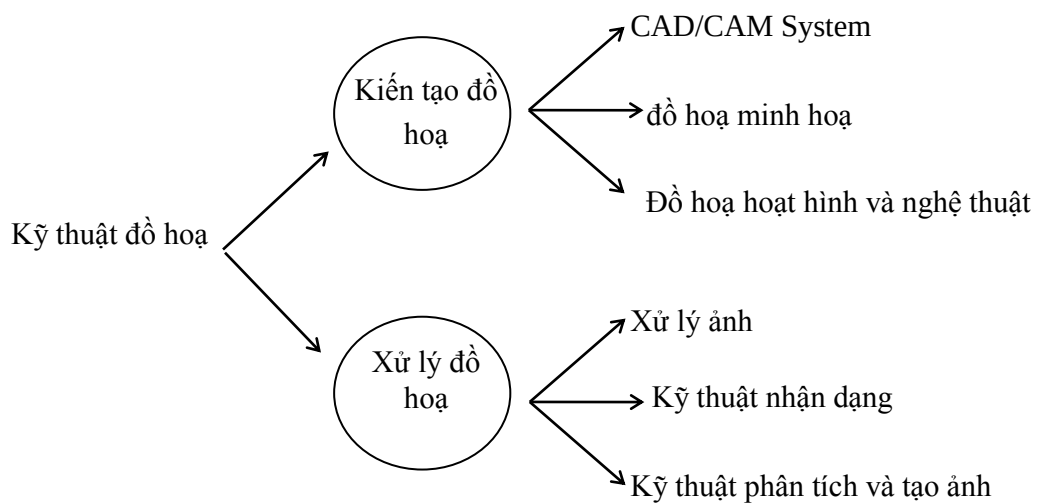
Chương 1: Tổng quan về kỹ thuật đồ họa

So sánh giữa Raster và Vector Graphics

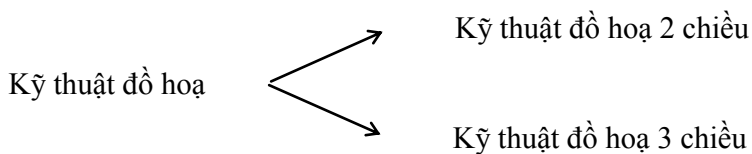
Đồ họa điểm(Raster Graphics) - Hình ảnh và mô hình của các vật thể được biểu diễn bởi tập hợp các điểm của lưới (grid) - Thay đổi thuộc tính của các pixel => thay đổi từng phần và từng vùng của hình ảnh. - Copy được các pixel từ một hình ảnh này sang hình ảnh khác.	Đồ họa vector(Vector Graphics) - Không thay đổi thuộc tính của từng điểm trực tiếp - Xử lý với từng thành phần hình học cơ sở của nó và thực hiện quá trình tô trát và hiển thị lại. - Quan sát hình ảnh và mô hình của hình ảnh và sự vật ở nhiều góc độ khác nhau bằng cách thay đổi điểm nhìn và góc nhìn.
---	--

1.2.3. Phân loại của đồ họa máy tính

Phân loại theo các lĩnh vực của đồ họa máy tính



Phân loại theo hệ tọa độ



▪ *Kỹ thuật đồ họa hai chiều:* là kỹ thuật đồ họa máy tính sử dụng hệ tọa độ hai chiều (hệ tọa độ phẳng), sử dụng rất nhiều trong kỹ thuật xử lý bản đồ, đồ thị.

▪ *Kỹ thuật đồ họa ba chiều:* là kỹ thuật đồ họa máy tính sử dụng hệ tọa độ ba chiều, đòi hỏi rất nhiều tính toán và phức tạp hơn nhiều so với kỹ thuật đồ họa hai chiều.

Các lĩnh vực của đồ họa máy tính:

Kỹ thuật xử lý ảnh (*Computer Imaging*): sau quá trình xử lý ảnh cho ta ảnh số của đối tượng. Trong quá trình xử lý ảnh sử dụng rất nhiều các kỹ thuật phức tạp: kỹ thuật khôi phục ảnh, kỹ thuật làm nổi ảnh, kỹ thuật xác định biên ảnh.

Kỹ thuật nhận dạng (*Pattern Recognition*): từ những ảnh mẫu có sẵn ta phân loại theo cấu trúc, hoặc theo các tiêu chí được xác định từ trước và bằng các thuật toán chọn lọc để có thể phân tích hay tổng hợp ảnh đã cho thành một tập hợp các ảnh gốc, các ảnh gốc này được lưu trong một thư viện và căn cứ vào thư viện này ta xây dựng được các thuật giải phân tích và tổ hợp ảnh.

Chương 1: Tổng quan về kỹ thuật đồ hoạ

Kỹ thuật tổng hợp ảnh (*Image Synthesis*): là lĩnh vực xây dựng mô hình và hình ảnh của các vật thể dựa trên các đối tượng và mối quan hệ giữa chúng.

Các hệ CAD/CAM (Computer Aided Design/Computer Aided Manufacture System): kỹ thuật đồ hoạ tập hợp các công cụ, các kỹ thuật trợ giúp cho thiết kế các chi tiết và các hệ thống khác nhau: hệ thống cơ, hệ thống điện, hệ thống điện tử....

Đồ hoạ trình bày (*Presentation Graphics*): gồm các công cụ giúp hiển thị các số liệu thí nghiệm một cách trực quan, dựa trên các mẫu đồ thị hoặc các thuật toán có sẵn.

Đồ hoạ hoạt hình và nghệ thuật: bao gồm các công cụ giúp cho các hoạ sĩ, các nhà thiết kế phim hoạt hình chuyên nghiệp làm các kỹ xảo hoạt hình, vẽ tranh... Ví dụ: phần mềm 3D Studio, 3D Animation, 3D Studio Max.

1.2.4. Các ứng dụng tiêu biểu của kỹ thuật đồ hoạ

Đồ hoạ máy tính là một trong những lĩnh vực lý thú nhất và phát triển nhanh nhất của tin học. Ngay từ khi xuất hiện nó đã có sức lôi cuốn mãnh liệt, cuốn hút rất nhiều người ở nhiều lĩnh vực khác nhau như khoa học, nghệ thuật, kinh doanh, quản lý... Tính hấp dẫn của nó có thể được minh hoạ rất trực quan thông qua các ứng dụng của nó.

▪ Xây dựng giao diện người dùng (*User Interface*)

Giao diện đồ hoạ thực sự là cuộc cách mạng mang lại sự thuận tiện và thoải mái cho người dùng ứng dụng. Giao diện WYSIWYG và WIMP đang được đa số người dùng ưu thích nhờ tính thân thiện, dễ sử dụng của nó.

▪ Tạo các biểu đồ trong thương mại, khoa học, kỹ thuật

Các ứng dụng này thường được dùng để tóm lược các dữ liệu về tài chính, thống kê, kinh tế, khoa học, toán học... giúp cho nghiên cứu, quản lý... một cách có hiệu quả.

▪ Tự động hoá văn phòng và chế bản điện tử

▪ Thiết kế với sự trợ giúp của máy tính (*CAD_CAM*)

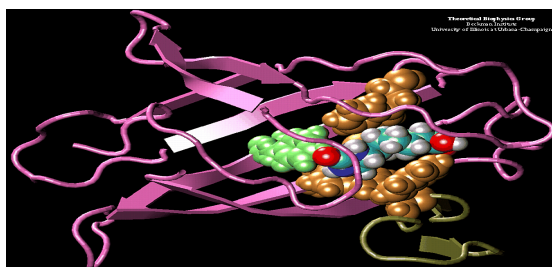
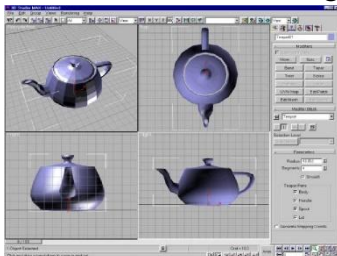
▪ Lĩnh vực giải trí, nghệ thuật và mô phỏng

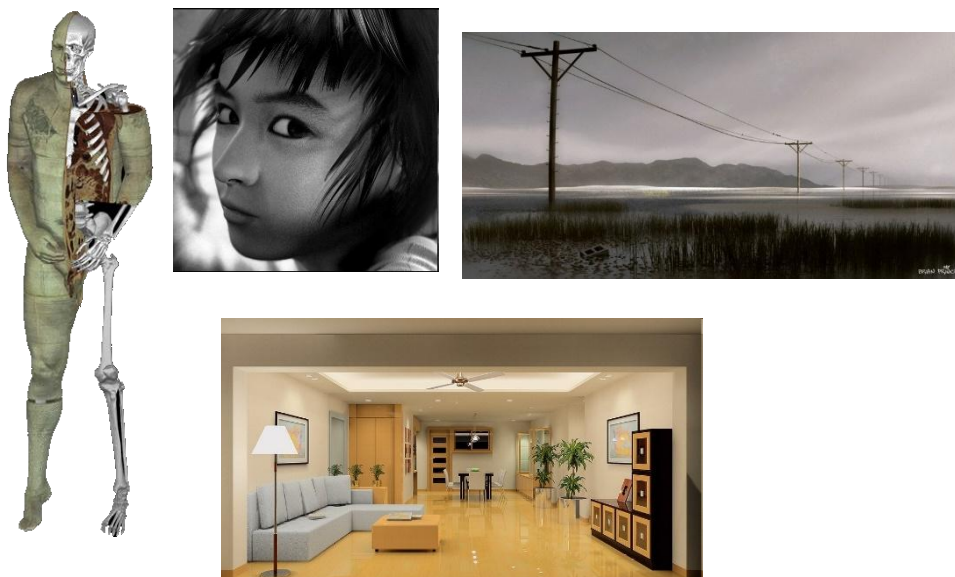
▪ Điều khiển các quá trình sản xuất (*Process Control*)

▪ Lĩnh vực bản đồ (*Cartography*)

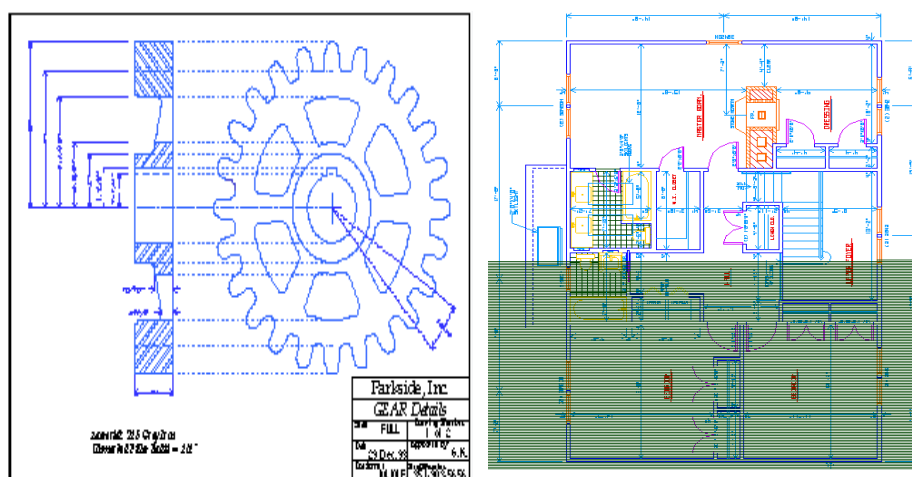
▪ Giáo dục và đào tạo

Một số ví dụ của ứng dụng kỹ thuật đồ hoạ:





Hình 1.4 Các ứng dụng của kỹ thuật đồ họa



Hình 1.5 Hệ ứng dụng CAD - CAM

1.2.5. Các chuẩn giao diện của hệ đồ họa

Mục tiêu căn bản của các chuẩn cho phần mềm đồ họa là đảm bảo tính tương thích. Khi các công cụ được thiết kế với hàm đồ họa chuẩn, phần mềm có thể được di chuyển một cách dễ dàng từ hệ phần cứng này sang hệ phần cứng khác và được dùng trong nhiều cài đặt và ứng dụng khác nhau.

GKS (Graphics Kernel System): chuẩn xác định các hàm đồ họa chuẩn, được thiết kế như một tập hợp các công cụ đồ họa hai chiều và ba chiều.

GKS Functional Description, ANSI X3.124 - 1985. GKS - 3D Functional Description, ISO Doc #8805:1988. CGI (Computer Graphics Interface System): hệ chuẩn cho các phương pháp giao tiếp với các thiết bị ngoại vi.

CGM (Computer Graphics Metafile): xác định các chuẩn cho việc lưu trữ và chuyển đổi hình ảnh.

Chương 1: Tổng quan về kỹ thuật đồ họa

VRML (Virtual Reality Modeling Language): ngôn ngữ thực tại ảo, một hướng phát triển trong công nghệ hiển thị được đề xuất bởi hãng Silicon Graphics, sau đó đã được chuẩn hóa như một chuẩn công nghiệp.

PHIGS (Programmers Hierarchical Interactive Graphics Standard): xác định các phương pháp chuẩn cho các mô hình thời gian thực và lập trình hướng đối tượng.

PHIGS Functional Description, ANSI X3.144 - 1985.+ Functional Description, 1988, 1992.

OpenGL thư viện đồ họa của hãng Silicon Graphics, được xây dựng theo đúng chuẩn của một hệ đồ họa năm 1993.

DIRECTX thư viện đồ họa của hãng Microsoft, Direct X/Direct3D 1997

1.3. PHẦN CỨNG ĐỒ HOẠ (GRAPHICS HARDWARE)

1.3.1. Các thành phần phần cứng của hệ đồ họa tương tác

CPU: thực hiện các chương trình ứng dụng.

Bộ xử lý hiển thị (Display Processor): thực hiện công việc hiển thị dữ liệu đồ họa.

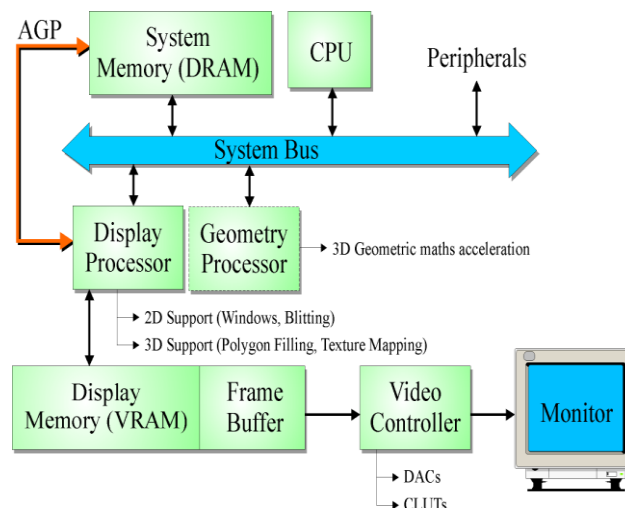
Bộ nhớ hệ thống (System Memory): chứa các chương trình và dữ liệu đang thực hiện.

Gói phần mềm đồ họa (Graphics Package): cung cấp các hàm đồ họa cho chương trình ứng dụng

Phần mềm ứng dụng (Application Program): phần mềm đồ họa ứng dụng.

Bộ đệm (Frame buffer): có nhiệm vụ chứa các hình ảnh hiển thị.

Bộ điều khiển màn hình (Video Controller): điều khiển màn hình, chuyển dữ liệu dạng số ở frame buffer thành các điểm sáng trên màn hình.



Hình 1.6 Các thành phần phần cứng của hệ đồ họa tương tác

1.3.2. Máy in

Dot size: đường kính của một điểm in bé nhất mà máy in có thể in được

Addressability: khả năng địa chỉ hoá các điểm in có thể có trên một đơn vị độ dài (dot per inch)

Chương 1: Tổng quan về kỹ thuật đồ hoạ

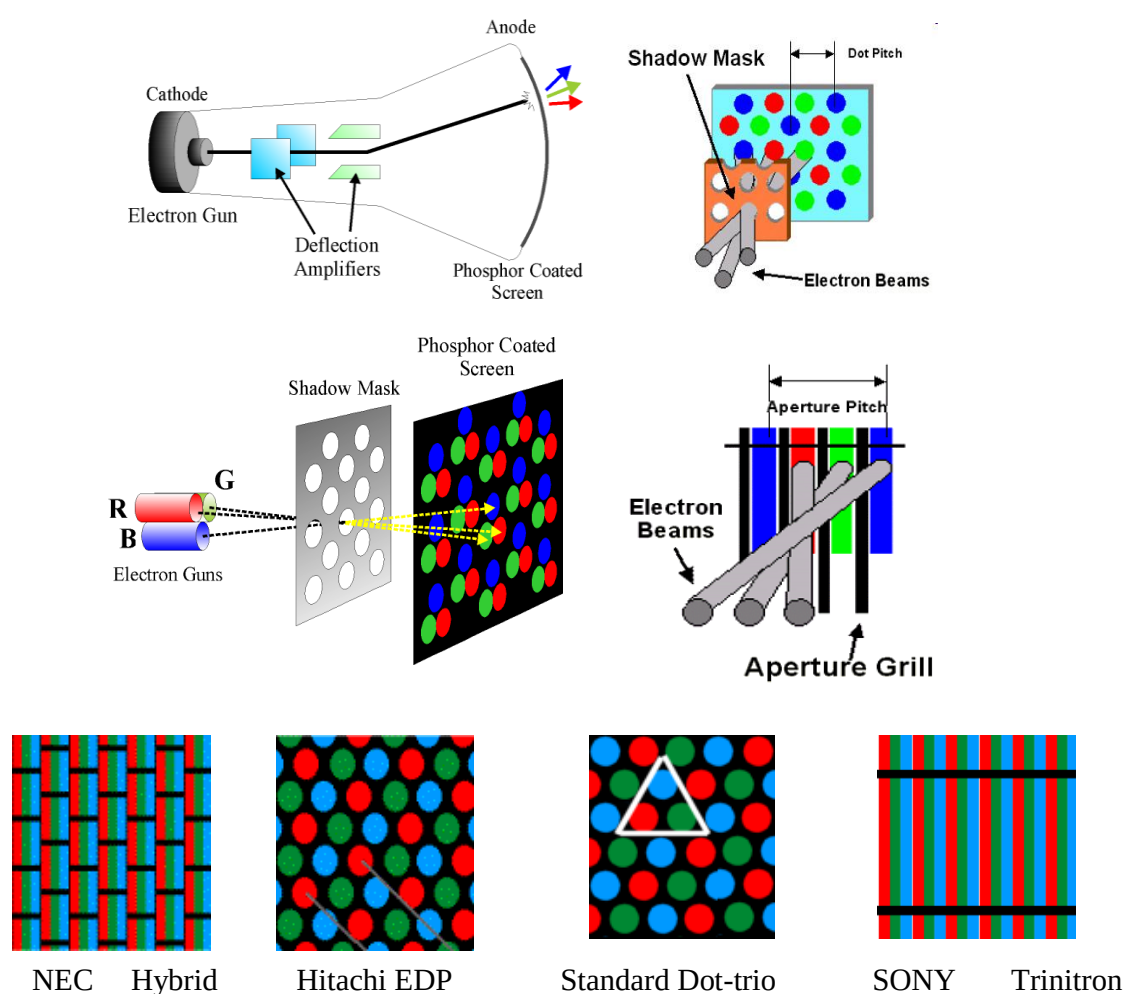
Dot size	Point per inch		
8 - 20/ 100 inch	200, 600		
5/1000 inch	1500	Máy vẽ	6,15/1000 inch

1000, 2000 **1.3.3. Màn hình CRT** Một chùm các tia điện tử (tia âm cực) phát ra từ một súng điện tử, vượt qua cuộn lái tia dẫn đến vị trí xác định trên màn hình được phủ một lớp phosphor. Tại mỗi vị trí tương tác với tia điện tử hạt phosphor sẽ phát lên một chấm sáng nhỏ. Nhưng chấm sáng sẽ mờ dần rất nhanh nên cần có cách nào nó duy trì ảnh trên màn hình. Một trong các cách là: lặp đi lặp lại nhiều lần việc vẽ lại ảnh thật nhanh bằng cách hướng các tia điện tử trở lại vị trí cũ. Gọi là làm tươi (refresh CRT).

Số lượng tối đa các điểm có thể hiển thị trên một CRT được gọi là độ phân giải (Resolution). Hay độ phân giải là số lượng các điểm có thể được vẽ theo chiều ngang và chiều dọc (được xem như tổng số điểm theo mỗi hướng) của màn hình.

Kích thước vật lý của màn hình đồ hoạ được tính từ độ dài của đường chéo màn hình. Thường dao động từ 12-27 inch, hoặc lớn hơn.

Thuộc tính khác của màn hình là tỷ số phương (aspect ratio). Nó là tỷ lệ của các điểm dọc và các điểm ngang cần để phát sinh các đoạn thẳng có độ dài đơn vị theo cả hai hướng trên màn hình. Màn hình có tỷ số phương khác một, thì hình vuông hiển thị trên đó thành hình chữ nhật còn hình tròn thành hình ellipse.



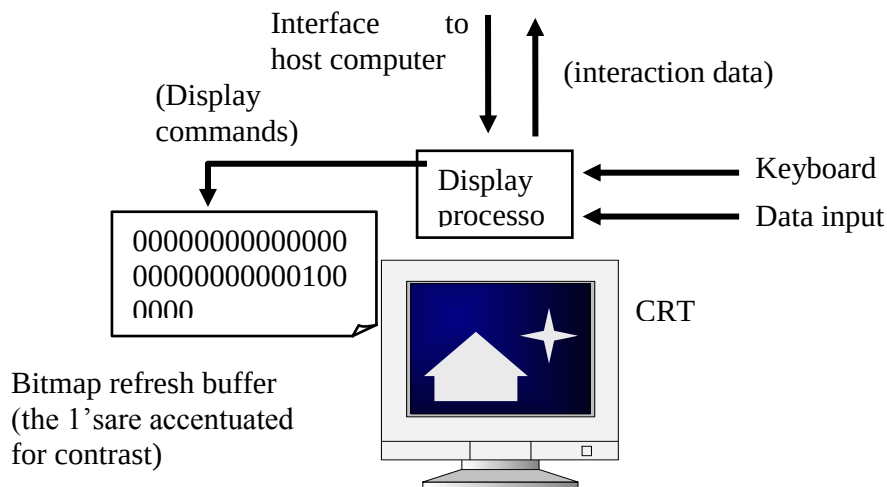
Hình 1.7 Công nghệ màn hình CRT

Chương 1: Tổng quan về kỹ thuật đồ hoạ

Màn hình dạng điểm (Raster Display): thường gặp nhất trong số các dạng màn hình sử dụng CRT trên công nghệ truyền hình. Mỗi điểm trên màn hình được gọi là pixel. Các thông tin về ảnh hiển thị trên màn hình được lưu trữ trong một vùng bộ nhớ gọi là vùng đệm làm tươi (Refresh buffer) hay là vùng đệm khung (Frame Buffer). Vùng lưu trữ tập các giá trị cường độ sáng của toàn bộ các điểm trên màn hình và luôn tồn tại một cách song ánh giữa mỗi điểm trên màn hình và mỗi phần tử trong vùng này.

Để tạo ra hình ảnh đen trắng, đơn giản chỉ cần lưu thông tin của mỗi Pixel là một bit (0,1) (xem hình 1.8). Trong trường hợp ảnh nhiều màu thì cần nhiều bit hơn, nếu thông tin mỗi pixel được lưu bằng b bit thì ta có thể có 2^b giá trị màu phân biệt cho pixel đó.

Ví dụ mô hình đồ hoạ điểm ngôi nhà và ngôi sao.

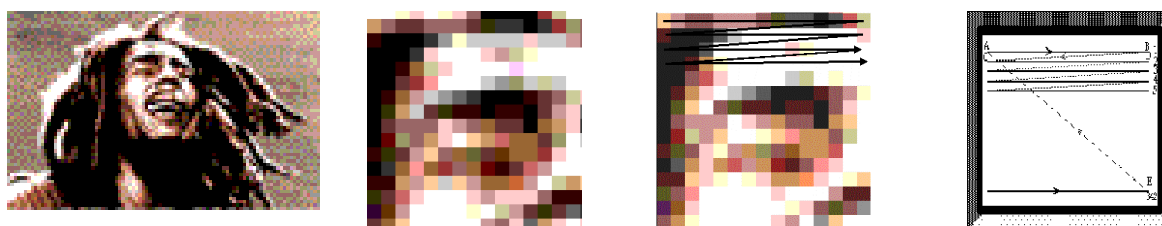


Hình 1.8 Song ánh giữa vùng đệm khung và màn hình

Trong các màn hình màu, người ta định nghĩa tập các màu làm việc trong một bảng tra (LookUp Table - LUT). Mỗi phần tử của LUT được định nghĩa một bộ ba giá trị (RGB) mô tả một màu nào đó. Khi cần sử dụng một màu, ta chỉ cần chỉ định số thứ tự (index) tương ứng của màu đó trong LUT, số phần tử trong bảng LUT chính là số màu có thể được hiển thị cùng một lúc trên màn hình.

X: 0 , Xmax2 màu/ 1 bit	640 x 480 x 16 → Video RAM = 0.59MB
Y: 0 , Ymax16 màu/ 4 bit ;256 màu/ 8bit	1024 x 1024 x 24 → Video RAM = 8MB
2^{16} màu/ 16 bit ; 2^{24} màu/ 24 bit	

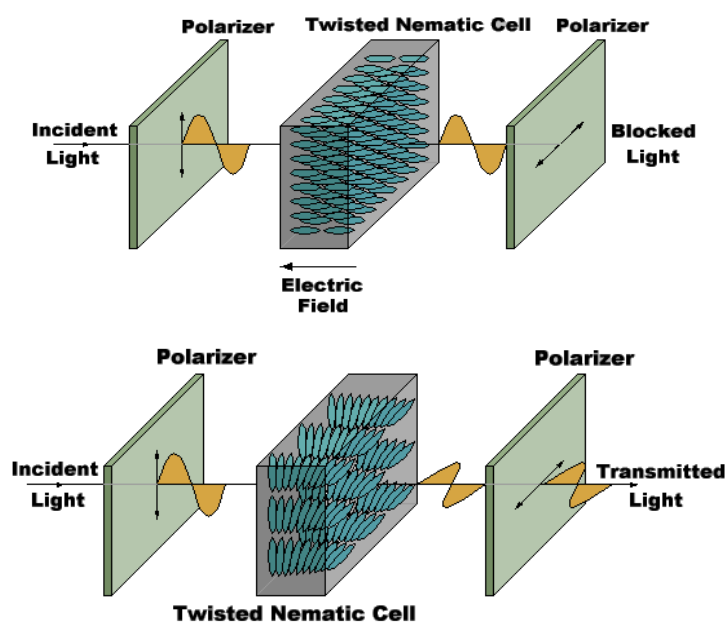
Việc làm tươi trên màn hình dạng này được thực hiện ở tốc độ 60 - 80 khung/giây. Đôi khi tốc độ làm tươi còn được biểu diễn bằng đơn vị Hertz (Hz - số chu kỳ trên/giây), trong đó một chu kỳ tương ứng với một khung (frame). Vậy tốc độ làm tươi 60 khung/giây đơn giản là 60 Hz. Khi đạt đến cuối mỗi dòng quét, tia điện tử quay trở lại bên trái của màn hình để bắt đầu dòng quét kế tiếp. Việc quay trở về bên trái màn hình sau khi làm tươi mỗi dòng quét được gọi là tia hồi ngang (Horizontal retrace). Và tới cuối mỗi frame, tia điện tử (tia hồi dọc - Vertical retrace) quay trở lại góc bên trái của màn hình để chuẩn bị bắt đầu frame kế tiếp.



Hình 1.9 Quét màn và quét dòng của màn hình CRT

1.3.4. Màn hình tinh thể lỏng (Liquid Crystal Display – LCD)

Dựa vào công nghệ truyền ánh sáng qua điện cực mà đặt giữa là cuộn dây xoắn. Khi chưa có từ trường (chưa có dòng điện) ở cuộn dây thì ánh sáng truyền thẳng, khi có từ trường thì ánh sáng truyền đổi chiều.



Hình 1.10 Công nghệ truyền ánh sáng trong màn hình tinh thể lỏng

CRT Displays (màn hình CRT)

Advantages (ưu điểm)	Disadvantages (nhược điểm)
Đáp ứng nhanh (có độ phân giải cao)	Lớn và nặng (typ. 70x70 cm, 15 kg)
Màu sắc đa dạng (Có độ sâu và rộng)	Tiêu tốn nguồn điện cao (typ. 140W)
Màu sắc bão hoà và tự nhiên	Có hại cho sức khoẻ vì trường điện từ và từ tính
Công nghệ không quá đắt và hoàn thiện	Màn hình nhấp nháy (at 50-80 Hz)
Góc nhìn rộng, tương phản và độ sáng cao	Hình hay bị méo tại 4 góc

LCD Displays (màn hình tinh thể lỏng)

Advantages (ưu điểm)	Disadvantages (nhược điểm)
Hình dáng nhỏ, trọng lượng nhẹ (approx 1/6 of CRT, typ. 1/5 of CRT)	Giá thành cao (presently 3x CRT)
Tiêu tốn nguồn thấp (typ. 1/4 of CRT)	Góc nhìn hẹp hơn (typ. +/- 50 degrees)
Màn hình phẳng tuyệt đối nên không méo tại các góc	độ tương phản thấp (typ. 1:100)
	độ chói (độ ngời) thấp hơn (typ. 200 cd/m ²)

Chương 1: Tổng quan về kỹ thuật đồ họa

Màu sắc đều, ảnh sinh động Không bị hiệu ứng điện từ trường Có thể màn hình vừa lớn vừa rộng (>20 inch)	
---	--

Tóm tắt chương:

Sự ra đời của đồ họa máy tính thực sự là cuộc cách mạng trong giao tiếp giữa người dùng và máy tính. Với lượng thông tin trực quan, đa dạng và phong phú được truyền tải qua hình ảnh. Các ứng dụng đồ họa máy tính đã lôi cuốn nhiều người nhờ tính thân thiện, dễ dùng, kích thích khả năng sáng tạo và tăng đáng kể hiệu suất làm việc.

Đồ họa máy tính ngày nay được ứng dụng rất rộng rãi trong nhiều lĩnh vực khoa học, kỹ thuật, nghệ thuật, kinh doanh, quản lý... Các ứng dụng đồ họa rất đa dạng, phong phú và phát triển liên tục không ngừng. Ngày nay, hầu như không có chương trình ứng dụng nào mà không sử dụng kỹ thuật đồ họa để làm tăng tính hấp dẫn cho mình.

Một hệ thống đồ họa bao giờ cũng gồm hai phần chính đó là phần cứng và phần mềm. Phần cứng bao gồm các thiết bị hiển thị (thiết bị xuất) và các thiết bị nhập. Tiêu biểu nhất là màn hình, có hai loại màn hình thông dụng là CRT và LCD.

Bài tập:

1. Cấu tạo và nguyên lý hoạt động của màn hình dạng điểm. Nêu các khái niệm vùng đệm khung, độ phân giải, tỷ số phương.... của màn hình loại này?
2. Ý nghĩa và hoạt động của bảng tra LUT?
3. Tính Video Ram của các màn hình lần lượt có độ phân giải là 640x480, 1024x768, 1280x1024 mà có mỗi pixel được mô tả lần lượt là 8bít, 12 bit, 24 bit.
4. Nếu chúng ta dùng các giá trị 12bit cho mỗi pixel trong một bảng tham chiếu lookup table, có bao nhiêu hạng mục mà lookup table có được?
5. Tại sao phải chuẩn hóa các phần mềm đồ họa? Liệt kê các chuẩn hóa đó.

CHƯƠNG 2: CÁC GIẢI THUẬT SINH THỰC THỂ CƠ SỞ

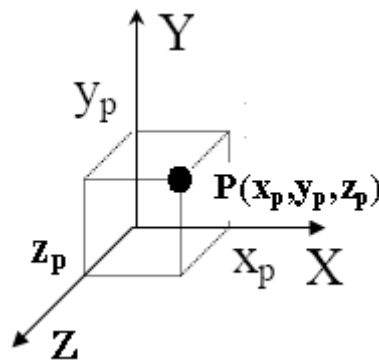
2.1. CÁC HỆ THỐNG TOẠ ĐỘ TRONG ĐỒ HOẠ

Trong lĩnh vực kỹ thuật đồ họa, chúng ta phải hiểu được rằng thực chất của đồ họa là làm thế nào để có thể mô tả và biến đổi được các đối tượng trong thế giới thực trên máy tính. Bởi vì, các đối tượng trong thế giới thực được mô tả bằng tọa độ thực. Trong khi đó, hệ tọa độ thiết bị lại sử dụng hệ tọa độ nguyên để hiển thị các hình ảnh. Đây chính là vấn đề cơ bản cần giải quyết. Ngoài ra, còn có một khó khăn khác nữa là với các thiết bị khác nhau thì có các định nghĩa khác nhau. Do đó, cần có một phương pháp chuyển đổi tương ứng giữa các hệ tọa độ và đối tượng phải được định nghĩa bởi các thành phần đơn giản như thế nào để có thể mô tả gần đúng với hình ảnh thực bên ngoài.

Hai mô hình cơ bản của ứng dụng đồ họa là dựa trên mẫu số hóa và dựa trên đặc trưng hình học. Trong ứng dụng đồ họa dựa trên mẫu số hóa thì các đối tượng đồ họa được tạo ra bởi lưới các pixel rời rạc. Các pixel này có thể được tạo ra bằng các chương trình vẽ, máy quét, ... Các pixel này mô tả tọa độ xác định vị trí và giá trị mẫu. Thuận lợi của ứng dụng này là dễ dàng thay đổi ảnh bằng cách thay đổi màu sắc hay vị trí của các pixel, hoặc di chuyển vùng ảnh từ nơi này sang nơi khác. Tuy nhiên, điều bất lợi là không thể xem xét đối tượng từ các góc nhìn khác nhau. Ứng dụng đồ họa dựa trên đặc trưng hình học bao gồm các đối tượng đồ họa cơ sở như đoạn thẳng, đa giác,.... Chúng được lưu trữ bằng các mô hình và các thuộc tính. Ví dụ : đoạn thẳng được mô hình bằng hai điểm đầu và cuối, có thuộc tính như màu sắc, độ dày. Người sử dụng không thao tác trực tiếp trên các pixel mà thao tác trên các thành phần hình học của đối tượng.

2.1.1. Hệ tọa độ thực (WCS – World Coordinate System)

Một trong những hệ tọa độ thực thường được dùng để mô tả các đối tượng trong thế giới thực là hệ tọa độ Descartes. Với hệ tọa độ này, mỗi điểm P được biểu diễn bằng một cặp tọa độ $P(x_p, y_p, z_p)$ với $x_p, y_p, z_p \in \mathbb{R}$



Hình 2.1 Hệ tọa độ thực.

Ox, Oy, Oz là trục tọa độ

x_p, y_p, z_p : tọa độ của P

2.1.2. Hệ tọa độ thiết bị (DCS – Device Coordinate System)

Hệ tọa độ thiết bị (device coordinates) được dùng cho một thiết bị xuất cụ thể nào đó, ví dụ như máy in, màn hình,.. Trong hệ tọa độ thiết bị thì các điểm cũng được mô tả bởi cặp tọa độ (x,y) . Tuy nhiên, khác với hệ tọa độ thực là $x, y \in \mathbb{N}$. Điều này có nghĩa là các điểm trong hệ tọa độ thực được định nghĩa liên tục, còn các điểm trong hệ tọa độ thiết bị là rời rạc. Ngoài ra, các tọa độ x, y của hệ tọa độ thiết bị chỉ biểu diễn được trong một giới hạn nào đó của $\mathbb{N}$. Ví dụ : Độ phân giải của màn hình trong chế độ đồ họa là 640x480. Khi đó, $x \in (0,639)$ và $y \in (0,479)$ (xem hình 2.2).



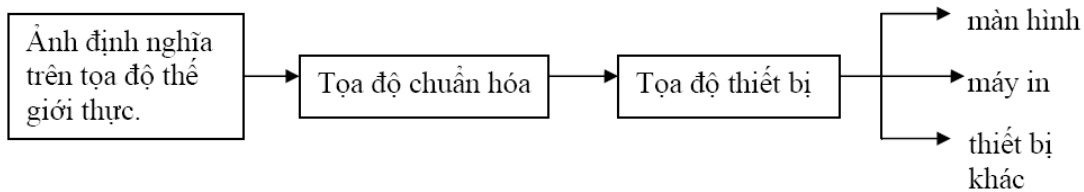
Hình 2.2 Hệ tọa độ trên màn hình

2.1.3. tọa độ thiết bị chuẩn (NDCS – Normalized Device Coordinate System)

Do cách định nghĩa các hệ tọa độ thiết bị khác nhau nên một hình ảnh hiển thị được trên thiết bị này là chính xác thì chưa chắc hiển thị chính xác trên thiết bị khác. Người ta xây dựng một hệ tọa độ thiết bị chuẩn đại diện chung cho tất cả các thiết bị để có thể mô tả các hình ảnh mà không phụ thuộc vào bất kỳ thiết bị nào.

Trong hệ tọa độ chuẩn, các tọa độ x, y sẽ được gán các giá trị trong đoạn từ $[0,1]$. Như vậy, vùng không gian của hệ tọa độ chuẩn chính là hình vuông đơn vị có góc trái dưới $(0, 0)$ và góc phải trên là $(1, 1)$.

Quá trình mô tả các đối tượng thực như sau (xem hình 2.3):



Hình 2.3 Hệ tọa độ trên màn hình.

2.2. ĐIỂM VÀ ĐOẠN THẲNG

2.2.1. Điểm

Trong hệ tọa độ hai chiều (mặt phẳng) thì điểm được biểu diễn $P(x,y)$, ngoài ra nó còn có tính chất màu sắc. Ví dụ để vẽ một điểm ta có hàm $putpixel(x,y,color)$.

2.2.2. Đoạn thẳng

▪ Biểu diễn tường minh: $y = f(x)$

Một đoạn thẳng được xác định nếu biết 2 điểm thuộc nó. Phương trình đoạn thẳng đi qua 2 điểm $P(x_1, y_1)$ và $Q(x_2, y_2)$ như sau:

$$(y - y_1) / (x - x_1) = (y_2 - y_1) / (x_2 - x_1)$$

$$(y-y_1)(x_2-x_1)=(x-x_1)(y_2-y_1)$$

$$(x_2-x_1)y=(y_2-y_1)x + y_1(x_2-x_1) - x_1(y_2-y_1)$$

$$y = ((y_2-y_1)/(x_2-x_1))x + y_1 - ((y_2-y_1)/(x_2-x_1))x_1$$

$$y = kx + b$$

$$k = (y_2-y_1)/(x_2-x_1) \text{ Độ dốc hay hệ số góc của đường}$$

$$b = y_1 - kx_1 \text{ Đoạn chắn trên trục y}$$

$$\Delta y = k\Delta x \text{ (tức là khi x thay đổi thì y thay đổi theo)}$$

▪ Biểu diễn không tường minh: $ax+by+c=0$

$$\text{Ta có: } (y_2-y_1)x - (x_2-x_1)y + (x_2-x_1)y_1 - (y_2-y_1)x_1 = 0$$

$$(y_2-y_1)x - (x_2-x_1)y + x_2y_1 - x_1y_2 = 0$$

$$\text{hay } ax + by + c = 0$$

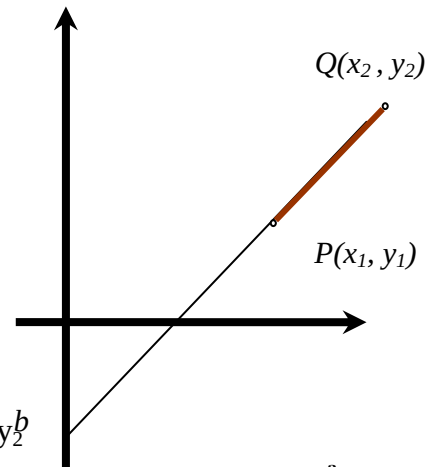
$$\text{Trong đó } a = (y_2-y_1), b = -(x_2-x_1) \text{ và } c = x_2y_1 - x_1y_2$$

▪ Biểu diễn thông qua tham số:

$$P(u) = P_1 + u(P_2 - P_1) \quad \text{có } u \in [0,1]$$

$$x(u) = x_1 + u(x_2 - x_1)$$

$$y(u) = y_1 + u(y_2 - y_1)$$



Hình 2.4 Vẽ đoạn thẳng PQ

2.3. CÁC GIẢI THUẬT XÂY DỰNG THỰC THỂ CƠ SỞ

2.3.1. Giải thuật vẽ đoạn thẳng thông thường

Nguyên lý chung: cho một thành phần tọa độ x hay y biến đổi theo từng đơn vị và tính độ nguyên còn lại sao cho gần với tọa độ thực nhất.

$$\text{Ta có } y = \frac{y_2 - y_1}{x_2 - x_1}(x - x_1) + y_1$$

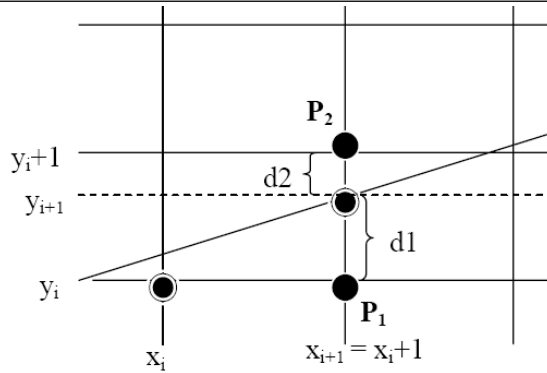
Cho x thay đổi tìm y, trong bài này cho x_1 thay đổi tiến tới x_2 ta chọn đơn vị nhỏ nhất của màn hình $\Delta x=1$.

Giải thuật thông thường:

```
void dline(int x1,int y1, int x2,int y2, int color) {
    float y;    int x;
    for (x=x1; x<=x2; x++) {
        y = y1 + (x-x1)*(y2-y1)/(x2-x1) ;
        putpixel(x, Round(y), color );
    } }
```

2.3.2. Giải thuật Bresenham

1960 Bresenham thuộc IBM tìm ra các điểm gần với đường thẳng dựa trên độ phân giải hữu hạn. Giải thuật này loại bỏ được các phép toán chia và phép toán làm tròn như ta đã thấy trong giải thuật trên. Xét đoạn thẳng với $0 < k < 1$



Hình 2.5 Mô tả giải thuật Bresenham ($0 < k < 1$)

Gọi (x_{i+1}, y_{i+1}) là điểm thuộc đoạn thẳng, ta có $y_{i+1} = k(x_{i+1}) + b$

$$d_1 = y_{i+1} - y_i = k(x_{i+1}) + b - y_i$$

$$d_2 = y_{i+1} - y_{i+1} = y_{i+1} - k(x_{i+1}) - b - \text{Nếu } d_1 \leq d_2 \Rightarrow y_{i+1} = y_i$$

$$\text{- Ngược lại } d_1 > d_2 \Rightarrow y_{i+1} = y_i + 1$$

$$\text{Đặt } D = d_1 - d_2 = 2k(x_{i+1}) - 2y_i + 2b - 1$$

$$\text{Có } k = \Delta y / \Delta x \text{ và đặt } P_i = \Delta x D = \Delta x (d_1 - d_2)$$

$$P_i = \Delta x (2\Delta y / \Delta x (x_{i+1}) - 2y_i + 2b - 1) = 2\Delta y x_{i+1} + 2\Delta y - 2\Delta x y_i + 2b\Delta x - \Delta x$$

Ta tính bước tiếp:

$$P_{i+1} = 2\Delta y x_{i+1} + 2\Delta y - 2\Delta x y_{i+1} + 2b\Delta x - \Delta x$$

$$P_{i+1} - P_i = -2\Delta x (y_{i+1} - y_i) + 2\Delta y (x_{i+1} - x_i)$$

Có $x_{i+1} = x_i + 1$ nên:

$$P_{i+1} - P_i = -2\Delta x (y_{i+1} - y_i) + 2\Delta y = 2\Delta y - 2\Delta x (y_{i+1} - y_i)$$

$$\text{Nếu } P_i \leq 0 \text{ thì } y_{i+1} = y_i \rightarrow P_{i+1} = P_i + 2\Delta y$$

$$\text{Nếu } P_i > 0 \text{ thì } y_{i+1} = y_i + 1 \rightarrow P_{i+1} = P_i + 2\Delta y - 2\Delta x$$

Tính giá trị đầu: P_1 ?

$$P_1 = \Delta x (d_1 - d_2) = \Delta x (2\Delta y / \Delta x (x_1 + 1) - 2y_1 + 2b - 1)$$

$$= 2\Delta y x_1 + 2\Delta y - 2\Delta x y_1 + 2b\Delta x - \Delta x$$

$$\text{Có } y_1 = kx_1 + b = \Delta y / \Delta x x_1 + b$$

$$P_1 = 2\Delta y x_1 + 2\Delta y - 2\Delta x ((\Delta y / \Delta x) x_1 + b) + 2b\Delta x - \Delta x$$

$$= 2\Delta y x_1 + 2\Delta y - 2\Delta y x_1 - 2b\Delta x + 2b\Delta x - \Delta x$$

$$P_1 = 2\Delta y - \Delta x$$

	<pre>void Bre_line(int x1, int y1, int x2, int y2, int c){</pre>
$x = x_1; y = y_1;$ $dx = x_2 - x_1;$ $dy = y_2 - y_1;$	22

Chương 2: Các giải thuật sinh thực thể cơ sở

Hình 2.6 Sơ đồ khối thuật toán Bresenham cho đường thẳng	<pre> int x, y, dx, dy, p; y = y1; dx = x2 - x1; dy = y2 - y1; p = 2*dy - dx; for (x=x1; x<=x2; x++) { putpixel(x, y, c); if (p <= 0) p+=2*dy; //p=p+2dy else { p+=2*(dy- dx); //p=p+2dy-2dx y++; } } </pre>
--	--

Xét 4 trường hợp của k:

0<k<1	k>1	-1<k<0	k<-1
int p=2*dy-dx;	int p=dy-2*dx;	int p=2*dy+dx;	int p=-dy-2*dx;
if (p>=0) { p+=2*dy-2*dx; y++; } else p+=2*dy;	if (p<=0) { p+=2*dy- 2*dx; x++; } else p+=-2*dx;	if (p<=0) { p+=2*dy+2*dx; y--; } else p+=2*dy;	if (p>=0) { p+=-2*dy- 2*dx; x--; } else p+=-2*dx;

2.3.3. Giải thuật trung điểm-Midpoint

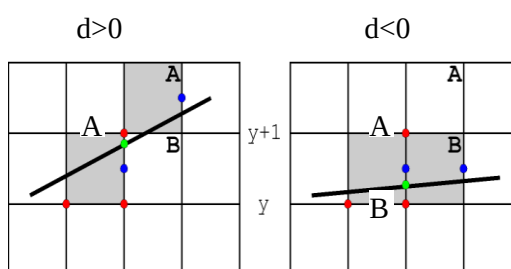
Jack Bresenham 1965/Pitteway 1967, áp dụng cho việc sinh các đường thẳng và đường tròn 1985. Xét trung điểm của đoạn AB (M)

Nếu M ở trên đoạn thẳng AB thì chọn B còn

M ở dưới đoạn thẳng AB chọn A

Công thức đơn giản hơn, tạo được các điểm tương tự như với Bresenham

$d = f(x_i + 1, y_i + 1/2)$ là trung điểm của đoạn AB



Hình 2.7 Mô tả giải thuật Midpoint

So sánh hay kiểm tra M sẽ được thay bằng việc xét giá trị d.

- $d < 0$ điểm B được chọn khi đó $y_{i+1} = y_i$
- $d > 0$ điểm A được chọn khi đó $y_{i+1} = y_i + 1$

Trường hợp $d = 0$ chúng ta có thể chọn điểm bất kỳ hoặc A, hoặc B. Sử dụng phương pháp biểu diễn không tường minh

$$f(x,y) = ax + by + c = 0 \quad (1) \quad dx = x_2 - x_1 \quad dy = y_2 - y_1$$

Biểu diễn tường minh:

$$y = (dy/dx)x + B \text{ hay } f(x,y)=0 = xdy - ydx + Bdx \quad (2)$$

So sánh (1) và (2) ta có $a=dy$, $b=-dx$ và $c=Bdx$

Có $f(x,y)=0$ với mọi (x,y) thuộc đường thẳng

Đặt $d_i = f(x_i+1, y_i+1/2) = a(x_i+1) + b(y_i+1/2) + c$

- Nếu chọn A ($d < 0$) thì M sẽ tăng theo 2 hướng x,y

$$d_{i+1} = f(x_i+2, y_i+3/2) = a(x_i+2) + b(y_i+3/2) + c$$

$$d_{i+1} - d_i = a + b \text{ Hay } d_{i+1} = d_i + dy - dx$$

- Nếu chọn B ($d > 0$) thì M sẽ tăng theo x

$$d_{i+1} = f(x_i+2, y_i+1/2) = a(x_i+2) + b(y_i+1/2) + c$$

$$d_{i+1} - d_i = a \text{ Hay } d_{i+1} = d_i + dy$$

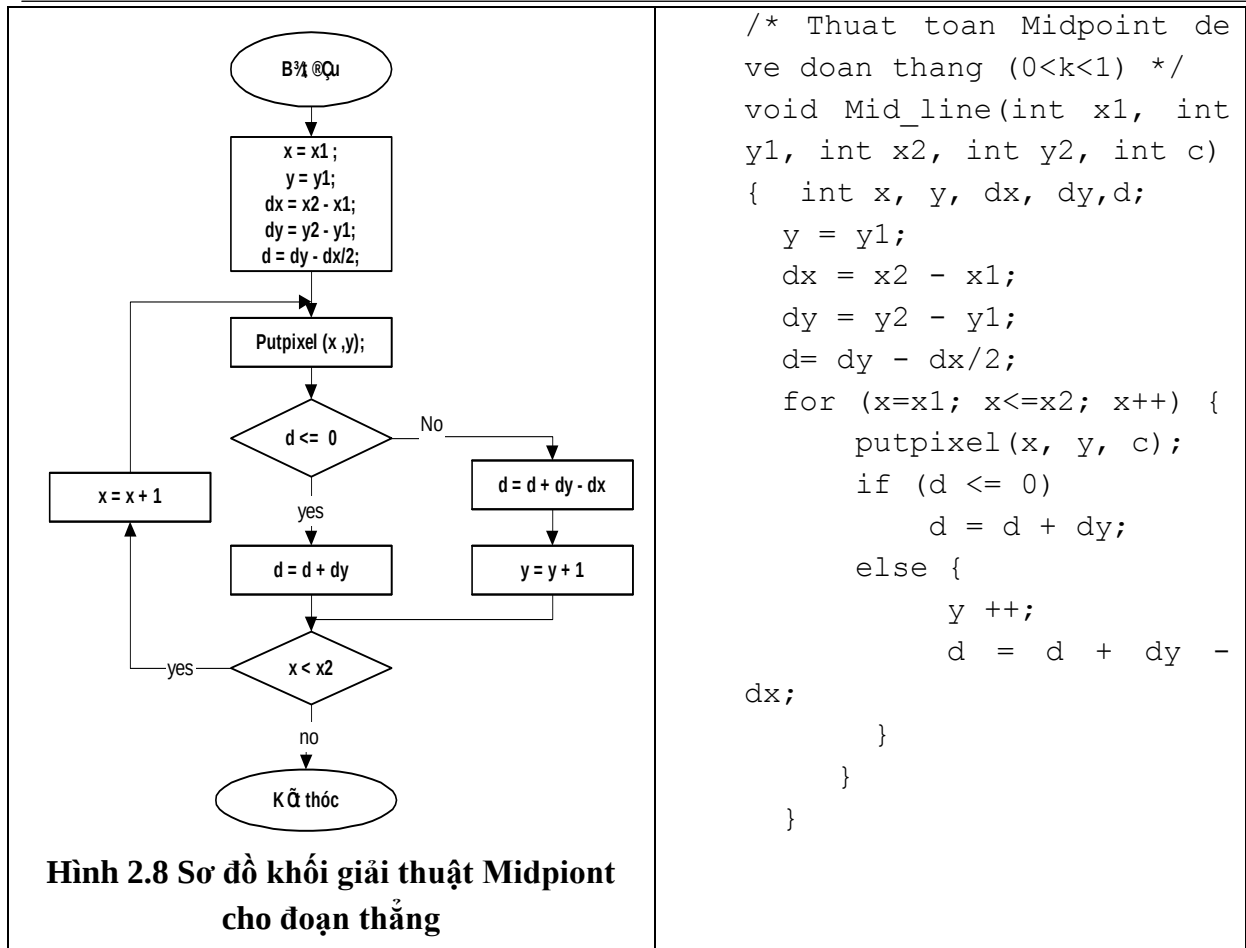
Tính d_1 ? $d_1 = f(x_1+1, y_1+1/2) = a(x_1+1) + b(y_1+1/2) + c$

$$= ax_1 + by_1 + c + a + 1/2 b = f(x_1, y_1) + a + b/2$$

Có (x_1, y_1) là điểm bắt đầu, nằm trên đoạn thẳng nên $f(x_1, y_1) = 0$

Vậy $d_1 = a + b/2 = dy - dx/2$

Chương 2: Các giải thuật sinh thực thể cơ sở

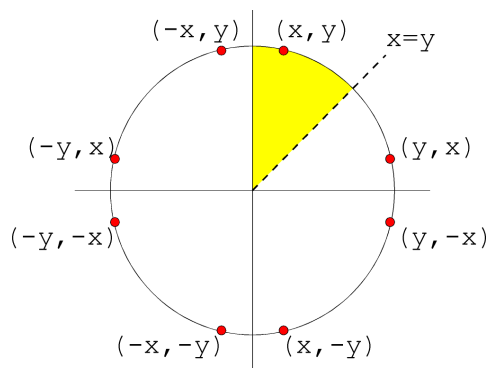


2.3.4. Giải thuật sinh đường tròn (Scan Converting Circles)(Bresenham)

- Phương trình đường tròn đi qua tâm có tọa độ (x_c, y_c) là:

$$(x - x_c)^2 + (y - y_c)^2 = r^2$$

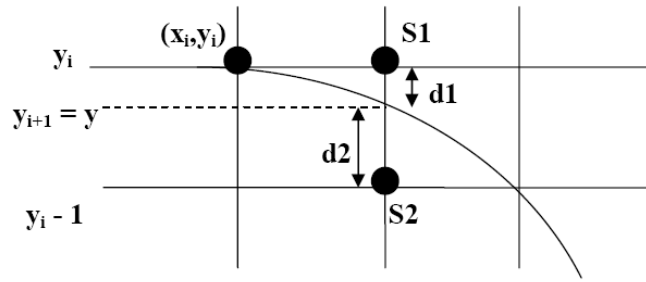
Hình tròn là hình đối xứng tám cách



Hình 2.9 Hình tròn đối xứng 8 phần

Để đơn giản ta xét tâm trùng gốc 0 thì phương trình đơn giản : $x^2 + y^2 = r^2$

Ta xét các điểm tạo ra từ góc phần tư thứ 2: từ 90° đến 45° , thực hiện theo hướng $+x, -y$



Hình 2.10 Mô tả giải thuật Bresenham

Giả sử bắt đầu x_i vậy $x_{i+1} = x_i + 1$

$$y^2 = r^2 - (x_i + 1)^2$$

$$d_1 = y_i^2 - y^2 = y_i^2 - r^2 - (x_i + 1)^2$$

$$d_2 = y^2 - (y_i - 1)^2 = r^2 - (x_i + 1)^2 - (y_i - 1)^2$$

$$p_i = d_1 - d_2 = 2(x_i + 1)^2 + y_i^2 + (y_i - 1)^2 - 2r^2$$

Xét: $p_i < 0$ ($d_1 < d_2$) chọn điểm nằm ngoài đường tròn $y_{i+1} = y_i$

$p_i \geq 0$ ($d_1 \geq d_2$) chọn điểm nằm trong đường tròn $y_{i+1} = y_i + 1$

$$p_i = 2(x_i + 1)^2 + 2y_i^2 - 2y_i - 2r^2$$

$$p_{i+1} = 2(x_i + 2)^2 + 2y_{i+1}^2 - 2y_{i+1} - 2r^2$$

$$p_{i+1} = p_i + 4x_i + 6 + 2y_{i+1}^2 - 2y_i^2 - 2y_{i+1} + 2y_i$$

$$p_{i+1} = p_i + 4x_i + 6 + 2(y_{i+1}^2 - y_i^2) - 2(y_{i+1} - y_i)$$

▪ Nếu $p_i < 0$ hay $y_{i+1} = y_i$

$$p_{i+1} = p_i + 4x_i + 6$$

▪ Nếu $p_i \geq 0$ hay $y_{i+1} = y_i + 1$

$$p_{i+1} = p_i + 4x_i + 6 - 4y_i + 2 + 2$$

$$p_{i+1} = p_i + 4(x_i - y_i) + 10$$

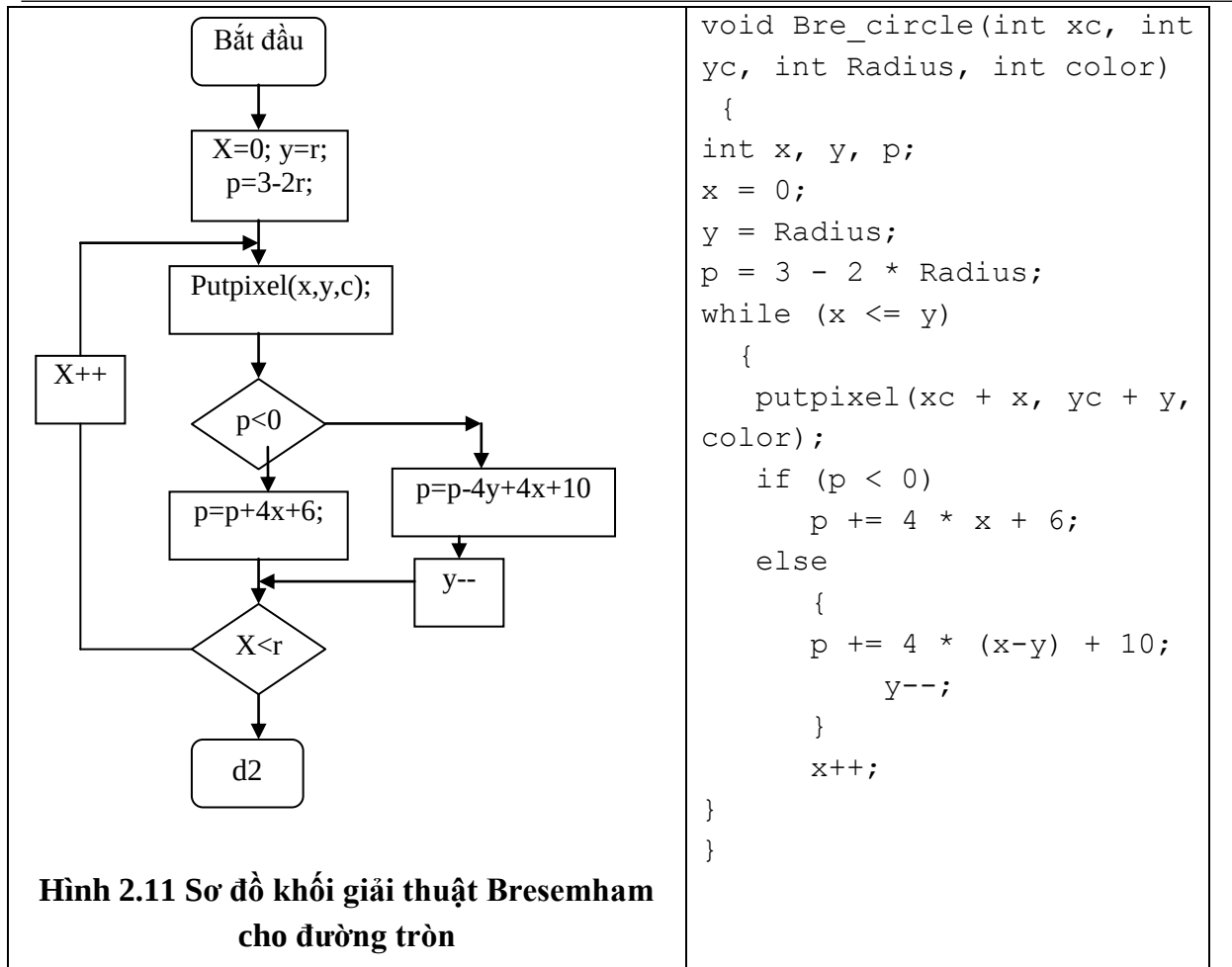
▪ Tính P_1 ? khi đó ứng với $x_1 = 0$ và $y_1 = r$

$$p_1 = 2(x_1 + 1)^2 + y_1^2 + (y_1 - 1)^2 - 2r^2$$

$$= 2 + r^2 + (r-1)^2 - 2r^2 = 3 - 2r$$

Giải thuật là:

Chương 2: Các giải thuật sinh thực thể cơ sở



Câu hỏi: lúc sử dụng tính đối xứng cho tám cách để vẽ một đường tròn đầy đủ từ các tọa độ pixel được tạo ứng với góc phần tư thứ hai. Một vài Pixel được vẽ hai lần, hiện tượng này gọi là Overstrike. Hãy chỉ định xem nơi nào xảy ra hiện tượng đó?

Trả lời: Tại (r,0) hoặc (0,r) và vị trí đường chéo: (αr , αr) trong đó $\alpha = 1/\sqrt{2} \approx 0.7071$

/* Thuật toán Bresenham để vẽ đường tròn */

```
#include <graphics.h>
```

```
#include <conio.h>
```

```
#define pc(xc,yc,x,y) {
    putpixel(xc + x, yc + y, color);
    putpixel(xc - x, yc - y, color);
    putpixel(xc -y, yc +x, color);
    putpixel(xc +y, yc -x, color);
}
```

```
void Bresenham_Circle(int xc, int yc, int Radius, int color){
    int x, y, p;
    x = 0;
    y = Radius;
    p = 3 - 2 * Radius;
    pc(xc,yc, Radius,0); //vẽ 4 điểm đặc biệt
    while (x < y){
        if (d < 0)
            p += 4 * x + 6;
```

```

else{
p += 4 * (x-y) + 10;
y--;
}
x++;
pc(xc,yc, x,y);
pc(xc,yc, y,x);
}
pc(xc,yc, y,y); // ve 4 diem phan giac x=y
}

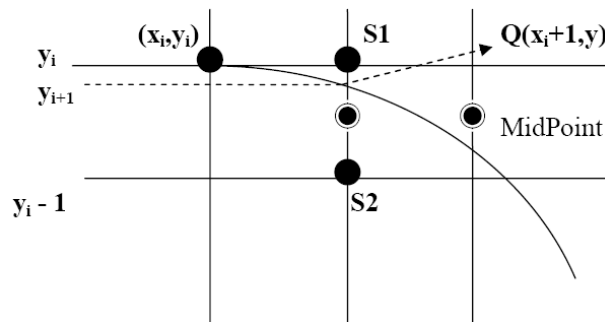
void main(){
int gr_drive = DETECT, gr_mode;
initgraph(&gr_drive, &gr_mode, "");
Bresenham_Circle(getmaxx() / 2, getmaxy() / 2, 150, 4);
getch();
closegraph();
}

```

2.3.5. Giải thuật sinh đường tròn Midpoint

Phương trình đường tròn không tường minh:

$$f(x,y) = x^2 + y^2 - R^2 = 0$$



Hình 2.12 Mô tả giải thuật Midpoint

Nếu $f(x,y) = 0$ thì nằm trên đường tròn

$f(x,y) > 0$ thì nằm bên ngoài đường tròn

$f(x,y) < 0$ thì nằm bên trong đường tròn

Thực hiện giải thuật trên 1/8 đường tròn và lấy đối xứng cho các góc còn lại.

Với M là điểm giữa của AB

Với d_i là giá trị của đường tròn tại một điểm bất kỳ

Ta có: $d_i = f(x_i+1, y_i - 1/2) = (x_i + 1)^2 + (y_i - 1/2)^2 - r^2$

- $d_i < 0$ chọn **A** thì điểm kế cận sẽ dịch chuyển theo x một đơn vị

$$d_{i+1} = f(x_i + 2, y_i - 1/2)$$

$$= (x_i + 2)^2 + (y_i - 1/2)^2 - r^2$$

$$d_{i+1} - d_i = (x_i + 2)^2 - (x_i + 1)^2 = 2x_i + 3$$

$$d_{i+1} = d_i + 2x_i + 3$$

Chương 2: Các giải thuật sinh thực thể cơ sở

- $d_i \geq 0$ chọn B thì điểm kế cận sẽ dịch chuyển theo x 1 đơn vị, theo y -1 đơn vị

$$d_{i+1} = f(x_i + 2, y_i - 3/2)$$

$$= (x_i + 2)^2 + (y_i - 3/2)^2 - r^2$$

$$d_{i+1} - d_i = 2x_i - 2y_i + 5$$

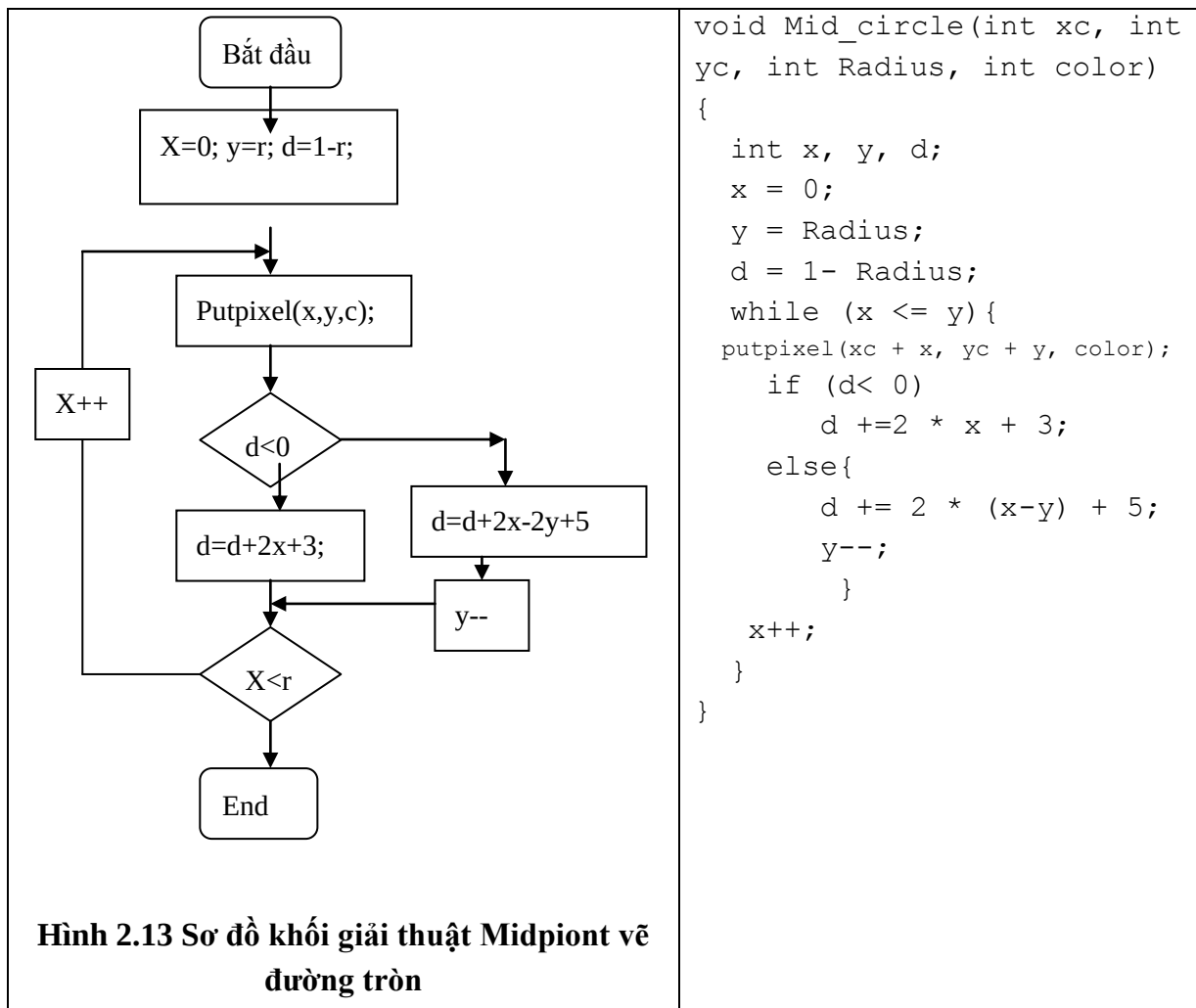
$$d_{i+1} = d_i + 2x_i - 2y_i + 5$$

Tính d_1 tại điểm (0, r)

$$d_1 = f(1, r-1/2) = 1^2 + (r-1/2)^2 - r^2$$

$$d_1 = 5/4 - r$$

Thuật toán như sau:



Hình 2.13 Sơ đồ khối giải thuật Midpiont vẽ đường tròn

```

void Midpoint_Circle(int xc, int yc, int Radius, int color){
    int x, y, d;
    x = 0;      y = Radius;
    d = 5 / 4 - Radius;
    while (x <= y) {
        putpixel(xc + x, yc + y, color);
        putpixel(xc - x, yc + y, color);
        putpixel(xc + x, yc - y, color);
        putpixel(xc - x, yc - y, color);
    }
}
    
```

```

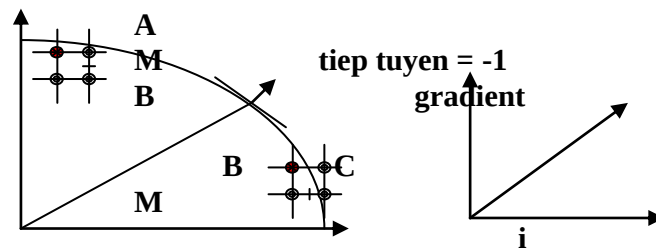
putpixel(xc + y, yc + x, color);
putpixel(xc - y, yc + x, color);
putpixel(xc + y, yc - x, color);
putpixel(xc - y, yc - x, color);
if (d < 0)
    d += 2 * x + 3;
else {
    d += 2 * (x-y) + 5;
    y--;
}
x++; }

```

2.3.6. Giải thuật sinh đường ellipse

Tính đối xứng được thực hiện trên 4 cách

$$F(x,y) = b^2x^2 + a^2y^2 - a^2b^2 = 0$$



Hình 2.14 Mô tả giải thuật sinh đường ellipse

Vector $\perp$ với tiếp tuyến gradient = 1

Ta có tiếp tuyến với cung tròn (độ dốc) = -1 = $dy/dx = -f_x/f_y$

Trong đó $f_x = 2b^2x$ đạo hàm riêng phần của $f(x,y)$ với x

Và $f_y = 2a^2y$ đạo hàm riêng phần của $f(x,y)$ với y

Giả sử ta chỉ xét trên góc phần tư thứ nhất: giả sử ta chia cung từ $(0,b)$ đến $(a,0)$ tại Q , có độ dốc -1

Trên phần 1: x thay đổi thì y thay đổi theo

Trên phần 2: y thay đổi thì x thay đổi theo

Xét trên phần 1:

Bắt đầu từ $(0,b)$, bước thứ i (x_i, y_i) chọn tiếp

$A(x_{i+1}, y_i)$

$B(x_{i+1}, y_{i-1})$

Tham số quyết định:

$$P_i = f(x_{i+1}, y_{i-1/2}) = b^2(x_{i+1})^2 + a^2(y_{i-1/2})^2 - a^2b^2$$

$$P_{i+1} = f(x_{i+1}+1, y_{i+1}-1/2) = b^2(x_{i+1}+1)^2 + a^2(y_{i+1}-1/2)^2 - a^2b^2$$

$$P_{i+1} - P_i = b^2((x_{i+1}+1)^2 - (x_{i+1})^2) + a^2((y_{i+1}-1/2)^2 - (y_{i-1/2})^2)$$

$$P_{i+1} = P_i + 2b^2x_{i+1} + b^2 + a^2((y_{i+1}-1/2)^2 - (y_{i-1/2})^2)$$

- Nếu $P_i < 0$ chọn A

$$x_{i+1} = x_i + 1$$

$$y_{i+1} = y_i$$

$$P_{i+1} = P_i + 2b^2x_i(x_i+1) + b^2 = P_i + 2b^2x_i + 3b^2$$

$$\text{Hay } P_{i+1} = P_i + b^2(2x_i + 3)$$

- Nếu $P_i \geq 0$ chọn B

$$x_{i+1} = x_i + 1$$

$$y_{i+1} = y_i - 1$$

$$P_{i+1} = P_i + 2b^2x_i(x_i+1) + b^2 + a^2((y_i-1 - 1/2)^2 - (y_i-1/2)^2)$$

$$= P_i + 2b^2x_i + 3b^2 + a^2(-3y_i + 9/4 + y_i - 1/4)$$

$$= P_i + 2b^2x_i + 3b^2 + a^2(-2y_i + 2)$$

$$\text{Hay } P_{i+1} = P_i + b^2(2x_i + 3) + a^2(-2y_i + 2)$$

- Tính P_1 ? tại $(0, b)$

$$P_1 = f(x_1+1, y_1-1/2) = b^2 + a^2(b-1/2)^2 - a^2b^2$$

$$P_1 = b^2 - a^2b + a^2/4$$

Xét trên phần 2:

Ta lấy toạ độ của Pixel sau cùng trong phần 1 của đường cong để tính giá trị ban đầu cho phần 2.

Giả sử pixel (x_k, y_k) vừa chuyển quét cuối cùng của phần 1 nhập vào bước j cho phần 2 (x_j, y_j) .

Pixel kế tiếp có thể là:

$$C(x_j, y_j-1)$$

$$D(x_j+1, y_j-1)$$

Tham số quyết định:

$$q_j = f(x_j+1/2, y_j-1) = b^2(x_j+1/2)^2 + a^2(y_j-1)^2 - a^2b^2$$

$$q_{j+1} = f(x_{j+1}+1/2, y_{j+1}-1) = b^2(x_{j+1}+1/2)^2 + a^2(y_{j+1}-1)^2 - a^2b^2$$

$$q_{j+1} - q_j = b^2((x_{j+1}+1/2)^2 - (x_j+1/2)^2) + a^2((y_{j+1}-1)^2 - (y_j-1)^2)$$

$$q_{j+1} = q_j + b^2((x_{j+1}+1/2)^2 - (x_j+1/2)^2) + a^2 - 2a^2y_{j+1}$$

- Nếu $q_j < 0$ chọn D

$$y_{j+1} = y_j - 1$$

$$x_{j+1} = x_j + 1$$

$$q_{j+1} = q_j + b^2((x_j+3/2)^2 - (x_j+1/2)^2) + a^2 - 2a^2(y_j - 1)$$

$$q_{j+1} = q_j + b^2(3x_j + 9/4 - x_j - 1/4) + 3a^2 - 2a^2y_j$$

$$\text{Hay } q_{j+1} = q_j + b^2(2x_j + 2) + a^2(-2y_j + 3)$$

- Nếu $q_j \geq 0$ chọn C

$$y_{j+1} = y_j - 1$$

$$x_{j+1} = x_j$$

$$q_{j+1} = q_j + a^2 - 2a^2(y_j - 1)$$

$$\text{Hay } q_{j+1} = q_j + a^2(3 - 2y_j)$$

▪ Tính q_1 ?

$$q_1 = f(x_k + 1/2, y_k - 1) = b^2(x_k + 1/2)^2 + a^2(y_k - 1)^2 - a^2b^2$$

Câu hỏi: lúc lấy đối xứng 4 cách để tìm 1 Ellipse hoàn chỉnh từ các tọa độ pixel được tạo ra với cung phần tư thứ 1. Có hiện tượng overstrike xảy ra hay không?

Trả lời: hiện tượng overstrike xảy ra tại:

(0,b); (0,-b); (a,0); (-a,0)

Thuật toán

```
#include <graphics.h>
#include <conio.h>
#define ROUND(a) ((long) (a+0.5))
void plot(int xc, int yc, int x, int y, int color){
    putpixel(xc+x, yc+y, color);
    putpixel(xc-x, yc+y, color);
    putpixel(xc+x, yc-y, color);
    putpixel(xc-x, yc-y, color);
}
void Mid_ellipse(int xc, int yc, int a, int b, int color){
    long x, y, fx, fy, a2, b2, p;
    x = 0;
    y = b;
    a2 = a * a;    //a2
    b2 = b * b;    //b2
    fx = 0;
    fy = 2 * a2 * y;    // 2a2y
    plot(xc, yc, x, y, color);
    p = ROUND(b2 - (a2*b) + (0.25*a));    // p=b2 - a2b + a2/4
    while (fx < fy){
        x++;
        fx += 2*b2;    //2b2
        if (p<0)
            p += b2*(2*x + 3);    // p=p + b2 (2x + 3)
        else{
            y--;
            p += b2*(2*x + 3) + a2*(-2*y + 2);    // p = p + b2(2x + 3) + a2(-2y + 2)
            fy -= 2*a2;    // 2a2
        }
        plot(xc, yc, x, y, color);
    }
}
```

```

p=ROUND(b2*(x+0.5)*(x+0.5) + a2*(y-1)*(y-1) -
a2*b2); //  $b^2(x+1/2)^2 + a^2(y-1)^2 - a^2b^2$ 
while (y>0){
y--;
fy -= 2*a2; //  $2a^2$ 
if (p>=0)
p+=a2*(3 - 2*y); //  $p = p + a^2(3-2y)$ 
else{
x++;
fx += 2*b2; //  $2b^2$ 
p += b2*(2*x+2) + a2*(-2*y + 3); //  $p = p + b^2(2x + 2) + a^2(-2y + 3)$ 
}
plot(xc, yc, x, y, color);
}
}
void main(){
int gr_drive = DETECT, gr_mode;
initgraph(&gr_drive, &gr_mode, "");
Mid_Ellipse(getmaxx() / 2, getmaxy() / 2, 150, 80, 4);
getch();
closegraph();
}

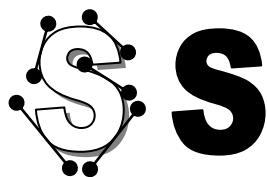
```

2.3.7. Giải thuật sinh ký tự

Trong màn hình text, truy xuất các ký tự trên màn hình được hỗ trợ bởi phần cứng. Các ký tự được lưu trữ trong bộ nhớ ROM, dưới dạng bitmap hay các ma trận ảnh. Phần cứng sẽ đưa ký tự lên màn hình tại vị trí xác định, tính toán cuộn trang và xuống dòng.

Trong đồ hoạ:

- Vector: định nghĩa các ký tự theo những đường cong mềm bao ngoài của chúng, tốn kém về mặt tính toán.

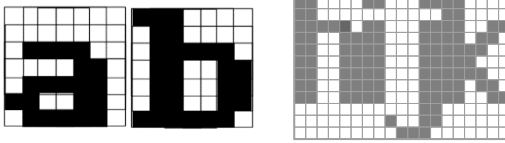


Hình 2.15 Ký tự vector

Ưu nhược điểm:

- phức tạp (tính toán phương trình)
- lưu trữ gọn nhẹ
- các phép biến đổi dựa vào công thức biến đổi
- Kích thước phụ thuộc vào môi trường (không có kích thước cố định)

- Bitmap: định nghĩa mỗi ký tự với 1 font chữ cho trước là 1 ảnh bitmap hình chữ nhật nhỏ.



Hình 2.16 Ký tự bitmap

- Đơn giản trong việc sinh ký tự (copypixel)
- Lưu trữ lớn
- Các phép biến đổi(I,B,U, scale) đòi hỏi lưu trữ thêm
- Kích thước không đổi

▪ bitmap: sử dụng hàm copypixel (copy điểm ảnh) được lưu trữ trong bộ nhớ cố định - Fontcache, đưa vào bộ nhớ đệm hiển thị. Mỗi 1 ký tự như 1 ma trận 2 chiều của các điểm ảnh - mặt nạ.

Hàm_sinh_ki_tu (mask)

```
{xmax, ymax, xmin, ymin      //các giới hạn của mặt nạ
  xo, yo                      //điểm gốc trên bộ đệm hiển thị
  for (i=ymin;i< ymax ;i++)
  for (j=xmin; j< xmax ; j++)
  if (mask(i,j) <> 0)
      copypixel ((mask(i,j), pixel(xo+j, yo+i));
  }
```

Ký tự fontcache bitmap đơn giản của SRGP lưu trữ các ký tự theo chuỗi liên tiếp nhau trong bộ nhớ. Nhưng độ rộng các ký tự khác nhau, truy nhập các fontcache thông qua bản ghi về cấu trúc cho từng ký tự.

Cấu trúc font chữtypedef struct {

```
int leftx;
int width;
} Charlocation;      //Vị trí của text
struct {
int CacheId;
int Height;          // Độ rộng chữ
int CharSpace;       // Khoảng cách giữa các ký tự
Charlocation Table [128]; //bảng chữ cái
} fontcache;
```

Ký tự vector Xây dựng theo phương pháp định nghĩa các ký tự bởi đường cong mềm bao ngoài của chúng để dàng thay đổi kích thước của ký tự cũng như nội suy ra các dạng của ký tự. Hoàn toàn độc lập với thiết bị.

- Tối ưu nhất: lưu trữ font dưới dạng đường bao. Khi các chương trình ứng dụng sử dụng là bitmap tương ứng với chúng.

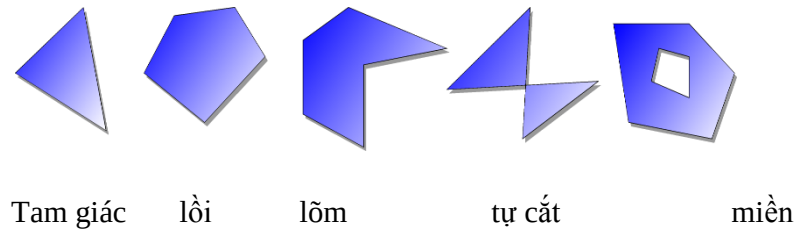
2.3.8. Giải thuật sinh đa giác (Polygon)

a. Thuật giải vẽ đường bao đa giác

Việc biểu diễn đa giác thông qua:

- Tập các đoạn thẳng
- Tập các điểm thuộc đa giác

Các loại đa giác:



Hình 2.17 Các loại đa giác

Đa giác lõm: là đa giác có đường thẳng nối bất kỳ 2 điểm bên trong nào của đa giác đều nằm trọn trong đa giác. Đa giác không lõm là đa giác lõm.

Các đường thẳng bao đa giác - cạnh của đa giác. Các điểm giao của cạnh - đỉnh của đa giác. Thông tin cần thiết để xác định đa giác:

- Số cạnh
- Tọa độ các đỉnh của đa giác

Giải thuật:

```
Polygon (arrayx, arrayy,n)
{ if (n<3          //không phải đa giác
  exit;
  for (i=1 ; i<= n-1; i++)
    line(arrayx[i],arrayy[i], arrayx[i+1], arrayy[i+1]);
    line(arrayx[i+1],arrayy[i+1], arrayx[1], arrayy[1]); }
```

b. Các thuật toán tô miền kín đa giác

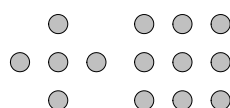
Lợi thế của hiển thị raster là: khả năng lưu trữ, copy, tô màu một vùng...Có hai dạng vùng tô thường gặp đó là: tô bằng một màu thuần nhất (solid fill), tô theo mẫu tô (fill pattern) nào đó.

Còn thiết bị vector thì hạn chế do các vùng tô màu tạo ra bởi một tập các đoạn thẳng sát nhau - làm chậm quá trình làm tươi.

Giải thuật đường biên (Boundary - fill Algorithm)

- Bắt đầu từ 1 điểm (x,y) trong vùng cần được tô màu:
 - Xác định màu điểm: getpixel(x,y,c)
 - Tô màu putpixel(x,y,c)
- Bước tiếp: kiểm tra thuộc tính màu các điểm lân cận
 - Điểm lân cận đã tô màu (exit)
 - Trùng với màu đường biên(exit)
 - Nếu không thì tô màu

Các phương pháp xác định điểm lân cận



4-connected 8-connected

Hình 2.18 Phương pháp tịnh tiến giải thuật

Giải thuật tô màu đường biên:

```
#include <graphics.h>
#include <conio.h>
void FloodFill (int x, int y, int in_color, int new_color){
    if (getpixel(x, y) == in_color){
        putpixel(x, y, new_color);
        FloodFill(x-1, y, in_color, new_color);
        FloodFill(x+1, y, in_color, new_color);
        FloodFill(x, y-1, in_color, new_color);
        FloodFill(x, y+1, in_color, new_color);
    }
}
void main() {
    int gr_drive = DETECT, gr_mode;
    initgraph(&gr_drive, &gr_mode, "");
    circle(getmaxx() / 2, getmaxy() / 2, 15);
    FloodFill(getmaxx() / 2, getmaxy() / 2, 0, 4);
    getch();
    closegraph(); }
```

- Giải thuật dòng quét (scanline) cho việc tô màu vùng

Giải thuật dựa trên ý tưởng sử dụng một đường quét trên trục y của màn hình đi từ $y_{\max}$ đến $y_{\min}$ của vùng cần được tô màu.

Với mỗi giá trị $y = y_i$ đường thẳng quét cắt các đường biên của vùng cần tô tạo ra đoạn thẳng $y = y_i$ với $x \in [x_{\min}, x_{\max}]$. Trên đoạn thẳng đó chúng ta tô màu các điểm tương ứng đi từ $x_{\min}$ đến $x_{\max}$ có các điểm tô $(x_i, y_i) \in y = y_i$.

- Đơn giản nhất ví dụ tô màu hình chữ nhật:

```
void scanline_rectg(x1,y1,x2,y2,c){ int i,j;
for(i=y1; i>=y2; i--)
    for(j=x1; j<= x2;j++)
        putpixel(i,j,c); }
```

- Phép tô màu 1 đa giác bất kỳ sẽ phức tạp hơn rất nhiều so với hình chữ nhật

Giả sử vùng tô được cho bởi 1 đa giác n đỉnh: $p_i(x_i, y_i)$, $i=0,1,\dots,n-1$. Đa giác này có thể là đa giác lồi, đa giác lõm hay đa giác tự cắt....

Các bước tóm tắt chính của thuật toán:

- Tìm y_{top} , y_{bottom} lần lượt là giá trị lớn nhất, nhỏ nhất của tập các tung độ của các đỉnh của đa giác đã cho.

$$y_{\text{top}} = \max\{y_i, (x_i, y_i) \in P\},$$

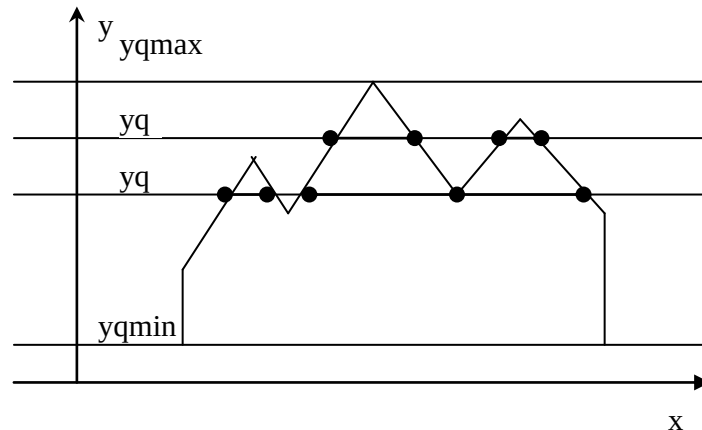
$$y_{\text{bottom}} = \min\{y_i, (x_i, y_i) \in P\}.$$

- Ứng với mỗi dòng quét $y=k$, với k thay đổi từ y_{bottom} đến y_{top} lặp:

- Tìm tất cả các hoành độ giao điểm của dòng quét $y=k$ với các cạnh của đa giác
- Sắp xếp các hoành độ giao điểm theo thứ tự tăng dần: $x_0, x_1, \dots$
- Tô màu các đoạn thẳng trên đường thẳng $y=k$ lần lượt được giới hạn bởi các cặp $(x_0, x_1), (x_2, x_3), \dots, (x_{2k}, x_{2k+1})$.

Chúng ta sẽ gặp 1 số vấn đề sau:

- Ứng với mỗi dòng quét không phải lúc nào tất cả các cạnh của đa giác cũng tham gia cắt dòng quét. Do đó để cải thiện tốc độ cần phải có một cách nào đó để hạn chế được số cạnh cần tìm giao điểm ứng với mỗi dòng quét.



Hình 2.19 Giải thuật scanline cho một đa giác bất kỳ

- Nếu số giao điểm tìm được giữa các cạnh đa giác và dòng quét là lẻ (điều này chỉ xảy ra khi dòng quét sẽ đi qua các đỉnh của đa giác) khi đó ta sẽ tính số điểm là 2 thì có thể tô không chính xác. Ngoài ra, việc tìm giao điểm của dòng quét với các cạnh nằm ngang là trường hợp đặt biệt...

Để giải quyết các vấn đề trên ta có các phương pháp sau:

- Danh sách các cạnh kích hoạt (AET - Active Edge Table)

Mỗi cạnh của đa giác được xây dựng từ 2 đỉnh kề nhau $P_i(x_i, y_i)$ và $P_{i+1}(x_{i+1}, y_{i+1})$ gồm các thông tin sau:

$y_{\min}$: giá trị nhỏ nhất trong 2 đỉnh của cạnh

$x_{\text{Intersect}}$: hoành độ giao điểm của cạnh với dòng quét hiện đang xét

$DxPerScan$: giá trị $1/m$ (m là hệ số góc của cạnh)

Δy : khoảng cách từ dòng quét hiện hành tới đỉnh $y_{\max}$

Danh sách các cạnh kích hoạt AET: danh sách này dùng để lưu các tập cạnh của đa giác có thể cắt ứng với dòng quét hiện hành và tập các điểm giao tương ứng. Nó có một số đặc điểm:

Các cạnh trong danh sách được sắp xếp theo thứ tự tăng dần của các hoành độ giao điểm để có thể tô màu các đoạn giao một cách dễ dàng.

Thay đổi ứng với mỗi dòng quét đang xét, do đó danh sách này sẽ được cập nhật liên tục trong quá trình thực hiện thuật toán. Đầu tiên ta có danh sách chứa toàn bộ các cạnh

của đa giác gọi là ET (Edge Table) được sắp xếp theo thứ tự tăng dần của $y_{\min}$, rồi sau mỗi lần dòng quét thay đổi sẽ di chuyển các cạnh trong ET thoả điều kiện sang AET.

Một dòng quét $y=k$ chỉ cắt 1 cạnh của đa giác khi và chỉ khi $k \geq y_{\min}$ và $\Delta y > 0$. Chính vì vậy mà với các tổ chức của ET (sắp theo thứ tự tăng dần của $y_{\min}$) điều kiện để chuyển các cạnh từ ET sang AET sẽ là $k \geq y_{\min}$; và điều kiện để loại một cạnh ra khỏi AET là $\Delta y \leq 0$

- Công thức tìm giao điểm nhanh

Nếu gọi x_k, x_{k+1} lần lượt là các hoành độ giao điểm của một cạnh nào đó với các dòng quét $y=k$ và $y=k+1$ ta có:

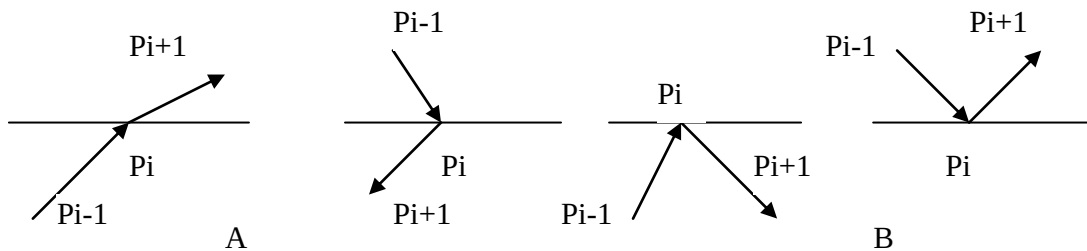
$$x_{k+1} - x_k = 1/m ((k+1) - k) = 1/m \text{ hay } x_{k+1} = x_k + 1/m$$

Như vậy nếu lưu hoành độ giao điểm ứng với dòng quét trước lại, cùng với hệ số góc của cạnh, ta xác định được hoành độ giao điểm ứng với dòng quét kế tiếp theo công thức trên. Nên thông tin của cạnh có 2 biến: $DxPerScan$, $x_{Intersect}$.

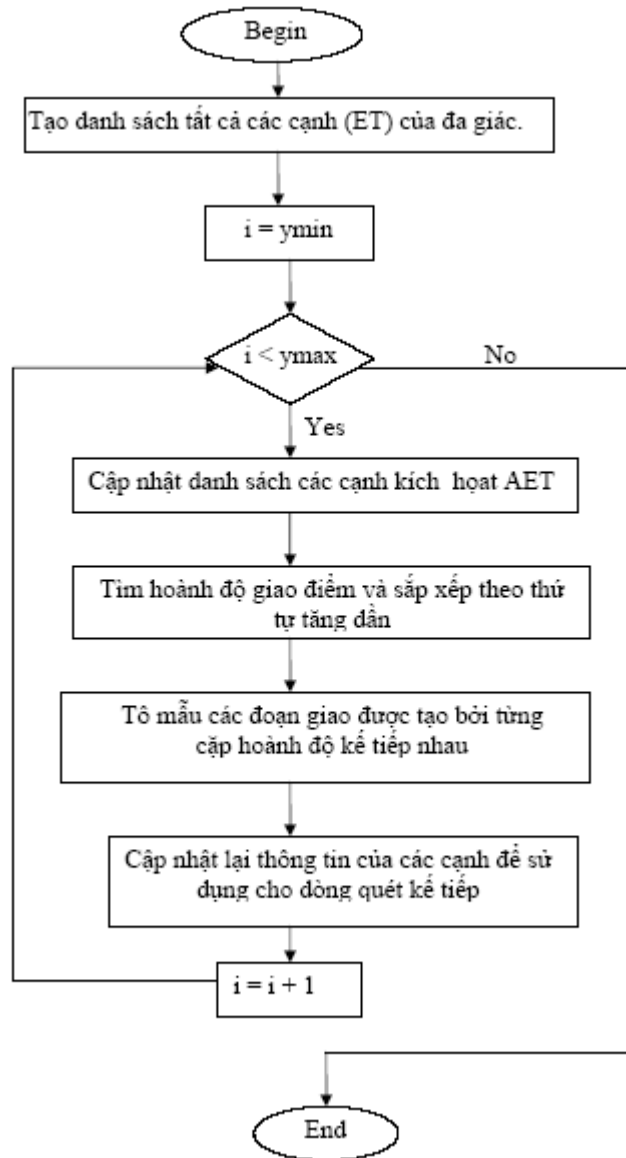
- Trường hợp dòng quét đi ngang qua một đỉnh:

Tính 1 giao điểm nếu chiều của 2 cạnh kề của đỉnh đó có xu hướng tăng hay giảm

Tính 2 giao điểm nếu chiều của 2 cạnh kề của đỉnh đó có xu hướng thay đổi, nghĩa là tăng-giảm hay giảm-tăng.



Hình 2.20 Qui tắc tính: một giao điểm (A) và hai giao điểm (B)



Hình 2.21 lưu đồ thuật toán scan - line

- Giải thuật tô vùng kín theo mẫu (Pattern filling)

☒ object (ảnh mẫu)

$A[m,n]$

Vấn đề: xác định điểm ở mẫu và nhiều điểm tương ứng với chúng trên màn hình.

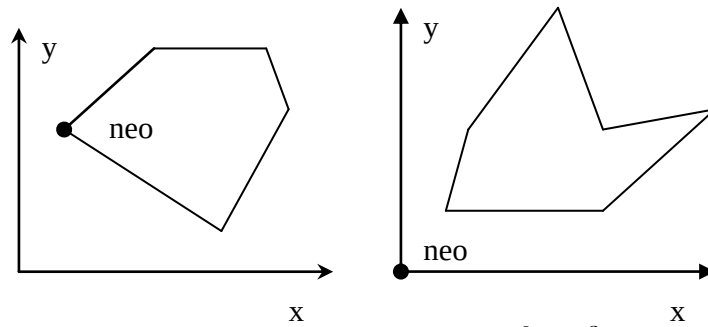
Phương pháp 1:

Tìm điểm neo ở đầu trái nhất hàng đầu tiên của đa giác.

Nhược điểm: không có điểm định vị trí phân biệt một cách rõ ràng cho mẫu tô trong một đa giác bất kỳ.

Phương pháp 2: sử dụng cho SRGP

Lấy điểm neo ở gốc toạ độ, giả sử ta coi cả màn hình được lát bởi màu tô các thực thể là các đường biên cho các vùng tô, vậy nếu ngoài các thực thể các màu tô không được phép thể hiện.



Hình 2.22 Phương pháp lấy điểm neo

Tóm tắt chương:

Để có thể hiển thị các đối tượng đồ họa trên thiết bị hiển thị dạng điểm mà điển hình là màn hình, cần phải có một quá trình chuyển các mô tả hình học của các đối tượng này trong hệ tọa độ thế giới thực về dãy các pixel tương ứng gần với chúng nhất trên tọa độ thiết bị. Quá trình này còn được gọi là quá trình chuyển đổi bằng dòng quét. Yêu cầu quan trọng nhất đối với quá trình này ngoài việc phải cho kết quả xấp xỉ tốt nhất còn phải cho tốc độ tối ưu.

Ba cách tiếp cận để vẽ đoạn thẳng gồm thuật toán DDA, thuật toán Bresenham, thuật toán Midpiont đều tập trung vào việc đưa ra cách chọn một trong hai điểm nguyên kế tiếp khi đã biết điểm nguyên ở bước trước. Thuật toán DDA đơn giản chỉ dùng thao tác làm tròn nên phải dùng các phép toán trên số thực, trong khi đó thuật toán Bresenham và Midpiont đưa ra cách chọn phức tạp hơn nhưng cho kết quả tốt hơn. Tương tự dùng hai giải thuật Bresenham và Midpiont để vẽ đường tròn và ellipse và một số đường cong khác.

Các thuật toán tô màu vùng gồm thuật toán loang (đệ qui) hay thuật toán dòng quét. Có thể tô cùng một màu hay tô theo mẫu.

Bài tập:

1. Chỉ định các vị trí mảnh nào sẽ được chọn bởi thuật toán Bresenham lúc chuyển quét một đường thẳng từ tọa độ pixel (1,1) sang tọa độ pixel (8,5).
2. Dùng giải thuật Bresenham viết hàm sinh đoạn thẳng (xét tất cả các trường hợp của hệ số góc).
3. Dùng giải thuật Midpiont viết hàm sinh đoạn thẳng (xét tất cả các trường hợp của hệ số góc).
4. Dùng giải thuật Midpiont viết hàm sinh đường tròn (tọa độ tâm (xc,yc) và bán kính r).
5. Dùng giải thuật Midpiont viết hàm sinh đường ellipse (tọa độ tâm (xc,yc) và bán kính rx và ry).
6. Từ hàm vẽ đường thẳng thiết kế và cài đặt hàm vẽ các hình sau: hình chữ nhật, đa giác, ngôi nhà....
7. Viết giải thuật tìm giao điểm hai đoạn thẳng.
8. Viết chương trình tô màu Floodfill (sử dụng đệ qui với 4-connected).

CHƯƠNG 3: CÁC PHÉP BIẾN ĐỔI ĐỒ HOẠ

3.1. CÁC PHÉP BIẾN ĐỔI HÌNH HỌC HAI CHIỀU

3.1.1. Phép biến đổi Affine (Affine Transformations)

Phép biến đổi Affine là phép biến đổi tuyến tính tọa độ điểm đặc trưng của đối tượng thành tập tương ứng các điểm mới để tạo ra các hiệu ứng cho toàn đối tượng.

Ví dụ: phép biến đổi tọa độ với chỉ 2 điểm đầu cuối của đoạn thẳng tạo thành 2 điểm mới mà khi nối chúng với nhau tạo thành đoạn thẳng mới. Các điểm nằm trên đoạn thẳng sẽ có kết quả là điểm nằm trên đoạn thẳng mới với cùng phép biến đổi thông qua phép nội suy.

3.1.2. Các phép biến đổi đối tượng

Các đối tượng phẳng trong đồ hoạ 2 chiều mô tả tập các điểm phẳng. Điểm trong đồ hoạ 2 chiều biểu diễn thông qua tọa độ, viết dưới dạng ma trận gọi là vector vị trí.

Có 2 dạng biểu diễn:

Một hàng và 2 cột: $[x \quad y]$

Hai hàng và 1 cột: $\begin{bmatrix} x \\ y \end{bmatrix}$

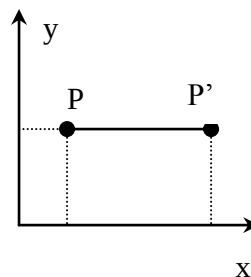
Trong giáo trình chúng ta chọn biểu diễn điểm là ma trận hàng (một hàng và 2 cột).

Tập các điểm được lưu trữ trong máy tính sẽ được viết dưới dạng ma trận vị trí của chúng. Chúng có thể là đường thẳng, đường cong, ảnh....thật dễ dàng kiểm soát các đối tượng thông qua các phép biến đổi chúng, thực chất các phép biến đổi đồ hoạ này được mô tả dưới dạng các ma trận.

3.1.2.1. Phép biến đổi vị trí

Giả sử ta có điểm $P = [x \quad y]$ trong mặt phẳng với $[x \quad y]$ là vector vị trí của P , kí hiệu là $[P]$. Gọi ma trận T là ma trận biến đổi sẽ có dạng:

$$T = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$



Hình 3.1 Phép biến đổi vị trí

Ta có điểm P sau phép biến đổi thành P' có giá trị $[x' \quad y']$.

Thực thi phép biến đổi đúng trên 1 điểm ảnh sẽ đúng với toàn bộ đối tượng.

$$[X]*[T]=[x \ y]*\begin{bmatrix} a & b \\ c & d \end{bmatrix}=[(ax+cy) \ (bx+dy)]=[x' \ y']$$

Hay ta có: $x' = ax + cy$

$$y' = bx + dy$$

Xét ma trận biến đổi T:

① Phép bất biến:

Khi đó: $a = d = 1$ và $b = c = 0$ và ma trận cho phép bất biến là:

$$T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$[X]*[T]=[x \ y]*\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}=[x \ y]=[x' \ y']$$

Vậy $x'=x$ và $y=y'$ hay là $P'=P$ chứng tỏ bất biến qua phép biến đổi.

② Phép biến đổi tỷ lệ (scaling):

- Nếu $d=1$ và $b = c = 0$ thì ma trận biến đổi là:

$$T = \begin{bmatrix} a & 0 \\ 0 & 1 \end{bmatrix}$$

$$x' = ax$$

$$y' = y$$

$$[X]*[T]=[x \ y]*\begin{bmatrix} a & 0 \\ 0 & 1 \end{bmatrix}=[(ax) \ y]=[x' \ y']$$

P' dịch chuyển theo trục x với tỷ lệ a xác định.

- Nếu $b = c = 0$ thì ma trận biến đổi là:

$$T = \begin{bmatrix} a & 0 \\ 0 & d \end{bmatrix}$$

$$[X]*[T]=[x \ y]*\begin{bmatrix} a & 0 \\ 0 & d \end{bmatrix}=[ax \ dy]=[x' \ y']$$

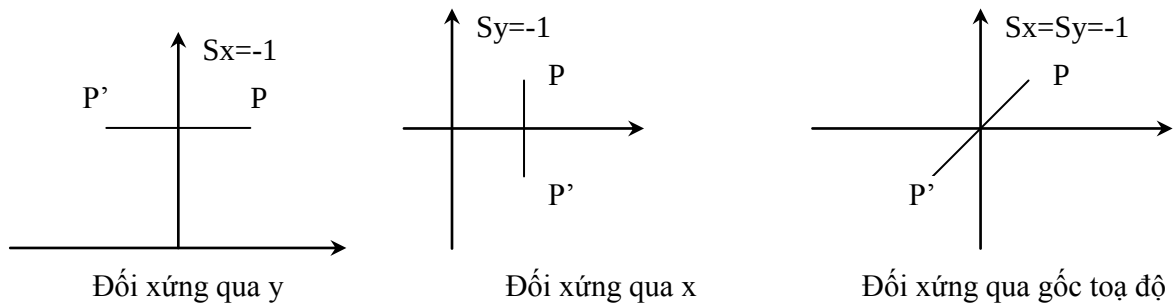
Hay tổng quát hơn gọi S_x, S_y lần lượt là tỷ lệ theo trục x và trục y, thì ma trận tỷ lệ sẽ là:

$$T = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix}$$

Khi $S_x, S_y > 1$ gọi là phép phóng to

Khi $S_x, S_y < 1$ gọi là phép thu nhỏ

- Các trường hợp đặc biệt:



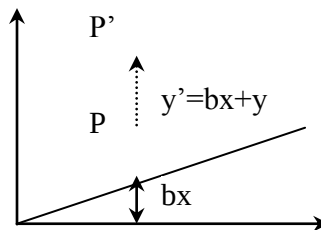
Hình 3.2 Các phép đối xứng trên 2D

③ Phép biến dạng

Khi $a = d = 1$ thì tọa độ của P' phụ thuộc vào thay đổi của b và c

- Xét $c = 0$

$$[X]^*[T] = [x \ y]^* \begin{bmatrix} 1 & b \\ 0 & 1 \end{bmatrix} = [x \ bx + y] = [x' \ y']$$

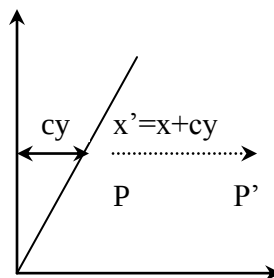


Hình 3.3 Phép biến dạng theo trục oy

Có P' không thay đổi giá trị tọa độ x , còn y' thay đổi phụ thuộc vào cả b và x

- Xét $b = 0$

$$[X]^*[T] = [x \ y]^* \begin{bmatrix} 1 & 0 \\ c & 1 \end{bmatrix} = [x + cy \ y] = [x' \ y']$$

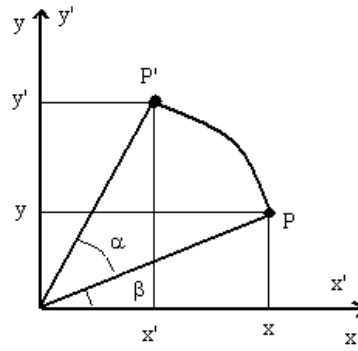


Hình 3.4 Phép biến dạng theo trục ox

Chú ý: điểm gốc tọa độ $P[0 \ 0]$ bất biến với mọi phép biến đổi

④ Phép quay

Có $\alpha > 0$ ngược chiều kim đồng hồ



Hình 3.5 Phép quay trên 2D

Ta có:

$$P[x \ y] = [r \cos \beta \ r \sin \beta]$$

$$P'[x' \ y'] = [r \cos(\alpha + \beta) \ r \sin(\alpha + \beta)]$$

$$\begin{aligned} P'[x' \ y'] &= [r(\cos \alpha \cos \beta - \sin \alpha \sin \beta) \ r(\cos \alpha \sin \beta + \sin \alpha \cos \beta)] \\ &= [(x \cos \alpha - y \sin \alpha) \ (x \sin \alpha + y \cos \alpha)] \end{aligned}$$

Vậy: $x' = x \cos \alpha - y \sin \alpha$

$$y' = x \sin \alpha + y \cos \alpha$$

Ta có:

$$[X'] = [X] * [T] = [(x \cos \alpha - y \sin \alpha) \ (x \sin \alpha + y \cos \alpha)]$$

Vậy T tổng quát khi quay đối tượng quanh gốc tọa độ 1 góc α bất kỳ là:

$$[T] = \begin{bmatrix} \cos \alpha & \sin \alpha \\ -\sin \alpha & \cos \alpha \end{bmatrix}$$

3.1.2.2. Phép biến đổi tổng hợp

Phương pháp biến đổi sử dụng phép nhân ma trận với tọa độ điểm thông qua các vector vị trí thật sự hiệu quả và đem lại công cụ mạnh về đồ hoạ cho người sử dụng. Nhưng thực tế các thao tác thường cần không chỉ một mà nhiều phép biến đổi khác nhau. Ta có phép hoán vị khi nhân ma trận là không thực hiện nhưng khả năng tổ hợp các phép nhân lại cho phép tạo ra một ma trận biến đổi duy nhất. Làm giảm bớt đáng kể khối lượng tính toán trong quá trình biến đổi, làm tăng tốc các chương trình ứng dụng và tạo điều kiện cho việc quản lý các biến đổi trong ứng dụng.

Giả sử ta có P với $[X] = [x \ y]$, có hai phép biến đổi $[T1]$ quay quanh gốc tọa độ 90° :

$$T1 = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

Và $[T2]$ lấy đối xứng P qua gốc tọa độ:

$$T2 = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}$$

Ta có:

$$[X'] = [X] * [T1] = \begin{bmatrix} x & y \end{bmatrix} * \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} = \begin{bmatrix} -y & x \end{bmatrix}$$

$$[X''] = [X'] * [T2] = \begin{bmatrix} -y & x \end{bmatrix} * \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix} = \begin{bmatrix} y & -x \end{bmatrix}$$

Giả sử [T3] là ma trận tổng hợp [T1] và [T2]

$$[X *] = [X] * [T3] = \begin{bmatrix} x & y \end{bmatrix} * \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} y & -x \end{bmatrix}$$

Kết luận: biến đổi qua nhiều ma trận thành phần sẽ tương đương với phép biến đổi qua ma trận tổng hợp từ các phép biến đổi đó.

3.2. TỌA ĐỘ ĐỒNG NHẤT VÀ CÁC PHÉP BIẾN ĐỔI

3.2.1. Tọa độ đồng nhất

Ta xét phép tịnh tiến:

$$x' = x + dx$$

$$y' = y + dy$$

$$\text{Vậy } P' = P + [T]$$

$$[T] = \begin{bmatrix} dx \\ dy \end{bmatrix}$$

Vậy phép biến đổi tổng hợp:

$$P'' = P' * [T'] + [T''] = (P + [T]) [T'] + [T'']$$

Rõ ràng không thể biểu diễn thông qua ma trận tổng hợp 2 chiều 2x2 được. Phép tịnh tiến đưa ra ma trận biến đổi tổng hợp là không thể.

Thế nào là phương pháp biểu diễn tọa độ đồng nhất ? là phương pháp biểu diễn mở rộng thông qua tọa độ đồng nhất của các vector vị trí không đồng nhất $\begin{bmatrix} x & y \end{bmatrix}$ là ứng dụng của phép biến đổi hình học mà ở đó tọa độ điểm được mô tả dưới ma trận $\begin{bmatrix} x^* & y^* & h \end{bmatrix}$ với $x = x^*/h$, $y = y^*/h$ có h là một số thực tùy ý. Vậy một vector vị trí $\begin{bmatrix} x & y \end{bmatrix}$ bất kỳ trên mặt phẳng xoy bằng tập vô số các điểm đồng nhất $\begin{bmatrix} hx & hy & h \end{bmatrix}$. Ví dụ: $\begin{bmatrix} 2 & 5 \end{bmatrix}$ sẽ biểu diễn bằng $\begin{bmatrix} 4102 \\ 6153 \end{bmatrix}$,..... đơn giản nhất là $\begin{bmatrix} 251 \end{bmatrix}$.

Vậy tọa độ đồng nhất của vector vị trí $[X] = \begin{bmatrix} x & y & 1 \end{bmatrix}$. Khi đó ma trận biến đổi sẽ là 3x3:

$$[T] = \begin{bmatrix} a & b & 0 \\ c & d & 0 \\ dx & dy & 1 \end{bmatrix}$$

$$P' = P * [T] = \begin{bmatrix} x & y & 1 \end{bmatrix} * \begin{bmatrix} a & b & 0 \\ c & d & 0 \\ dx & dy & 1 \end{bmatrix}$$

$$x' = ax + cy + dx$$

$$y' = bx + dy + dy$$

$$[X'] = [x' \ y' \ 1]$$

Trong đó dx, dy là khoảng tịnh tiến theo trục x và y.

3.2.2. Phép biến đổi với tọa độ đồng nhất

$$[T] = \begin{bmatrix} a & b & 0 \\ c & d & 0 \\ m & n & 1 \end{bmatrix}$$

Ma trận biến đổi đồng nhất

① Phép tịnh tiến

Có a=d=1 và b=c=0

$$[T] = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ m & n & 1 \end{bmatrix}$$

$$[x' \ y' \ 1] = [x \ y \ 1] \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ m & n & 1 \end{bmatrix} = [x+m \ y+n \ 1]$$

Chú ý: trong mặt phẳng tọa độ, mọi điểm kể cả gốc tọa độ đều có thể biến đổi.

Phép biến đổi tổng hợp của hai phép tịnh tiến theo khoảng $[m1 \ n1]$ và $[m2 \ n2]$ bằng phép biến đổi duy nhất một khoảng có giá trị bằng tổng của hai phép biến đổi $[m1+m2 \ n1+n2]$.

Thật vậy:

$$[T1] * [T2] = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ m1 & n1 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ m2 & n2 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ m1+m2 & n1+n2 & 1 \end{bmatrix}$$

② Phép tỷ lệ

Tương tự ma trận tỷ lệ:

$$[Ts] = \begin{bmatrix} Sx & 0 & 0 \\ 0 & Sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad [x' \ y' \ 1] = [x \ y \ 1] \begin{bmatrix} Sx & 0 & 0 \\ 0 & Sy & 0 \\ 0 & 0 & 1 \end{bmatrix} = [x.Sx \ y.Sy \ 1]$$

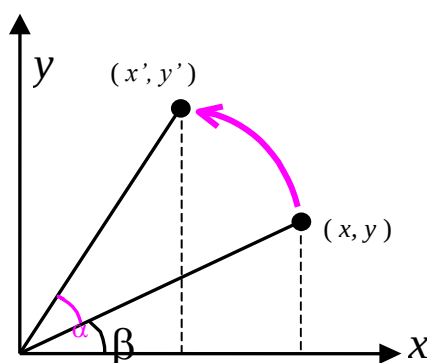
Chú ý: Phép biến đổi tổng hợp của hai phép tỷ lệ $Sx1, Sx2$ và $Sy1, Sy2$ bằng phép biến đổi duy nhất có tỷ lệ là tích hai phép biến đổi trên $Sx1*Sx2, Sy1*Sy2$.

Cài đặt c/c++ cho phép tỷ lệ đoạn thẳng tọa độ từ $(x1, y1)$ đến $(x2, y2)$:

`x11=int(x1*1.1);    x22=int(x2*0.9);`

`y11=int(y1*1.1);    y22=int(y2*0.9);`

③ Phép quay



Hình 3.6 Phép quay

$$\begin{bmatrix} x' & y' & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} \cos \alpha & \sin \alpha & 0 \\ -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} (x \cos \alpha - y \sin \alpha) & (x \sin \alpha + y \cos \alpha) & 1 \end{bmatrix}$$

Chú ý: Phép biến đổi tổng hợp của hai phép quay θ_1 và θ_2 là

$$M = \begin{pmatrix} \cos \theta_1 & \sin \theta_1 & 0 \\ -\sin \theta_1 & \cos \theta_1 & 0 \\ 0 & 0 & 1 \end{pmatrix} * \begin{pmatrix} \cos \theta_2 & \sin \theta_2 & 0 \\ -\sin \theta_2 & \cos \theta_2 & 0 \\ 0 & 0 & 1 \end{pmatrix} =$$

$$\begin{pmatrix} \cos(\theta_1 + \theta_2) & \sin(\theta_1 + \theta_2) & 0 \\ -\sin(\theta_1 + \theta_2) & \cos(\theta_1 + \theta_2) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Cài đặt c/c++ cho phép quay tam giác quanh gốc tọa độ :

```
void quay(int &x,int tamgiac(x1,y1,x2,y2,x3,y3,4);
&y,float goc){ quay(x1,y1,goc);
goc=goc*3.14/180; quay(x2,y2,goc);
int t=x; quay(x3,y3,goc);
x=x*cos(goc)- tamgiac(x1,y1,x2,y2,x3,y3,mau);
y*y*sin(goc);

y=t*sin(goc)+y*cos(goc);
}
```

3.2.3. Cài đặt c/c++ cho phép quay tam giác quanh 1 điểm (xq,yq):

```
#define RADS 0.017453293
void Quay(int x1,int y1, int x2, int y2, int x3, int y3,int xq, int
yq,float goc ){
float x11,y11,x22,y22,x33,y33;
float anpha = RADS *goc;
x11=int(x1*cos(anpha)-y1*sin(anpha)+(1-cos(anpha))*xq +
sin(anpha)*yq);
y11=int(x1*sin(anpha)+y1*cos(anpha)-sin(anpha)*xq+(1-
cos(anpha))*yq);
```

Chương 3: Các phép biến đổi đồ họa

```

x22=int(x2*cos(anpha)-y2*sin(anpha)+(1-cos(anpha))*xq +
sin(anpha)*yq);
y22=int(x2*sin(anpha)+y2*cos(anpha)-sin(anpha)*xq+(1-
cos(anpha))*yq);
x33=int(x3*cos(anpha)-y3*sin(anpha)+(1-cos(anpha))*xq +
sin(anpha)*yq);
y33=int(x3*sin(anpha)+y3*cos(anpha)-sin(anpha)*xq+(1-
cos(anpha))*yq);
tamgiac(x11,y11,x22,y22,x33,y33,12);
}

```

3.3. CÁC PHÉP BIẾN ĐỔI HÌNH HỌC BA CHIỀU

Các phép biến đổi chuyển vị - translation, tỉ lệ-scaling và quay-rotation sử dụng trong không gian 2D đều có thể mở rộng trong không gian 3D.

3.3.1. Biểu diễn điểm trong không gian 3 chiều

$$[x^* \ y^* \ z^* \ h] \text{ hay } [x^*/h \ y^*/h \ z^*/h \ 1]$$

Viết gọn hơn: $[x \ y \ z \ 1]$

Ma trận biến đổi tổng quát trong không gian 3D với tọa độ đồng nhất (4x4)

$$[T] = \begin{bmatrix} a & b & c & p \\ d & e & f & q \\ g & i & j & r \\ l & m & n & s \end{bmatrix} \quad \text{Hay } T = \begin{bmatrix} a & b & c & 0 \\ d & e & f & 0 \\ g & i & h & 0 \\ dx & dy & dz & 1 \end{bmatrix}$$

3.3.2. Phép tịnh tiến

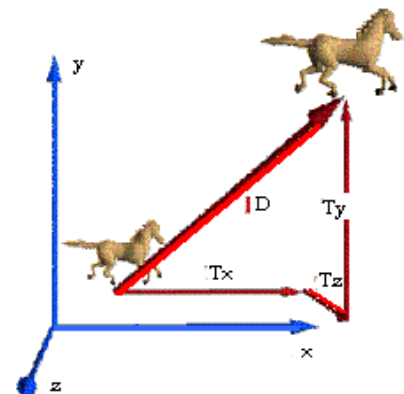
Đây là phép biến đổi đơn giản nhất, mở rộng từ phép biến đổi trong không gian 2D ta có:

$$[T(dx, dy, dz)] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ dx & dy & dz & 1 \end{bmatrix}$$

$$[X'] = [X] \cdot [T(dx, dy, dz)]$$

$$[x' \ y' \ z' \ 1] = [x \ y \ z \ 1] \cdot [T(dx, dy, dz)]$$

$$= [x+dx \ y+dy \ z+dz \ 1]$$



Hình 3.7 Phép tịnh tiến trên 3D

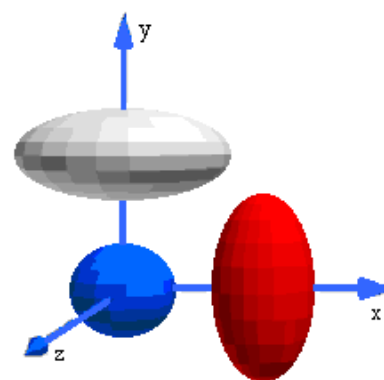
3.3.3. Phép tỉ lệ

Tương tự trong 2D ta có phép tỉ lệ trong 3D là :

$$T_s = \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Ta có S_x, S_y và S_z là các hệ số tỉ lệ trên các trục tọa độ

$$\begin{aligned} [X'] &= [X] \cdot [T(S_x, S_y, S_z)] \\ [x' \ y' \ z' \ 1] &= [x \ y \ z \ 1] \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ &= [x \cdot S_x \ y \cdot S_y \ z \cdot S_z \ 1] \end{aligned}$$



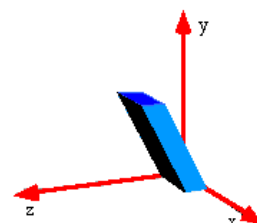
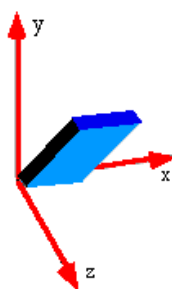
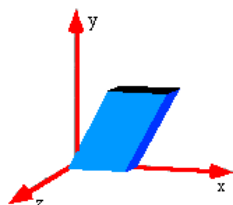
Hình 3.8 Phép tỷ lệ trên 3D

3.3.4. Phép biến dạng

- Ta có tất cả các phần tử nằm trên đường chéo chính bằng 1
- Các phần tử chiếu và tịnh tiến bằng 0

$$[X'] = [X] \cdot [T_{sh}]$$

$$\begin{aligned} [x' \ y' \ z' \ 1] &= [x \ y \ z \ 1] \begin{bmatrix} 1 & b & c & 0 \\ d & 1 & f & 0 \\ g & i & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ &= [x + yd + gz \quad bx + y + iz \quad cx + fy + z \quad 1] \end{aligned}$$



Hình 3.9 Các phép biến dạng trên 3D

3.3.5. Phép lấy đối xứng

Đối xứng qua trục ox

$$M_{ox} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Đối xứng qua mặt xoy

$$M_{xoy} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

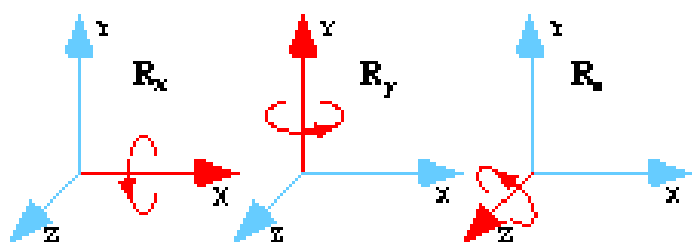
Đối xứng qua gốc O

$$M_o = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3.3.6. Phép quay 3 chiều

3.3.6.1. Quay quanh các trục toạ độ

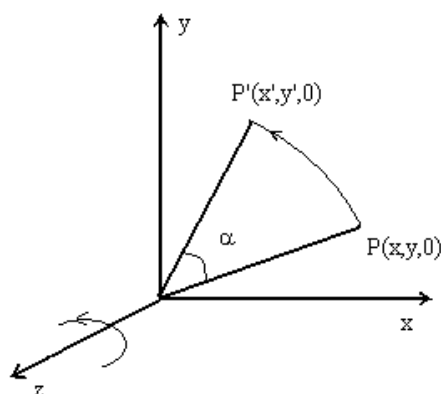
Đơn giản nhất là phép quay quanh các trục toạ độ ox, oy và oz với góc dương:



Hình 3.10 Xác định góc quay dương trên 3 trục toạ độ

Khi này phép quay lại đưa về phép quay không gian 2D quanh gốc toạ độ

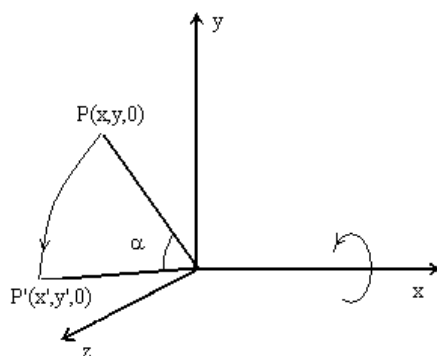
- Quay quanh trục oz:



$$\begin{cases} x' = x \cos \alpha - y \sin \alpha \\ y' = x \sin \alpha + y \cos \alpha \\ z' = z \end{cases} \quad [T_z] = \begin{bmatrix} \cos \varphi & \sin \varphi & 0 & 0 \\ -\sin \varphi & \cos \varphi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Hình 3.11 Quay quanh trục oz

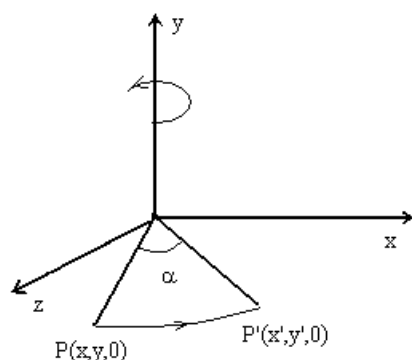
- Quay quanh trục ox:



$$\begin{cases} x' = x \\ y' = y \cos \alpha - z \sin \alpha \\ z' = y \sin \alpha + z \cos \alpha \end{cases} \quad [T_x] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \phi & \sin \phi & 0 \\ 0 & -\sin \phi & \cos \phi & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Hình 3.12 quay quanh trục ox

- Quay quanh trục oy:



$$\begin{cases} x' = x \cos \alpha + z \sin \alpha \\ y' = y \\ z' = -x \sin \alpha + z \cos \alpha \end{cases} \quad [Ty] = \begin{bmatrix} \cos \theta & 0 & -\sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ \sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Hình 3.13 quay quanh trục oy

Ghi chú: phép quay trong không gian 3D sử dụng phép nhân ma trận biến đổi không có khả năng hoán vị các ma trận.

3.3.6.2. Quay quanh một trục bất kỳ song song với các trục tọa độ

- Đầu tiên chuyển dịch đối tượng cho đến khi toạ độ địa phương của đối tượng trùng với trục tọa độ mà trục địa phương song song.

Quay đối tượng xung quanh trục của nó (chính là trục tọa độ)

- Đưa đối tượng về toạ độ trước khi dịch chuyển ta có ma trận tổng hợp là:

$$[T_{\alpha//}] = [T_{tt-}] \cdot [T_{\alpha}] \cdot [T_{tt+}]$$

Ví dụ: quay đối tượng xung quanh một trục // với trục z với khoảng dịch chuyển là x,y và góc quay là α .

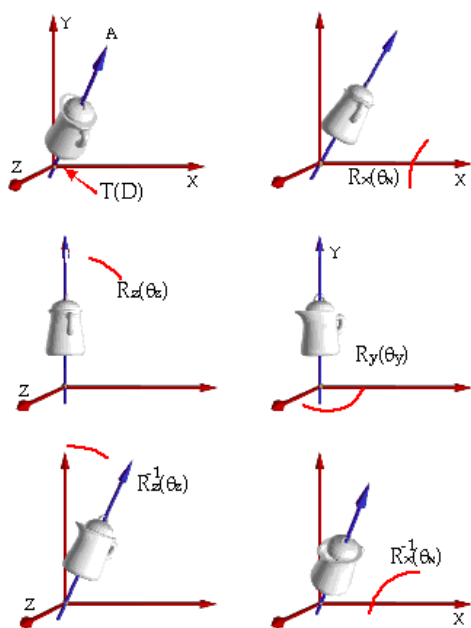
Giải:

$$\begin{aligned} [T_{\alpha//z}] &= [T_{tt-}] [T_{\alpha}] [T_{tt+}] \\ &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -x & -y & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos \alpha & \sin \alpha & 0 & 0 \\ -\sin \alpha & \cos \alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ x & y & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} \cos \alpha & \sin \alpha & 0 & 0 \\ -\sin \alpha & \cos \alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ (-x \cos \alpha + y \sin \alpha) & (-x \sin \alpha - y \cos \alpha) & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ x & y & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} \cos \alpha & \sin \alpha & 0 & 0 \\ -\sin \alpha & \cos \alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ x(1 - \cos \alpha) + y \sin \alpha & y(1 - \cos \alpha) - x \sin \alpha & 0 & 1 \end{bmatrix} \end{aligned}$$

3.3.6.3. Quay đối tượng quanh một trục bất kỳ

① Xét bài toán sau, hãy tìm ma trận biến đổi để đưa một trục bất kỳ có hướng: $V = ax + by + cz$ về trùng với trục oz theo chiều dương.

Chương 3: Các phép biến đổi đồ họa

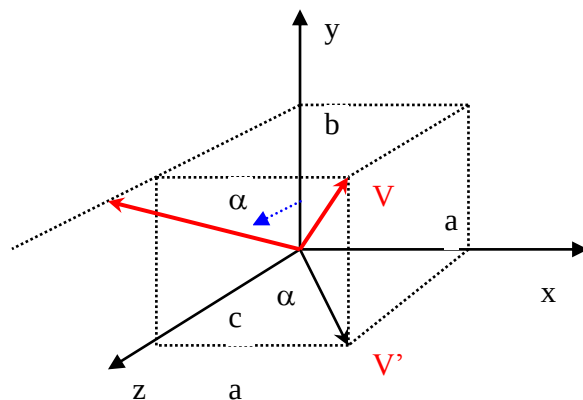


Hình 3.14 Quay đổi trục quanh một trục bất kỳ đi qua tâm

Ta thực hiện các bước như sau:

- Quay V quanh trục y một góc $(-\alpha)$ sao cho nằm trên mặt phẳng (y, z) được V_1
- Quay V_1 quanh trục x một góc β sao cho trùng với trục z được V_2

Bước 1:



Hình 3.15 Quay $V=ax+by+cz$ quanh trục oy

Ta tính α ?

Chiếu V trên mặt phẳng (x,z) được V' . Ta có:

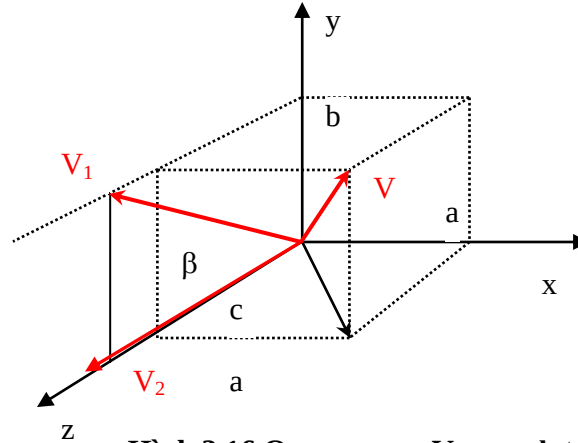
$$\sin \alpha = \frac{a}{\sqrt{a^2 + c^2}}$$

$$\cos \alpha = \frac{c}{\sqrt{a^2 + c^2}}$$

$$[T_{-oy}] = \begin{bmatrix} \cos \alpha & 0 & -\sin(-\alpha) & 0 \\ 0 & 1 & 0 & 0 \\ \sin(-\alpha) & 0 & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \frac{c}{\sqrt{a^2 + c^2}} & 0 & \frac{a}{\sqrt{a^2 + c^2}} & 0 \\ 0 & 1 & 0 & 0 \\ -\frac{a}{\sqrt{a^2 + c^2}} & 0 & \frac{c}{\sqrt{a^2 + c^2}} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Vector: $V_1(0, b, \sqrt{a^2 + c^2})$

Bước 2:



Hình 3.16 Quay vector V_1 quanh trục ox

$$\sin \beta = \frac{b}{\sqrt{a^2 + b^2 + c^2}}$$

$$\cos \beta = \frac{\sqrt{a^2 + c^2}}{\sqrt{a^2 + b^2 + c^2}}$$

$$[T_{\beta x}] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \beta & \sin \beta & 0 \\ 0 & -\sin \beta & \cos \beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \frac{\sqrt{a^2 + c^2}}{\sqrt{a^2 + b^2 + c^2}} & \frac{b}{\sqrt{a^2 + b^2 + c^2}} & 0 \\ 0 & -\frac{b}{\sqrt{a^2 + b^2 + c^2}} & \frac{\sqrt{a^2 + c^2}}{\sqrt{a^2 + b^2 + c^2}} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Vậy $[T_v] = [T_{-\alpha y}] \cdot [T_{\beta x}]$

Nếu đưa ngược lại ta được:

$$[T_v^{-1}] = ([T_{-\alpha y}] [T_{\beta x}])^{-1} = [T_{-\beta x}] [T_{\alpha y}]$$

② Cho trục quay L (vector $V = ax + by + cz$) và một điểm P nằm trên trục L . Hãy tìm ma trận biến đổi quay đối tượng xung quanh trục L một góc θ .

Giải:

Ta thực hiện như sau:

1. Tịnh tiến P về gốc tọa độ.
2. Chuyển trục L về trùng với trục OZ
3. Quay đối tượng xung quanh trục OZ một góc θ .
4. Thực hiện ngược lại 2,1

Vậy:

$$[T_{\theta, L}] = [T_{tt-}] [T_v] [T_{\theta, Z}] [T_v^{-1}] [T_{tt+}]$$

3.3.7. Cài đặt bằng c/c++ như sau:

- Đổi 3D sang 2D

```
#define RADS 0.017453293
```

```
struct point{
    int x,y,z;
}
point Diem3d(int &x, int &y, int &z) {
    point p;
    p.x = int(getmaxx()/2+ y - x*cos(RADS*45));
    p.y = int(getmaxy()/2 - z + x*cos(RADS*45));
    return p;
}
```

▪ Quay quanh các trục toạ độ

```
point quay(int x,int y,int z,float goc,int truc){
    point p;
    if (truc==1){ //Quay quanh OX
        p.y=y*cos(RADS*goc)-z*sin(RADS*goc);
        p.z=y*sin(RADS*goc)+z*cos(RADS*goc);
        p.x=x;
    }
    if (truc==2){ //Quay quanh OY
        p.x=x*cos(RADS*goc)+z*sin(RADS*goc);
        p.z=-x*sin(RADS*goc)+z*cos(RADS*goc);
        p.y=y;
    }
    if (truc==3){
        p.x=x*cos(RADS*goc)-y*sin(RADS*goc);
        p.y=x*sin(RADS*goc)+y*cos(RADS*goc);
        p.z=z;
    }
    return p;
}
```

Tóm tắt:

Các phép biến đổi hình học cho phép dễ dàng thao tác trên các đối tượng đã được tạo ra. Chúng làm thay đổi mô tả về toạ độ của các đối tượng, từ đó đối tượng sẽ được thay đổi về hướng, kích thước và hình dạng. Các phép biến đổi hình học cơ sở bao gồm tịnh tiến, quay và biến đổi tỷ lệ. Ngoài ra một số phép biến đổi khác cũng thường được áp dụng đó là phép đối xứng và biến dạng.

Các phép biến đổi hình học 2D đều được biểu diễn dưới dạng ma trận đồng nhất 3x3 để tiện cho việc thực hiện các thao tác kết hợp giữa chúng. Trong hệ toạ độ đồng nhất, toạ độ của một điểm được mô tả bởi một vector dòng bao gồm ba giá trị, hai giá trị đầu tương ứng với toạ độ Descartes của điểm đó, và giá trị thứ ba là 1. Với cách biểu diễn này, ma trận của phép biến đổi có được từ sự kết hợp của các phép biến đổi cơ sở sẽ bằng tích của các ma trận của các phép biến đổi thành phần.

Phép biến đổi hình học 3D là sự mở rộng của phép biến đổi 2D. Tương tự nó cũng có các phép: tịnh tiến, tỷ lệ, biến dạng và quay. Phức tạp nhất là phép quay, nên chúng ta

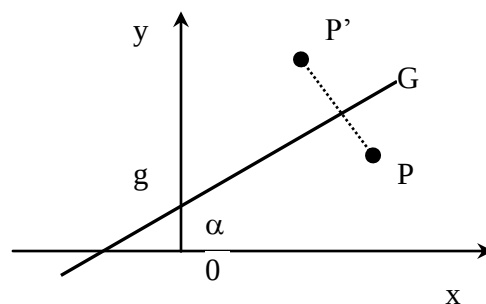
Chương 3: Các phép biến đổi đồ họa

khảo sát lần lượt từ đơn giản đến phức tạp: quay đối tượng quanh các trục toạ độ, quanh 1 trục song song với trục toạ độ, quanh 1 trục bất kỳ....Do khảo sát các phép biến đổi affine với biểu diễn dạng ma trận đồng nhất (4×4 với 3D) nên công việc khá đơn giản và nhất quán.

Lưu ý phép tịnh tiến và quay có chung thuộc tính là: sau khi biến đổi hình dạng và kích thước của đối tượng không thay đổi mà chúng chỉ bị thay đổi vị trí và định hướng trong không gian.

Bài tập:

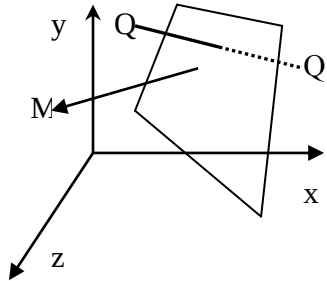
1.
 - a. Hãy tìm ma trận biểu thị phép quay một đối tượng một góc 60° quanh gốc toạ độ.
 - b. Tìm toạ độ mới của $P(-3,3)$ sau khi thực hiện phép quay trên?
2.
 - a. Hãy viết dạng tổng quát của ma trận điều chỉnh tỷ lệ tương ứng với một điểm cố định $Q(a,b)$?
 - b. Phóng lớn tứ giác có các đỉnh $A(0,0)$, $B(1,3)$, $C(4,2)$ và $D(3,1)$ lên hai lần kích thước ban đầu của nó trong khi vẫn giữ điểm $D(3,1)$?
3.
 - a. Mô tả phép biến đổi nhằm quay 1 đối tượng một góc α xung quanh một tâm cố định $Q(a,b)$?
 - b. Thực hiện phép quay tam giác ABC có $A(0,0)$, $B(1,1)$ và $C(4,2)$ một góc 45° xung quanh điểm $(-1, -1)$?
4. Cho ΔABC có các toạ độ đỉnh là $A(2,2)$, $B(3,1)$ và $C(4,3)$. Xác định ma trận biến đổi affine biến đổi tam giác này thành $A'B'C'$ biết ảnh $A'(4,3)$, $B'(4,5)$ và $C'(7,3)$.
5.
 - a. hãy tìm một ma trận dành cho phép phản chiếu đối xứng gương xung quanh một đường thẳng G với độ dốc m và tung độ gốc $(0,g)$?



- b. Tạo phản xạ đối xứng gương đa giác mà đỉnh của nó: $A(-1,0)$, $B(0,-2)$, $C(1,0)$ và $D(0,2)$ xung quanh đường G trong các trường hợp sau:
 - i. $x=2$
 - ii. $y=3$
 - iii. $y=2x+3$

Chương 3: Các phép biến đổi đồ họa

6. Cho hình chóp ABCD có các tọa độ $A(0,0,0)$, $B(1,0,0)$, $C(0,1,0)$ và $D(0,0,1)$. Quay hình chóp quanh đường L ($v=x+y+z$) đi qua điểm C một góc 45° . Tìm tọa độ hình chóp mới.
- 7.
- a. Hãy tìm phép biến đổi dành cho phép đối xứng gương tương ứng với một mặt phẳng đã cho (có vector pháp tuyến M , trên đó có một điểm P).



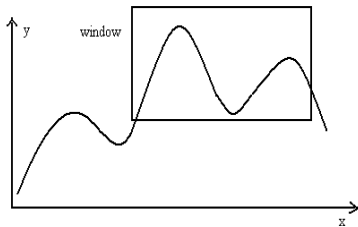
- b. Áp dụng tìm ma trận cho phép đối xứng gương với mặt phẳng đi qua gốc tọa độ và vector pháp tuyến có hướng $M=x+y+z$.
8. Viết chương trình với đối tượng (đường thẳng, tam giác, tứ giác...) trong mặt phẳng 2D
- a. Dịch chuyển (dùng các phím dịch chuyển)
- b. Phóng lớn, thu nhỏ (dùng phím dịch chuyển hay gõ vào từ bàn phím)
- c. Quay với góc quay được gõ vào từ bàn phím (góc gõ vào được tính bằng độ)
9. Viết chương trình với đối tượng (hình kim cương, hình lập phương ...) trong 3D
- a. Dịch chuyển
- b. Phóng lớn, thu nhỏ
- c. Quay một góc

CHƯƠNG 4: CÁC GIẢI THUẬT ĐỒ HOẠ CƠ SỞ

4.1. MÔ HÌNH CHUYỂN ĐỔI GIỮA BA HỆ THỐNG TOẠ ĐỘ

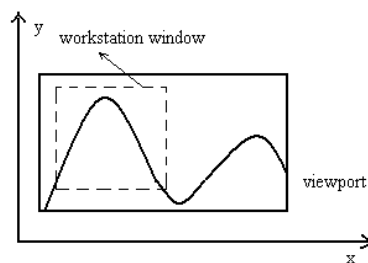
4.1.1. Mô hình chuyển đổi

Hệ tọa độ Descartes là dễ thích ứng cho các chương trình ứng dụng để miêu tả các hình ảnh (picture) trên hệ tọa độ thế giới thực (World Coordinate System). Các hình ảnh được định nghĩa trên hệ tọa độ thế giới thực này sau đó được hệ đồ họa vẽ lên các hệ tọa độ thiết bị (Device Coordinate). Để hiển thị, một vùng đồ họa cho phép người sử dụng xác định vùng nào của hình ảnh sẽ được hiển thị và bạn muốn đặt nó ở nơi nào trên hệ tọa độ thiết bị. Một vùng đơn lẻ hoặc vài vùng của hình ảnh có thể được chọn. Những vùng này có thể được đặt ở những vị trí tách biệt, hoặc một vùng có thể được chèn vào một vùng lớn hơn. Quá trình biến đổi này liên quan đến những thao tác như tịnh tiến, biến đổi tỷ lệ vùng được chọn và xóa bỏ những phần bên ngoài vùng được chọn.



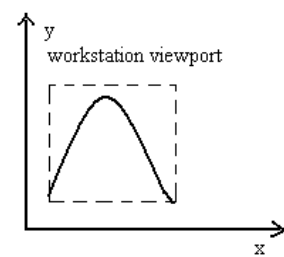
Hình 4.1 WCS

World Coordinate System



Hình 4.2 NDCS Normalized

Device Coordinate System

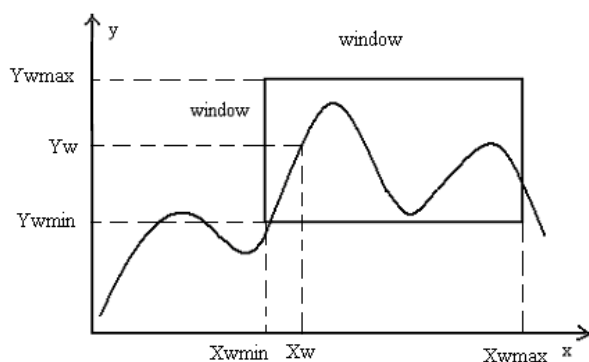


Hình 4.3 DCS

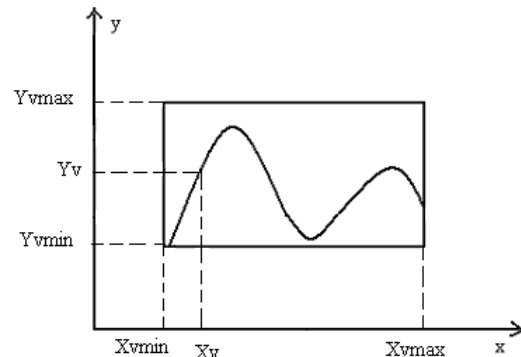
Device Coordinate System

- Qui trình để chuyển đổi các đối tượng trong WCS sang NDCS được gọi là phép ánh xạ cửa sổ sang khung nhìn hay phép biến đổi chuẩn hoá (Window to Viewport mapping or Normalization Transformation)
- Qui trình có thể áp các tọa độ thiết bị hiển thị chuẩn hoá sang các thiết bị rời rạc được gọi là phép biến đổi trạm làm việc (Workstation Transformation)

4.1.2. Phép ánh xạ từ cửa sổ vào khung nhìn



Hình 4.4 Đối tượng trong cửa sổ



Hình 4.5 Đối tượng tại khung nhìn

- Một cửa sổ (window) được chỉ định bởi bốn tọa độ thực (WCS): X_{wmin} , X_{wmax} , Y_{wmin} , Y_{wmax}

Chương 4: Các giải thuật đồ hoạ cơ sở

- Một khung nhìn (viewport) được mô tả bởi bốn tọa độ thiết bị chuẩn hoá (NDCS): X_{vmin} , X_{vmax} , Y_{vmin} , Y_{vmax}

Mục đích của phép ánh xạ này là chuyển đổi các tọa độ thực (X_w, Y_w) của một điểm tùy ý sang thiết bị chuẩn hoá tương ứng (X_v, Y_v). Để giữ lại khoảng cách của điểm trong khung nhìn bằng với khoảng cách của điểm trong cửa sổ, với yêu cầu:

$$\frac{X_v - X_{vmax}}{X_{vmax} - X_{vmin}} = \frac{X_w - X_{wmin}}{X_{wmax} - X_{wmin}}$$
$$\frac{Y_v - Y_{vmax}}{Y_{vmax} - Y_{vmin}} = \frac{Y_w - Y_{wmin}}{Y_{wmax} - Y_{wmin}}$$

Ta có:

$$X_v = \frac{(X_{vmax} - X_{vmin})}{(X_{wmax} - X_{wmin})} (X_w - X_{wmin}) + X_{vmax}$$
$$Y_v = \frac{(Y_{vmax} - Y_{vmin})}{(Y_{wmax} - Y_{wmin})} (Y_w - Y_{wmin}) + Y_{vmax}$$

Ta có tám giá trị ở trên để xác định cửa sổ và khung nhìn đều là hằng số. Vậy chúng ta hoàn toàn tính được (X_v, Y_v) từ (X_w, Y_w) qua phép biến đổi:

$$[X_v Y_v 1] = [X_w Y_w 1] \cdot [T]$$

$$[T] = [T_{tt-}] \cdot [T_s] \cdot [T_{tt}]$$

- Ma trận tịnh tiến theo window:

$$[T_{tt-}] = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -x_w & -y_w & 1 \end{bmatrix}$$

- Ma trận chuyển đổi tỷ lệ window vào viewport là:

$$[T_s] = \begin{bmatrix} \frac{X_{vmax} - X_{vmin}}{X_{wmax} - X_{wmin}} & 0 & 0 \\ 0 & \frac{Y_{vmax} - Y_{vmin}}{Y_{wmax} - Y_{wmin}} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Ma trận tịnh tiến theo tọa độ viewport:

$$[T_{tt}] = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ x_v & y_v & 1 \end{bmatrix}$$

Vậy ma trận biến đổi từ cửa sổ vào khung nhìn:

$$[T] = \begin{bmatrix} \frac{x_{vmax} - x_{vmin}}{x_{wmax} - x_{wmin}} & 0 & 0 \\ 0 & \frac{y_{vmax} - y_{vmin}}{y_{wmax} - y_{wmin}} & 0 \\ x_{vmin} - x_{wmin} \cdot \frac{x_{vmax} - x_{vmin}}{x_{wmax} - x_{wmin}} & y_{vmin} - y_{wmin} \cdot \frac{y_{vmax} - y_{vmin}}{y_{wmax} - y_{wmin}} & 1 \end{bmatrix}$$

4.2. CÁC GIẢI THUẬT XÉN TỈA (CLIPPING)

4.2.1. Khái niệm

Xén tỉa là tiến trình xác định các điểm của một đối tượng nằm trong hay ngoài cửa sổ hiển thị. Nằm trong được hiển thị, nằm ngoài loại bỏ.

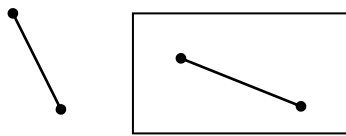
Việc loại từng điểm ảnh của đối tượng thường chậm nhất là khi đối tượng mà phần lớn nằm ngoài cửa sổ hiển thị.

4.2.2. Clipping điểm

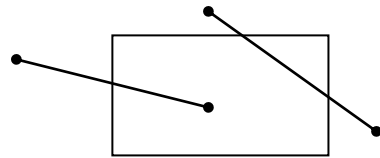
Giả sử (x,y) là tọa độ của một điểm, vậy điểm đó được hiển thị khi thỏa mãn:

$$Xmin \leq x \leq Xmax$$

$$Ymin \leq y \leq Ymax$$



Các đoạn thẳng không cắt cửa sổ thì: hoặc nằm trong hoàn toàn, hoặc nằm ngoài hoàn toàn



Nếu đoạn thẳng cắt cửa sổ thì phân chia qua điểm cắt phần nằm trong và phần nằm ngoài

4.2.3. Xén tỉa đoạn thẳng

4.2.3.1. Thuật toán Cohen-Sutherland

Ý tưởng: Các đoạn thẳng có thể rơi vào các trường hợp sau

- Hiển thị (visible): cả hai đầu cuối của đoạn thẳng đều nằm bên trong cửa sổ
- Không hiển thị (not visible): đoạn thẳng xác định nằm ngoài cửa sổ. Điều này xảy ra khi đoạn thẳng từ (x1,y1) đến (x2,y2) thỏa mãn bất kỳ một trong bốn bất đẳng thức sau:

$$x_1, x_2 > x_{max} \vee y_1, y_2 > y_{max}$$

$$x_1, x_2 < x_{min} \vee y_1, y_2 < y_{min}$$

- Xén tỉa: đoạn thẳng cần xén tỉa

Việc cài đặt giải thuật chia làm hai bước:

Bước 1 : Gán mã vùng 4-bit cho mỗi điểm cuối của đoạn thẳng

1001	1000	1010
0001	0000	0010
0101	0100	0110

Window

Hình 4.6 Mặt phẳng mã trong các trường hợp clipping đoạn thẳng

Mã vùng được xác định theo 9 vùng của mặt phẳng mà các điểm cuối nằm vào đó. Một bit được cài đặt true (1) hoặc false (0).

Bít 1: điểm cuối ở bên trên cửa sổ $s_o = \text{sign}(y - y_{\max})$

Bít 2: điểm cuối ở bên dưới cửa sổ $s_o = \text{sign}(y_{\min} - y)$

Bít 3: điểm cuối ở bên phải cửa sổ $s_o = \text{sign}(x - x_{\max})$

Bít 4: điểm cuối ở bên trái cửa sổ $s_o = \text{sign}(x_{\min} - x)$

Qui ước $\text{sign}(a) = 1$ nếu a dương

$= 0$ nếu a âm

Bước 2: Quá trình kiểm tra vị trí của đoạn thẳng so với cửa sổ. Tất cả điểm đầu và điểm cuối của đoạn thẳng đã có mã.

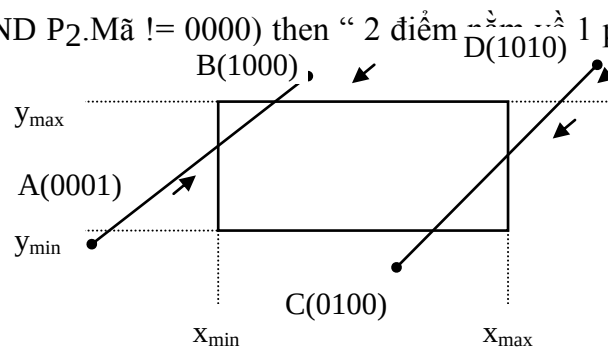
Giải thuật như sau:

Bước 1: Nếu mã của P₁ hoặc P₂ đều = 0000 thì toàn bộ đoạn thẳng thuộc phần hiển thị.

If (P₁.Mã OR P₂.Mã == 0000) then “ cả đoạn thẳng thuộc cửa sổ hiển thị”

Bước 2: Nếu mã của P₁ và P₂ có cùng một vị trí mà ở đó P₁ và P₂ => cùng phía

If (P₁.Mã AND P₂.Mã != 0000) then “ 2 điểm D(1010) 1 phía của cửa sổ” **Bước 3**: Xét giao điểm:



Hình 4.7 Mô tả thuật toán Cohen Sutherland outcode

Tìm giao điểm của đường thẳng với cửa sổ, chính xác hơn là với phần mở rộng của đường biên.

Chú ý: các đường biên mà điểm cuối được chọn sẽ “đẩy ngang qua” nhằm thay đổi mã “1” thành “0”

Nếu:

Bít 1 là 1: cắt $y = y_{\max}$

Bít 2 là 1: cắt $y = y_{\min}$

Chương 4: Các giải thuật đồ hoạ cơ sở

Bít 3 là 1: cắt $x=x_{\max}$

Bít 4 là 1: cắt $x=x_{\min}$

Nhìn trên hình ta có: gọi điểm cuối của đoạn (x_1, y_1)

Nếu C được chọn thì đường $y=y_{\min}$ chọn để tính phần cắt nhau (bít 2 = 1)

Nếu D được chọn thì $y=y_{\max}$ hoặc $x=x_{\max}$ (bít 1 và bít 3 = 1)

Toạ độ cắt:

$$\begin{cases} x_i = x_{\min} / x_{\max} \\ y_i = y_1 + m(x_i - x_1) \end{cases} \left\| \begin{array}{l} x = x_{\min} \\ x = x_{\max} \end{array} \right.$$

Hoặc:

$$\begin{cases} x_i = x_1 + (y_i - y_1) / m \\ y_i = y_{\min} / y_{\max} \end{cases} \left\| \begin{array}{l} y = y_{\min} \\ y = y_{\max} \end{array} \right.$$

Có m là độ dốc của đoạn thẳng

$$m = (y_2 - y_1) / (x_2 - x_1)$$

Bây giờ thay điểm cuối (x_1, y_1) với điểm cắt (x_i, y_i)

Ví dụ: Cho cửa sổ cắt tia hình chữ nhật có góc trái dưới L(-3,1), góc phải trên R(2,6)

Hãy tìm mã vùng dành cho các điểm cuối của các đoạn AB có A(-2,3), B(1,2) ; CD có C(-4,7), D(-2,10) ; IK có I(-4,2), K(-1,7)

Tìm hạng mục cắt tia của chúng

Giải:

▪ Mã vùng

$$\text{Bít 1} = \text{sign}(y - y_{\max}) = \text{sign}(y - 6)$$

$$\text{Bít 2} = \text{sign}(y_{\min} - y) = \text{sign}(1 - y)$$

$$\text{Bít 3} = \text{sign}(x - x_{\max}) = \text{sign}(x - 2)$$

$$\text{Bít 4} = \text{sign}(x_{\min} - x) = \text{sign}(-3 - x)$$

$$\begin{aligned} \text{Qui ước } \text{sign}(a) &= 1 \text{ nếu } a \text{ dương} \\ &= 0 \text{ nếu } a \text{ âm} \end{aligned}$$

Vậy:

A(-2,3) có (0000)

B(1,2) có (0000)

C(-4,7) có (1001)

D(-2,10) có (1000)

I(-4,2) có (0001)

K(-1,7) có (1000)

▪ Hạng mục cắt tia

AB có mã vùng đều là (0000) nên nó nằm hoàn toàn trong

CD có (1001) and (1000) = (1000) (!= (0000)) nên hoàn toàn nằm ngoài

IK có (0001) or (1000) = (1001) và (0001) and (1000) = (0000) nên phải xén tia.

Xét I(0001) dựa trên đường biên $x_{\min} = -3$

$$\begin{cases} x_c = x_{\min} \\ y_c = y_1 + m(x_c - x_1) \end{cases}$$

$$x_{\min} = -3$$

Chương 4: Các giải thuật đồ hoạ cơ sở

$$m=(y_2-y_1)/x_2-x_1=(7-2)/(-1-(-4))=5/3$$

$$y_c=2+5/3(-3-(-4))=11/3$$

Thay I bởi $I_1(-3,11/3)$ (có mã vùng 0000)

Xét $K(1000)$ cắt $y_{\max}=6$

$$\begin{cases} x_c = x_1 + (y_c - y_1)/m \\ y_c = y_{\max} \end{cases}$$

$$y_c=6$$

$$x_c=-1+(6-7)/(5/3)=-8/5$$

Thay K bởi $K_1(-8/5,6)$ có mã vùng 0000

Vậy sau khi xén tia đoạn IK thu được I_1K_1

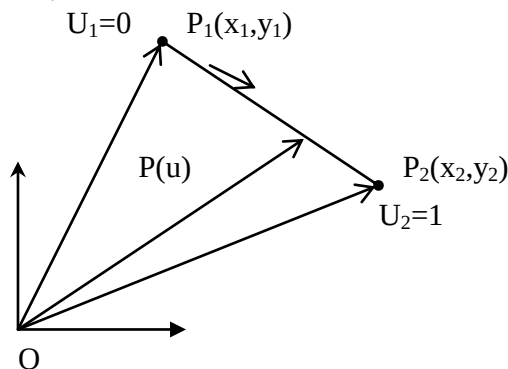
Cài đặt bằng c/c++ là :

```
#define LEFT_EDGE    0x1
#define RIGHT_EDGE   0x2
#define TOP_EDGE     0x8
#define BOTTOM_EDGE  0x4
#define INSIDE(a)    (!a)
#define REJECT(a,b) (a&b)
#define ACCEPT(a,b) (!(a|b))

unsigned char encode(double x, double y, double xmin, double
ymin,double xmax, double ymax){
    unsigned char code = 0x00;
    if (x < xmin)
        code = code | LEFT_EDGE;
    if (x > xmax)
        code = code | RIGHT_EDGE;
    if (y < ymin)
        code = code | TOP_EDGE;
    if (y > ymax)
        code = code | BOTTOM_EDGE;
    return code;
}
```

4.2.3.2. Giải thuật Lyangbarsky

:



Hình 4.8 Phương trình tham số cho đoạn thẳng

Chương 4: Các giải thuật đồ hoạ cơ sở

Biểu diễn đoạn thẳng theo tham biến: đoạn thẳng bất kỳ đi qua hai điểm $P_1(x_1, y_1)$ và $P_2(x_2, y_2)$ chúng ta có phương trình tham biến :

$$x = x(u) \quad \text{có } 0 \leq u \leq 1$$

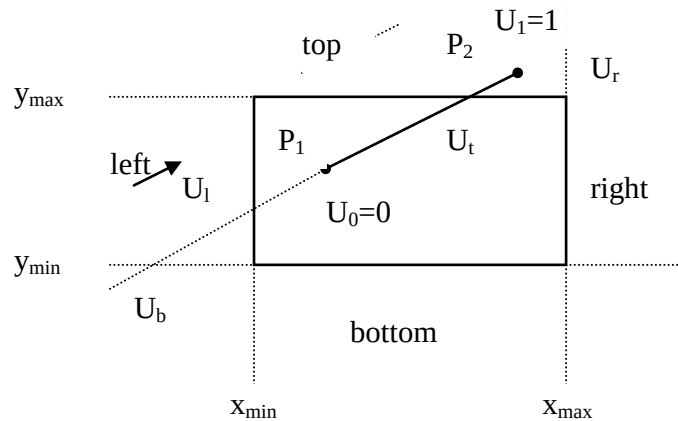
$$y = y(u)$$

Vì vector chỉ có một chiều nên vector vị trí:

$$P(u) = P_1 + (P_2 - P_1) \cdot u$$

$$x = x_1 + (x_2 - x_1)u = x_1 + \Delta x \cdot u$$

$$y = y_1 + (y_2 - y_1)u = y_1 + \Delta y \cdot u$$



Hình 4.9 Mô tả giải thuật LyaBarsky

- Khi ta chuyển động dọc theo đường mở rộng với u tăng từ $-\infty$ tới $+\infty$ thì ta phải:
 - Chuyển từ bên ngoài vào bên trong hai đường biên của cửa sổ cắt tia (ở dưới và bên trái)
 - Sau đó chúng ta di chuyển từ trong ra bên ngoài của hai đường biên khác (từ trên và từ bên phải)
- Với điểm (x, y) nằm bên trong cửa sổ cắt tia:

$$x_{\min} \leq x_1 + \Delta x \cdot u \leq x_{\max}$$

$$y_{\min} \leq y_1 + \Delta y \cdot u \leq y_{\max}$$

Suy ra:

$$-\Delta x \cdot u \leq x_1 - x_{\min} \quad k=1$$

$$\Delta x \cdot u \leq x_{\max} - x_1 \quad k=2$$

$$-\Delta y \cdot u \leq y_1 - y_{\min} \quad k=3$$

$$\Delta y \cdot u \leq y_{\max} - y_1 \quad k=4$$

Viết tổng quát:

$$P_k \cdot u \leq q_k \quad \text{có } k=1, 2, 3, 4$$

Trong đó:

$$P_1 = -\Delta x, q_1 = x_1 - x_{\min} \quad (\text{phải})$$

$$P_2 = \Delta x, q_2 = x_{\max} - x_1 \quad (\text{trái})$$

$$P_3 = -\Delta y, q_3 = y_1 - y_{\min} \quad (\text{trên})$$

$$P_4 = \Delta y, q_4 = y_{\max} - y_1 \quad (\text{dưới})$$

Xét:

▪ Nếu $P_k = 0$: điều đó tương đương với việc đoạn thẳng đang xét song song với cạnh thứ k của hình chữ nhật clipping.

a) Nếu $q_k < 0$ đoạn thẳng nằm ngoài cửa sổ (hệ bất phương trình trên vô nghiệm)

b) Nếu $q_k \geq 0$ thì đoạn thẳng nằm trong hoặc nằm trên cạnh của cửa sổ clipping.

Yêu cầu khảo sát tiếp.

▪ Nếu $P_k \neq 0$: đoạn thẳng đang xét sẽ cắt cạnh k tương ứng của cửa sổ clipping tại vị trí trên đoạn thẳng $u_k = q_k/P_k$

a) $P_k < 0$ đoạn thẳng có dạng đi từ ngoài vào trong của đường biên tương ứng

b) $P_k > 0$ thì đoạn thẳng mở rộng tiến hành từ trong ra ngoài của đường biên tương

ứng.

Kết hợp lại ta có các bước sau:

(Với $u_0 \leq u \leq u_1$)

Bước 1: Nếu $P_k = 0$

$q_k < 0$ ứng với bất kỳ giá trị nào của k . Loại bỏ đoạn thẳng $\Rightarrow$ Exit

$q_k \geq 0$ thực hiện bước tiếp theo

Bước 2: Tính:

$$U_0 = \max \left(\{0\} \cup \left\{ u_k : u_k = \frac{q_k}{P_k}, P_k < 0 \right\} \right)$$

$$U_1 = \min \left(\{1\} \cup \left\{ u_k : u_k = \frac{q_k}{P_k}, P_k > 0 \right\} \right)$$

u_0 là các giá trị cực đại của tập hợp đang chứa 0 và các giá trị u_k được tính

u_1 là các giá trị cực tiểu của tập hợp đang chứa 1 và các giá trị u_k được tính

Ví dụ: cho cửa sổ cắt tia hình chữ nhật có góc trái dưới $L(1,2)$ và góc phải trên $R(9,8)$. Có các đoạn AB tọa độ $A(11,10)$ và $B(11,6)$, đoạn CD tọa độ $C(3,2)$ và $D(8,4)$, đoạn IJ tọa độ $I(-1,7)$ và $J(11,1)$. Tìm các hạng mục cắt tia.

Giải:

▪ Xét AB

$$P_1 = 0, q_1 = 10$$

$$P_2 = 0, q_2 = -2$$

$$P_3 = -4, q_3 = 4$$

$$P_4 = 4, q_4 = 2$$

Có $P_2 = 0$ mà $q_2 = -2 < 0$ nên AB nằm hoàn toàn ngoài

▪ Xét CB

$$P_1 = -5, q_1 = 2 \quad u_1 = -2/5 \quad (u_k = q_k/p_k)$$

$$P_2 = 5, q_2 = 6 \quad u_2 = 6/5$$

Chương 4: Các giải thuật đồ hoạ cơ sở

$$P_3 = -2q_3=0 \quad u_3=0$$

$$P_4 = 2 q_4=6 \quad u_4=3$$

Vậy $U_0=\max(0,-2/5,0)=0$ (với $P_k<0$)

$$U_1=\min(1,6/5,3)=1(\text{với } P_k>0)$$

Vậy CD nằm hoàn toàn trong cửa sổ

▪ Xét IJ

$$P_1 = -12q_1=-2 \quad u_1=1/6$$

$$P_2 = 12 q_2=10 \quad u_2=5/6$$

$$P_3 = 6q_3=5 \quad u_3=5/6$$

$$P_4 = -6 q_4=1 \quad u_4=-1/6$$

Vậy $u_0=\max(0,1/6,-1/6)=1/6$ (với $P_k<0$)

$$u_1=\min(1,5/6,5/6)=5/6(\text{với } P_k>0)$$

Vì $u_0<u_1$ nên hai điểm cuối của đường cắt tia là:

$$X_{c1}=x_1 + \Delta x.u_0 = -1 + (11-(-1)).1/6 = 1$$

$$Y_{c1}=y_1 + \Delta y.u_0 = 7 + (1-7).1/6 = 6$$

$$X_{c2}=x_1 + \Delta x.u_1 = -1 + (11-(-1)).5/6 = 9$$

$$Y_{c2}=y_1 + \Delta y.u_1 = 7 + (1-7).5/6 = 2$$

Vậy đường sau khi cắt tia là: I_1I_2 có $I_1(1,6)$ và $I_2(9,2)$

Cài đặt bằng c/c++ là :

```
#define TRUE 1
#define FALSE 0
int cliptest(double p, double q, double *u0, double *u1) {
    double r;int retVal=TRUE;
    if (p < 0.0) {
        r = q / p;
        if (r > *u1)
            retVal = FALSE;
        else
            if (r > *u0)
                *u0 = r;    }
    else
        if ( p > 0.0) {
            r = q / p;
            if (r < *u0)
                retVal = FALSE;
            else
                if (r < *u1)
                    *u1 = r;
        }
    Else {
        if (q < 0.0)
            retVal = FALSE;
```

```

    }
    return (retVal);
}

void clipline(double x1, double y1, double x2, double y2,
double xmin, double ymin, double xmax, double ymax){
    double u0 = 0.0, u1 = 1.0, dx, dy;
    double oldx1, oldy1, oldx2, oldy2;
    oldx1 = x1;   oldy1 = y1;
    oldx2 = x2;   oldy2 = y2;
    dx = x2 - x1;
    if (cliptest(-dx, x1 - xmin, &u0, &u1))
        if (cliptest(dx, xmax-x1, &u0, &u1)) {
            dy = y2 - y1;
            if (cliptest(-dy, y1 - ymin, &u0, &u1))
                if (cliptest(dy, ymax - y1, &u0, &u1)){
                    if (u1 < 1.0){
                        x2 = x1 + u1 * dx;
                        y2 = y1 + u1 * dy;
                    }
                    if (u0 > 0.0){
                        x1 += u0 * dx;
                        y1 += u0 * dy;  }
                    setcolor(RED);
                    line(oldx1, oldy1, x1, y1);
                    line(x2, y2, oldx2, oldy2);
                    setcolor(YELLOW);
                    line(x1, y1, x2, y2);
                }
        }
}
}
}

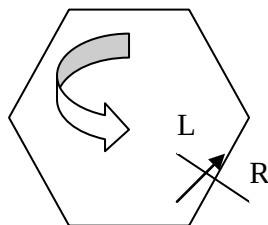
```

4.2.4. Giải thuật xén tỉa đa giác (Sutherland Hodgman)

4.2.4.1. Khái niệm

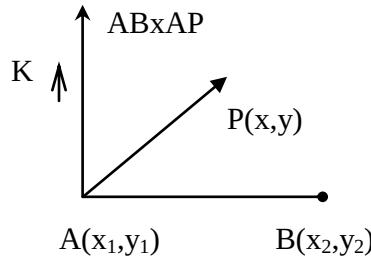
Theo qui ước: một đa giác với các đỉnh $P_1, \dots, P_N$ (các cạnh là $P_{i-1}P_i$ và P_NP_1) được gọi là theo hướng dương nếu các hình theo thứ tự đã cho tạo thành một mạch ngược chiều kim đồng hồ.

Nếu bàn tay trái của một người đọc theo bất kỳ cạnh $P_{i-1}P_i$ hoặc P_NP_1 cũng chỉ về bên trong đa giác.



Hình 4.10 Qui tắc tìm hướng dương của một đa giác hướng

Khảo sát một điểm so với một đường thẳng:



Hình 4.11 Khảo sát một điểm so với đường thẳng

Có điểm $A(x_1, y_1)$, $B(x_2, y_2)$ và $P(x, y)$

Theo định nghĩa thì tích của hai vector ($AB \times AP$) chỉ theo hướng của vector $K \perp$ với mặt phẳng (xoy). Có:

$$AB = (x_2 - x_1)i + (y_2 - y_1)j$$

$$AP = (x - x_1)i + (y - y_1)j$$

$$\text{Vậy } AB \times AP = [(x_2 - x_1)(y - y_1) - (y_2 - y_1)(x - x_1)] \cdot K$$

Do đó hướng được xác định như sau:

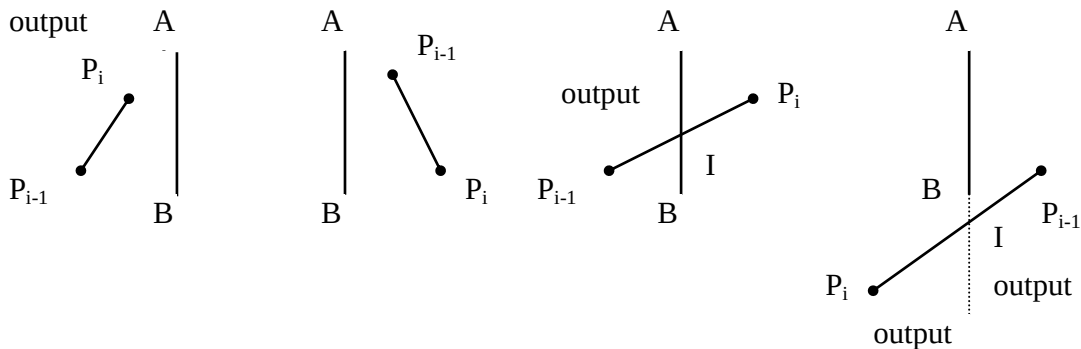
$$\bar{C} = (x_2 - x_1)(y - y_1) - (y_2 - y_1)(x - x_1)$$

Nếu C dương thì P nằm bên trái AB

Nếu C âm thì P nằm bên phải AB

4.2.4.2. Giải thuật Hodgman

Cho $P_1, \dots, P_N$ là danh sách các đỉnh của đa giác đã bị cắt tỉa. Cho cạnh AB (điểm A, B) là cạnh bất kỳ. Đa giác cắt tỉa lỗi hướng dương.

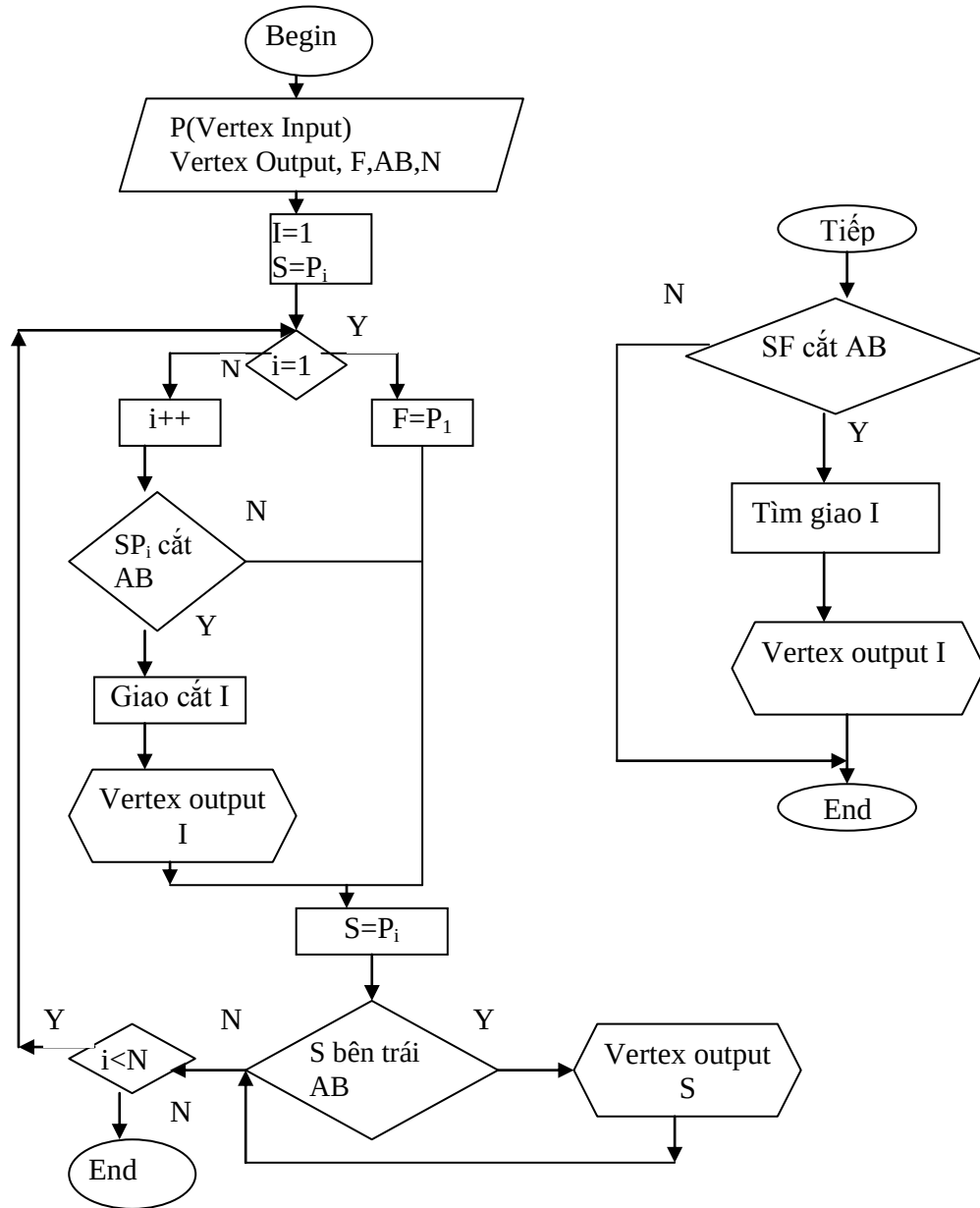


Hình 4.12 Các trường hợp trong giải thuật Hodgman

- Nếu cả P_{i-1} và P_i đều nằm bên trái của cạnh này thì P_i được lưu lại (đưa vào output) của đa giác cắt tỉa.
- Nếu cả P_{i-1} và P_i đều nằm bên phải của cạnh thì không có đỉnh nào được lưu lại.
- Nếu P_{i-1} nằm bên trái và P_i nằm bên phải của cạnh thì giao điểm I của $P_{i-1}P_i$ với cạnh sẽ được lưu lại.
- Nếu P_{i-1} nằm bên phải và P_i nằm bên trái thì cả giao điểm I và P_i đều được lưu lại.

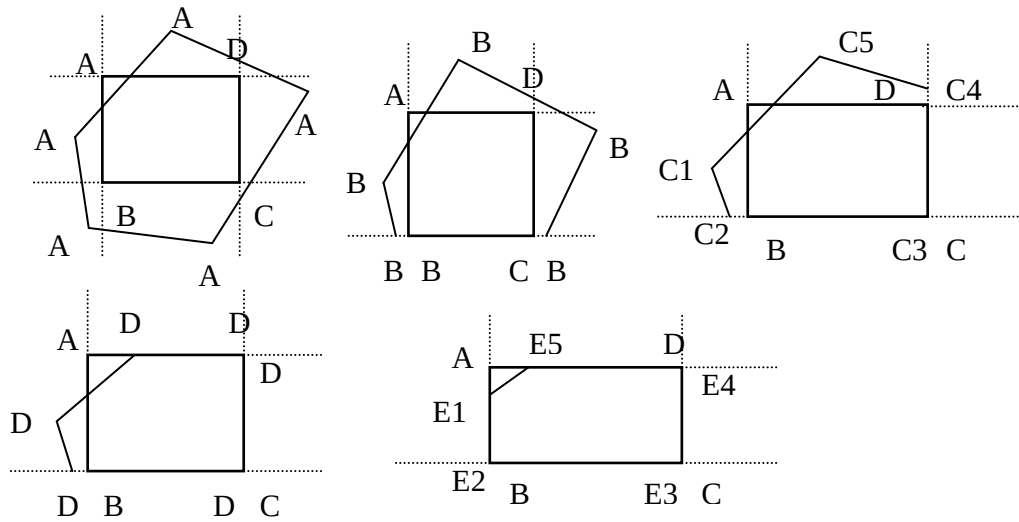
4.2.4.3. Minh họa thuật toán Sutherland – Hodgman

$F=P_1$ (F là đỉnh được đưa vào đầu tiên, S là đỉnh đưa vào trước P - đỉnh được đưa). Sơ đồ bên phải khép kín đa giác bởi $P_N P_1$



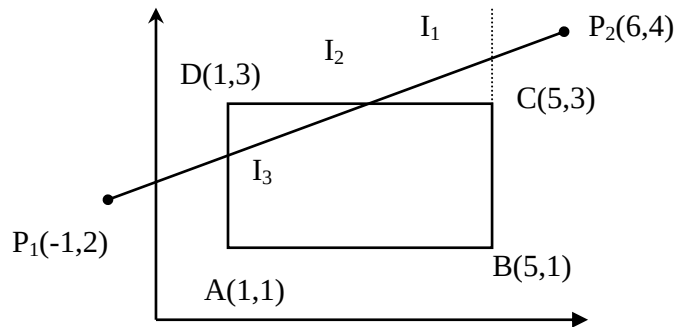
Hình 4.13 Sơ đồ khối thuật toán Hodgman

4.2.4.4. Ví dụ minh họa:



Hình 4.14 Ví dụ dùng giải thuật Hodgman xén tia đa giác A1...A5

Ví dụ 2: hãy sử dụng thuật toán Hodgman để cắt xén đoạn thẳng nối $P_1(-1,2)$ đến $P_2(6,4)$ trên cửa sổ $A(1,1)$, $B(5,1)$, $C(5,3)$ và $D(1,3)$



Hình 4.15 Ví dụ dùng thuật toán Hodgman xén tia đoạn P_1P_2

Theo thuật toán Hodgman ta xén P_1P_2 dựa trên từng cạnh.

▪ AB: ta xét $C=(x_2-x_1)(y-y_1) - (y_2-y_1)(x-x_1)$

Điểm P_1 : $C=(5-1)(2-1)-(1-1)(-1-1) = 4 > 0$ nằm bên trái

Điểm P_2 : $C= (5-1)(4-1)-(1-1)(-1-1) = 12 > 0$ nằm bên trái

Vậy P_1P_2 đều được lưu

▪ BC

Điểm P_1 : $C= (5-5)(2-1)-(3-1)(-1-5)= 12 > 0$ nằm bên trái

Điểm P_2 : $C= - (3-1)(6-5) = -2 < 0$ nằm bên phải

Giao điểm I_1 : $m = \frac{4-2}{6-(-1)} = \frac{2}{7}$

$$\begin{cases} x_c = 5 \\ y_c = y_1 + (x_c - x_1)m = 2 + (5 - (-1)) \cdot \frac{2}{7} = 3\frac{5}{7} \end{cases}$$

$I_1(5, 26/7)$ vậy P_1I_1 lưu lại

▪ CD

Điểm P_1 : $C= (1-5)(2-3) = 4 > 0$ nằm bên trái

Điểm I_1 : $C=(1-5)(26/7 - 3) = -20/7 < 0$ nằm bên phải

Giao điểm I_2 :

$$\begin{cases} y_c = 3 \\ x_c = x_1 + (y_c - y_1)/m \\ = -1 + (3 - 2) \cdot \frac{7}{2} = \frac{5}{2} \end{cases}$$

Vậy $I_2(5/2, 3)$ có P_1I_2 được lưu

▪ DA

Điểm P_1 : $C = -(1-3)(-1-1) = -4$ nằm bên phải

Điểm I_2 : $C = -(1-3)(5/2-1) = 7/2$ nằm bên trái

Giao điểm I_3 :

$$\begin{cases} x_c = 1 \\ y_c = y_1 + (x_c - x_1)m \\ = 2 + (1 - (-1)) \cdot \frac{2}{7} = 2\frac{4}{7} \end{cases} \quad I_3(1, 18/7)$$

Tóm lại: Cắt xén đoạn P_1P_2 được đoạn I_2I_3 có $I_2(5/2, 3)$ và $I_3(1, 18/7)$

Tóm tắt chương:

Hiện thị đối tượng là quá trình đưa các mô tả đối tượng từ thế giới thực sang một thiết bị xuất cụ thể nào đó. Mỗi đối tượng có thể được quan sát ở nhiều vị trí và góc độ khác nhau. Thông thường mỗi hình ảnh mà chúng ta quan sát được trên màn hình thiết bị được gọi là một góc nhìn (view) của đối tượng.

Quá trình loại bỏ các phần hình ảnh nằm ngoài một vùng cho trước được gọi là xén tia. Vùng được dùng để xén tia gọi là cửa sổ xén tia.

Các thuật toán xén tia đoạn thẳng Cohen-Sutherland, LyaBarsky đều tập trung giải quyết hai vấn đề chính nhằm tối ưu tốc độ thuật toán đó là: loại bỏ việc tìm giao điểm đối với các đoạn thẳng chắc chắn không cắt cửa sổ (như nằm hoàn toàn trong, nằm hoàn toàn ngoài), và với đoạn thẳng có khả năng cắt cửa sổ, cần phải tìm cách hạn chế số lần tìm giao điểm với các biên không cần thiết. Thuật toán Cohen-Sutherland giải quyết hai ý này thông qua kiểu dữ liệu mã vùng mà về bản chất đó chỉ là sự mô tả vị trí tương đối của vùng đang xét so với các biên của cửa sổ. Còn thuật toán LyaBarsky thì tuy dựa vào phương trình tham số của đoạn thẳng để lập luận nhưng thực chất là dựa trên việc xét các giao điểm có thể có giữa đoạn thẳng kéo dài với các biên của cửa sổ (cũng được kéo dài). Các giao điểm này tương ứng với các giá trị $r_k = q_k/p_k$. Cả hai thuật toán này đều có thể được mở rộng cho việc xén hình trong đồ hoạ 3D.

Bài tập:

1. Tìm phép biến đổi chuẩn hoá tạo ánh xạ cho một cửa sổ mà góc bên trái phía dưới của nó (1,1) và góc bên phải trên (3,5) vào:
 - a. Là màn hình thiết bị được chuẩn hoá toàn bộ.
 - b. Một khung nhìn có góc bên trái dưới (0,0) và góc bên trái trên (1/2, 1/2)

2. Cho cửa sổ cắt tỉa hình chữ nhật có góc bên trái dưới $(-3,-2)$ và góc phải trên $(2,3)$, dùng giải thuật Cohen Sutherland xén tỉa đoạn thẳng AB có toạ độ là A(3,2) và B(-2,-4).
3. Cho cửa sổ cắt tỉa hình chữ nhật có góc bên trái dưới $(-3,-2)$ và góc phải trên $(2,3)$, dùng giải thuật LyaBarsky xén tỉa đoạn thẳng AB có toạ độ là A(3,2) và B(-2,-4).
4. Cho cửa sổ cắt tỉa hình chữ nhật có góc bên trái dưới $(-3,-2)$ và góc phải trên $(2,3)$, dùng giải thuật Hodgman xén tỉa đoạn thẳng AB có toạ độ là A(3,2) và B(-2,-4).

CHƯƠNG 5: PHÉP CHIẾU –PROJECTION

5.1. KHÁI NIỆM CHUNG

5.1.1. Nguyên lý về 3D (three-Dimension)

Đồ họa 3 chiều (3D computer graphics) bao gồm việc bổ xung kích thước về chiều sâu của đối tượng, cho phép ta biểu diễn chúng trong thế giới thực một cách chính xác và sinh động hơn.

Tuy nhiên các thiết bị truy xuất hiện tại đều là 2 chiều, Do vậy việc biểu diễn được thực thi thông qua phép tô chất (render) để gây ảo giác (illusion) về độ sâu

Đồ họa 3D là việc chuyển thế giới tự nhiên dưới dạng các mô hình biểu diễn trên các thiết bị hiển thị thông qua kỹ thuật tô cát (rendering).

5.1.2. Đặc điểm của kỹ thuật đồ họa 3D

Có các đối tượng phức tạp hơn các đối tượng trong không gian 2D

- Bao bởi các mặt phẳng hay các bề mặt
- Có các thành phần trong và ngoài

Các phép biến đổi hình học phức tạp

Các phép biến đổi hệ tọa độ phức tạp hơn

Thường xuyên phải bổ xung thêm phép chiếu từ không gian 3D vào không gian 2D

Luôn phải xác định các bề mặt hiển thị

5.1.3. Các phương pháp hiển thị 3D

Với các thiết bị hiển thị 2D thì chúng ta có các phương pháp sau để biểu diễn đối tượng 3D:

- Kỹ thuật chiếu (projection): Trục giao (orthographic)/phối cảnh (perspective)
- Kỹ thuật đánh dấu độ sâu (depth cueing)
- Nét khuất (visible line/surface identification)
- Tô chất bề mặt (surface rendering)
- Cắt lát (exploded/cutaway scenes, cross-sections)

Các thiết bị hiển thị 3D:

- Kính stereo - Stereoscopic displays*
- Màn hình 3D – Holograms

Chương 5: Phép chiếu

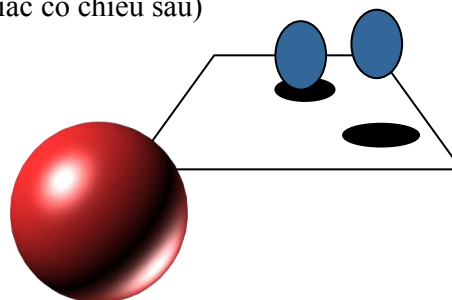
Exploded/cutaway scenes (cắt lát)



Perspective (phối cảnh) and Depth of Field (chiều sâu)



Shadows as depth cues (tạo bóng cảm giác có chiều sâu)



Hình 5.1 Các cách mô tả đối tượng 3D



Hình chiếu cạnh

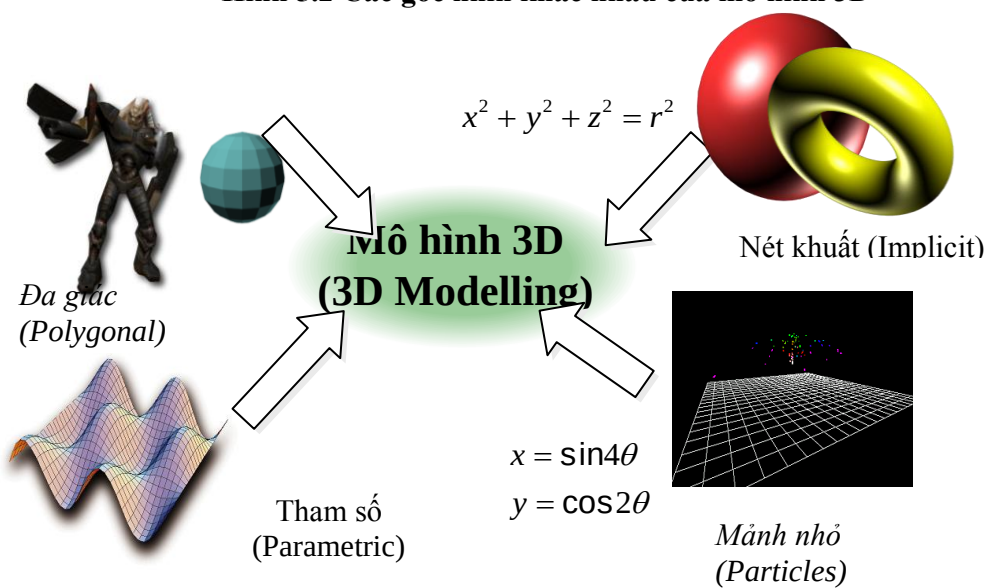


Hình chiếu bằng



Hình chiếu đứng

Hình 5.2 Các góc nhìn khác nhau của mô hình 3D



Hình 5.3 Các cách mô tả đối tượng 3D

5.2. PHÉP CHIẾU

Định nghĩa về phép chiếu

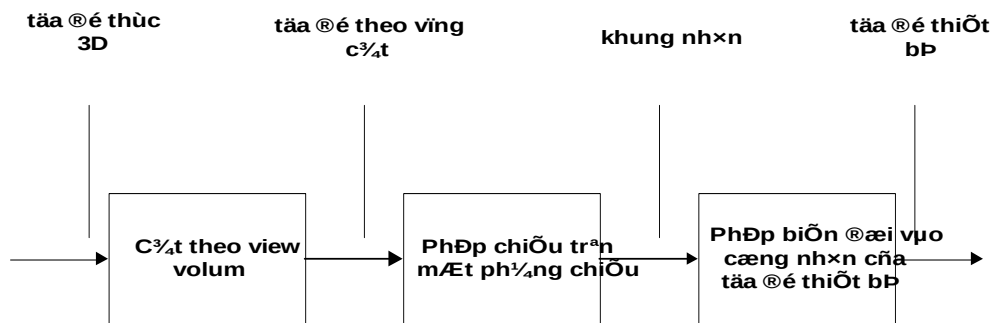
Một cách tổng quát, phép chiếu là phép chuyển đổi những điểm của đối tượng trong hệ thống tọa độ n chiều thành những điểm trong hệ thống tọa độ có số chiều nhỏ hơn n .

Định nghĩa về hình chiếu

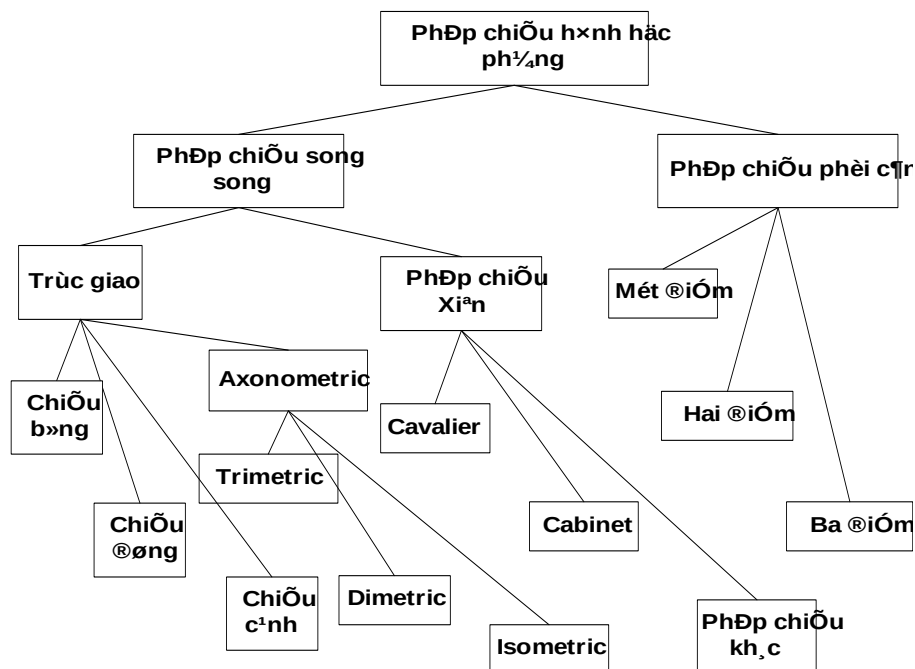
Ảnh của đối tượng trên mặt phẳng chiếu được hình thành từ phép chiếu bởi các đường thẳng gọi là tia chiếu (projector) xuất phát từ một điểm gọi là tâm chiếu (center of projection) đi qua các điểm của đối tượng giao với mặt chiếu (projection plan).

Các bước xây dựng hình chiếu

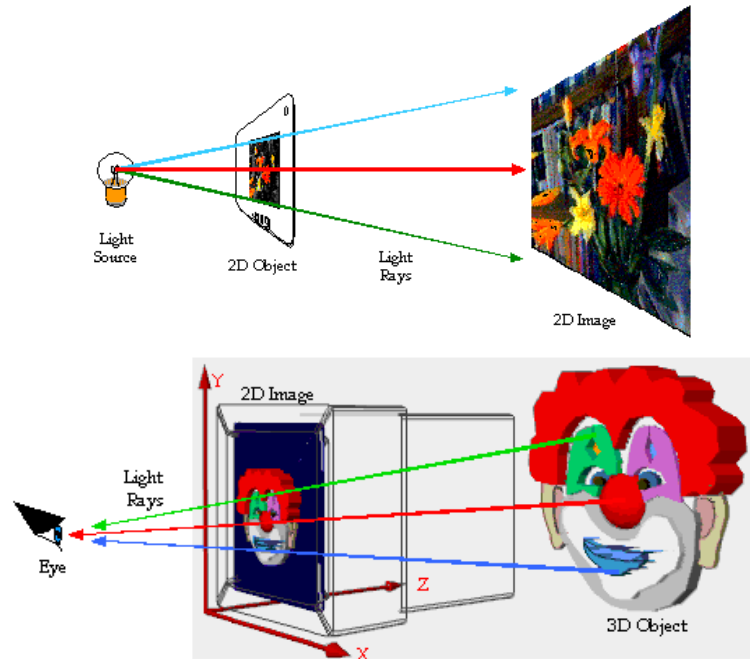
1. Đối tượng trong không gian 3D với tọa độ thực được cắt theo một không gian xác định gọi là view volume.
2. View volume được chiếu lên mặt phẳng chiếu. Diện tích choán bởi view volume trên mặt phẳng chiếu đó sẽ cho chúng ta khung nhìn.
3. Là việc ánh xạ khung nhìn vào trong một cổng nhìn bất kỳ cho trước trên màn hình để hiển thị hình ảnh.



Hình 5.4 Mô hình nguyên lý của tiến trình biểu diễn đối tượng 3D



Hình 5.5 Phân loại các phép chiếu



Hình 5.6 Ví dụ minh họa các phép chiếu phối cảnh

5.3. PHÉP CHIẾU SONG SONG (Parallel Projections)

Phép chiếu song song (Parallel Projections) là phép chiếu mà ở đó các tia chiếu song song với nhau hay xuất phát từ điểm vô cùng.

Phân loại phép chiếu song song dựa trên hướng của tia chiếu (*Direction Of Projection*) và mặt phẳng chiếu (*projection plane*).

5.3.1. Phép chiếu trực giao (Orthographic projection)

Là phép chiếu song song và tia chiếu vuông góc với mặt phẳng chiếu. Về mặt toán học, phép chiếu trực giao là phép chiếu với một trong các mặt phẳng tọa độ có giá trị bằng 0. Thường dùng mặt phẳng $z=0$, ngoài ra $x=0$ và $y=0$.

Ứng với mỗi mặt phẳng chiếu ta có một ma trận chiếu tương ứng.

$$[T_y] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad [T_x] = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad [T_z] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Hình 5.7 Phép chiếu trực giao

Thông thường thì người ta không sử dụng cả 6 mặt phẳng để suy diễn ngược hình của một đối tượng mà chỉ sử dụng một trong số chúng như: hình chiếu bằng, đứng, cạnh.

Cả sáu góc nhìn đều có thể thu được từ một mặt phẳng chiếu thông qua các phép biến đổi hình học như quay, dịch chuyển hay lấy đối xứng.

Ví dụ: giả sử chúng ta có hình chiếu bằng trên mặt phẳng $z=0$, với phép quay đối tượng quanh trục một góc 90° sẽ cho ta hình chiếu cạnh.

Đối với các đối tượng mà các mặt của chúng không song song với một trong các mặt phẳng hệ tọa độ thì phép chiếu này không cho hình dạng thật của vật thể. Muốn nhìn vật thể chính xác hơn người ta phải hình thành phép chiếu thông qua việc quay và dịch chuyển đối tượng sao cho mặt phẳng đó song song với các trục tọa độ.

Hình của đối tượng quá phức tạp cần thiết phải biết các phần bên trong của đối tượng đôi lúc chúng ta phải tạo mặt cắt đối tượng.

5.3.2. Phép chiếu trục lượng (Axonometric)

Phép chiếu trục lượng là phép chiếu mà hình chiếu thu được sau khi quay đối tượng sao cho ba mặt của đối tượng được trông thấy rõ nhất (thường mặt phẳng chiếu là $z=0$).

Có 3 phép chiếu

- Phép chiếu Trimetric
- Phép chiếu Dimetric
- Phép chiếu Isometric

5.3.2.1. Phép chiếu Trimetric

Là phép chiếu hình thành từ việc quay tự do đối tượng trên một trục hay tất cả các trục của hệ tọa độ và chiếu đối tượng đó bằng phép chiếu song song lên mặt phẳng chiếu (thường là mặt phẳng $z = 0$).

Ngoại trừ những mặt phẳng của đối tượng song song với mặt phẳng chiếu không thay đổi. Các mặt khác của đối tượng hay hình dạng của đối tượng thường biến dạng sau phép chiếu. Tuy nhiên tỉ lệ co (Shortening Factor - SF) là tỷ số của độ dài đoạn thẳng chiếu so với độ dài thực tế của đối tượng. Trên cơ sở SF phép chiếu trục lượng được chia làm ba loại sau:

- Phép chiếu Trimetric
- Phép chiếu Dimetric
- Phép chiếu Isometric

Việc tính các giá trị SF này của các trục tương ứng dựa vào công thức $[U]^* [T]$.

$$[U] = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \quad [T] = \begin{bmatrix} x'_x & y'_x & 0 & 1 \\ x'_y & y'_y & 0 & 1 \\ x'_z & y'_z & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

[U]: là ma trận vector đơn vị của các trục x, y, z bất biến

[T]: là ma trận chiếu tổng hợp tương ứng

SF- tỉ lệ co theo các trục là:

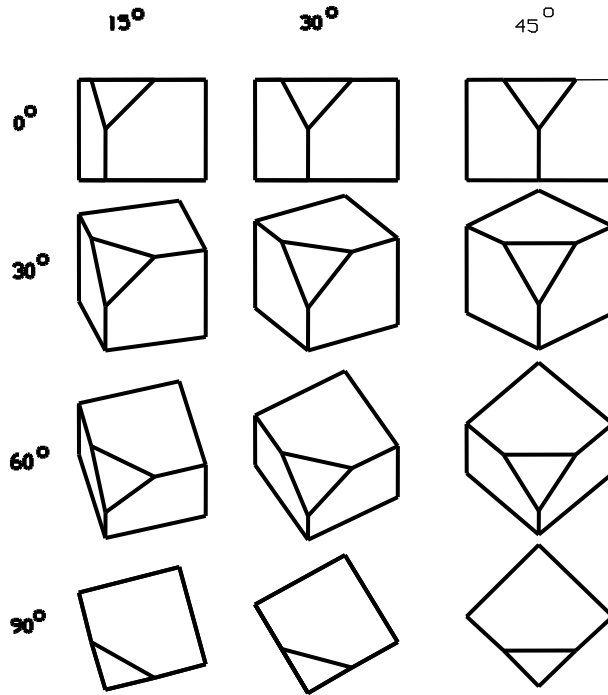
$$f_x = \sqrt{x'^2_x + y'^2_x} \quad f_y = \sqrt{x'^2_y + y'^2_y} \quad f_z = \sqrt{x'^2_z + y'^2_z}$$

5.3.2.2. Phép chiếu Dimetric

Là phép chiếu Trimetric với 2 hệ số tỉ lệ co bằng nhau, giá trị thứ 3 còn lại là tùy ý.

Chương 5: Phép chiếu

Phép chiếu được xây dựng bằng cách quay đối tượng quanh trục y theo một góc ϕ , tiếp đó quanh trục ox một góc φ và sau cùng là phép chiếu trên mặt phẳng $z=0$ với tâm chiếu tại vô tận.



$$[T] = [Ry][Rx][Pz]$$

$$= \begin{bmatrix} \cos \phi & 0 & -\sin \phi & 0 \\ 0 & 1 & 0 & 0 \\ \sin \phi & 0 & \cos \phi & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi & 0 \\ 0 & -\sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T] = \begin{bmatrix} \cos \phi & \sin \phi \sin \varphi & 0 & 0 \\ 0 & \cos \varphi & 0 & 0 \\ \sin \phi & -\cos \phi \sin \varphi & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} x'_x & y'_x & 0 & 0 \\ x'_y & y'_y & 0 & 0 \\ x'_z & y'_z & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Hình 5.8 Ảnh của phép chiếu Trimetric với các tham số góc xoay thay đổi

$$f_x^2 = (x_x'^2 + y_x'^2) = \cos^2 \phi + \sin^2 \phi \cdot \sin^2 \varphi \quad f_y^2 = (x_y'^2 + y_y'^2) = \cos^2 \varphi$$

Tỷ lệ co trên x và y bằng nhau nên ta có:

$$\cos^2 \varphi = \cos^2 \phi + \sin^2 \phi \cdot \sin^2 \varphi$$

$$1 - \sin^2 \varphi = 1 - \sin^2 \phi + \sin^2 \phi \cdot \sin^2 \varphi$$

$$\sin^2 \phi (\sin^2 \varphi - 1) = -\sin^2 \varphi$$

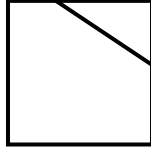
$$f_z^2 = (x_z'^2 + y_z'^2) = \sin^2 \phi + \cos^2 \phi \sin^2 \varphi$$

$$= \sin^2 \phi + \sin^2 \varphi (1 - \sin^2 \phi) = \sin^2 \varphi + \sin^2 \phi (1 - \sin^2 \varphi)$$

$$\sin^2 \phi = \frac{f_z^2 - \sin^2 \varphi}{1 - \sin^2 \varphi} = \frac{\sin^2 \varphi}{1 - \sin^2 \varphi} \quad \sin^2 \phi = \frac{\sin^2 \varphi}{1 - \sin^2 \varphi} \quad \phi = \sin^{-1} \left(\pm \frac{f_z}{\sqrt{2 - f_z^2}} \right)$$

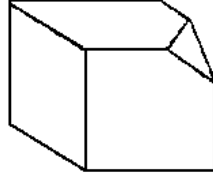
$$\sin^2 \varphi = \frac{f_z}{2} \quad \varphi = \sin^{-1} \left(\pm \frac{f_z}{\sqrt{2}} \right)$$

Ta thấy tỷ lệ co thuộc khoảng $[0 \ 1]$, với mỗi f_z ta có bốn khả năng của phép chiếu.



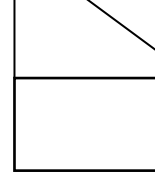
$$f_z = 0$$

$$f_z = 0$$



$$f_z = 1/2$$

$$f_z = 1/2$$



$$f_z = 1$$

$$f_z = 1$$

Hình 5.9 Phép chiếu hình hộp với $f=0$, $f=1/2$ và $f=1$

Khi $f_z = 1/2$ thì:

$$\varphi = \sin^{-1} \left(\pm \frac{1}{2\sqrt{2}} \right) \approx \sin^{-1} (\pm 0.35355) \approx \pm 20.705^\circ$$

$$\phi = \sin^{-1} \left(\pm \frac{1/2}{\sqrt{7/4}} \right) = \sin^{-1} (\pm 0.378) \approx \pm 22.208^\circ$$

5.3.2.3. Phép chiếu Isometric

Là phép chiếu trục lượng mà ở đó hệ số co cạnh trên 3 trục là bằng nhau.

Góc quay tương ứng là 35.26° và 45°

Được ứng dụng nhiều trong việc xây dựng các góc quan sát chuẩn cho đối tượng trong các hệ soạn thảo đồ hoạ.

$$\sin^2 \phi = \frac{1 - 2 \sin^2 \varphi}{1 - \sin^2 \varphi} \quad \sin^2 \phi = \frac{\sin^2 \varphi}{1 - \sin^2 \varphi}$$

$$\sin \varphi = \pm \frac{1}{\sqrt{3}}$$

$$\varphi = \pm 35.26^\circ$$

$$\sin^2 \phi = \frac{\sin^2 \varphi}{1 - \sin^2 \varphi} = \frac{1/3}{1 - 1/3} = 1/2$$

$$\sin \phi = \pm \frac{1}{\sqrt{2}}$$

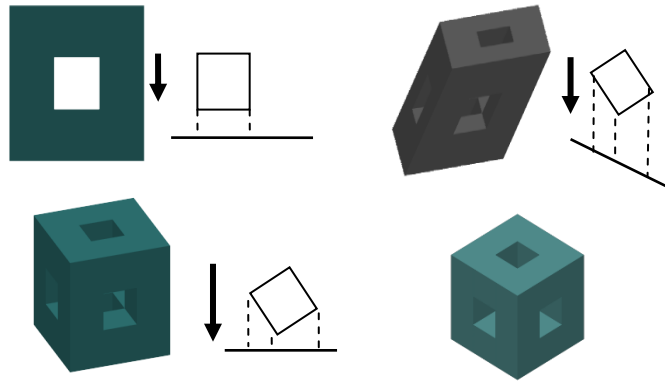
$$\phi = \pm 45^\circ$$

$$\varphi = \pm 35.26^\circ$$

$$\phi = \pm 45^\circ$$

$$f = \sqrt{\cos^2 \varphi} = \sqrt{2/3} = 0.8165$$

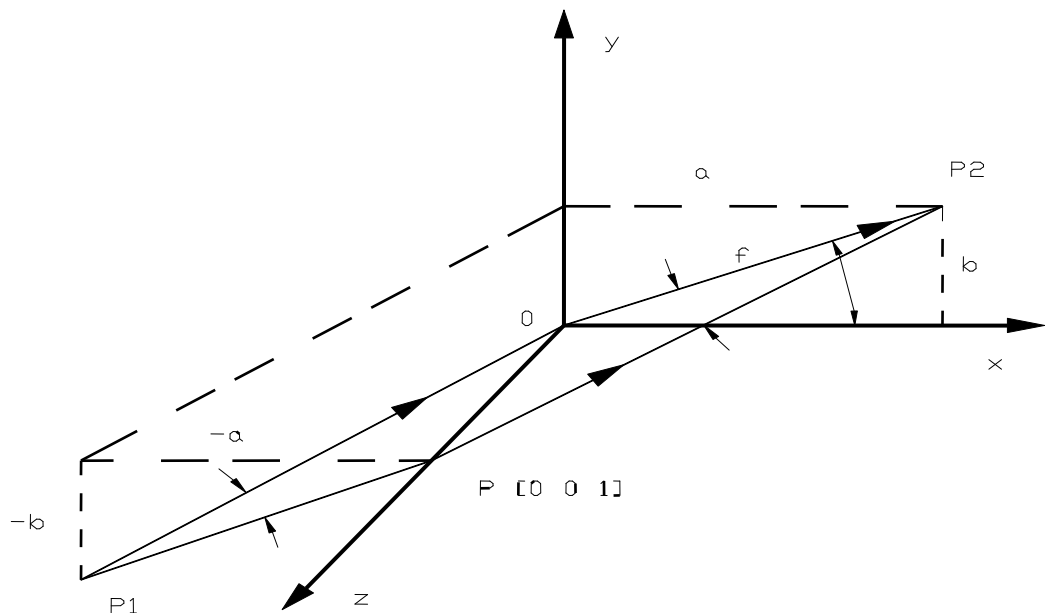
Ví dụ các phép chiếu song song:



Hình 5.10 Ví dụ các phép chiếu song song

5.3.3. Phép chiếu xiên - Oblique

- Phép chiếu Cavalier
- Phép chiếu Cabinet



Hình 5.11 Phép chiếu với tâm chiếu trên trục oz

5.3.3.1. Phép chiếu Cavalier

Phép chiếu cavalier là phép chiếu xiên được tạo thành khi các tia chiếu làm thành với mặt phẳng chiếu một góc 45°

Hệ số co trên các hệ trục tọa độ bằng nhau.

Ta có vectơ đơn vị theo trục z là $P=[0 \ 0 \ 1]$ tia chiếu làm với mặt phẳng một góc và p được chiếu lên mặt phẳng chiếu.

Ta có P_1O và PP_2 là 2 tia chiếu của phép chiếu xiên với góc β với mặt phẳng chiếu.

P_2 là ảnh của P

P_1 là điểm dịch chuyển của P tạo ra tia chiếu song song PP_2

Tia chiếu P_1O có thể thu được từ phép biến đổi P đến điểm $[-a-b]$

Trong không gian hai chiều dịch chuyển sẽ là:

$$T' = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -a & -b & 1 \end{bmatrix}$$

Trong không gian ba chiều, f là chiều dài của hình chiếu vector đơn vị trục z trên mặt phẳng chiếu, đó chính là hệ số co.

$$[T'] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -a & -b & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \begin{matrix} a = f \cos \alpha \\ b = f \sin \alpha \end{matrix} \quad \hookrightarrow \quad [T] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -f \cos \alpha & -f \sin \alpha & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$f = 0, b = 90^\circ$ phép chiếu sẽ trở thành phép chiếu trục giao.

Còn với $f = 1$ kích thước của hình chiếu bằng kích thước của đối tượng $\Rightarrow$ cavalier

Phép chiếu Cavalier cho phép giá trị của a biến đổi một cách tự do $a = 30^\circ$ và 45°

5.3.3.2. Phép chiếu Cabinet

Phép chiếu xiên với hệ số co tỉ lệ $f = 1/2$.

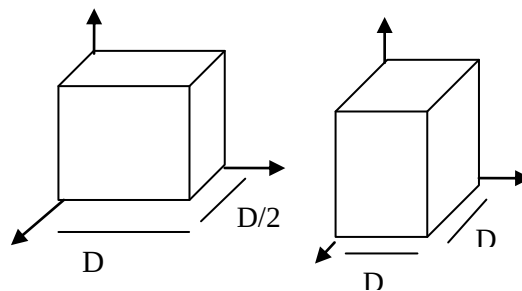
Với phép chiếu cabinet, giá trị của α có thể thay đổi tùy ý. Thông thường thì các giá trị hay được sử dụng là 30° và 45° .

Theo phép chiếu xiên một số mặt của đối tượng có thể được hiển thị như hình dạng thật cho nên rất phù hợp với việc mô tả các đối tượng có hình dạng tròn hay các bề mặt

$$\begin{aligned} \beta &= \cos^{-1} \left(\frac{f}{\sqrt{1^2 + f^2}} \right) \\ &= \cos^{-1} \left(\frac{1/2}{\sqrt{1^2 + (1/2)^2}} \right) = 63.435^\circ \end{aligned}$$

cong.

Ví dụ về phép chiếu xiên (Oblique Projections)



Hình 5.12 Ví dụ các phép chiếu xiên hình hộp

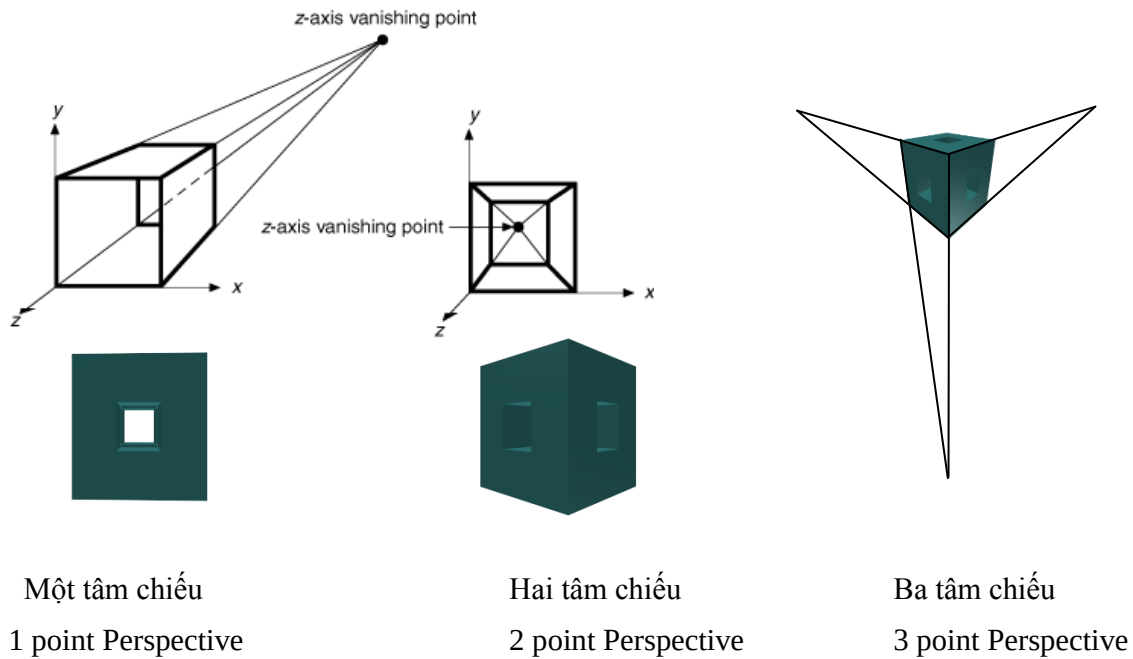
5.4. PHÉP CHIẾU PHỐI CẢNH (Perspective Projection)

Phép chiếu phối cảnh là phép chiếu mà các tia chiếu không song song với nhau mà xuất phát từ một điểm gọi là tâm chiếu. Phép chiếu phối cảnh tạo ra hiệu ứng về luật xa gần tạo cảm giác về độ sâu của đối tượng trong thế giới thật mà phép chiếu song song không lột tả được.

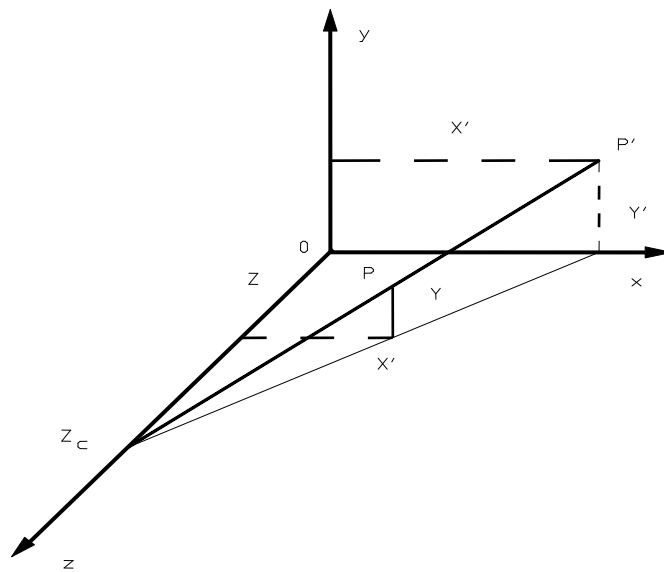
Các đoạn thẳng song song của mô hình 3D sau phép chiếu hội tụ tại một điểm gọi là điểm triệt tiêu (vanishing point).

Chương 5: Phép chiếu

Phân loại phép chiếu phối cảnh dựa vào tâm chiếu - Centre Of Projection (COP) và mặt phẳng chiếu projection plane



Hình 5.13 Phép biến đổi phối cảnh



Hình 5.14 Phép chiếu với tâm chiếu trên trục z

Phép chiếu phối cảnh của các điểm trên đối tượng lên trên mặt phẳng 2D thu được từ phép chiếu trực giao và phép biến đổi phối cảnh.

5.4.1. Phép chiếu phối cảnh một tâm chiếu

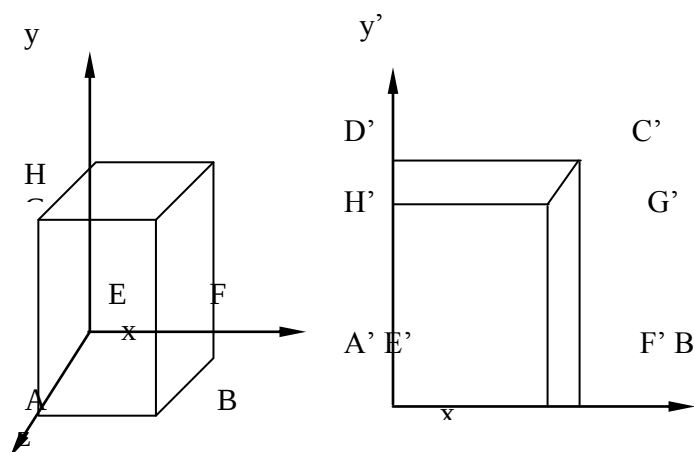
Giả sử khi mặt phẳng được đặt tại $z = 0$ và tâm phép chiếu nằm trên trục z , cách trục z một khoảng $z_c = -1/r$.

Nếu đối tượng cũng nằm trên mặt phẳng $z = 0$ thì đối tượng sẽ cho hình ảnh thật.

Phương trình biến đổi:

$$\begin{bmatrix} x & y & z & 1 \end{bmatrix} [Tr] = \begin{bmatrix} x & y & z & rz+1 \end{bmatrix}$$

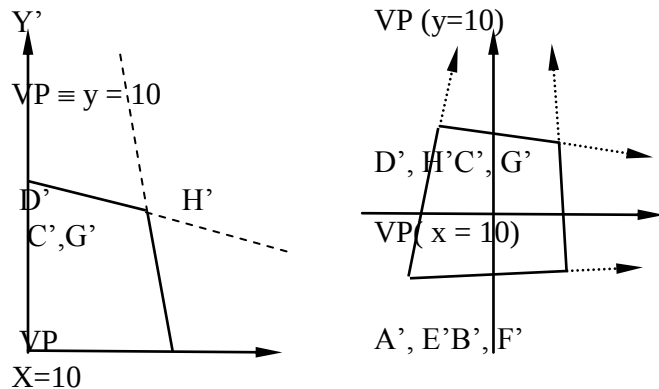
ma trận biến đổi một điểm phối cảnh $[Tr]$ có dạng:



5.15 Phép chiếu phối cảnh một tâm chiếu

$$\begin{aligned}
 \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & r \\ 0 & 0 & 0 & 1 \end{bmatrix} &= \begin{bmatrix} x & y & 0 & rz+1 \end{bmatrix} \\
 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & r \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & r \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} \frac{x}{rz+1} & \frac{y}{rz+1} & 0 & 1 \end{bmatrix}
 \end{aligned}$$

5.4.2. Phép chiếu phối cảnh hai tâm chiếu



5.16 Phép chiếu phối cảnh hai tâm chiếu

$$[T_c] = [T_{pq}][T_z]$$

$$= \begin{bmatrix} 1 & 0 & 0 & p \\ 0 & 1 & 0 & q \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & p \\ 0 & 1 & 0 & q \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Chương 5: Phép chiếu

Hai tâm chiếu: $[-1/p \ 0 \ 0 \ 1]$ và $[0 \ -1/q \ 0 \ 1]$

Điểm triệt tiêu (VP -Vanishing point) tương ứng trên 2 trục x và y là điểm: $[1/p \ 0 \ 0 \ 1]$ và $[0 \ 1/q \ 0 \ 1]$.

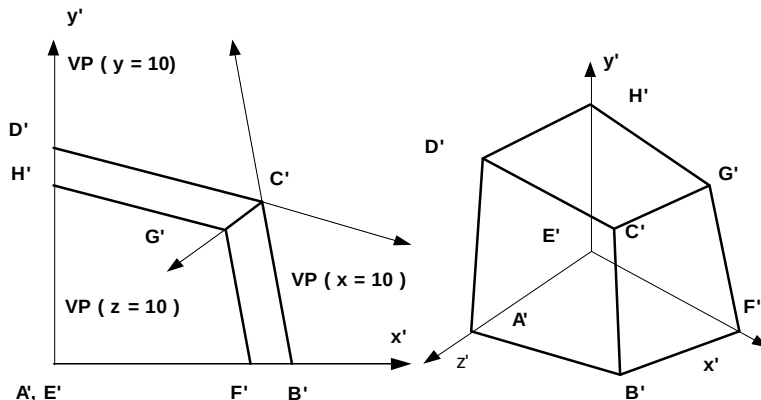
$$\begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & p \\ 0 & 1 & 0 & q \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} x & y & 0 & (px+qy+1) \end{bmatrix}$$

$$\begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & p \\ 0 & 1 & 0 & q \\ 0 & 0 & 1 & r \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} x & y & 0 & (px+qy+1) \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} x & y & 0 & (px+qy+1) \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & p \\ 0 & 1 & 0 & q \\ 0 & 0 & 1 & r \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T_c] = [T_{pqr}] [T_z] = \begin{bmatrix} 1 & 0 & 0 & p \\ 0 & 1 & 0 & q \\ 0 & 0 & 1 & r \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & p \\ 0 & 1 & 0 & q \\ 0 & 0 & 0 & r \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & p \\ 0 & 1 & 0 & q \\ 0 & 0 & 0 & r \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} x & y & 0 & (px+qy+rz+1) \end{bmatrix}$$

$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} \frac{x}{(px+qy+rz+1)} & \frac{y}{(px+qy+rz+1)} & 0 & 1 \end{bmatrix}$$



Hình 5.17 Phép chiếu phối cảnh ba tâm chiếu

5.4.3. Phép chiếu phối cảnh ba tâm chiếu

$$[T_{pqr}] = [T_p] [T_q] [T_r]$$

Ba tâm chiếu: trên trục x tại điểm $[-1/p \ 0 \ 0 \ 1]$,

y tại điểm $[0 \ -1/q \ 0 \ 1]$ và z tại điểm $[0 \ 0 \ -1/r \ 1]$

Điểm triệt tiêu -VP sẽ tương ứng với các giá trị:

$$[1/p \ 0 \ 0 \ 1], [0 \ 1/q \ 0 \ 1] [0 \ 0 \ 1/r \ 1]$$

Cài đặt bằng c/c++: Chiếu trên các mặt phẳng:

Chương 5: Phép chiếu

```
point hinhchieu(int x,int y,int z,int mfchieu) {
    point p;
    if (mfchieu==1){      //XOY
        p.x=int(getmaxx()/2+y-x*cos(RADS*45));
        p.y=int(getmaxy()/2+x*cos(RADS*45));
        p.z=z;
    }
    if (mfchieu==2) {      //XOZ
        p.x=int(getmaxx()/2+y-x*cos(RADS*45));
        p.y=int(getmaxy()/2-z+x*cos(RADS*45));
        p.z=z;
    }
    if (mfchieu==3){      //YOZ
        p.x=int(getmaxx()/2+y-
x*cos(RADS*45)+x*sin(RADS*45));
        p.y=int(getmaxy()/2-z);
        p.z=z;
    }
    return p; }
}
```

Tóm tắt chương:

Chúng ta xét hai phép chiếu song song và phối cảnh, trong phép chiếu song song thì phân ra các loại: trực giao, trục lượng và phép chiếu xiên. Phép chiếu trực giao chỉ đơn giản là bỏ đi một trong ba tọa độ của điểm chiếu bằng cách cho các tia chiếu song song với một trong các trục tọa độ. Phép chiếu trục lượng thì quay đối tượng khi thấy được rõ nhất đối tượng rồi mới cho các tia chiếu song với các trục tọa độ. Phép chiếu xiên thì tia chiếu không song song với trục tọa độ mà thay bằng nó làm với các mặt phẳng chiếu một góc không vuông.

Phép chiếu phối cảnh thì sử dụng một điểm cố định gọi là tâm chiếu và hình chiếu của các điểm được xác định bằng giao điểm của tia chiếu (nối điểm chiếu và tâm chiếu) với mặt phẳng quan sát. Phép chiếu phối cảnh hội tụ tại mắt nên đối tượng càng xa trông càng nhỏ và ngược lại.

Bài tập:

1. Cho Hình vuông ABCD có các tọa độ là: A(0,0,0), B(0,2,0), C(2,2,2) và D(2,0,2). Tính tọa độ mới của hình vuông sau khi chiếu nó bởi phép chiếu Isometric?
2. Cho Hình vuông ABCD có các tọa độ là: A(0,0,0), B(0,2,0), C(2,2,2) và D(2,0,2). Tính tọa độ mới của hình vuông sau khi chiếu nó bởi phép chiếu Dimetric với $fz=1/2$ (tỷ lệ co theo trục z)?
3. Cho tam giác ABC có các tọa độ là A(2,3,1), B(0,4,6) và C(5,2,7), Hãy tính tọa độ mới của hình tam giác đó sau khi chiếu phối cảnh sau:
 - Một tâm chiếu tại P(0,0,10)
 - Hai tâm chiếu tại M(5,0,0) và N(0,-8,0)

Chương 5: Phép chiếu

- Ba tâm chiếu tại $M(4,0,0)$, $N(0,-6,0)$ và $P(0,0,12)$
- 4. Viết chương trình chiếu hình kim cương (ABCD) với phép chiếu trực giao.
- 5. Viết chương trình chiếu hình lập phương với phép chiếu
 - Trimetric
 - Dimetric
 - Isometric
- 6. Viết chương trình chiếu hình kim cương (ABCD) với phép chiếu xiên
 - Cavalier
 - Cabinet
- 7. Viết chương trình chiếu hình lập phương với phép chiếu phối cảnh.

CHƯƠNG 6: MÀU SẮC TRONG ĐỒ HOẠ

6.1. ÁNH SÁNG VÀ MÀU SẮC (light and color)

6.1.1. Quan niệm về ánh sáng

- Ánh sáng đem đến sự sống cho con người
- Ánh sáng đem đến màu sắc cho con người

Màu sắc là cảm giác mà nó xảy ra khi có năng lượng của ánh sáng, xuất hiện trên võng mạc và nhận biết được nhờ não.

- Hạnh phúc của con người là cảm nhận được màu sắc
- Nguyên tắc của ánh sáng dựa trên hai góc độ
 - Vật lý - physics
 - Sinh lý - physiology

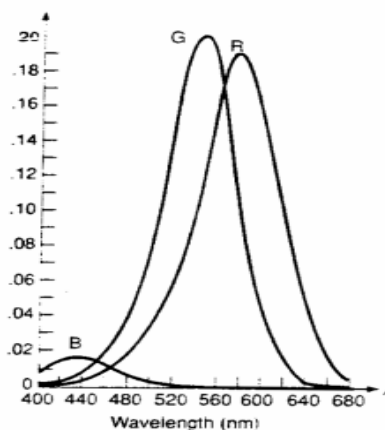
6.1.2. Yếu tố vật lý

Ánh sáng phụ thuộc vào mức năng lượng được truyền hay bước sóng của ánh sáng. Ánh sáng trắng hay dải sóng mà mắt người có thể cảm nhận được, sau khi phân tích qua lăng kính thành các phổ màu: tím, chàm, lam, lục, vàng, da cam, đỏ....Ánh sáng là sóng điện từ có bước sóng λ đi từ 400nm – 700nm.

Frequency $f(\text{Hz})$	Color	Wavelength $\lambda(\text{nm})$
7×10^{14}	Violet	400
6×10^{14}	Blue	500
5×10^{14}	Green	600
4×10^{14}	Yellow	700
	Orange	
	Red	

Hình 6.1 Tần số, màu sắc và bước sóng của ánh sáng nhìn thấy

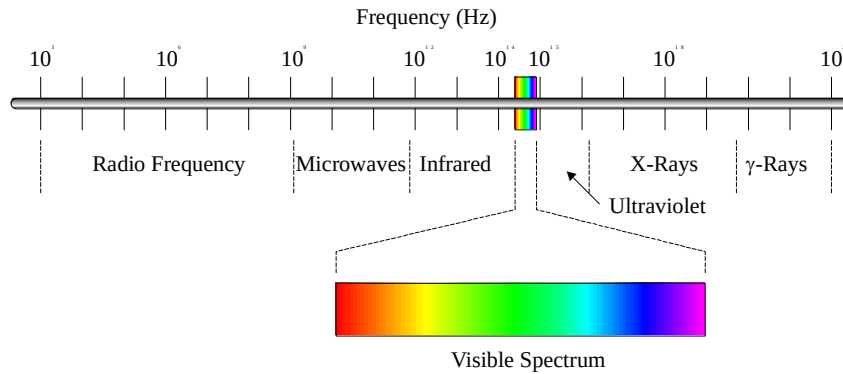
Tổng năng lượng đặc trưng cho từng loại bước sóng được biểu diễn bằng hàm phân bố năng lượng phổ $P(\lambda)$.



Hình 6.2 Đồ thị phân bố ba màu

Chương 6: Màu sắc trong đồ hoạ

Nguyên lý pha màu với các sắc màu cơ bản là đỏ, lục, lam (Red, Green, Blue). Theo nguyên lý ba màu này, một màu bất kỳ đều có thể được tạo ra từ ba màu cơ bản.



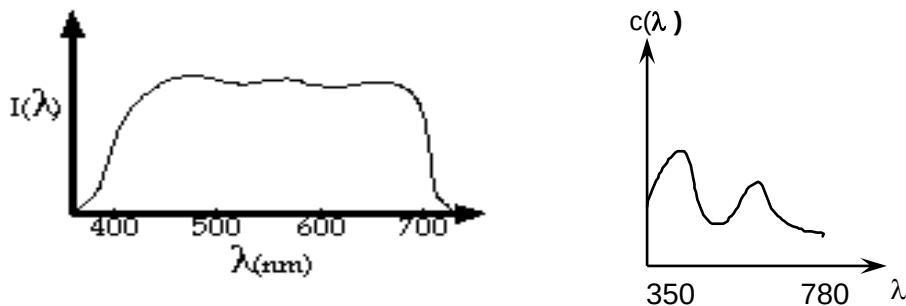
Hình 6.3 Vùng ánh sáng thấy được

Phổ của ánh sáng (Spectrum)

- Ánh sáng xuất phát từ nguồn sáng được xác định bởi phổ $I(\lambda)$ của nó - *spectrum*, phổ $I(\lambda)$ này được đo bởi năng lượng của ánh sáng với bước sóng cho trước đi qua một đơn vị diện tích trong một khoảng thời gian.
- Thuật ngữ khác phổ công suất - *power spectrum*, với đơn vị là watts/m^2 .
- Phổ công suất được dùng để đo cường độ phát sáng của nguồn - *emission intensity*
- Hay còn gọi cường độ truyền dẫn - *transmission intensity* của ánh sáng theo luồng trong không gian, hay cường độ phát sáng- *illumination intensity* của ánh sáng đập lên bề mặt.

Màu sắc

- Isaac Newton - ánh sáng trắng đi qua thấu kính thuỷ tinh sẽ phát tán ra thành phổ các màu cầu vồng
- Ngược lại, thấu kính có thể kết hợp các phổ ánh sáng để tạo thành ánh sáng trắng.
- Chùm sáng khi phân tách thành phổ màu có liên quan đến phổ năng lượng $I(\lambda)$.
- Phổ điện từ đó có bước sóng từ 350 tới 780 nm và màu được đặc trưng bởi $c(\lambda)$



Hình 6.4 Phổ điện từ của ánh sáng

6.1.3. Cảm nhận màu sắc của con người (Physiology - Sinh lý - Human Vision)

Hai mắt chỉ là những bộ phận thu hình giống như chiếc máy ảnh, còn não mới phân tích, tổng hợp, kết hợp những thông tin của hàng triệu tế bào cảm quang gửi về để tạo nên cảm nhận hình ảnh. Chính ở não mới tái hiện rõ rệt các hình thái, sắc màu mà hai mắt đã ghi nhận được. Phải chăng vì vậy, nhiều lúc con người chỉ “trông” mà không “nhìn” thấy. Nói cách khác, mắt “trông” và não “nhìn”.

Cấu tạo hệ quan sát của con người gồm 2 loại tế bào cảm thụ - sensors

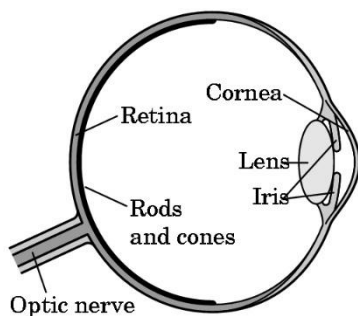
- Rods (tế bào que): nhạy cảm với cường độ ánh sáng thấp hay trong bóng tối
- Cones - tế bào hình nón
- 125 triệu tế bào que và 6 triệu tế bào nón

Nhạy cảm với ánh sáng màu sắc

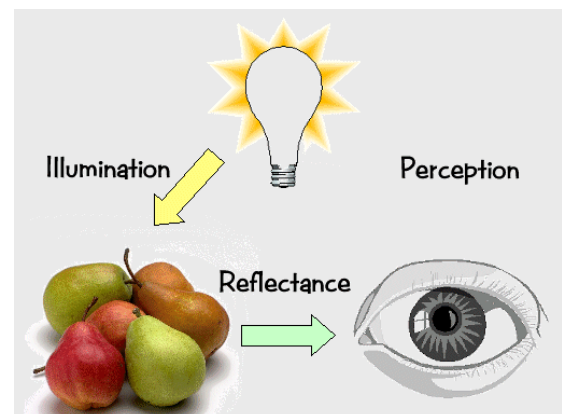
Chia làm 3 loại nón - cone

Ba loại sẽ có ba giá trị gọi là tristimulus values cảm nhận tương ứng trên 3 màu cơ bản và gửi đến não những tín hiệu tạo ra cảm nhận về màu sắc S-M-L.

Để đạt được một sự cảm nhận về một màu bất kỳ ta phải xác định giá trị của 3 đại lượng này



Hình 6.5 Cấu tạo mắt con người

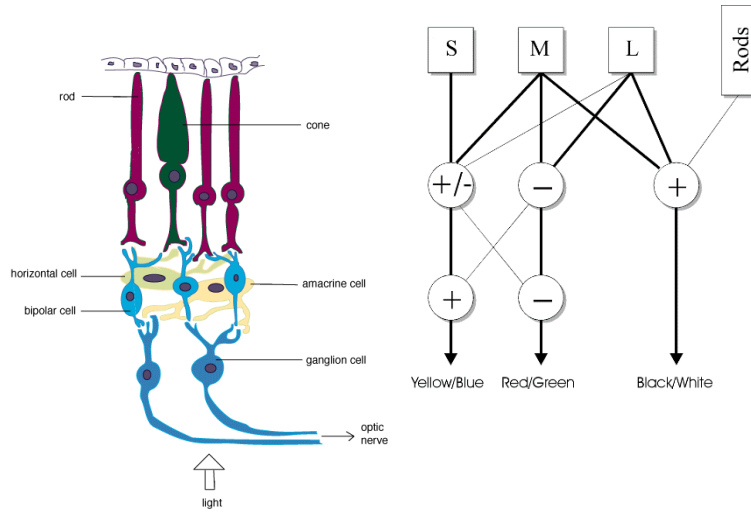


Hình 6.6 Con người cảm nhận màu sắc

Ba loại tế bào nón sẽ có độ nhạy cảm với 3 màu và các bước sóng khác nhau như:

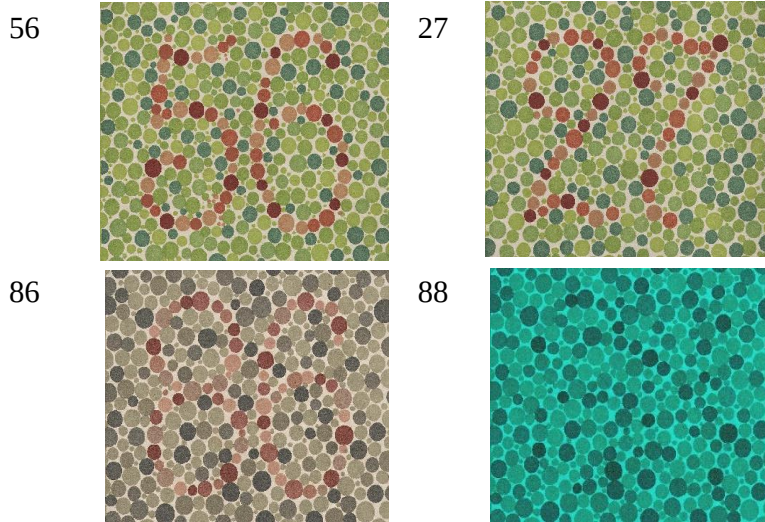
- L or R, hầu như nhạy cảm với ánh sáng đỏ (610 nm)
- M or G, nhạy cảm với ánh sáng lục (560 nm)
- S or B, nhạy cảm với ánh sáng lam (430 nm)
- Vậy ta có người bị mù màu chẳng qua là mất tế bào nón

S:M:L tỷ lệ = 1:20:40 $\Rightarrow$ từ đó ta thấy con người nhạy cảm với màu đỏ hơn là màu xanh lam.



Hình 6.7 Cấu tạo và nguyên lý hoạt động các tế bào mắt

- Nó không chỉ đơn giản là RGB cộng với ánh sáng
- Kết hợp tế bào que và nón mang lại cảm nhận cả màu sắc và ánh sáng
- Tế bào đáp ứng thay đổi với cường độ:
 - Khi ánh sáng yếu: thích ứng với nhìn tối, tế bào que trội hơn cảm nhận màu sắc không đáng kể
 - Khi ánh sáng là trung bình: thì cả hai là mức trung bình
 - Ánh sáng cao: xử lý màu sắc, tế bào nón trội hơn



Hình 6.8 Cảm nhận màu sắc của con người

Ta thấy màu đỏ tươi (bão hoà) khác màu đỏ tái (chưa bão hoà).

Yếu tố cảm nhận sinh lý:

- Hue - sắc màu: dùng để phân biệt sự khác nhau giữa các màu như xanh, đỏ, vàng ...
- Saturation - độ bão hoà: chỉ ra mức độ thuần của một màu hay khoảng cách của màu tới điểm có cường độ cân bằng.
- Lightness - độ sáng: hiện thân về mô tả cường độ sáng từ ánh sáng phản xạ nhận được từ đối tượng.
- Brightness - độ phát sáng: cường độ ánh sáng tự đối tượng phát ra chứ không phải do phản xạ từ các nguồn sáng khác.
- Hệ thống màu được sử dụng rộng rãi đầu tiên do A.H.Munsell đưa ra vào những năm 1976 không gian ba chiều bao gồm ba yếu tố Hue, Lightness và Saturation.

Sắc màu trong hội hoạ: thường được xác định mẫu trên góc độ sắc thái (tints), sắc độ (shade), tông màu (tone) từ các màu nguyên chất hay bão hoà. Sắc thái là được hình thành từ việc thêm sắc tố trắng vào các màu nguyên chất để giảm độ bão hoà. Sắc độ hay còn gọi là độ giảm màu được tạo ra bằng cách thêm màu đen vào các màu nguyên chất để giảm đi độ sáng của màu. Còn tông màu là kết quả của cả hai quá trình trên khi thêm cả màu trắng lẫn màu đen vào các màu nguyên chất.

6.1.4. Các đặc trưng cơ bản của ánh sáng

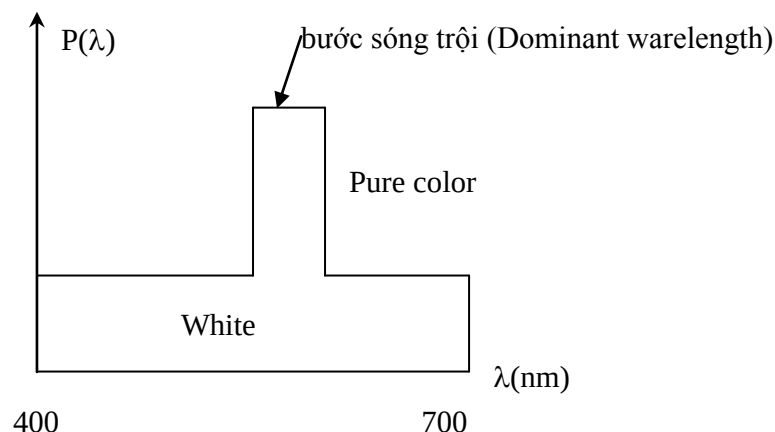
Ánh sáng có thể được mô tả bằng ba thuật ngữ:

- Độ sáng (Lightness): dựa vào tính chất vật lý của nó, hay còn gọi là tính phát sáng (brightness). Tính phát sáng đo lường năng lượng toàn phần trong ánh sáng. Nó tỷ lệ với diện tích giới hạn bởi $P(\lambda)$ và trục λ trong dãy 400 đến 700 nm. Diện tích này được tính như sau:

$$\int_{\lambda} P(\lambda) d\lambda$$

Tính phát sáng càng cao, thì độ sáng càng sáng hơn đối với người quan sát.

- Sắc độ (shade): để phân biệt ánh sáng trắng với ánh sáng đỏ với ánh sáng xanh. Đối với ánh sáng có sự phân bố quang phổ lý tưởng như hình dưới, sắc độ thích ứng với một tính chất vật lý khác được gọi là bước sóng trội của sự phân bố.



Hình 6.9 Phân bố quang phổ của ánh sáng

- Độ bão hoà (Saturation): mô tả mức độ chói lọi của ánh sáng. Ví dụ hai ánh sáng màu đỏ có thể khác nhau ở tính phát sáng/độ sáng và chúng có thể khác nhau ở mức độ chói lọi (ví dụ màu đỏ tươi/bão hoà khác với màu đỏ tái/ không bão hoà). Chúng ta có:

màu tươi

$$\text{Độ bão hoà} = \frac{\text{màu tươi} - \text{màu trắng}}{\text{màu tươi} - \text{màu trắng}}$$

Màu tươi + màu trắng

6.2. ÁNH SÁNG ĐƠN SẮC

Không cảm nhận được các sắc màu khác như vàng, đỏ, tím... khi quan sát trên màn hình đen trắng

Chương 6: Màu sắc trong đồ họa

Định lượng là thuộc tính duy nhất của các tia sáng đơn sắc và về mặt vật lý nó được tính bằng năng lượng của tia sáng được mô tả cường độ (intensity) hay độ chiếu sáng (luminance).

Dưới góc độ cảm nhận về mặt tâm lý thì cường độ của tia sáng chính là độ sáng của vật (brighness)

Sử dụng phổ kế - photometer để đo độ sáng thấp nhất (min) và cao nhất (max) của màn hình. Và đó là khoảng động.

Khoảng cường độ nhận giá trị min là I_0 , đến max là 1.0. Làm thế nào để thể hiện được 256 mức xám khác nhau?

6.2.1. Cường độ sáng và cách tính

Cường độ của nguồn sáng sẽ thay đổi trong khoảng từ 0 đến 1: 0 qui ước cho màu đen và 1 cho màu trắng.

khoảng tăng của cường độ sáng sẽ phân chia theo hàm logarit

$$I_0 = I_0, I_1 = r I_0, I_2 = r I_1 = r^2 I_0, \dots, I_{255} = r^{255} I_0 = 1$$

$$r = (1/I_0)^{1/255}, I_j = r^j I_0 = I_0^{(255-j)/255}$$

$$I = k.N^\gamma$$

Với k và γ là các hằng số (có γ từ 2.2 -> 2.6), N số lượng hạt tại một thời điểm phát ra từ cathode trong một chùm tia điện tử.

$I_0 = I_0$
$I_1 = r I_0$
$I_2 = r I_1 = r^2 I_0$
...
$I_{255} = r I_{254} = r^{255} I_0$

6.2.2. Phép hiệu chỉnh gama

Ta có $I = K.V^\gamma$ hay $V = (I / K)^{1/\gamma}$

Trong đó V điện áp tỉ lệ với N trên mỗi điểm ảnh.

Giả sử chúng ta có một cường độ sáng I thì bước đầu tiên ta phải làm là tìm ra giá trị I_j gần nhất qua phép làm tròn. Giá trị j tìm được $I = r^j I_0$ vậy $r^j = I/I_0$ suy ra $j = \text{ROUND}(\log_r(I / I_0))$.

Thay j vào công thức ta có:

$$I_j = r^j \cdot I_0$$

Bước tiếp theo của tiến trình là xây dựng mức điện áp V_j cho điểm ảnh mà cường độ ánh sáng có giá trị tương ứng là I_j .

$$V_j = \text{ROUND}(I_j / K)^{1/\gamma}$$

Ta thấy để cường độ sáng là như nhau cho các màn hình (hay ảnh là như nhau), thì chỉ còn thay đổi giá trị gama. Giá trị gama là số mũ của hàm lũy thừa, giá trị đó đối với loại phim nhựa 35mm trong phòng tối là 1.5. Nhưng hệ số gama của CRT là loại thiết bị độ sáng phụ thuộc vào ống phóng tia điện tử. Thực tế giá trị gama của CRT dao động từ 2.3 đến 2.6.

Ta có sự phản hồi tuyến tính của CRT có thể được bù bởi phần cứng và phép bù này gọi là phép hiệu chỉnh gama (Gama correction). Việc sử dụng I_j làm chỉ số trong Lookup table (LUT) để tìm ra cường độ sáng cho các điểm ảnh trên màn hình gọi là phép hiệu chỉnh gama với bảng LUT. Vậy bao nhiêu khoảng sẽ là đủ nhỏ cho việc thể hiện một điểm ảnh đen trắng là liên tục? Theo tính toán thì $r=1.01$ là mức ngưỡng phân biệt của mắt. Nếu $r<1.01$ thì mắt sẽ không phân biệt được sự khác lệnh giữa hai cường độ lân cận nhau I_j và I_{j+1} .

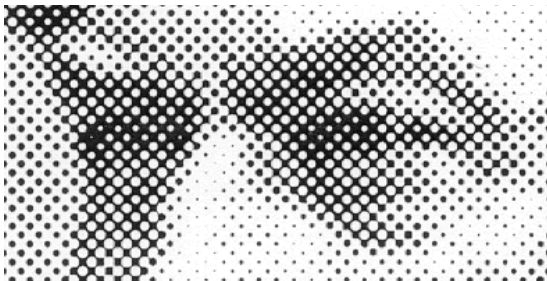
$r=(1/I_0).1/n=1.01$ vậy $n=\log_{1.01}(1/I_0)$ với $(1/I_0)$ là khoảng biến động của thiết bị phần cứng.

6.2.3. Xấp xỉ bán tông - halftone

Sử dụng đen trắng để mô tả ảnh nhiều màu ?

Phương pháp trên dựa vào cấu tạo mắt của người cũng như nguyên lý thu nhận ảnh của mắt khi nhìn những vùng nhỏ ở khoảng cách xa. Lúc đó mắt không thể phân biệt được các vật một cách cụ thể mà chỉ ghi nhận cường độ trung bình của vùng ảnh đó. Phương pháp này được gọi là xấp xỉ bán tông.

Phương pháp này cho phép đạt được độ phân giải trong in ảnh báo vào khoảng từ 60->80 dpi, còn trong tạp chí và sách cao hơn là khoảng từ 110 -> 120 dpi.

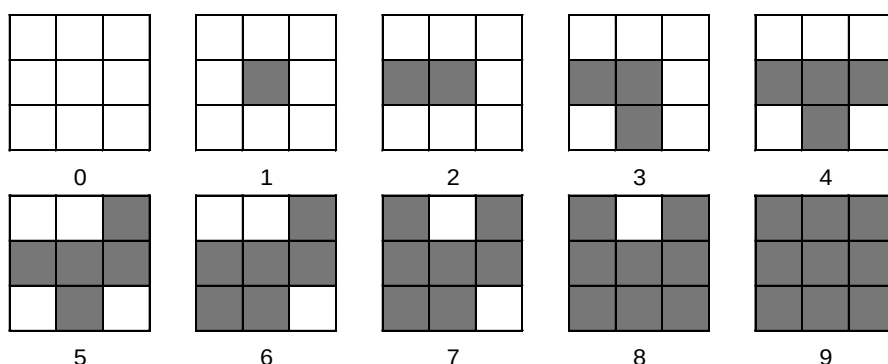


Hình 6.10 Dùng đen trắng để thể hiện ảnh màu

Ta có giải thuật phân ngưỡng (Thread Hold). Phân ngưỡng là lấy một giá trị trung bình của cả vùng ảnh làm ngưỡng và so sánh nó với mức sáng của từng điểm ảnh trong ô. Nếu giá trị của nó lớn hơn ngưỡng thì điểm đó được bật (on), nếu ngược lại thì tắt (off).

Ta thấy với phương pháp này mất đi nhiều thông tin của ảnh gây ra một số hiệu ứng phụ cho ảnh. Để giải quyết ta dùng phương pháp sau:

Mẫu tô: ta biểu diễn một điểm ảnh trên màn hình theo các mẫu tô. Đơn vị nhỏ nhất của ảnh lưới là 2×2 ta có 5 mức độ để thể hiện cường độ sáng của vùng đơn vị. Ma trận lưới kích thước $n \times n$ chúng ta có n^2+1 độ phân giải khác nhau. Hình dưới đây là ma trận 3×3 và các đơn vị mã là 0 đến 9.

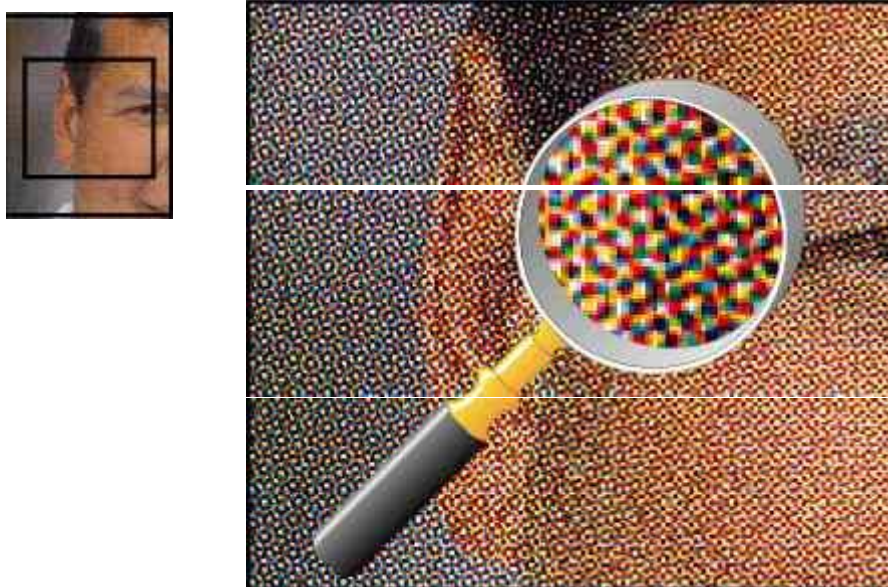


Hình 6.11 Phân bố các điểm trong vùng theo thứ tự tăng dần

Việc thể hiện cường độ vùng ảnh I bây giờ chỉ còn đơn thuần là bật tắt cả các vị trí $< I$

- Thứ nhất: Không dùng ma trận mẫu có dạng đường thẳng ngang
- Thứ hai: Các mẫu phải được hình thành theo chuỗi các bước liên tiếp nhau sao cho mọi điểm ảnh có mật độ thể hiện ngưỡng a đều phải có mặt để thể hiện mọi ngưỡng b với $b > a$.
- Thứ ba: Các mẫu phải được phát triển theo quy tắc từ tâm đi dần ra xung quanh. Nhờ đó sẽ gây được cho người sử dụng hiệu ứng tăng kích thước điểm.
- Thứ tư: Với một số các thiết bị in như máy in laser hay các thiết bị ghi hình, vấn đề về các điểm độc lập tuyệt đối là rất khó có khả năng đạt được. Khi mà đại đa phần các điểm ảnh được bật cho một cường độ sáng thì chúng sẽ gây ra các thay đổi cho các điểm còn lại.

Xấp xỉ bán tông với ảnh màu: Ta lấy mỗi cell không phải là một đơn vị nữa mà là ba đơn vị nhỏ đặc trưng cho ba màu (Red, Green, Blue).



Hình 6.12 Màu sắc trong ảnh màu

6.2.4. Ma trận Dither và phép lấy xấp xỉ bán tông

Bayer năm 1973 đã đưa ra dạng ma trận dither mà nhờ đó tăng được độ mịn của ảnh khi hiển thị.

Ma trận 2×2 ma trận dither có ký hiệu $D^{(2)}$:

Tính các ma trận $D^{(2n)}$ thông qua $D^{(n)}$:

$$D^{(2)} = \begin{bmatrix} 0 & 2 \\ 3 & 1 \end{bmatrix} \quad D^{(n)} = \begin{bmatrix} 4D^{(n/2)} + D_{00}^{(2)}U^{(n/2)} & 4D^{(n/2)} + D_{01}^{(2)}U^{(n/2)} \\ 4D^{(n/2)} + D_{10}^{(2)}U^{(n/2)} & 4D^{(n/2)} + D_{11}^{(2)}U^{(n/2)} \end{bmatrix}$$

$U^{(n)}$ là ma trận $n \times n$ với tất cả các phần tử = 1

Với $n = 4$ và kết quả từ $D^{(2)}$

$$D^{(4)} = \begin{bmatrix} 4 \begin{bmatrix} 0 & 2 \\ 3 & 1 \end{bmatrix} + 0 \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} & 4 \begin{bmatrix} 0 & 2 \\ 3 & 1 \end{bmatrix} + 2 \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \\ 4 \begin{bmatrix} 0 & 2 \\ 3 & 1 \end{bmatrix} + 3 \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} & 4 \begin{bmatrix} 0 & 2 \\ 3 & 1 \end{bmatrix} + 1 \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \end{bmatrix}$$

$$D^{(4)} = \begin{bmatrix} 0 & 8 & 2 & 10 \\ 12 & 4 & 14 & 6 \\ 3 & 11 & 1 & 9 \\ 15 & 7 & 13 & 5 \end{bmatrix}$$

Để xác định điểm (x,y) là bật hay tắt, ta cần xác định vị trí điểm tương ứng với vị trí ma trận Dither để so sánh cường độ sáng trung bình S với giá trị đó trong ma trận. Nếu $S > D_{ij}$ thì bật.

6.3. CÁC HỆ MÀU TRONG MÀN HÌNH ĐỒ HỌA

Mô hình màu - color model: là hệ thống có quy tắc cho việc tạo khoảng màu từ tập các màu cơ bản.

Có 2 loại mô hình màu là:

- Màu thêm (additive): mô hình màu thêm sử dụng ánh sáng - light để hiển thị màu. Màu sắc của mô hình này là kết quả của ánh sáng truyền dẫn - transmitted
- Màu bù (subtractive): mô hình màu bù sử dụng mực in - printing inks. Màu sắc cảm nhận được là từ ánh sáng phản xạ - reflected light (lấy màu trội).

Khoảng màu mà chúng ta tạo ra với tập các màu cơ bản gọi là gam màu hệ thống (system's color gamut)

Mỗi mô hình màu có khoảng màu hay gam màu riêng gamut (range) của những màu mà nó có thể hiển thị hay in.

Mỗi mô hình màu được giới hạn khoảng của phổ màu nhìn được. Gam màu hay khoảng còn được gọi là không gian màu "color space". Ảnh hay đồ họa vector có thể nói: sử dụng không gian màu RGB hay CMY hay bất cứ không gian màu nào khác.

Một số ứng dụng đồ họa cho phép người dùng sử dụng nhiều mô hình màu đồng thời để soạn thảo hay thể hiện đối tượng hình học. Điểm quan trọng là hiểu và để cho đúng mô hình cần thiết cho công việc.

Có ba hệ màu định hướng phần cứng:

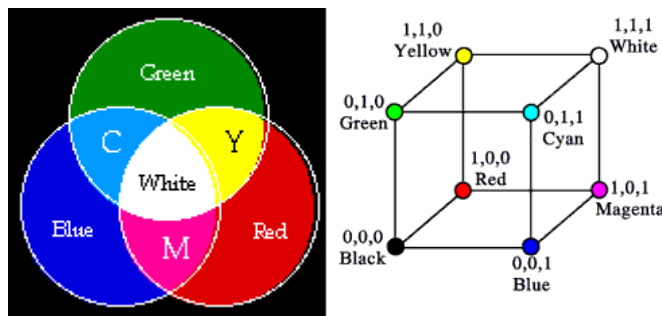
- RGB (Red, Green, Blue) dùng với CRT
- YIQ trong hệ thống tivi màu băng tần rộng
- CMY (Cyan, Magenta, Yellow) sử dụng một số thiết bị in màu

Không có một mô hình màu nào trong các mô hình thực tế trên có tính dễ sử dụng, vì chúng không có mối liên hệ trực tiếp với ý niệm màu trực giác của con người. Màu mà con người cảm nhận: Hue (sắc màu), Saturation (độ bão hoà), Lightness (độ sáng).

Các mô hình màu khác nhau được phát triển nhằm sử dụng cho một tiêu chí nhất định.

6.3.1. Mô hình màu RGB (Red, Green, Blue - đỏ, lục, lam)

Gam màu thể hiện trong màn hình CRT xác định bằng những đặc tính của hiện tượng phát quang các chất phát pho trong màn hình CRT. Mô hình không gian màu RGB được sắp xếp theo khối lập phương đơn vị. Đường chéo chính của khối lập phương với sự cân bằng về số lượng từng màu gốc tương ứng với các mức độ xám với đen là (0,0,0) và trắng (1,1,1).



Hình 6.13 Mô hình không gian màu RGB

$$C = rR + gG + bB$$

Trong đó C = màu hoặc ánh sáng kết quả. (r,g,b) = tọa độ màu trong miền $[0\ 1]$, (R,G,B) = các màu cơ bản đỏ, lục và lam.

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} X_r & X_g & X_b \\ Y_r & Y_g & Y_b \\ Z_r & Z_g & Z_b \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

Nếu hai màu tạo ra cùng một giá trị kích thích thì chúng ta không thể phân biệt được hai màu. Không gian màu RGB dựa theo chuẩn ITU-R BT.709, với $\gamma = 2.2$ và điểm trắng của mô hình là 6500 degrees K.

6.3.2. Mô hình màu CMY (Cyan, Magenta, Yellow - xanh tím, Đỏ tươi, vàng)

Đây là mô hình màu bù (*Subtractive color models*) hiển thị ánh sáng và màu sắc phản xạ từ mực in. Bỏ xung thêm mực đồng nghĩa với ánh sáng phản xạ càng ít.

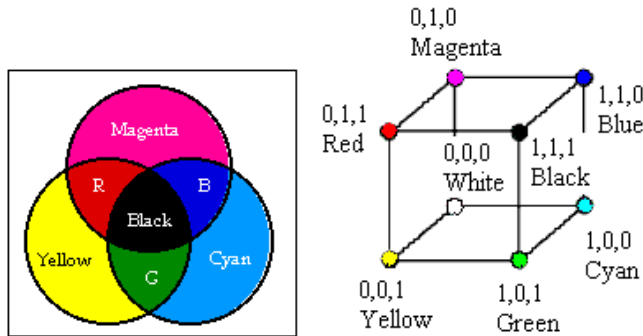
Khi bề mặt không phủ mực thì ánh sáng phản xạ là ánh sáng trắng - white.

Khi 3 màu có cùng giá trị cho ra màu xám. Khi các giá trị đạt max cho màu đen.

$$\text{Color} = cC + mM + yY$$

Ta có Red +Cyan = Black ; Green +Magenta = Black ; Blue + Yellow = Black

$$Black = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} C \\ M \\ Y \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$



Hình 6.14 Mô hình không gian màu CMY

Mô hình màu CMY- K

Mô hình mở rộng của CMY ứng dụng trong máy in màu. Giá trị đen bổ xung vào thay thế cho hàm lượng màu bằng nhau của 3 màu cơ bản.

Công thức chuyển đổi:

$$K = \min(C, M, Y) ;$$

$$C = C - K ;$$

$$M = M - K ;$$

$$Y = Y - K ;$$

C-Cyan, M-Magenta, Y-Yellow; K-black

6.3.3. Mô hình màu YIQ

Mô hình màu YIQ là mô hình màu được ứng dụng trong truyền hình màu bằng tần rộng tại Mỹ, và do đó nó có mối quan hệ chặt chẽ với màn hình đồ họa màu raster. YIQ là sự thay đổi của RGB cho khả năng truyền phát và tính tương thích với ti vi đen trắng thế hệ trước. Tín hiệu truyền sử dụng trong hệ thống NTSC (National Television System Committee).

Sự biến đổi RGB thành YIQ được xác định theo công thức sau:

$$\begin{bmatrix} Y \\ I \\ Q \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ 0.596 & -0.275 & -0.321 \\ 0.212 & -0.523 & 0.311 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

Y độ chói, I & Q đại lượng về màu sắc

Chú ý: Y giống như Y trong mô hình CIE's

Nó hoàn toàn tương thích với đen/trắng (B/W) của TV

Những đại lượng trong ma trận biến đổi được tìm bằng cách sử dụng các phosphor NTSC RGB chuẩn có các tọa độ sắc phổ là R(0.67 0.33), G (0.21 0.71) và B(0.14 0.08). Người ta cũng giả định rằng điểm trắng nằm ở $x_w=0.31$, $y_w=0.316$ và $Y_w=1.0$.

3.4. Mô hình màu HSV (Hue, Saturation, Value) - Mỹ thuật

Yếu tố cảm nhận màu sắc:

- Hue - sắc màu: dùng để phân biệt sự khác nhau giữa các màu như xanh, đỏ, vàng...
- Saturation - độ bão hoà: chỉ ra mức độ thuần của một màu hay khoảng cách của màu tới điểm có cường độ cân bằng(màu xám)
- Lightness - độ sáng: hiện thân về mô tả cường độ sáng từ ánh sáng phản xạ nhận được từ đối tượng.
- Brightness - độ phát sáng: cường độ ánh sáng mà tự đối tượng phát ra chứ không phải do phản xạ từ các nguồn sáng khác.

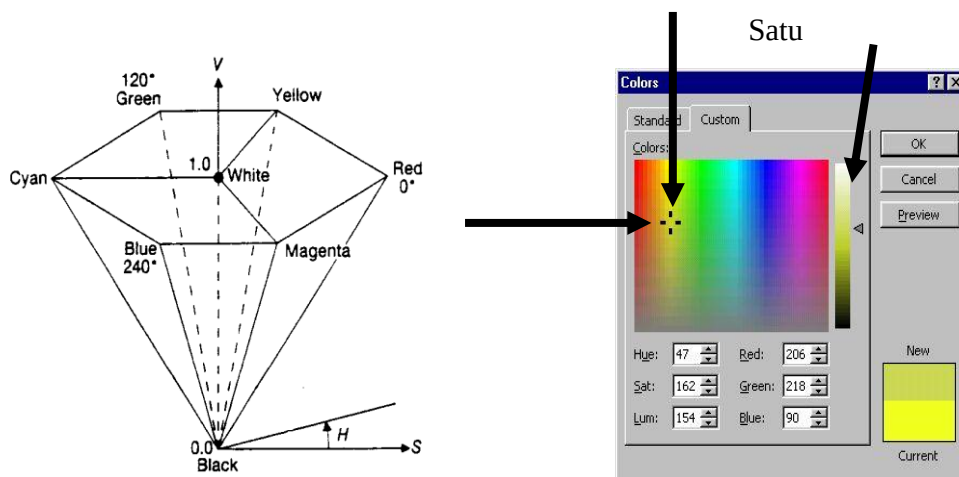
Mô hình màu RGB, CMY, YIQ được định hướng cho phần cứng

HSV (Hue, Saturation, Value)=HSB(Hue, Saturation, Brightness) định hướng người sử dụng dựa trên cơ sở về trực giác về tông màu, sắc độ và sắc thái mỹ thuật

Mô hình màu HSV được Alvey Ray Smith đưa ra 1978. Hue: màu sắc 0° - 360° đo bởi góc quay xung quanh trục đứng với màu đỏ là 0° , màu lục là 120° , màu lam là 240° . Các màu bổ sung cho hình chóp ở 180° đối diện với màu khác.

Value-Brightness:(độ sáng) 0-1 đường cao V với đỉnh là các điểm gốc toạ độ (0,0). Điểm ở đỉnh là màu đen và giá trị $V=0$, tại các điểm này giá trị của H và S không liên quan đến nhau. Khi điểm có $S=0$ và $V=1$ là điểm màu trắng, những giá trị trung gian của V đối với $S=0$ (trên đường thẳng qua tâm) là các màu xám. Khi $S=0$ giá trị của H phụ thuộc được gọi bởi các qui ước không xác định. Ngược lại khi S khác 0 giá trị H sẽ là phụ thuộc.

Saturation: Độ bão hoà 0-1, giá trị của S là tập các giá trị từ 0 trên đường trục tâm (trục V) đến 1 trên các mặt bên tại đỉnh của chóp 6 cạnh.

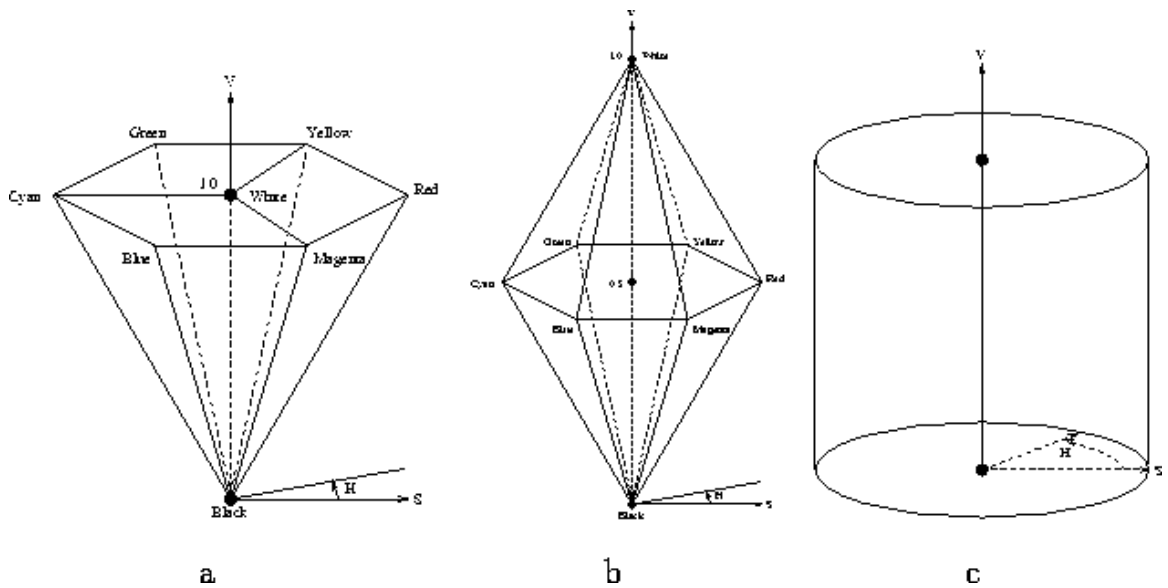


Hình 6.15 Mô hình màu HSV

Mô hình màu HLS (Hue, Lightness, Saturation Model) – không gian màu trực quan

Mô hình thường được sử dụng trong kỹ thuật đồ hoạ. Ưu điểm là rất trực giác ví dụ ta có thể chọn màu, thay đổi độ sáng và thay đổi độ bão hoà. Nhược điểm là khi chuyển

đối với không gian màu RGB sẽ có sai số (cube stood on end) thay đổi trên các loại màn hình khác nhau, rõ ràng không cảm nhận đều các màu.



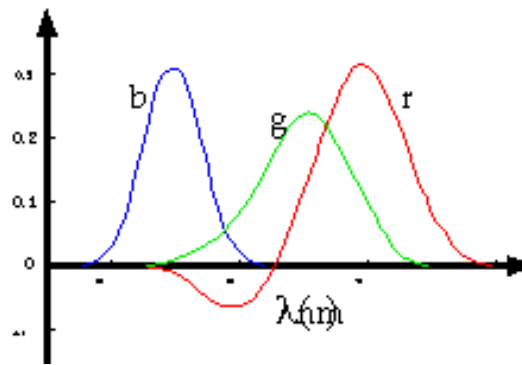
Hình 6.16 Mô hình màu hình chóp sáu cạnh đôi HLS

Chúng ta có thể coi mô hình HLS như một sự biến dạng của mô hình RGB mà trong đó mô hình này màu trắng được kéo hướng lên hình chóp sáu cạnh phía trên từ mặt $V=1$. Như với mô hình chóp sáu cạnh đơn, phần bổ sung của màu sắc được đặt ở vị trí 180° hơn là xung quanh hình chóp sáu cạnh đôi, sự bão hoà được đo xung quanh trục đứng, từ không trên trục tới 1 trên bề mặt. Độ sáng (Lightness)=0 cho màu đen (tại điểm mút thấp nhất của hình chóp sáu cạnh đôi) và bằng 1 cho màu trắng (tại đầu mút cao nhất).

6.3.5. Biểu đồ màu CIE (1931 – Commission Internationale de l'Eclairage)

Nhược điểm của RGB:

- Kết quả thực nghiệm cho thấy rất nhiều những ánh sáng mẫu không thể tạo thành từ 3 thành phần màu cơ sở với nguyên nhân do vỏ của võng mạc - retinal cortex.
- Với màu Cyan: cường độ của ánh sáng 2 màu green và blue kích thích cảm nhận màu đỏ trong mắt ngăn không cho thu được màu chính xác
- Cách duy nhất để thu được màu này là loại bớt phần màu đỏ bằng cách thêm ánh sáng đỏ vào mẫu ban đầu.
- Bằng cách thêm từ từ ánh sáng đỏ vào thu được (test + red) sẽ cho ra màu đúng bằng (blue + green)
- $C + rR = gG + bB \Leftrightarrow C = gG + bB - rR$
- Vấn đề đặt ra là việc phức tạp trong phân tích màu và chuyển đổi màu với đại lượng âm của ánh sáng đỏ độc lập thiết bị.



Hình 6.17 Hàm phân bố ba màu cơ sở (qua thực nghiệm - phụ thuộc vào mắt người)

CIE stands for *Comission Internationale de l'Eclairage* (International Commission on Illumination)

Commission thành lập 1913 tạo một diễn đàn quốc tế về trao đổi ý tưởng và thông tin cũng như tập chuẩn - set standards cho những vấn đề liên quan đến ánh sáng.

Mô hình màu CIE color phát triển trên cơ sở hoàn toàn độc lập thiết bị

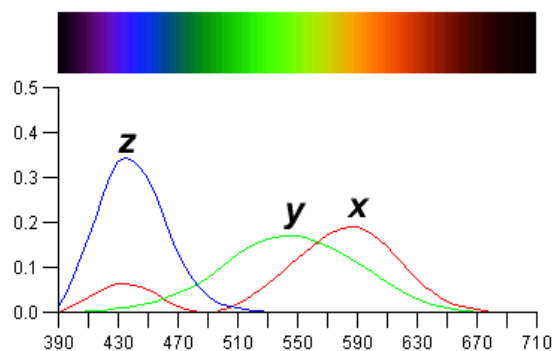
Dựa trên sự cảm nhận của của mắt người về màu sắc.

Yếu tố cơ bản của mô hình CIE định nghĩa trên chuẩn về nguồn sáng và chuẩn về người quan sát.

CIE XYZ - Color Space

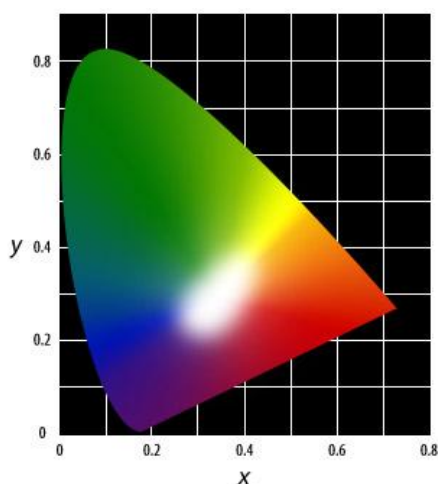
- CIE - Cambridge, England, 1931. Với ý tưởng 3 đại lượng ánh sáng - lights màu X, Y, Z cùng phổ tương ứng.
- Mỗi sóng ánh sáng λ có thể cảm nhận được bởi sự kết hợp của 3 đại lượng X,Y,Z
- Mô hình - là khối hình không gian 3D X,Y,Z gồm gam màu (gamut) của tất cả các màu có thể cảm nhận được.
- $\text{Color} = X'X + Y'Y + Z'Z$
- Các giá trị XYZ thay thế cho 3 đại lượng truyền thống RGB
- Màu được hiểu trên 2 thuật ngữ (Munsell's terms): màu sắc và sắc độ
- Ưu điểm của 3 loại màu nguyên lý cơ bản là có thể sinh ra các màu trên cơ sở tổng các đại lượng dương của màu mới thành phần.
- Việc chuyển đổi từ không gian màu 3D tọa độ (X,Y,Z) vào không gian 2D xác định bởi tọa độ (x,y), theo công thức dưới phân số của của tổng 3 thành phần cơ bản.
- $x = X/(X+Y+Z)$, $y = Y/(X+Y+Z)$, $z = Z/(X+Y+Z)$

Có: $x + y + z = 1$, ở đây tọa độ z không được sử dụng



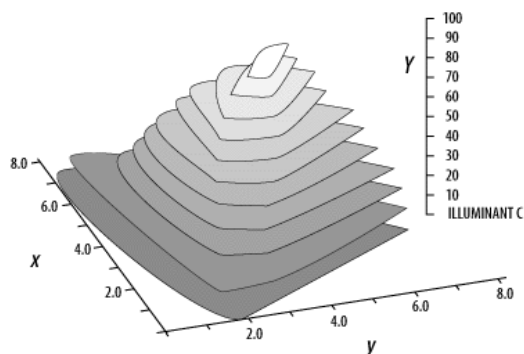
Hình 6.18 Hàm phân bố của các đại lượng CIE cơ sở

- Chuẩn CIE xác định 3 màu giả thuyết *hypothetical colors*, X, Y, and Z làm cơ sở cho phép trộn màu theo mô hình 3 thành phần kích thích.
- Không gian màu hình móng ngựa - horseshoe-shaped là kết hợp của không gian tọa độ 2D màu x, y và độ sáng.
- $\lambda_x = 700 \text{ nm}$; $\lambda_y = 543.1 \text{ nm}$; $\lambda_z = 435.8 \text{ nm}$
- Thành phần độ sáng hay độ chói được chỉ định chính bằng giá trị đại lượng Y trong tam kích tố của màu sắc



Hình 6.19 Không gian màu hình móng ngựa

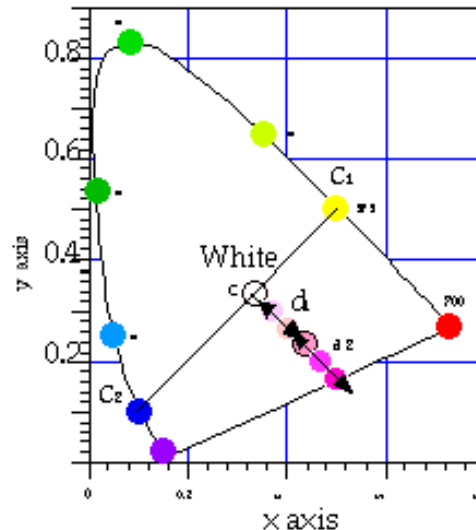
Mô hình CIE xyY



Hình 6.20 Mô hình màu CIE xyY

- Thang đo của Y xuất phát từ điểm trắng trên đường thẳng vuông góc với mặt phẳng x,y với giá trị từ 0 tới 100.

- Khoảng màu lớn nhất khi $Y=0$ tại điểm trắng và bằng CIE Illuminant C. Đây là đáy của hình.
- Khi Y tăng màu trở nên sáng hơn và khoảng màu hay gam màu giảm diện tích trên tọa độ x,y cũng giảm theo.
- Tại điểm trên không gian với $Y=100$ màu có sắc xám bạc và khoảng màu ở đây là bé nhất.



Hình 6.21 Không gian màu hình móng ngựa

6.4. CHUYỂN ĐỔI GIỮA CÁC HỆ MÀU

Việc xây dựng và thể hiện màu sắc của các đối tượng trên màn hình đồ hoạ chỉ thực hiện qua mô hình ba màu mà phần cứng hỗ trợ RGB. Vậy khi phần mềm ứng dụng có sử dụng đến các mô hình màu khác, ta phải chuyển đổi giữa chúng.

6.4.1. Chuyển đổi HSV - RGB

Hệ HSV có H (sắc màu) chạy từ 0^0 đến 360^0 với màu đỏ tại 0^0 .

S (độ bão hoà) và V (giá trị cường độ ánh sáng) thuộc khoảng $[0\ 1]$

If ($S=0$)//H không tham gia - đen trắng

R = V;

G = V;

B = V;

Else // Khi đó $S > 0$ trường hợp màu

// Màu của điểm ảnh được xác định thông qua 3 biến phụ M,N và K

if ($H=360$)

H=0;

Else

H = H/60;

I = (int)H; // lấy giá trị nguyên

F = H - I;

M = $V*(1 - S)$;

N = $V*(1 - S*F)$;

```

K = V*(1- S*(1- F)) ;
if I == 0 then (R, G, B) = (V, K, M) ;
if I == 1 then (R, G, B) = (N, V, M) ;
if I == 2 then (R, G, B) = (M, V, K) ;
if I == 3 then (R, G, B) = (M, N, V) ;
if I == 4 then (R, G, B) = (K, M, V) ;
if I == 5 then (R, G, B) = (V, M, N) ;

```

6.4.2. Chuyển đổi RGB sang XYZ

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

Các nhà sản xuất cung cấp các tọa độ XYZ cho phốt pho tương ứng với màu RGB mà màn hình hiển thị: (X_r, Y_r, Z_r) , (X_g, Y_g, Z_g) và (X_b, Y_b, Z_b)

Hay viết lại:

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} X_r & X_g & X_b \\ Y_r & Y_g & Y_b \\ Z_r & Z_g & Z_b \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

Có $R=1$

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \text{ suy ra } \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} X_r \\ Y_r \\ Z_r \end{bmatrix}$$

Tương tự $G=1$

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \text{ suy ra } \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} X_g \\ Y_g \\ Z_g \end{bmatrix}$$

Tương tự $B=1$

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \text{ suy ra } \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} X_b \\ Y_b \\ Z_b \end{bmatrix}$$

Thường $[X \ Y \ Z]$ cho mỗi phốt pho (phosphor) thì không được cung cấp, nên chúng ta tính với trường hợp điểm trắng (whitepoint) khi $R=G=B=1$.

Bây giờ chúng ta cần biết độ chói của điểm trắng được gọi bởi Y_w . Ta đặt:

$$E_r = X_r + Y_r + Z_r \text{ suy ra } x_r = X_r / E_r \text{ vậy } X_r = x_r \cdot E_r ; Y_r = y_r \cdot E_r ; Z_r = (1 - x_r - y_r) \cdot E_r$$

Tương tự:

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} x_r E_r & x_g E_g & x_b E_b \\ y_r E_r & y_g E_g & y_b E_b \\ (1 - x_r - y_r) E_r & (1 - x_g - y_g) E_g & (1 - x_b - y_b) E_b \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

Ta có điểm trắng:

$$\begin{bmatrix} X_w \\ Y_w \\ Z_w \end{bmatrix} = [M] \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

Mà ta có theo NTSC: RGB chuẩn (x_w, y_w, Y_w) như sau: $x_w = 0.31, y_w = 0.316$ và $Y_w = 1.0$
Có:

$$y_w = \frac{Y_w}{X_w + Y_w + Z_w} (X_w + Y_w + Z_w) = \frac{Y_w}{y_w}$$

$$x_w = \frac{X_w}{X_w + Y_w + Z_w} \quad X_w = x_w (X_w + Y_w + Z_w) \quad X_w = x_w \frac{Y_w}{y_w}$$

$$\text{Và: } Z_w = (1 - x_w - y_w) \frac{Y_w}{y_w}$$

Ta có $R + G + B = W$ nên:

$$\begin{bmatrix} X_r \\ Y_r \\ Z_r \end{bmatrix} + \begin{bmatrix} X_g \\ Y_g \\ Z_g \end{bmatrix} + \begin{bmatrix} X_b \\ Y_b \\ Z_b \end{bmatrix} = \begin{bmatrix} X_w \\ Y_w \\ Z_w \end{bmatrix} \quad \text{suy ra } X_w = X_r + X_g + X_b = x_r E_r + x_r E_g + x_b E_b$$

Từ đó:

$$\begin{bmatrix} X_w \\ Y_w \\ Z_w \end{bmatrix} = \begin{bmatrix} x_r & x_g & x_b \\ y_r & y_g & y_b \\ (1-x_r-y_r) & (1-x_g-y_g) & (1-x_b-y_b) \end{bmatrix} \begin{bmatrix} E_r \\ E_g \\ E_b \end{bmatrix}$$

Ta hoàn toàn tính được: E_r, E_g và E_b

Tóm tắt:

Chương này chúng ta nghiên cứu cấu tạo của ánh sáng (về mặt vật lý), rồi xét bộ phận cảm nhận ánh sáng của con người là mắt. Đưa ra các hệ màu định hướng cho phần cứng như: RGB dùng cho máy tính, CMYK dùng cho máy in và YIQ dùng cho truyền hình. Tất cả các hệ màu này con người đều không cảm nhận được, con người chỉ cảm nhận được hệ màu HSV hay HLS. Từ tất cả các ưu nhược điểm của các hệ màu trên, từ sự cảm nhận màu sắc của con người phụ thuộc vào cấu tạo của các tế bào mắt nên năm 1913 tổ chức quốc tế về ánh sáng đã đưa ra hệ màu chuẩn thuần nhất CIE. Hệ màu này có thể bao hàm tất cả các hệ màu trên, nó giải quyết được các nhược điểm của hệ màu RGB. Cuối cùng chúng ta đưa ra các công thức để chuyển đổi giữa các hệ màu với nhau.

Bài tập:

1. Giả sử rằng môi trường trọng tâm là không khí (hoặc chân không) hãy mô tả dải quang phổ hiển thị bằng một dải tần số.
2. Tại sao mọi thứ trông có vẻ màu xám hoặc đen trong một phòng tối nơi chúng ta hầu như không thể nhìn thấy được?

3. Hãy thiết lập một công thức đơn giản để tính diện tích bị giới hạn bởi hàm phân bố $P(\lambda)$. (Xem hình 6.9 sách Kỹ thuật đồ hoạ)
4. Từ hàm phân bố $P(\lambda)$ hãy lập công thức tính độ bão hoà từ hình 6.9 - sách kỹ thuật đồ hoạ.
5. Sự khác nhau giữa Y trong CMY và Y trong YIQ là gì?
6. Giả sử rằng một màn hình hiển thị tạo nên những gì được gọi là màu trắng chuẩn với $x_w = 0.313$ $y_w = 0.329$ và $Y_w = 1.0$ ($R=G=B=1$). Và các tọa độ sắc phổ của photon giống như tọa độ được tìm thấy ở mô hình màu.
 $R(0.62 \ 0.34)G(0.290.59)B(0.150.06)$
Hãy tìm ma trận biến đổi màu M cho màn hình hiển thị.
7. Hãy kiểm nghiệm rằng Y trong mô hình CIE XYZ tương tự Y trong mô hình màu NTSC YIQ. Ta có NTSC chuẩn thì: $x_w = 0.31$, $y_w = 0.316$ và $Y_w = 1.0$ và sử dụng các tọa độ sắc phổ của các photon NTSC chuẩn: $R(0.67 \ 0.33)$, $G(0.21 \ 0.71)$ và $B(0.14 \ 0.08)$.

CHƯƠNG 7: ĐƯỜNG CONG VÀ MẶT CONG TRONG 3D

7.1. ĐƯỜNG CONG - CURVE

Trong các ứng dụng của đồ họa máy tính, hầu như các thực thể là đường cong mềm và mặt cong, chúng dùng để mô tả thế giới thực: nhà cửa, xe cộ, núi non....hay xây dựng nên các thực thể đang được thiết kế. Nhưng ta thấy sử dụng các phương trình đường cong không thể hiện được hình ảnh thực hay ý tưởng của người thiết kế, còn nếu ta dùng tập hợp các điểm thì thường cần nhiều dung lượng nhớ để lưu trữ cũng như tốc độ tính toán.

Ta có quỹ đạo chuyển động của một điểm trong không gian thì tạo thành đường cong. Trong chương này sẽ đưa ra phương pháp tổng thể về những mô hình toán học để biểu diễn và xây dựng các loại đường và mặt cong trong không gian 3D trên máy tính.

7.1.1. Điểm biểu diễn đường cong (curve represents points)

Ta thấy qua hai điểm vẽ được một đường thẳng. Qua ba điểm vẽ được một đường cong trong mặt phẳng. Qua bốn điểm vẽ được một đường cong trong không gian. Dùng các phương trình đường cong như Hypebol, parabol... thì tính toán phức tạp và không thể hiện được hình ảnh thực hay ý tưởng của người thiết kế.

Chọn đường cong như thế nào để phù hợp với máy tính? Biểu diễn và điều khiển đường cong thông qua điểm điều khiển. Đường cong là các đối tượng cơ bản thường là kết quả của tiến trình thiết kế và các điểm đóng vai trò là công cụ để kiểm soát và mô hình hoá đường cong. Cách tiếp cận này là cơ sở của lĩnh vực thiết kế mô hình hình học nhờ máy tính (*Computer Aided Geometric Design - CAGD*).

Các cách để biểu diễn đường cong:

- Tường minh (Explicit functions):

$$y = f(x), z = g(x)$$

- Không tường minh (Implicit equations):

$$f(x,y,z) = 0$$

- Biểu diễn các đường cong tham biến (Parametric representation)

$$x = x(t), y = y(t), z = z(t) \text{ trong đó } t \in [0 \ 1]$$

Hạn chế:

- Hệ đồ hoạ ứng dụng chỉ mô tả bó hẹp trong đoạn nào đấy
- Đường cong bậc cao với mỗi giá trị của x ta luôn có 2 tập giá trị của y (thực tế chỉ cần 1)
- Chúng ta cần biểu diễn đường cong mềm (chỉ biểu diễn đường “cong gãy”)

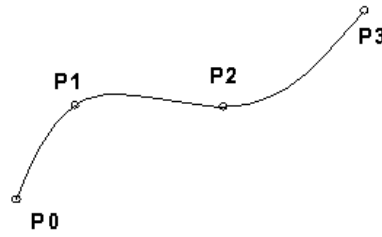
7.1.2. Đường cong đa thức bậc ba tham biến

Phải đảm bảo là đường cong không gian với 3 trục tọa độ x, y, z. Tránh được những tính toán phức tạp và những phần nhấp nhô ngoài ý muốn xuất hiện ở những đường đa thức bậc cao.

Công thức mô tả:

Tường minh : $y = f^3(x), z = g^3(x)$

Không tường minh: $f^3(x,y,z) = 0$



Hình 7.1 Đường cong đa thức bậc ba

Biểu diễn các đường cong tham biến (Parametric representation):

$$x = f^3(u), y = f^3(u), z = f^3(u) \text{ trong đó } u \in [0 \ 1]$$

Theo Lagrange:

$$x = a_1 + b_1u + c_1u^2 + d_1u^3$$

$$y = a_2 + b_2u + c_2u^2 + d_2u^3$$

$$z = a_3 + b_3u + c_3u^2 + d_3u^3$$

Ở đây ba phương trình với 12 ẩn số

Với 4 điểm p_0, p_1, p_2, p_3 phương trình xác định (vì 4 điểm thì xác định 1 đường cong trong không gian).

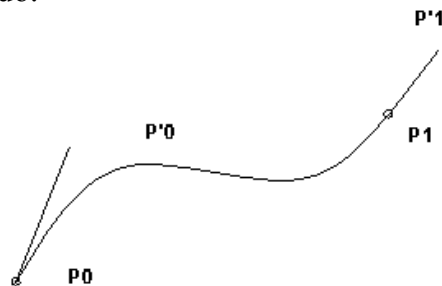
$$\text{Mỗi 1 điểm cho ta cặp 3 giá trị: } P_0 = \begin{bmatrix} x_0 \\ y_0 \\ z_0 \end{bmatrix} \quad P_1 = \begin{bmatrix} x_1 \\ y_1 \\ z_1 \end{bmatrix} \quad P_2 = \begin{bmatrix} x_2 \\ y_2 \\ z_2 \end{bmatrix} \quad P_3 = \begin{bmatrix} x_3 \\ y_3 \\ z_3 \end{bmatrix}$$

Cả thảy có 12 phương trình, thay vào 3 phương trình trên ta tính được 12 ẩn $a_1, \dots, d_3$

Ghi chú: rõ ràng có sự thay đổi một chút về đường cong thì ta lại phải giải lại hệ phương trình để tính các tham số cho đường cong, dẫn đến tính toán chậm.

7.1.3. Đường cong Hermite

Phương pháp Hermite dựa trên cơ sở của cách biểu diễn Ferguson hay Coons năm 60. Với phương pháp của Hermite đường bậc ba sẽ xác định bởi hai điểm đầu và cuối cùng với hai góc nghiêng tại hai điểm đó.



Hình 7.2 Đường cong Hermite

Theo công thức toán học hàm bậc ba được biểu diễn dưới dạng:

$$p = p(u) = k_0 + k_1u + k_2u^2 + k_3u^3$$

$$p(u) = \sum k_i u_i \in n \text{ (với } k_i \text{ là các tham số chưa biết)}$$

Độ dốc của đường cong được đo bằng $p'(u)$

$$p' = p'(u) = k_1 + 2k_2u + 3k_3u^2$$

p_0 và p_1 ta có hai độ dốc p_0' và p_1' với $u = 0$ và $u = 1$ tại hai điểm đầu cuối của đoạn $[0,1]$.

$$p_0(u=0)=k_0$$

$$p'_0(u=0)=k_1$$

$$p_1(u=1)=k_0+k_1+k_2+k_3$$

$$p'_1(u=1)=k_1+2k_2+3k_3$$

hay

$$k_0=p_0 \text{ và } k_1=p'_0$$

$$k_2=3(p_1 - p_0) - 2p'_0 - p'_1 \text{ và } k_3 = 2(p_0-p_1) + p'_0 + p'_1$$

Khi đã có k_0, k_1, k_2, k_3 thay vào:

$$p = p(u) = k_0 + k_1u + k_2u^2 + k_3u^3$$

$$p_0(1-3u^2+2u^3) + p_1(3u^2-2u^3) + p'_0(u-2u^2+u^3) + p'_1(-u^2+u^3)$$

$$p = p(u) = \begin{bmatrix} 1 & u & u^2 & u^3 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -3 & 3 & -2 & -1 \\ 2 & 2 & 1 & 1 \end{bmatrix} \begin{bmatrix} p_0 \\ p_1 \\ p'_0 \\ p'_1 \end{bmatrix}$$

Thay đổi của các điểm hay các góc nghiêng (thay đổi 2 vector) dẫn đến sự thay đổi hình dạng của đường.

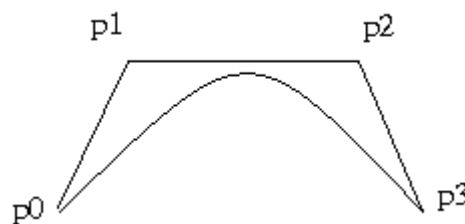
7.1.4. Đường cong Bezier

Việc sử dụng điểm với các vector kiểm soát được độ dốc của đường cong tại những điểm mà nó đi qua. Tuy nhiên không được thuận lợi cho việc thiết kế tương tác, không tiếp cận với các độ dốc của đường cong bằng các giá trị số (Hermite).

Paul Bezier, nhân viên hãng RENAULT vào năm 1970 đi đầu trong việc ứng dụng máy tính cho việc xây dựng các bề mặt. Hệ thống UNISURF của ông được áp dụng trong thực tế vào năm 1972 được thiết kế và kiểm xe Mezesez hay Renault.

Bezier đã sử dụng đa giác kiểm soát cho đường cong tại những đỉnh của đa giác và tiếp tuyến tại đó (p_0, p_1, p_2, p_3).

Ta có p_0, p_3 tương đương với p_0, p_1 trên đường Hermite, điểm trung gian p_1, p_2 được xác định bằng 1/3 theo độ dài của vector tiếp tuyến tại điểm p_0 và p_3



Hình 7.3 Đa giác kiểm soát Bezier

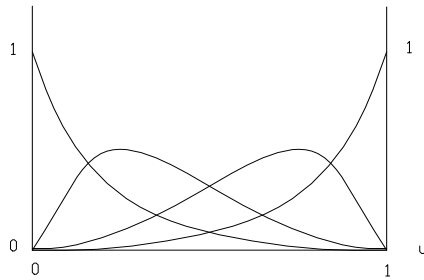
$$p_0' = 3(p_1 - p_0)$$

$$p_1' = 3(p_3 - p_2)$$

$$p = p(u) = p_0(1-3u^2+2u^3) + p_1(3u^2-2u^3) + p_0'(u-2u^2+u^3) + p_1'(-u^2+u^3)$$

$$p = p(u) = p_0(1 - 3u + 3u^2 - u^3) + p_1(3u-6u^2+3u^3) + p_2(3u^2 - 3u^3) + p_3u^3$$

$$p = p(u) = \begin{bmatrix} 1 & u & u^2 & u^3 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ -3 & 3 & 0 & 0 \\ 3 & -6 & 3 & 0 \\ -1 & 3 & -3 & 1 \end{bmatrix} \begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ p_3 \end{bmatrix}$$



Hình 7.4 Hàm hợp của đường cong Bezier

Ưu điểm:

Đễ dàng kiểm soát hình dạng của đường cong hơn vector tiếp tuyến tại p_0' và p_1' của Hermite.

Nằm trong đa giác kiểm soát với số điểm trung gian tùy ý (số bậc tùy ý),
có số bậc = số điểm kiểm soát - 1.

Đi qua điểm đầu và điểm cuối của đa giác kiểm soát, tiếp xúc với cặp hai vector của đầu cuối đó.

Biểu thức Bezier-Bernstein

Đường Bezier cũng có thể được biết đến như biểu thức Bezier Bernstein bởi kỹ thuật mà Bezier sử dụng là áp dụng công thức hoá các vector trong phép tính đa giác xấp xỉ được Bernstein phát triển gần đây. Phép toán đại số được xác định như sau:

$$0 \leq u \leq 1$$

$$0! = 1, u_i = 1 \text{ khi } i = 0$$

$$P(u) = \sum_{i=0}^n B_{i,n}(u) P_i$$

$$B_{i,n}(u) = C(n, i) u^i (1-u)^{n-i}$$

$$C(n, i) = \frac{n!}{i!(n-i)!}$$

Trong đó $P_0, \dots, P_n$: vector vị trí của đa giác $(n+1)$ đỉnh.

7.1.5. Đường cong B-spline

7.1.5.1. Đường cong bậc ba Spline

Trong công thức của Bezier, chúng ta sử dụng hàm hợp liên tục để xác định điểm kiểm soát tương đối. Với các điểm nội suy thì mức độ tương đối sẽ khác nhau mà trong đó một chuỗi các phần tử nhỏ sẽ kết hợp với nhau tạo ra đường cong đa hợp. Theo tính toán thì đường bậc ba sẽ đa thức bậc thấp nhất có thể để biểu diễn một đường cong trong không gian và chuỗi điểm Hermite sẽ phù hợp nhất đối với việc xây dựng nên đường cong đa hợp này.

Việc yêu cầu người sử dụng đưa vào các vector tiếp tuyến tại mỗi điểm trong tập hợp các điểm là cực kỳ bất tiện cho nên thường trong các đường bậc ba đa hợp ta sử dụng các điều kiện biên liên tục trong phép đạo hàm bậc một và hai tại điểm nối giữa. và đường cong được xác định như trên gọi là đường spline bậc ba với phép đạo hàm liên tục bậc hai. Giá trị đạo hàm của đường cong sẽ xác định độ cong tại mỗi điểm nút và nó cũng đưa ra điều kiện biên cho mỗi đoạn trên đường cong.

Vậy đường bậc ba spline có ưu điểm là không phải xác định độ dốc của đường tại các nút nhưng nhược điểm của nó là chỉ tạo ra sự thay đổi toàn cục khi ta thay đổi vị trí của điểm.

Đường cong – Spline đi qua n điểm cho trước mà mỗi đoạn là các đường cong bậc ba độc lập có độ dốc và độ cong liên tục tại mỗi điểm kiểm soát hay điểm nút. Với n điểm ta có $(n-1)$ đoạn với mỗi đoạn gồm bốn vector hệ số hay $4(n-1)$ cho $n-1$ đoạn, và $2(n-1)$ điều kiện biên và $(n-2)$ điều kiện về độ dốc cùng $(n-2)$ về độ cong.

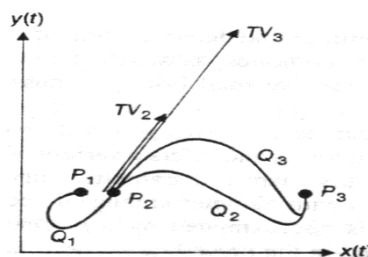
Để xây dựng nên đường spline có tham số với n điểm nút ta có một dãy các giá trị tham số mà ta gọi là vector nút.

$$u_0, \dots, u_{n-1} \text{ trong đó } u_{i+1} > u_i$$

Cần lựa chọn tại mỗi nút, cách lựa chọn đơn giản nhất là theo cách đơn điệu có nghĩa là với giá trị 0 tại điểm đầu và tăng lên 1 tại những điểm kế tiếp. tuy vậy phương pháp này dẫn đến độ cong không mong muốn tại các điểm vì vậy việc tham số hoá sẽ đưa vào chiều dài, nhưng phương pháp này cũng không được chính xác khi mà đường cong chưa xác định chiều dài. Tuy nhiên thông thường người ta sử dụng việc tích lũy của các dây cung với:

$$u_0 = 0 \text{ và } u_{i+1} = u_i + d_{i+1} \text{ trong đó } d_i \text{ là khoảng cách giữa 2 điểm } p_{i-1} \text{ và } p_i$$

Trong các trường hợp đường cong có bậc lớn hơn ba có thể dùng cho đường spline. Thông thường đường spline bậc n sẽ được xây dựng trên các phần nhỏ liên tục của các biến độc lập.



Hình 7.5 Kết nối hai đường cong

Hình trên cho thấy hai đoạn cong có chung điểm nối mà đường cong liên tục tại điểm đó, việc biểu diễn tính liên tục của đường cong thông qua chữ cái C-Continue. C_0 để đảm bảo không có sự gián đoạn giữa hai đoạn cong. C_1 tính liên tục bậc nhất hay đạo hàm bậc nhất tại điểm nối. C_2 đạo hàm bậc hai liên tục của đường cong tại điểm nối.

Giả sử khi biểu diễn đường cong mềm thông qua các đoạn cong q_1, q_2, q_3 (mỗi đoạn có 4 vector hệ số) cần thỏa mãn:

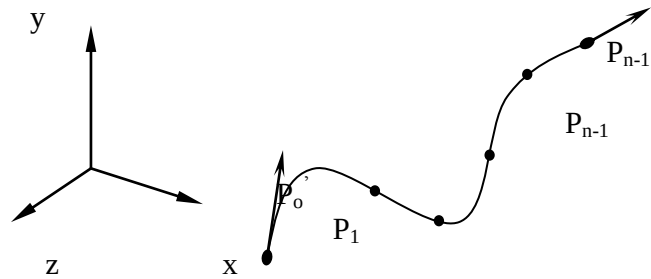
Liên tục tại điểm nối hay $C^1_0 = C^2_0$

Độ dốc (hay vector tiếp tuyến) tại điểm nối (điểm cuối của q_1 và đầu q_2) là như nhau: $C^1_1 = C^2_1$ (đạo hàm bậc nhất)

Thỏa mãn liên tục trên tại điểm nối (đạo hàm bậc 2 liên tục tại điểm nối) $C^1_2 = C^2_2$

Việc kết hợp các đoạn cong Hermite bậc ba để mô tả một đường cong mềm theo kiểu phân đoạn spline là phương pháp đơn giản nhất hay còn gọi là phương pháp Hermite nội suy. Với phương pháp này thì tham biến u_i cho mỗi đoạn cong i của tập các đoạn cong Hermite sẽ biến đổi trong khoảng từ 0 đến 1 và luôn tồn tại đạo hàm bậc nhất của các đoạn cong tại các điểm nối. Phương trình cho mỗi đoạn cong được sử dụng lúc này là phương trình đường cong bậc ba Hermite:

$$p = p(u) = \begin{bmatrix} 1 & u & u^2 & u^3 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -3 & 3 & -2 & -1 \\ 2 & 2 & 1 & 1 \end{bmatrix} \begin{bmatrix} p_0 \\ p_1 \\ p'_0 \\ p'_1 \end{bmatrix}$$



Hình 7.6 Phân đoạn của đường cong Spline - Hermite

Theo Hermite các đoạn là các đường cong, tính liên tục của đạo hàm bậc hai tại các điểm nối có thể dễ dàng đạt được bằng cách đặt $P''_{i-1}(u_{i-1}=1)$ là đạo hàm bậc hai tại điểm cuối của đoạn $(i-1)$ bằng với $P''_i(u_i=0)$ đạo hàm bậc hai tại điểm đầu của đoạn thứ i .

$$P''_{i-1}(1) = P''_i(0)$$

Có phương trình:

$$P_i(u) = k_{0i} + k_{1i}u + k_{2i}u^2 + k_{3i}u^3$$

Đạo hàm bậc hai sẽ là:

$$P''_i(u) = 2k_{2i} + 6k_{3i}u$$

$$P''_{i-1}(1) = P''_i(0) \text{ nên } 2k_{2(i-1)} + 6k_{3(i-1)} = 2k_{2i}$$

Vì điểm cuối của đoạn $i-1$ trùng với điểm đầu của đoạn thứ i ($P_i(0) = P_{i-1}(1)$)

Theo Hermite: $k_2 = 3(p_1 - p_0) - 2p_0' - p_1'$ và

$$k_3 = 2(p_0 - p_1) + p_0' + p_1'$$

$$2(3(P_i - P_{i-1}) - 2P'_{i-1} - P'_i) + 6(2(P_{i-1} - P_i) + P'_{i-1} + P'_i) = 2(2(P_{i-1} - P_i) + P'_{i-1} + P'_i)$$

Hay: $P'_{i-1} + 4P'_i + P'_{i+1} = 3(P_{i+1} - P_i)$ (*)

Với phương trình (*) này thì phương trình dạng tổng quát của đường cong Spline là tập của các đoạn cong Hermite sẽ xác định với điều kiện ban đầu cho là tập các điểm kiểm soát của đường cong và hai vector tiếp tuyến tại hai điểm đầu cuối của đường cong đó. Sử dụng (*) ta có thể tính được các giá trị của các vector tiếp tuyến tại từng điểm kiểm soát của đường cong.

$$\begin{bmatrix} 1 & 0 & . & . & . & . \\ 1 & 4 & 1 & 0 & . & . \\ 0 & 1 & 4 & 1 & 0 & . \\ . & . & . & . & . & . \\ . & . & 0 & 1 & 4 & 1 \\ . & . & . & . & 0 & 1 \end{bmatrix} \begin{bmatrix} P'_0 \\ P'_1 \\ . \\ . \\ P'_{n-2} \\ P'_{n-1} \end{bmatrix} = \begin{bmatrix} P'_0 \\ 3(P_2 - P_0) \\ . \\ . \\ 3(P_{n-1} - P_{n-3}) \\ P'_{n-1} \end{bmatrix}$$

Tương đương với:

$$\begin{bmatrix} P'_0 \\ P'_1 \\ . \\ . \\ P'_{n-2} \\ P'_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 0 & . & . & . & . \\ 1 & 4 & 1 & 0 & . & . \\ 0 & 1 & 4 & 1 & 0 & . \\ . & . & . & . & . & . \\ . & . & 0 & 1 & 4 & 1 \\ . & . & . & . & 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} P'_0 \\ 3(P_2 - P_0) \\ . \\ . \\ 3(P_{n-1} - P_{n-3}) \\ P'_{n-1} \end{bmatrix}$$

7.1.5.2. Đường B-spline

Với Bezier hay spline đều không cho ta thay đổi đường cong một cách cục bộ, việc thay đổi vị trí các điểm kiểm soát hay các vector tiếp tuyến không chỉ ảnh hưởng trực tiếp đến độ dốc của đường cong lân cận quanh điểm kiểm soát mà còn kéo theo ảnh hưởng đến các phần còn lại của đường cong. Đường Bezier thêm vào đó là khi tính xấp xỉ ở bậc cao sẽ rất phức tạp còn khi liên kết nhiều đoạn Bzier hay Hermite bậc thấp (bậc ba) có thể đem lại ích lợi khi tính toán nhưng yếu tố ràng buộc về tính liên tục của đạo hàm bậc cao tại các điểm nối không cho điều khiển cục bộ như mong muốn.

Việc kết hợp luôn phiên các đoạn cong tổng hợp, thông qua các đa thức tham số xác định riêng rẽ trên một số điểm kiểm soát lân cận với số bậc tùy ý không phụ thuộc vào số lượng các điểm kiểm soát, cho phép tạo nên đường cong trơn mềm B-spline. Đường cong này đã khắc phục được các nhược điểm mà các dạng đường cong trước chưa đạt được. Có nghĩa là khi dịch chuyển điểm kiểm soát của đường cong thì chỉ một vài phân đoạn lân cận của điểm kiểm soát đó bị ảnh hưởng chứ không phải toàn bộ đường cong.

Với $n+1$ số điểm kiểm soát P_i ta có:

$$P(u) = \sum_{i=0}^n N_{i,k}(u) \cdot P_i$$

Trong đó $N_{i,k}(u)$ là hàm hợp B-Spline bậc $k-1$ và sự khác biệt giữa B-spline và Bezier sẽ được thể hiện trên đó. Trong đường Bezier bậc của đa thức được xác định bởi số đoạn cong trên đường cong đó, còn với B-spline bậc được thỏa mãn độc lập với số điểm kiểm soát của đường.

Hơn nữa hàm hợp của Bezier khác 0 trên toàn bộ khoảng của tham số u còn B-spline chỉ khác 0 trên đoạn ngắn của các tham số. Mỗi đoạn trên hàm hợp chỉ tương ứng với một điểm thì chỉ dẫn tới sự thay đổi cục bộ trong khoảng mà trên đó tham số của hàm hợp khác 0.

Biểu diễn toán học của B-spline, với hàm B-spline có bậc $k-1$ xác định thì:

$$N_{i,k}(u) = \frac{(u - U_{i+1-k})}{(U_i - U_{i+1-k})} N_{i-1,k-1}(u) + \frac{(U_{i+1} - u)}{(U_{i+1} - U_{i+2-k})} N_{i,k-1}(u)$$

$$N_{i,1}(u) = \begin{cases} 1 & u \in [u_i, u_{i+1}] \\ 0 & \text{others} \end{cases}$$

Trong đó u_i là giá trị tại nút p_i với biến số là u được gọi là các vector nút.

Tất cả các giá trị nút đồng thời xác định trên vector nút và các nút nguyên thường sử dụng dễ dàng. Trong trường hợp này các hàm hợp bậc k sẽ khác 0 trong khoảng k của vector nút và toàn bộ các giá trị trên vector cho một tập hợp điểm bằng $n+1+k$.

Không như Bezier, đường B-spline không đi qua hai điểm đầu và cuối trừ khi hàm hợp được dùng là tuyến tính.

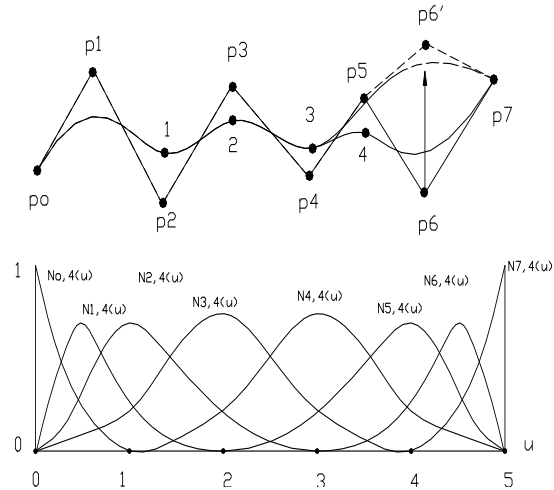
Đường B-spline có thể được tạo qua hai điểm đầu, cuối và tiếp xúc với vector đầu và cuối của đa giác kiểm soát. Bằng cách thêm vào các nút tại vị trí của các nút cuối của vector tuy nhiên các giá trị giống nhau không nhiều hơn bậc của đường cong.

Giống như đường cong Bezier, tính chất bao lồi của đa giác kiểm soát và tính chất chuẩn được thỏa mãn. Vậy có:

$$\sum_{i=0}^n N_{i,k}(u) = 1$$

Trong đường cong B-spline, số lượng các nút, bậc của đường cong và số điểm điều khiển luôn có các quan hệ ràng buộc:

$$0 \leq u \leq n - k + 2$$



Hình 7.7 Đường cong B-spline

Vậy việc xác định các vector nút sẽ phụ thuộc vào sự phân loại của chính bản thân chúng và điều đó sẽ ảnh hưởng đến hình dạng của đường cong được mô tả. Phân loại sẽ dựa trên loại của đường cong như sau:

Đều tuần hoàn (periodic)

Không tuần hoàn (open or unperodic)

Không đều (non-uniform)

a. B Spline - Đều và tuần hoàn

Vector nút là đều khi giá trị của chúng cách đều nhau một khoảng ∇ xác định.

Ví dụ: [0 1 2 3 4 5] với ∇ xác định = 1

[-2-1/2 1 5/2 4] với ∇ xác định = 3/2

[-1-0.6 -0.2 0.2 0.6 1] với ∇ xác định = 0.4

Trong các bài toán thực tế, thông thường thì khoảng xác định của tham biến nằm trong khoảng từ 0 đến 1 hay từ 0^0 đến 360^0 thì việc chọn giá trị của các vector nút được chuẩn hoá trong khoảng [0 1] hay $[0^0 360^0]$ đó.

[0 0.2 0.4 0.6 0.8 1] với ∇ xác định = 0.2

[$0^0 120^0 240^0 360^0$] với ∇ xác định = 120^0

Bậc (k-1)	Cấp (k)	Vector nút (m=n+k)	Khoảng tham số $(k-1) \leq t \leq (n+1)$
1	2	[0 1 2 3 4 5 6 7]	$1 \leq t \leq 6$
2	3	[0 1 2 3 4 5 6 7 8]	$2 \leq t \leq 6$
3	4	[0 1 2 3 4 5 6 7 8 9]	$3 \leq t \leq 6$

Các vector nút gọi là đều và tuần hoàn khi các hàm B-spline đối với mỗi phân đoạn có thể chuyển đổi lẫn nhau. Bảng trên chỉ ra sự thay đổi của miền tham số và vector nút khác nhau của các đường cong B-spline khi bậc của đường cong thay đổi. Số lượng của

vector nút được qui định bởi biểu thức $m-n+k$ và số lượng các điểm kiểm soát tính qua biểu thức $(n+1)$ bằng 6.

Tính chất:

Ảnh hưởng của mỗi hàm cơ sở được giới hạn trong k đoạn là cấp của đường cong cần thể hiện. Vậy chúng ta sử dụng đường cong bậc ba thì ảnh hưởng của hàm cơ sở trải dài trên bốn đoạn của đường cong.

Đường B-spline tuần hoàn hoàn toàn không đi qua các điểm đầu và cuối của đa giác kiểm soát ngoại trừ với đường bậc 1 ($k=2$) mà khi đó đường cong chuyển dạng thành đường thẳng.

Ví dụ về các đường B-spline tuần hoàn có các bậc khác nhau có cùng các điểm và đa giác kiểm soát. Khi $k=2$ đường cong bậc một trùng với các cạnh của đa giác kiểm soát.

Khi $k=3$, đường cong B-spline bậc 2, bắt đầu tại trung điểm của cạnh thứ nhất và kết thúc tại trung điểm của cạnh cuối cùng của đa giác kiểm soát.

b. Không tuần hoàn (Open – Non Uniform)

Một vector không tuần hoàn hoặc mở là vector nút có giá trị nút tại các điểm đầu cuối lặp lại với số lượng các giá trị lặp lại này bằng chính cấp k của đường cong và các giá trị nút trong mỗi điểm lặp này là bằng nhau

Nếu một trong hai điều kiện này hoặc cả hai điều kiện không được thoả mãn thì vecto nút là không đều.

Ví dụ, xét một đa giác kiểm soát với bốn đỉnh. Các đường cong B-spline cấp 2,3,4 được xây dựng dựa trên đa giác kiểm soát có số lượng các nút $m=n+k$ sẽ có vector nút như sau:

Cấp (k)	số lượng nút ($m = n + k$)	Vector nút không tuần hoàn
2	6	[0 0 1 2 3 3]
3	7	[0 0 0 1 2 2 2]
4	8	[0 0 0 0 1 1 1 1]

Các biểu thức phải được thoả mãn đối với nút u_i trên vector nút không tuần hoàn bắt đầu tại u_0 .

Danh sách các vector nút không tuần hoàn đã đưa ra ở mục này đều thoả mãn các biểu thức sau:

$$u_i = 0 \quad 1 \leq i \leq k$$

$$u_i = i - k \quad k+1 \leq i \leq n+1$$

$$u_i = n - k + 2 \quad n+1 \leq i \leq n+k+1$$

Các vector nút không tuần hoàn cung cấp các hàm cơ sở được định nghĩa trong một miền tham số phức tạp và không có sự mất mát như với loại vector tuần hoàn và vì vậy đường cong B-spline loại này luôn đi qua các điểm đầu và cuối của đa giác kiểm soát.

Ví dụ: hàm hợp bậc ba tính xấp xỉ cho 8 khoảng sẽ xác định trên vector nút là 00001234555. Ở đây chúng ta còn thấy sự thay đổi cục bộ trên đường cong khi ta thay đổi vị trí mỗi điểm.

Đường cong Bezier là trường hợp đặc biệt của B-spline không tuần hoàn, trong đó số lượng các đỉnh sử dụng bằng với cấp của đường cong. Vector nút trong trường hợp này là:

[0 0 0 1 1 1 1]

kk

Đường cong B-spline bậc ba với bốn điểm kiểm soát và vector không tuần hoàn [0 0 0 0 1 1 1 1] cũng chính là đường cong Bezier.

c. Không đều

Trong vector nút không tuần hoàn, giá trị các nút xuất hiện tại các biên được lặp lại và các nút bên trong các bước nút bằng nhau. Nếu một trong hai điều kiện này hoặc cả hai điều kiện này không được thỏa mãn thì vector nút là không đều.

Ví dụ các nút không đều có thể tạo ra bằng cách đặt các giá trị lặp lại đối với các nút ở khoảng giữa [0 1 2 3 3 4 5]

Hay tạo ra bước nhảy không bằng nhau giữa các nút [0.0 0.2 0.5 0.75 1.0]

Các vector nút loại đều cho phép người sử dụng dễ hình dung và xử lý trong các phép toán nhưng trong một số các trường hợp bước nút không đều lại có những ưu điểm đặc biệt. Ví dụ như trong việc điều khiển hình dạng của đường cong trong tiến trình thiết kế khi các sai lệch không mong muốn có thể xuất hiện mà việc sử dụng đường cong B-spline đều với các dữ liệu điểm có các khoảng cách tương đối lớn mà không đều nhau.

Kết luận

- B-spline là một dòng của Bezier
- Thực tế khi ta chọn bậc k cho tập hợp k điểm thì B-spline chuyển thành Bezier
- Khi bậc của đa thức giảm sự ảnh hưởng cục bộ của mỗi điểm nút càng rõ ràng hơn.
- Khi tồn tại ảnh hưởng cục bộ càng lớn và đường cong phải đi qua điểm đó.
- Chúng ta có thể thay đổi hình dạng đường cong B-spline bằng cách:
 - Thay đổi kiểu vector nút: đều tuần hoàn, mở, không đều
 - Thay đổi cấp k của đường cong
 - Thay đổi số đỉnh và vị trí các đỉnh đa giác kiểm soát
 - Sử dụng các điểm kiểm soát trùng nhau

7.2. MÔ HÌNH BỀ MẶT (Surface) VÀ CÁC PHƯƠNG PHÁP XÂY DỰNG

7.2.1. Các khái niệm cơ bản

Mặt cong (surface): là quỹ đạo chuyển động của một đường cong tạo nên. Biểu diễn tham biến cho mặt cong:

- ✓ Dựa vào việc xây dựng và tạo bề mặt toán học trên những điểm dữ liệu

- ✓ Dựa trên việc xây dựng nên bề mặt phụ thuộc vào biến số có khả năng thay đổi một cách trực diện thông qua các tương tác đồ họa.

Ta có:

$$x=x(u,v,w) \quad u,v,w \in [0, 1]$$

$$y=y(u,v,w) \quad u + v + w = 1$$

$$z=z(u,v,w)$$

$$Q(u,v,w) = Q[x=x(u,v,w) \ y=y(u,v,w) \ z=z(u,v,w)]$$

Biểu diễn theo mảnh

- ✓ Biểu diễn miếng tứ giác - quadrilatera Patches
- ✓ Biểu diễn miếng tam giác - Triangular Patches

7.2.2. Biểu diễn mảnh tứ giác

Phương trình:

$$x=x(u,v)$$

$$y=y(u,v) \quad u,v \in [0, 1]$$

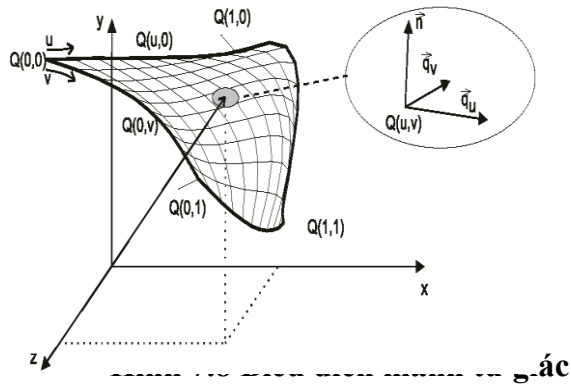
$$z=z(u,v)$$

$$Q(u,v) = Q[x=x(u,v) \ y=y(u,v) \ z=z(u,v)]$$

Thành phần

u,v là các tham biến

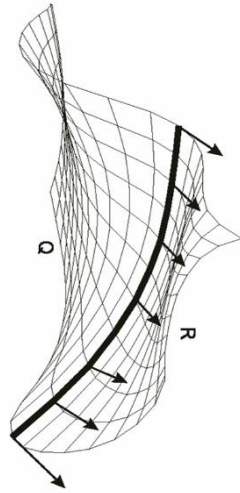
Các điểm $Q(0,0)$, $Q(0,1)$, $Q(1,0)$, $Q(1,1)$ là cận của mảnh, các đường cong $Q(1,v)$, $Q(0,v)$, $Q(u,0)$, $Q(u,1)$ là các biên của mảnh. Đạo hàm riêng tại điểm $Q(u,v)$ xác định vector tiếp tuyến theo hướng u và v



$$\partial Q(u,v) / \partial u = Q[\partial x(u,v) / \partial u, \partial y(u,v) / \partial u, \partial z(u,v) / \partial u]$$

$$\partial Q(u,v) / \partial v = Q[\partial x(u,v) / \partial v, \partial y(u,v) / \partial v, \partial z(u,v) / \partial v]$$

7.2.2.1. Kết nối mảnh tứ giác



Hình 7.9 Kết nối mảnh tứ giác

Thực thể hình học biểu diễn thông qua các mảnh cùng dạng, các mảnh có thể nối với nhau theo các hướng u, v khi hai mảnh cùng hướng đó. Nếu mọi điểm trên biên của hai mảnh bằng nhau, hay hai biên bằng nhau. Hai mảnh liên tục bậc C_0 . Nếu hai biên bằng nhau và đạo hàm bằng nhau trên cùng một hướng thì hai mảnh gọi là kết nối bậc C_1 .

7.2.2.2. Hệ tọa độ Barycentric Coordinates

Tập các điểm $P_1, P_2 \dots P_n$, tập các tổ hợp của các điểm đó

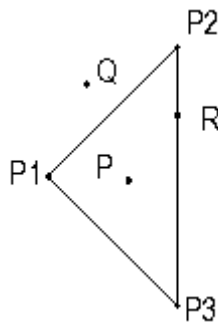
$$k_1 P_1 + k_2 P_2 + k_3 P_3 \dots + k_n P_n$$

Với $k_1 + k_2 + k_3 + \dots + k_n = 1$

Các điểm tạo thành không gian affine với các giá trị tọa độ nates

$k_1, k_2, k_3, \dots, k_n$ được gọi là hệ tọa độ barycentric.

7.2.2.3. Tam giác – Triangular



Hình 7.10 Mảnh tam giác

Trong tam giác các điểm có dạng P_1, P_2, P_3

Hệ số: $k_1, k_2, k_3 \in [0, 1]$

$$k_1 + k_2 + k_3 = 1$$

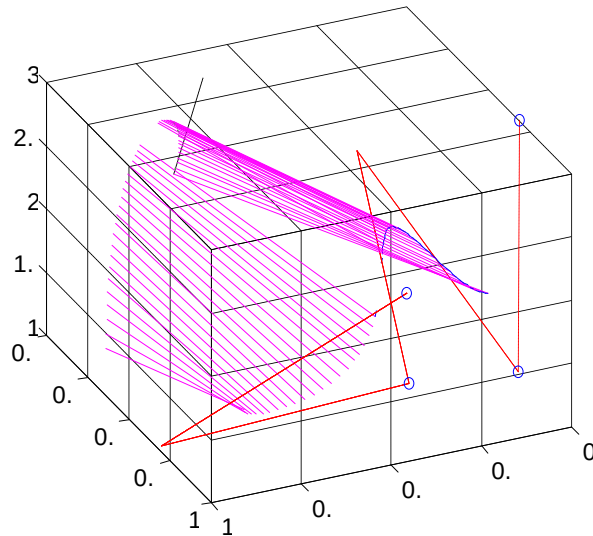
$$P = k_1 P_1 + k_2 P_2 + k_3 P_3$$

Nếu Hệ số $k_i > 1$ hoặc < 0 điểm P sẽ nằm ngoài tam giác (Q)

Nếu Hệ số $k_i = 1$ hoặc $= 0$ điểm P sẽ nằm trên cạnh tam giác (R)

7.2.3. Mô hình hoá các mặt cong (Surface Patches)

7.2.3.1. Mặt kẻ (Ruled Surface) Bề mặt được xây dựng bằng cách cho trượt một đoạn thẳng trên hai đường cong. Các mặt kẻ nhận được bằng phép nội suy tuyến tính từ hai đường cong biên cho trước tương ứng với hai biên đối diện của mặt kẻ $P_1(u)$ và $P_2(u)$



Phương trình mặt kẻ:

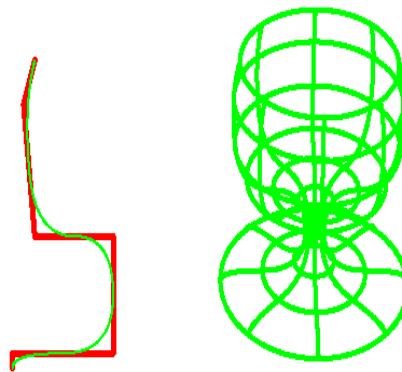
$$Q(u,v) = P_2(u)v + P_1(u)(1-v)$$

Nếu hai đường cong cho trước tương ứng là $P_1(v)$ và $P_2(v)$

Thì mặt kẻ có phương trình:

$$Q(u,v) = P_1(v)(1-u) + P_2(v)u = \begin{bmatrix} 1-u \\ u \end{bmatrix} \begin{bmatrix} P_1(v) \\ P_2(v) \end{bmatrix}$$

7.2.3.2. Mặt tròn xoay (Revolution surface)



Hình 7.12 Mô hình mặt tròn xoay

Mặt được xây dựng bởi đường thẳng hay một đường cong phẳng, quanh một trục trong không gian.

Giả sử đường cong phẳng có dạng:

$$P(t)=[x(t) \ y(t) \ z(t)] \ 0 \leq t \leq t_{\max}$$

Ví dụ: quay quanh trục x một thực thể nằm trên mặt phẳng xoy, phương trình bề mặt là

$$Q(t, f) = [x(t) \ y(t) \ \cos f z(t) \ \sin f]$$

$$0 \leq \phi \leq 2\pi$$

Ví dụ: Mặt tròn xoay $P_1[1 \ 1 \ 0]$ và $P_2[6 \ 2 \ 0]$ nằm trong mặt phẳng xoy. Quay đường thẳng quanh trục ox sẽ được một mặt nón. Xác định điểm của mặt tại $t=0.5, f=p/3$.

Phương trình tham số cho đoạn thẳng từ P_1 tới P_2 là:

$$P(t) = [x(t) \ y(t) \ z(t)] = P_1 + (P_2 - P_1)t \quad 0 \leq t \leq 1$$

Với các thành phần Đề-các:

$$x(t) = x_1 + (x_2 - x_1)t = 1 + 5t$$

$$y(t) = y_1 + (y_2 - y_1)t = 1 + t$$

$$z(t) = z_1 + (z_2 - z_1)t = 0$$

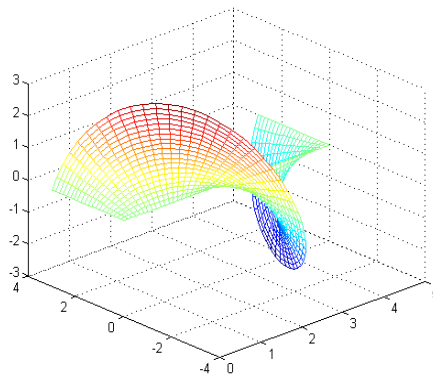
Dùng phương trình:

$$Q(1/2, p/3) = [1 + 5t(1+t)\cos f \ (1+t)\sin f]$$

$$= \left[\frac{7}{2} \quad \frac{3}{2} \cos \frac{\pi}{3} \quad \frac{3}{2} \sin \frac{\pi}{3} \right]$$

$$= \left[\frac{7}{2} \quad \frac{3}{4} \quad \frac{3\sqrt{3}}{4} \right]$$

7.2. 3.3. Mặt trượt (Swept Surface)



Sweep surface là mặt được tạo bởi bằng cách trượt một thực thể.

Ví dụ: một đường thẳng, đa giác, một đường cong, một hình... dọc theo một đường trong không gian.

$$Q(u,v) = P(u) * [T(v)]$$

$P(u)$ thực thể cần trượt

$[T(v)]$ là ma trận biến đổi ($[T(v)]$ có thể là ma trận tịnh tiến, quay, hay tỉ lệ ... hoặc là kết hợp của nhiều phép biến đổi đó).

Ví dụ:

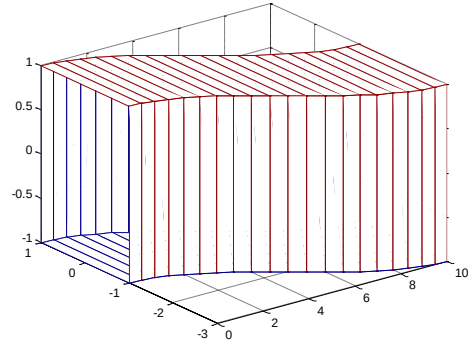
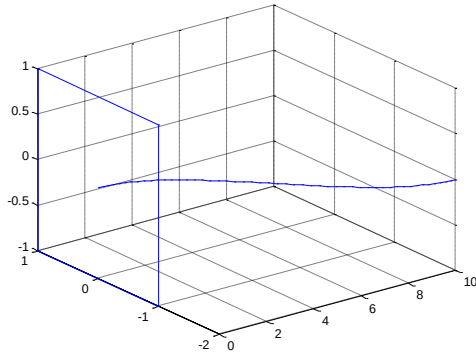
$$P_1[000], P_2[03 \ 0]$$

$$P(t) = P_1 + (P_2 - P_1) * u = [0 \ 3u \ 0]$$

$$0 \leq u, v \leq 1$$

$$|T(v)| = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(2\pi v) & \sin(2\pi v) & 0 \\ 0 & -\sin(2\pi v) & \cos(2\pi v) & 0 \\ 10v & 0 & 0 & 1 \end{bmatrix}$$

Ví dụ về mặt trượt (Sweep Extrusion)



Hình 7.14 Hình thành mặt trượt

Hình vuông xác định bởi 4 đỉnh:

$$P_1[0 \ -10], P_2[0 \ -1 \ -1]$$

$$P_3[0 \ 1 \ -1], P_4[0 \ 1 \ 1]$$

Đường cong trượt:

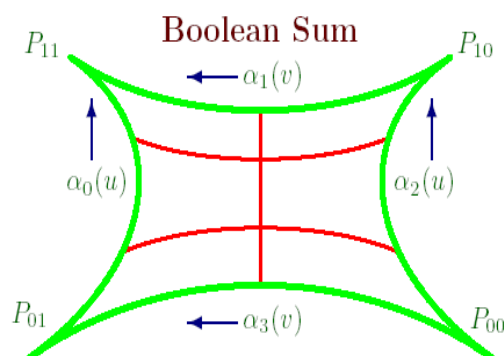
$$|P(u)| = \begin{bmatrix} P_1 \\ P_2 \\ P_3 \\ P_4 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 1 & 1 \\ 0 & -1 & -1 & 1 \\ 0 & 1 & -1 & 1 \\ 0 & 1 & 1 & 1 \\ 0 & -1 & 1 & 1 \end{bmatrix} \quad |T(v)| = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 10v & \cos(\pi v) - 1 & 0 & 1 \end{bmatrix}$$

$$x = 10vy = \cos(\pi v) - 1$$

Quay 1 góc khi trượt:

$$\begin{bmatrix} \cos(\varphi) & \sin(\varphi) & 0 & 0 \\ -\sin(\varphi) & \cos(\varphi) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 10v & \cos(\pi v) - 1 & 0 & 1 \end{bmatrix}$$

7.2.3.4. Mặt nội suy trên bốn đường biên (Boolean sum surface) Mặt được xây dựng trên 4 điểm và các đường cong biên, $S(u, v)$ mặt nội suy trên 4 đường biên



Hình 7.15 Mô hình mặt cong Boolean Sum

$$S(u, v) = S_1(u, v) + S_2(u, v) - P(u, v)$$

Với:

$$P(u, v) = (1-u)(1-v)P_{00} + (1-u)vP_{01} + u(1-v)P_{10} + uvP_{11}$$

$$S_1(u, v) = va_0(u) + (1-v)a_2(u)$$

$$S_2(u, v) = ua_1(v) + (1-u)a_3(v);$$

P là các đỉnh của mảnh 4

$a_i(u)$ là các phương trình đường biên

Ví dụ về mặt Boolean Sum:

Với $u = 0$

$$\begin{aligned} S(0, v) &= S_1(0, v) + S_2(0, v) - P(0, v) \\ &= v a_0(0) + (1 - v)a_2(0) + 0a_1(v) + 1 a_3(v) - (1 - v)P_{00} - v P_{01} \\ &= v P_{01} + (1 - v)P_{00} + a_3(v) - (1 - v)P_{00} - v P_{01} \\ &= a_3(v) \end{aligned}$$

7.2.4. Mặt từ các đường cong

7.2.4.1. Mặt cong bậc ba Hermite

Mặt cong tham biến được tạo bởi bề mặt qua tại 4 điểm dữ liệu tại 4 góc và các đường cong có phương trình bậc ba qua chúng, như vậy 16 vector điều kiện hay tương đương với 48 giá trị đại số cần thiết để xác định các hệ số phương trình.

Khi những hệ số là 4 điểm dữ liệu góc và 8 vector tiếp tuyến tại các điểm đó theo các hướng u, v tương ứng cùng 4 vector xoắn thì mặt cong tạo thành là mặt cong Hermite. Phương trình có dạng:

$$Q(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 C_{ij} u^i v^j \quad 0 \leq u, v \leq 1$$

$$Q(u, v) = [U] [C] [V]^T \quad 0 \leq u, v \leq 1$$

$$\text{Với: } u = [u^3 u^2 u 1], v = [v^3 v^2 v 1]$$

Và ma trận hệ số $[C]$ là:

$$[C] = \begin{bmatrix} C_{00} & C_{01} & C_{02} & C_{03} \\ C_{10} & C_{11} & C_{12} & C_{13} \\ C_{20} & C_{21} & C_{22} & C_{23} \\ C_{30} & C_{31} & C_{32} & C_{33} \end{bmatrix}$$

Cuối cùng ta thu được các hệ số theo phương trình mới có dạng:

$$Q(u, v) = [U][M_H] [B] [M_H]^T [V]^T \quad u, v \in [0,1]$$

$$[M_H] = \begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

Và B là ma trận điều kiện biên:

$$[B] = \begin{bmatrix} P_{00} & P_{01} & P_{v00} & P_{v01} \\ P_{10} & P_{11} & P_{v10} & P_{v11} \\ P_{u00} & P_{u01} & P_{uv00} & P_{uv01} \\ P_{u10} & P_{u11} & P_{uv10} & P_{uv11} \end{bmatrix}$$

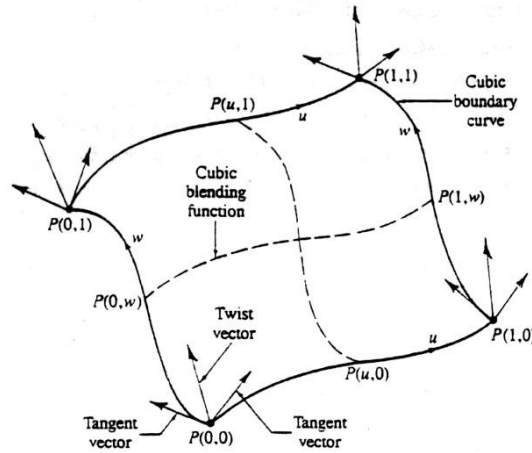
Hay với dạng thức rút gọn của ma trận [B] theo các ma trận điều kiện biên tương ứng:

[B] ma trận các giá trị tham số

[P_u], [P_v] các vector tiếp tuyến theo u,v tương ứng.

[P_{uv}] ma trận xoắn trên u,v

$$[B] = \begin{bmatrix} [P] & [P_v] \\ [P_u] & [P_{uv}] \end{bmatrix}$$



Hình 7.16 Mặt cong Hermite và các điểm dữ liệu

Các vector tiếp tuyến và vector xoắn của bề mặt cong được biểu diễn qua phương trình sau:

$$Q_u(u,v) = [U] [M_H]^u [B] [M_H]^T [V]^T$$

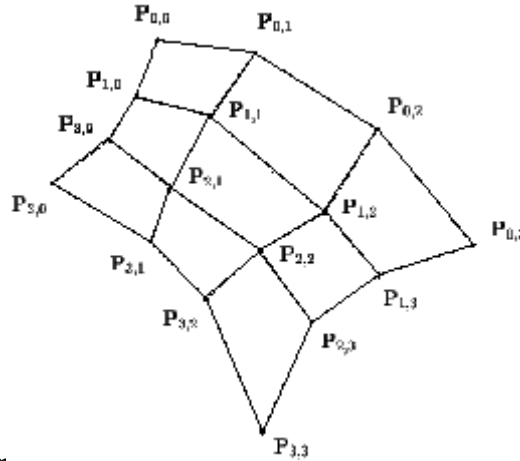
$$Q_v(u,v) = [U] [M_H]^v [B] [M_H]^T [V]^T$$

$$Q_{uv}(u,v) = [U] [M_H]^{uv} [B] [M_H]^T [V]^T$$

Với:

$$[M_H]^U \& [M_H]^V = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 6 & -6 & 3 & 3 \\ -6 & 6 & -4 & -2 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

7.2.4.2. Mặt cong Bezier



Mảnh-patch Bézier

Hình 7.17 Mặt cong Bezier

Mảnh Bezier được hình thành trên phép trượt của đường cong Bezier. Việc xây dựng nên mảnh Bezier dưới các điểm kiểm soát, tạo nên đa diện kiểm soát.

$$\{P_{i,j} : 0 \leq i \leq n, 0 \leq j \leq m\}$$

Phương trình tổng quát của mặt cong tham biến Bezier có dạng:

$$P(u,v) = \sum_{j=0}^m \sum_{i=0}^n P_{i,j} B_{i,n}(u) B_{j,m}(v) \text{ trong đó } u,v \in [0, 1]$$

Mảnh Bezier bậc ba:

Mặt cong Bezier bậc ba là mặt phổ biến nhất trong CG, vì độ đơn giản của nó. Nó hình thành trên 4x4 điểm kiểm soát, công thức có dạng Đa thức Bernstein có dạng:

$$Q(u,v) = \sum_{i=0}^3 \sum_{j=0}^3 B_{n,i}(u) B_{m,j}(v) P_{ij}$$

$$B_0(t) = (1-t)^3$$

$$B_1(t) = 3t(1-t)^2$$

$$B_2(t) = 3t^2(1-t)$$

$$B_3(t) = t^3$$

Tính chất của mảnh Bézier Tính bao lồi: Mặt cong Bezier luôn nằm trong đa diện lồi của các điểm kiểm soát

- Mặt cong đi qua 4 điểm cận $P_{00}, P_{01}, P_{10}, P_{11}$ hay chính xác

$$Q(0,0)=P_{00}, Q(0,1)=P_{01}, Q(1,0)=P_{10}, Q(1,1)=P_{11}$$

- Đường cong biên của Mặt Bezier là đường cong Bezier Mặt cong là liên tục và đạo hàm riêng các bậc tồn tại của nó cũng liên tục.

- Đạo hàm riêng của mặt cong có dạng:

$$Q(u,v) = [U]^3 [N]^v [B]^1 [M]^T [V]^T$$

$$[U] = [u^3 \ u^2 \ u \ 1]$$

$$\partial Q(0,0) / \partial u = 3(P_{01} - P_{00})$$

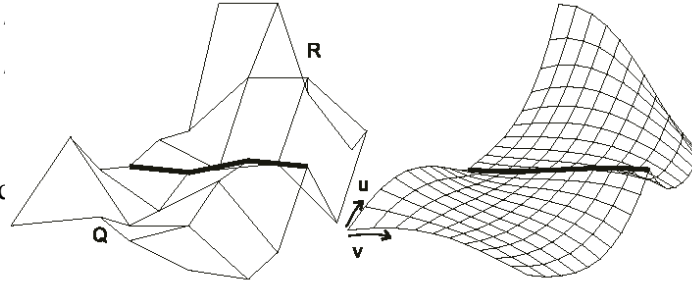
$$\partial Q(0,0) / \partial v = 3(P_{01} - P_{00})$$

$$\partial Q(0,1)$$

$$\partial Q(1,0)$$

$Q(u,v)$ là mọi điểm nằm trên mặt cong và $[N]$ và

$[M]$ được biểu c



zier Bậc 3(Bi-

cubic)

$$Q(u,v) = [u^3 \ u$$

$$\begin{bmatrix} P_{00} & P_{01} & P_{02} & P_{03} \\ P_{10} & P_{11} & P_{12} & P_{13} \\ P_{20} & P_{21} & P_{22} & P_{23} \\ P_{30} & P_{31} & P_{32} & P_{33} \end{bmatrix} \begin{bmatrix} P_{00} & P_{01} & P_{02} & P_{03} \\ P_{10} & P_{11} & P_{12} & P_{13} \\ P_{20} & P_{21} & P_{22} & P_{23} \\ P_{30} & P_{31} & P_{32} & P_{33} \end{bmatrix} \begin{bmatrix} -3 & 3 & -1 \\ 3 & -6 & 3 \\ 0 & 3 & -3 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v^3 \\ v^2 \\ v \\ 1 \end{bmatrix}$$

Hình 7.18 Nối hai mảnh Bezier bậc ba

Hai mảnh Q và R cùng chung tham biến tại biên (Giả sử u), hai đường cong biên phải bằng nhau $Q(1,v)=R(0,v)$. Hệ số của cột cuối ma trận Q = cột đầu ma trận R, tương tự: nếu theo hướng của v thì hàng sẽ thay cột ma trận.

Bậc của mặt cong theo mỗi hướng của tham biến bằng số điểm kiểm soát trừ 1. Tính liên tục hay đạo hàm của mặt theo mỗi tham biến bằng số điểm kiểm soát trừ 2. Hình dạng của mặt biến đổi theo các cạnh của đa giác kiểm soát. Mặt lưới chỉ đi qua các điểm góc cạnh của đa giác kiểm soát, nó chỉ nằm trong phần giới hạn bởi lưới của đa giác lồi kiểm soát và không thay đổi dưới tác động của các phép biến đổi affine. Mỗi đường biên của mặt Bezier là một đường cong Bezier với mặt cong bậc ba Bezier các đường cong biên luôn đảm bảo là các đường Bezier bậc 3. Như vậy lưới đa giác cho bề mặt sẽ là 4 x

4.7.2.4.3. Mặt cong B-Spline

Phương trình mặt B-spline:

$$Q(u, w) = \sum_{i=1}^n \sum_{j=1}^m N_{i,k}(u) \cdot M_{j,h}(w) \cdot P_{i,j}$$

$$N_{i,k}(u) = \begin{cases} 1 & x_i \leq u < x_{i+1} \\ 0 & \text{otherwise} \end{cases} \quad N_{i,k}(u) = \frac{(u - x_i)N_{i,k-1}(u)}{x_{i+k-1} - x_i} + \frac{(x_{i+k} - u)N_{i+1,k-1}(u)}{x_{i+k} - x_{i+1}}$$

$$\begin{cases} x_i = 0 (1 \leq i \leq k) \\ x_i = i - k (k+1 \leq i \leq n) \\ x_i = n - k + 1 (n+1 \leq i \leq n+k) \end{cases}$$

P_{ij} là điểm kiểm soát

N và M là đa thức B-spline

Với các mặt cong mở mặt cong phụ thuộc vào các nút vector

Đặc điểm của mặt cong B-Spline Số bậc cao nhất của bề mặt theo mỗi hướng thì bằng số điểm kiểm soát - 1 theo hướng đó.

- Đạo hàm riêng của phương trình bề mặt theo mỗi tham biến có bậc bằng số điểm kiểm soát theo tham biến đó trừ 2.
- Bề mặt B-spline thì không chịu ảnh hưởng của phép biến đổi affine. Bề mặt sẽ thay đổi nếu ta thay đổi đa giác kiểm soát.
- Ảnh hưởng của một điểm kiểm soát đơn được giới hạn bởi $+ - k/2$ khoảng đối với mỗi tham số.
- Nếu số đỉnh của đa giác kiểm soát bằng số bậc theo mỗi tham biến và không có điểm kép nào thì mặt B-spline sẽ chuyển thành mặt Bezier.
- Nếu các đa giác kiểm soát có dạng tam giác thì lưới đa giác kiểm soát sẽ có hình dáng gần giống với bề mặt cong.
- Mỗi mặt B-Spline luôn nằm trong bao lồi của đa giác kiểm soát.
- Mỗi mặt B-Spline có dáng điệu luôn bám theo hình dáng của đa giác kiểm soát.

Tóm tắt:

Việc tạo ra các đường cong theo ý muốn cũng là vấn đề thường gặp khi làm việc với đồ họa máy tính. Chúng ta khảo sát cách tiếp cận vẽ đường cong bằng Hermite, Bezier và B-spline. Các cách tiếp cận này dựa trên cơ sở vẽ đường cong bằng một tập điểm mô tả hình dáng của đường cong gọi là tập điểm kiểm soát. Khi thay đổi tập điểm này, hình dáng của đường cong sẽ thay đổi theo. Cách tiếp cận này cho thấy sự thuận lợi và linh hoạt khi cần phải vẽ các đường cong phức tạp và do đó nó được dùng nhiều trong thiết kế.

Một nhược điểm trong cách vẽ đường cong bằng Bezier là khi một phần của đường cong đã đạt yêu cầu, nhưng khi hiệu chỉnh phần còn lại sẽ mất đi phần đã đạt yêu cầu, hay việc nối trơn các đường cong sẵn có. Để khắc phục các vấn đề này ta có cách tiếp cận cải tiến vẽ đường cong bằng B-spline.

Tương tự như vậy việc biểu diễn các mặt cong trong đồ họa máy tính cũng là một vấn đề cần thiết để mô tả đối tượng trong thế giới thực. Chúng ta khảo sát về các phương pháp biểu diễn mặt cong thông qua phương trình tham số. Trong đó, phương trình tham số của một mặt có dạng là một phương trình tham số hai biến $p(u, v)$ và một điểm bất kỳ trên

mặt sẽ được biểu diễn dưới dạng $p(u,v) = (x(u,v), y(u,v), z(u,v))$. Chúng ta khảo sát một số mặt đơn giản như: mặt kẻ, mặt tròn xoay, mặt trượt và mặt Boolean Sum.

Trên cơ sở các đường cong bằng Hermite, Bezier và B-spline chúng ta cũng xây dựng được các mặt Hermite, Bezier và B-spline.

Bài tập:

1. Cho 4 điểm $P_0(1,3,6)$, $P_1(6,0,3)$, $P_2(-1,3,-2)$ và $P_3(5,4,1)$ dùng 4 điểm trên làm điểm kiểm soát đưa ra phương trình của đường cong Bezier.
2. Cho hai điểm $P_1[-2 \ 4 \ 5]$ và $P_2[0 \ 1 \ -6]$ nằm trong mặt phẳng xoy. Quay đường thẳng quanh trục ox sẽ được một mặt nón. Xác định điểm của mặt tại $t=0.8$, $\phi=\pi/4$.
3. Cài đặt thuật toán vẽ đường cong bằng Bezier cho phép người dùng định nghĩa tập điểm kiểm soát mô tả hình dạng đường cong và cho phép người dùng hiệu chỉnh một số điểm kiểm soát mô tả hình dạng đường cong và cho phép người dùng hiệu chỉnh một số điểm kiểm soát để hiệu chỉnh đường cong theo ý muốn.
4. Viết phương trình vẽ đường Spline.

CHƯƠNG 8: ÁNH SÁNG

8.1. GIỚI THIỆU

8.1.1. Mục tiêu chính trong đồ họa máy tính

Có hai mục tiêu chính trong đồ họa máy tính mà các nhà đồ họa luôn muốn chinh phục đó là thời gian thực và tính trực quan.

Thời gian thực:

Thời gian thực qui định mức ảnh hưởng thường xuyên đến việc hình ảnh có nằm trong mức dung sai cho phép hay không. Tất nhiên, mức độ ảnh hưởng của thời gian ở mỗi một ứng dụng là khác nhau, như: thời gian đối với một người làm công tác kiểm định mã code được biểu diễn bằng thời gian vào khoảng một vài phút; trong khi đó, trong môi trường đồ họa 3D thì lại qui định thời gian thực cập nhật trong khi lập thiết kế.

Tính trực quan:

Kỹ thuật viên thiết kế đồ họa 3D luôn phải làm việc với một loạt đường biên để hiển thị phối cảnh mà anh ta đang tạo dựng. Cụ thể hơn, lấy ví dụ: nhà đồ họa làm việc trong một công ty sản xuất phim, anh ta phải đưa ra bản biểu diễn phối cảnh xác thực nhất cho hoạt cảnh mà anh ta thiết kế.

Trong một thời gian dài hai mục tiêu này luôn là cái đích mà chúng ta muốn đạt tới. Tuy nhiên, cùng sự phát triển của phần cứng và các công nghệ khoa học, các sản phẩm đồ họa đã dần đạt tới ngưỡng của thời gian thực. Nếu nói theo lĩnh vực của công nghệ phần cứng, trong vòng sáu tháng phần cứng của đồ họa đã phát triển với tốc độ gấp đôi. Các hình mô tả dưới đây được minh họa từ trò chơi Doom3.



Hình 8.1 hình minh họa từ trò chơi Doom3

Vấn đề chủ yếu ở đây là việc tập trung tận dụng được ánh sáng cục bộ và tạo bóng. Ngoài ra việc khuếch tán bóng của hình ảnh được minh họa như khung hình cornell phía dưới đây, là điều không thể thực hiện được với hệ thống phần cứng đơn lẻ hiện có.

8.1.2. Các giải pháp trong đồ họa máy tính

Những điều mà chúng ta muốn thực hiện lại không như trong thực tế phát tán ánh sáng được gây bởi sự tương tác của các bề mặt, bóng của ánh sáng được gây nên bởi nhiều nguồn sáng, rồi việc hiệu chỉnh sự phản chiếu ánh sáng... Thuật toán chiếu sáng toàn cục (the Global illumination Algorithm) đã thoả mãn được các điều kiện đem lại những bức

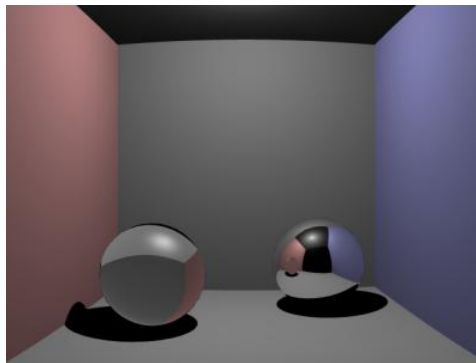
ảnh có độ trung thực cao. Chương này đi sâu vào nghiên cứu chi tiết ba giải pháp của thuật toán chiếu sáng toàn cục:

- Raytracing
- Radiosity
- Photon mapping

8.1.2.1. Raytracing - dựng đồ thị tia

Về cơ bản của Raytracing hoạt động dựa trên phương pháp dựng các tia đồ thị qua các điểm ảnh trong vùng thắp của màn hình và khung nhìn được tác động đến. Giá trị ánh sáng của mỗi điểm ảnh phụ thuộc vào vector pháp tuyến chuẩn hoá và vị trí điểm giao nhau của mỗi đối tượng cũng như vị trí của ánh sáng.

Cơ bản phương pháp Raytracing không hoàn toàn là phương pháp chiếu sáng toàn cục bởi phương thức của ánh sáng chỉ là cục bộ. Nó hoạt động dựa trên việc đưa nhiều nguồn sáng vào việc tạo bóng, kết hợp với đa dạng ánh sáng phản chiếu, độ khúc xạ,... từ đó có thể cho ta những bức ảnh vô cùng sắc nét mà có thể đánh lừa thị giác của những người xem dễ tính. Một dấu hiệu dễ nhận biết phương pháp raytracing đó là hầu hết các bức ảnh đều trông rất thật, như : bóng đồ cứng và khung cảnh được hợp thành bởi nhiều yếu tố lung linh. Do vậy phương pháp Raytracing có thể mở rộng thành phương pháp chiếu sáng toàn cục đầy đủ, như path tracing - lần theo dấu vết đường dẫn.



Hình 8.2 Raytraced Cornell box

8.1.2.2. Radiosity

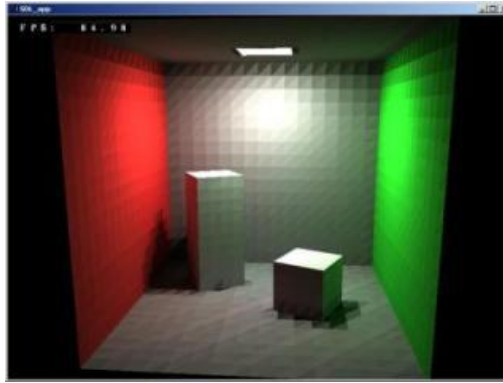
Radiosity về phương pháp thì ngược với raytracing, vì nó chỉ giải quyết các phần về khuếch tán trong thuật toán bố trí ánh sáng.

Thuật giải của radiosity chính là việc chia nhỏ tất cả bề mặt của vật thành một số mảnh hình chữ nhật. Giá trị ánh sáng của một mảnh sẽ là tổng các giá trị ánh sáng do nó phát ra cộng với tất cả các ánh sáng nó nhận được một cách trực tiếp hoặc gián tiếp từ các mảnh (patch) khác. Vì quá trình này được áp dụng lặp đi lặp lại cho tất cả các mảnh, cuối cùng nó đồng qui về giải pháp chiếu sáng toàn cục.

Vì phương pháp này chỉ nghiên cứu về giải pháp khuếch tán ánh sáng, và giải thuật này về mặt tổng quan vốn đã được mô tả một cách độc lập. Điều đó có nghĩa là giải thuật này có thể được xử lý trước, sau đó nó được hiển thị dưới mọi góc độ. Tất nhiên, nếu bắt

kỳ một ánh sáng hay đối tượng nào trong tổng thể bị dịch chuyển thì giải thuật này sẽ phải tính toán lại.

Với những nguyên nhân được trình bày ở trên, Radiosity không hoàn toàn là giải thuật chiếu sáng toàn cục. Một giải pháp chưa hoàn toàn chính xác nhưng vẫn có thể đáng tin cậy, nếu nó chuẩn theo kỹ thuật tiền xử lý của radiosity với việc thực hiện một raytracing trong suốt quá trình tô trát (render).



Hình 8.3 Phương pháp radiosity

8.1.2.3. Photon Mapping

Photon mapping là thuyết minh về chiếu sáng toàn cục gần đây nhất. Phương pháp này giải quyết được phần việc của tán xạ ánh sáng và các vấn đề của phản chiếu ánh sáng. Tuy nhiên, xử lý raytracing lần thứ hai phụ thuộc hoàn toàn vào giải pháp bắt điểm phản chiếu.

Khái niệm của giải pháp này khá đơn giản: mỗi ánh sáng phát ra x hạt photon. Khi các hạt này gặp phải một vật gì đó thì vị trí và giá trị của ánh sáng của chúng sẽ được lưu lại trong một cấu trúc dữ liệu thông minh. Dựa vào cơ sở dữ liệu trên cùng với kỹ thuật render, từ các hạt này ta sẽ lần ngược lại dấu vết ánh sáng trong thế giới thực thể để đánh giá lại cường độ ánh sáng tại từng điểm xác định trong khoảng đang xét.



Hình 8.4 ánh xạ photon

8.2. CÁC KỸ THUẬT CHIẾU SÁNG TRONG ĐỒ HỌA MÁY TÍNH

8.2.1. Đánh giá về cường độ ánh sáng

Khái niệm	Định nghĩa
P	P là điểm mà ta tính toán các giá trị màu sắc

Chương 8: Ánh sáng

N	N là vector pháp tuyến chuẩn hoá của bề mặt (vector trục chuẩn) tại điểm P
Q	Q: vị trí của ánh sáng
L	L là vécto ánh sáng từ điểm Q tới P. Do vậy giá trị của L sẽ được tính như sau: $L = P - Q$
V	V là vector quan sát.
R	R là giá trị phản chiếu xung quanh V, nó được tính toán như sau: $R = N - V * ((N * V) * 2)$
H	Trong thời gian trước, khi tốc độ của máy tính chưa nhanh được như ngày nay, để tính được giá trị R đòi hỏi chi phí nhiều về thời gian và tiền bạc nên người ta dùng một giá trị xấp xỉ để thay thế là H: $H = \frac{(L + V)}{2}$

8.2.2. Cường độ ánh sáng

Trong nghiên cứu, thiết kế cường độ ánh sáng có hai phương pháp:

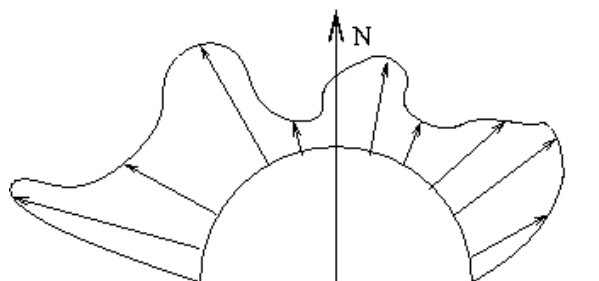
Phương pháp thứ nhất được đại diện bằng một bộ 3 vector đơn, mỗi giá trị được giới hạn từ 0-1. Trong phương pháp này, cùng lúc 3 vector được dùng để xác định tất cả các thuộc tính màu của vật.

Phương pháp thứ hai, mỗi thuộc tính ánh sáng sẽ được biểu diễn bởi một bộ 3 vector. Do vậy mỗi một giá trị ánh sáng sẽ bao gồm một giá trị màu bao quanh, một giá trị màu khuếch tán ...

Phương pháp thứ nhất thì mô tả thật hơn, trong khi phương pháp thứ hai lại đạt hiệu quả cao trong việc tạo hiệu ứng nghệ thuật. Hiện nay thì xu hướng chung không còn quá coi trọng việc mô tả chính xác sự vật hiện tượng.

Hàm phân phối phản xạ hai chiều – BRDF

Nếu nhìn từ cạnh của vật, ánh sáng nằm rải rác tại từng điểm đơn lẻ trên khắp bề mặt với cường độ sáng sẽ thay đổi liên tục qua các góc.



Hình 8.5 BRDF

Việc cố gắng đưa một mô hình ánh sáng về một mẫu chính là hàm phân phối hệ số phản xạ hai chiều (BRDF). Chức năng của BRDF được định nghĩa như sau :

$$p_{\lambda}''(\lambda_r, \theta_r, \phi_r, \theta_i, \phi_i) = \frac{I_{\lambda,r}(\lambda_r, \theta_r, \phi_r, \theta_i, \phi_i)}{I_{\lambda,r}(\lambda_r, \theta_i, \phi_i) \cos(\theta_i d\omega_i)}$$

Các thuật ngữ được định nghĩa như sau :

Khái niệm Định nghĩa

I_r	Cường độ phản chiếu của ánh sáng hướng ra
I_i	Cường độ phản chiếu của ánh sáng hướng vào
λ	Bước sóng
ρ''	Hệ số Quang Phổ của tia phản xạ
(θ_r, ϕ_r)	hướng tia đi ra
(θ_i, ϕ_i)	Hướng tia đi vào

BRDF được định nghĩa là tỷ lệ của cường độ ánh sáng đi ra theo một hướng đặc biệt và cường độ luồng sáng đi vào được đánh dấu bằng điểm P.

Theo công thức ở trên thì nó chưa hoàn toàn hữu ích cho những công việc mang tính thực hành cao. Do vậy các nhà nghiên cứu đã bỏ nhiều công sức vào việc nghiên cứu tìm ra những mô tả, ứng dụng mang tính thực tiễn cao hơn của BRDF.

Màu sắc truyền thống của mỗi vật chất tại điểm P sẽ được chia ra thành 3 yếu tố riêng biệt: Bao quanh (ambient), phản chiếu (specular) và khuếch tán (diffuse).

8.2.3. Những thuộc tính bao quanh của vật chất

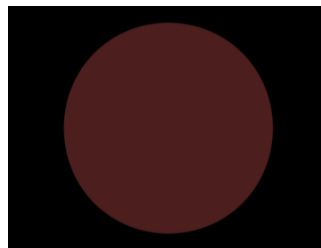
Thuộc tính bao quanh không phải là một thuộc tính reallife thực sự, nhưng nó thường được ứng dụng cao trong nghệ thuật và việc gỡ lỗi (debug). Màu bao quanh có thể được mô tả bằng màu đồng nhất của chúng, mà không thay đổi cường độ trên toàn bộ bề mặt.

Vì thế giá trị ánh sáng của một đối tượng bao quanh tại một bề mặt được xây dựng trong đoạn giả mã sau:

Ambient Color Calculation

```
Surface S = Surface that P lies on;
Color C = S.Material.Ambient;
return C;
```

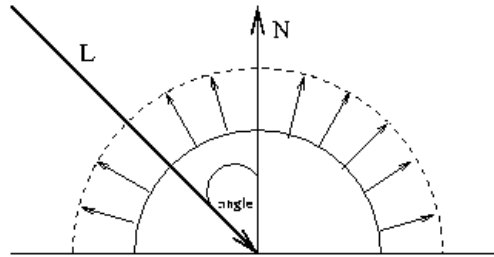
Và chúng ta có thể nhận thấy rằng, ánh sáng toàn cục là tổng thể của những tác động tự nhiên của sự chiếu sáng và các điều kiện của bề mặt. Và là một ánh sáng liên tục.



Hình 8.6 hình cầu Raytraced chỉ với thành phần bao quanh.

8.2.4. Thuộc tính khuếch tán của vật chất

Màu khuếch tán là sự trải ra như nhau theo mọi phương hướng, và vì thế chỉ phụ thuộc trên vectơ ánh sáng đầu vào L , và vectơ pháp tuyến trực chuẩn N . Hình vẽ sau cho thấy ý tưởng đó:

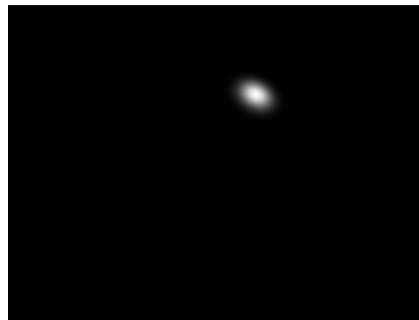


Hình 8.7 Mô tả ánh sáng khuếch tán

Mô hình Phong sau khi thực hiện thành phần phản chiếu:

Specular Color Calculation

```
ReflectionVector R
IntersectionNormal N
IntersectionPoint P
Surface S = Surface P lies on
float glossiness = S.material.glossiness
float angle = R.dot(N)
float intensity = Light.Color * S.material.specular
return intensity * pow(angle,glossiness)
```



Hình 8.8 hình cầu Raytraced chỉ với thành phần phản chiếu

Mô Hình Phong:

Đa số vật thể không chỉ được mô tả đơn thuần bằng mỗi thuộc tính phản xạ hay khuếch tán, mà nó là sự kết hợp của cả hai yếu tố này thậm chí là cả các thuộc tính bao quanh nếu cần.

Mô hình Phong được định nghĩa trong BRDF như sau:

$$\rho''_{\lambda}(\lambda, \Theta_r, \phi_r, \Theta_i, \phi_i) = k_{diffuse} + k_{specular} \cos^n \phi$$

Khái niệm

Định nghĩa

Diffuse

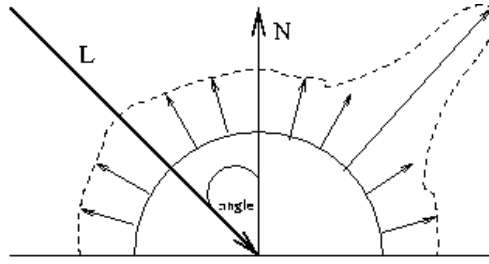
Phần nhỏ năng lượng phản chiếu

Specular

Phần nhỏ của năng lượng khuếch tán phản chiếu

ϕ

Góc giữa hướng tia tới R và hướng nhìn V



Hình 8.9 Phân phối của Phong.

Đa số việc mở rộng thực thi đều đưa tới:

$$I_{\lambda, \phi}(\lambda, \phi) = I_{\text{ambient}} k_{\text{ambient}}(\lambda) + I_{\text{diffuse}} k_{\text{diffuse}}(\lambda)(L \cdot N) + I_{\text{specular}} k_{\text{specular}}(\lambda)(R \cdot V)^n$$

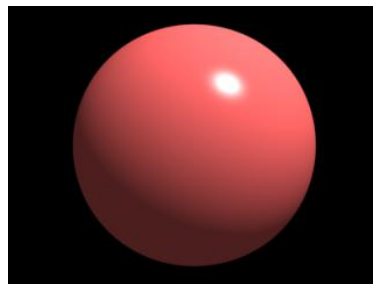
Đoạn mã rút gọn:

Phong Calculation

return

AmbientColorCalculation()+DiffuseColorCalculation()+SpecularColorCalculation();

Lưu ý: nếu ta dùng màu sắc theo hướng trực quan hoá thì sẽ cần dùng đến phương pháp chồng chất màu.



Hình 8.10 Hình cầu Raytraced với ánh sáng Phong

8.2.5. Sự tương tác bề mặt/ánh sáng

Mỗi bề mặt tương tác ánh sáng theo những cách khác nhau tùy theo thuộc tính riêng của nó.

Absorbance

Trong thế giới thật một bề mặt luôn thu hút một vài luồng ánh sáng khác nhau nào đó. Điều đó lý giải cho sự tán sắc nhiệt. Và đa số những thuộc tính này đều bị bỏ qua khi render. Đôi khi quá trình render cũng phải sử dụng một số vật thay thế ngẫu nhiên (fake) để tránh các vòng đệ quy vô hạn.

Reflectance

Một bề mặt phản chiếu ánh sáng dựa trên các yếu tố như: bề mặt bình thường của vật, hướng nhìn, và góc tới của ánh sáng. Tham số hệ số phản xạ chính là giá trị cường độ ánh sáng trong khu vực xem xét được mô tả ở trên.

8.2.6. Sự khúc xạ và sự truyền sáng

Bất kỳ một vật thể nào với một độ trong suốt nhất định thì đều truyền sáng. Đó là hiện tượng ánh sáng đi qua các điểm cong P, tuân theo luật Snell. Hiện tượng này dễ thấy nhất

khi ta quan sát một cốc nước, nơi mà các đối tượng ở một mặt khác hay ở dưới bề mặt quan sát có vẻ được chuyển vị trí. Vì thế nếu một tia tới trong môi trường vật chất m_1 (lúc này không khí và chân không đều được coi là vật chất) với hệ số khúc xạ n_1 ở một góc θ_1 , đến môi trường vật chất m_2 có hệ số khúc xạ n_2 ; thì luật Snell định nghĩa là:

$$n_1 \cos(\theta_1) = n_2 \cos(\theta_2)$$

Đây được gọi là phương trình khúc xạ của tia T.

$$T = ((\frac{n_1}{n_2}) * R) + (\frac{n_1}{n_2} * (N \cdot R) * \sqrt{1 - (\frac{n_1}{n_2})^2 * (1 - (N \cdot R)^2)}) * N$$

R là tia tới của ánh sáng.

Biên giới giữa hai môi trường vật chất được gọi là giao diện.

8.3. CÁC CÔNG NGHỆ

8.3.1. Raytracing

8.3.1.1. Giới thiệu

Raytracing bắt đầu được sử dụng trong đồ họa máy tính từ khoảng 1980 như là một sự thử nghiệm sớm nhất trong việc giải quyết vấn đề chiếu sáng tổng thể. Sau đó đã có vô số cải tiến được đưa ra, nhưng chúng ta chỉ diễn tả một ý tưởng cơ bản và những phần liên quan. Phần này có nhiều ưu điểm hơn và cũng chứa một vài kỹ thuật mới hơn, tuy không cải thiện chất lượng hình ảnh nhưng lại tăng tốc độ trong việc xử lý biểu diễn.

8.3.1.2. Thuật toán

Ý tưởng cơ bản trong raytracing là lần theo dấu vết của các photon ánh sáng ngược chiều với nguồn phát sáng, ví dụ từ mắt đến khung cảnh. Khi một tia sáng tiếp xúc với đối tượng trong khung cảnh có ánh sáng cường độ mạnh tại điểm giao nhau được tính toán cho phương đối diện với tia sáng. Cường độ này chính là “màu” của tia sáng.

Để tính toán cường độ, chúng ta sử dụng công thức sau:

$$I = I_{\text{local}} + k_r * I_{\text{reflected}} + k_t * I_{\text{transmitted}}$$

Trong đó:

I_{local} : ánh sáng trực tiếp

K_r : nhân tố phụ thuộc vật chất đại diện sự phản chiếu

$I_{\text{reflected}}$: cường độ ánh sáng đến từ nguồn phản chiếu

K_t : nhân tố phụ thuộc vật chất đại diện độ chắn sáng

$I_{\text{transmitted}}$: cường độ ánh sáng truyền xuyên qua đối tượng

I_{local} (ánh sáng trực tiếp) có thể được tính toán sử dụng bất kỳ sự phản chiếu mô hình cục bộ nào, nhưng trong thực tế điều này đa số luôn luôn là mô hình Phong được mô tả trong phần trước của tài liệu. Mô hình thường được tăng cường và được gọi là “bóng nháy sáng” (Shadow feelers). Bóng nháy sáng là tia sáng được ném xuyên qua khung cảnh từ điểm giao nhau đến mỗi nguồn sáng để xác định nếu có tầm nhìn (line of sight) hoặc nếu ánh sáng bị chặn lại bởi một vài đối tượng khác. Nếu một nguồn sáng bị chặn lại thì nó bị

bỏ qua trong mô hình cục bộ. Mô hình Phong có thể được tính toán từ điểm giao nhau và tại điểm bề mặt thông thường.

Cả $I_{\text{reflected}}$ và $I_{\text{transmitted}}$ đều được xác định bằng việc bắn một tia sáng mới xuyên qua khung cảnh cũng giống như tia sáng được bắn xuyên qua điểm ảnh của hình ảnh. Điểm bắt đầu của chuỗi tia sáng này là điểm giao nhau, có phương là phương phản chiếu của $I_{\text{reflected}}$ và hướng khúc xạ của $I_{\text{transmitted}}$. Việc tính toán những phương hướng này được biểu diễn (covered) bên dưới:

Phương hướng được tính toán như sau:

$$R = 2(N * L)N - L$$

Trong đó: N : pháp tuyến trực chuẩn tại điểm giao nhau

L : phương đối diện với tia sáng đến

Điều khiển việc truyền tia sáng (khúc xạ):

$$T = (n_1/n_2)L - (\cos(\varphi) + (n_1/n_2)(L * N))N$$

Trong đó: n_1 : chỉ số khúc xạ của vật chất đến

n_2 : chỉ số khúc xạ của vật chất đi

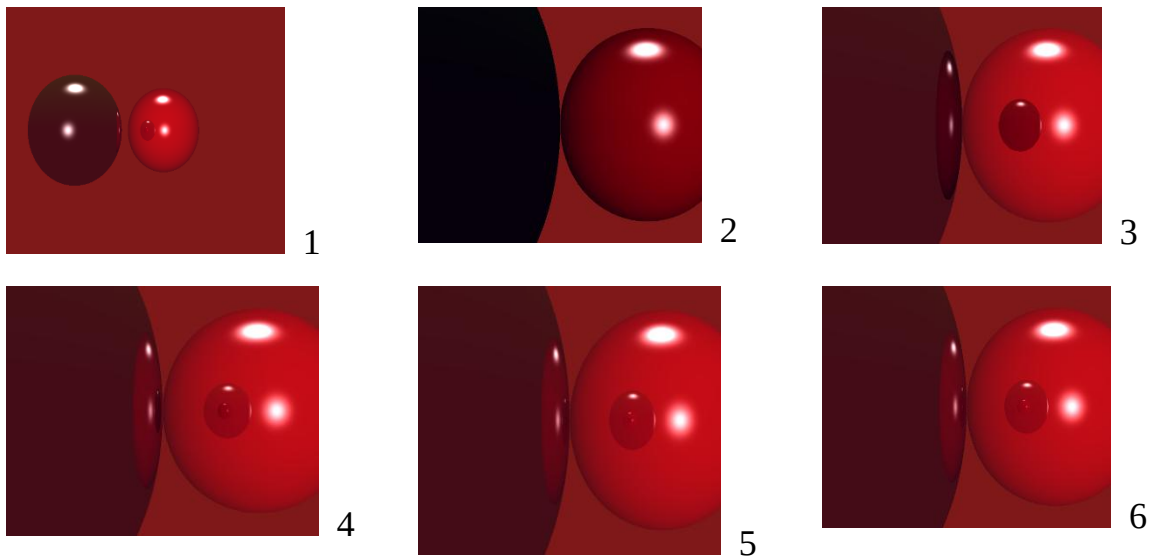
φ : được xác định bằng công thức sau:

$$(\sin \varphi / \sin \theta) = (n_1/n_2) \text{ và } \theta \text{ là góc giữa } L \text{ và } N$$

Một khi bắn một tia sáng mới tới mỗi điểm giao nhau (để tính toán $I_{\text{reflected}}$ và $I_{\text{transmitted}}$) raytracing sử dụng một hàm đệ quy. Điều kiện giới hạn cho đệ quy có thể hoặc là định trước chiều sâu phép đệ quy cực đại có thể đạt đến hoặc khi tia sáng không chạm tới bất kỳ đối tượng nào trong khung cảnh trong trường hợp nào đó được cấp phát một màu nền.

Rõ ràng độ sâu hơn mà ta cho phép phép đệ quy sẽ làm thời gian dịch lâu hơn nhưng chiều sâu phép đệ quy cực tiểu có thể làm cho một phần nhất định của hình ảnh bị mất. Việc tìm chính xác chiều sâu của phép đệ quy cực đại để sử dụng là một vấn đề của việc vận và kiểm tra cho đến khi kết quả hình ảnh nhìn được tốt.

Một ví dụ làm cách nào để xác định trước chiều sâu phép đệ quy có thể ảnh hưởng đến hình ảnh cuối cùng có thể được nhìn thấy trong ảnh dưới. Hình ảnh đầu tiên hiển thị một cảnh tổng quan với độ sâu phép đệ quy là 5. Trong những ảnh tiếp theo, ta có cái nhìn cận cảnh một phần bên trong thú vị - tức là ở đâu có 2 hình cầu tương tác với nhau. Ảnh 2 cho thấy chiều sâu phép đệ quy 0, ví dụ chỉ mô hình phản xạ cục bộ (Phong trong trường hợp này) được sử dụng. Ảnh 3 biểu diễn chiều sâu phép đệ quy 1. Ở đây ta có thể nhìn thấy 2 hình cầu phản chiếu lẫn nhau một lần. Hình cầu cực tả (leftmost) cũng sáng hơn khi màu nền của cảnh được cho phép nảy ra khỏi nó. Ảnh còn lại cho thấy chiều sâu phép đệ quy 2, 3 và 4. Chú ý thông số tương ứng của hình cầu phản chiếu lẫn nhau và thật khó để nhận ra sự khác nhau giữa 2 lần cuối.



Hình 8.11 Hình các quả cầu phản chiếu

Raytracing có thể được nhìn như một cây- xử lý tạo thành từ mỗi va chạm tia làm phát sinh ra nhiều hơn 2 tia: 1 phương phản chiếu và 1 phương khúc xạ. Những tia này được giả thiết rất mỏng và không dần trải. Kết quả này trong một số vấn đề được mô tả phía dưới nhưng cũng có nghĩa 1 raytracer có thể phải đủ nhanh một khi số điểm giao nhau trên 1 pixel có hạn bởi 2^n với n là chiều sâu của phép đệ quy. Có thể thấy chiều sâu của phép đệ quy không phải rất cao- thông thường giữa 4 và 8 là đủ. 256 điểm giao nhau bởi vậy là cực đại và trong trường hợp những đối tượng mờ đục chỉ có 1 tia trên 1 điểm giao nhau thì sinh ra 1 khối lượng lớn số giao điểm trên 1 pixel tới 8. Điều này có nghĩa là những cảnh có độ phức tạp vừa phải có thể biểu diễn lại trong 10 phần giây (tens of second) trên phần cứng hiện tại và thậm chí khung hình thời gian thực (realtime framerates) là có thể nếu thuật giải được tối ưu hóa.

Chú ý: cả 3 thành phần trong công thức tính cường độ có thể được tính toán từ điểm giao nhau và vectơ pháp tuyến trực chuẩn trên điểm đó.

Mã giả cho toàn bộ thuật giải như sau:

```
Color traceRay(Vector start, Vector direction, int depth){
    if (depth < 0) return BLACK;
    Object hitObject;
    Vector hitPoint;
    if (rayHit(start, direction, &hitObject, &hitPoint)) {
        Color result;
        result = calculateLocalContribution(hitObject, hitPoint);
        result += calculateReflection(hitObject, hitPoint,
            traceRay(hitPoint, reflectedDirection, depth - 1));
        result += calculateRefraction(hitObject, hitPoint,
            traceRay(hitPoint, refractedDirection, depth - 1));
        return result;
    }
    else
        return BACKGROUND_COLOR;
}

Image raytraceImage(Vector eye) {
```

```
Image image;
for(each scanline Y) {
    for(each pixel X) {
        image[X][Y] = traceRay(eye,directionFromEyeToXY, RECURSION_DEPTH);
    }
}
return image;
}
```

8.3.1.3. Màu gốc (Primitives)

Cảnh raytraced phải bao gồm những đối tượng (được gọi là những màu gốc) mà có thể tính toán được giao điểm với một tia và vectơ pháp tuyến trực chuẩn tại điểm đó. Mặt khác chúng yêu cầu một raytracer và bởi vậy nó dễ dàng được thêm vào những màu gốc-chỉ một trong số đó phải viết hàm để thực hiện nhiệm vụ này.

8.3.1.4. Màu gốc chính xác (Mathematical primitives)

Những đối tượng như những hình cầu và mặt phẳng thì dễ dàng để điều khiển trong một raytracer từ tia sáng – sự giao nhau và thông thường rất dễ để tính toán. Những điều này cũng có thể được tính toán trong một khoảng thời gian không thay đổi và với độ chính xác tùy ý (đây không phải là trường hợp cho màu gốc được mô tả bên dưới). Ở đây không đi vào chi tiết về tia sáng – tính toán cắt ngang (intersect), chúng có thể được tìm thấy trong [6] và việc thực hiện có thể được tìm thấy trong code.

8.3.1.5. Lưới đa giác (Polygon meshes)

Lưới đa giác rất hữu ích để mô hình hóa những đối tượng tùy chọn và rất thích hợp để hỗ trợ như việc vẽ lại màu gốc trong một tia vạch nếu muốn vẽ lại khung cảnh hiển thị đối tượng từ thế giới thực như là ô tô, cái bàn, máy bay hay ngôi nhà... - tất cả những đối tượng khó hoặc không thể mô hình hóa thực sự mà chỉ sử dụng một thuật toán đối tượng như là hình cầu, mặt phẳng... Trong trường hợp ở đây chỉ làm việc với những lưới đa giác nơi mà tất cả đa giác đều là tam giác. Đây là cách truyền thống để biểu diễn một lưới đa giác và như vậy cũng không bị quá hạn chế.

Với bất kỳ màu gốc nào trong một tia vạch tất cả chúng ta cần có thể vẽ lại nó là một cách để tính toán điểm giao nhau với một tia sáng đã cho và một bề mặt bình thường mà có điểm đó bên trong. Với những đối tượng toán học được miêu tả ở trên ta có thể nhanh chóng tính toán chính xác. Với một lưới đa giác có phần khó hơn và ta sẽ hướng vào 2 nhiệm vụ lần lượt ở bên dưới.

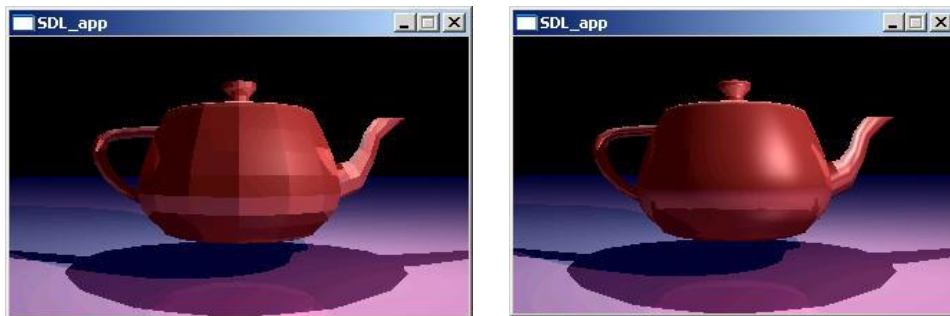
Vấn đề tính toán giao điểm giữa một lưới đa giác và một tia sáng có thể bị phá vỡ trong việc tính toán giao điểm giữa tia sáng và tất cả các hình tam giác mà tạo ra lưới. Cái gần nhất của những điều này là rất nhiều sự giao nhau là một cái mà chúng ta sử dụng điểm giao nhau giữa lưới và tia sáng.

Như vậy chúng ta gặp phải thách thức với việc tính toán giao điểm giữa một tam giác và một tia sáng. Những đặc trưng được làm trước bởi việc tính toán giao điểm giữa tia sáng và mặt phẳng mà có tam giác ở đó. Một điểm như vậy sẽ luôn tồn tại trừ phi tia sáng song song tuyệt đối với mặt phẳng tam giác. Nếu trường hợp chúng ta có thể quyết định ngay tam giác đó không được cắt ngang bởi tia sáng. Nếu tia sáng đó cắt mặt phẳng

tam giác thì chúng ta tiếp tục kiểm tra nếu điểm đó nằm trong hình tam giác. Có rất nhiều cách để làm việc này và giải thuật mà chúng ta chọn để sử dụng chi tiết về cơ bản nó tạo một nửa mặt phẳng từ mỗi hình tam giác 3 đoạn, tất cả hướng sao cho những điểm trực giao (normal) của chúng hoặc chỉ đi vào trong tam giác hoặc ra khỏi từ nó. Một điểm trong vấn đề là bên trong tam giác nếu nó hoặc nằm giữa mặt phẳng hoặc đằng sau tất cả 3 nửa mặt phẳng (phụ thuộc vào cách chọn phương hướng của nửa mặt phẳng).

Bây giờ chúng ta trở lại với nhiệm vụ tính toán trực giao như là một giao điểm. Ý tưởng hiển nhiên và trước hết là có thể đơn giản trả lại trực giao của mặt phẳng có tam giác nằm trên đó. Một hướng khác có thể phản đối rằng điều này sẽ thực sự là đúng trực giao của hầu hết những điểm bên trong hình tam giác đó. Tuy nhiên, lưới tam giác cũng thường được sử dụng để làm mềm hơn mô hình, uốn cong bề mặt và sau đó làm nhẵn bề ngoài của mặt được nhìn thấy của lưới tam giác. Nếu sử dụng những mặt phẳng tam giác trực giao như là giao cắt thông thường thì kết quả ta được giữa các cạnh với các hình tam giác đơn lẻ.

Giải pháp phổ biến nhất là tính toán cái gọi là đỉnh trực giao cho lưới. Điều này có nghĩa là mỗi tam giác sẽ có 2 trực giao liên quan đến nó – 1 cho mỗi đỉnh trong tam giác – và những hình tam giác đó sẽ chia sẻ 1 đỉnh cũng sẽ chia sẻ những đỉnh trực giao tương ứng. Đỉnh trực giao được tính toán bằng cách lấy trung bình của mặt phẳng trực giao cho những tam giác mà có đỉnh được sử dụng. Hiệu ứng này có nghĩa là đỉnh đó đặt trên 1 cạnh (mép) giữa 2 hình tam giác mà ở 1 góc nhất định đối với mỗi góc khác sẽ chỉ trong 1 phương có phần giữa mặt phẳng trực giao của tam giác và sẽ làm mềm ở ngoài ánh sáng ở mép. Trực giao cho 1 điểm đã cho bên trong hình tam giác được tính toán bởi phép nội suy tuyến tính giữa 3 đỉnh trực giao (xem thêm chi tiết cho giải thuật này trong code).



Hình 8.12 lưới đa giác

TỔNG KẾT: điểm giao nhau được tính toán bởi việc kiểm tra mỗi tam giác trong lưới cho sự giao nhau với tia sáng và sau đó sử dụng giao điểm gần nhất. Trực giao cho mỗi giao điểm được tính toán bởi phép nội suy giữa 3 đỉnh trực giao của hình tam giác.

8.3.2. Radiosity

8.3.2.1. Giới thiệu

Phương pháp Radiosity là một phần giải pháp làm sáng tỏ vấn đề toàn cầu, tập trung vào giải quyết sự phản xạ trực tiếp (interreflection) giữa những bề mặt khuếch tán bên trong một môi trường đóng. Phương pháp không gửi những vấn đề với khuếch tán – phản chiếu, phản chiếu – khuếch tán hoặc phản chiếu – phản xạ phản chiếu trực tiếp.

8.3.2.2. Tổng quan về phương pháp Radiosity (Overview of the radiosity method)

Một khái niệm chính của phương pháp radiosity là những miếng vá (patches), mà là những vùng hình chữ nhật được phát và nhận radiosity của những miếng vá khác, như vậy mô phỏng sự tương tác nhẹ.

Hình học của môi trường được chia ra thành những miếng vá. Những miếng vá, mà không phải là những nguồn sáng, thoát tiên cho phép cường độ là không, trong khi những miếng vá có nguồn sáng được đặt ở một lượng năng lượng ban đầu, phụ thuộc vào cường độ của nguồn sáng.

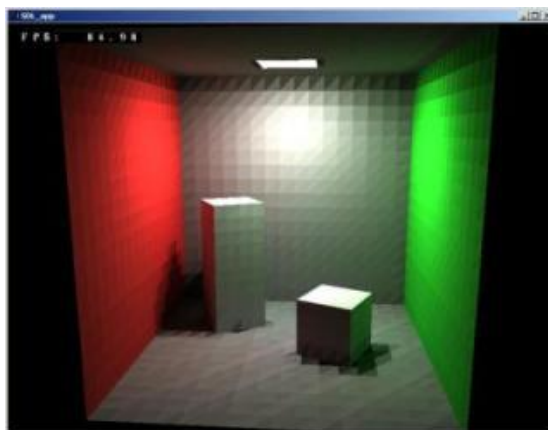
Mục đích là tính toán tổng lượng bức xạ của mỗi miếng vá nhận được từ môi trường, như đây là một ước lượng của cường độ ánh sáng phân tán. Điều này được làm lặp đi lặp lại (iteratively), bằng cách tính toán radiosity tại tất cả các miếng vá cho đến lượng năng lượng tối đa được tách ra dưới một ngưỡng nhất định. Điểm tận cùng được đảm bảo bởi thực tế, một lượng nhất định năng lượng tỏa ra được nhận lại thì lại được hút lại tạo mỗi tương tác.

Sự tương tác bức xạ giữa những miếng vá phải tính đến vài thứ: khoảng cách, góc và diện tích miếng vá, và xa hơn nữa bất kỳ sự can thiệp nào vào những miếng vá cũng mang lại những giải pháp.

Những vấn đề này được đặt ra bởi Form Factors. Form Factors miêu tả phần nhỏ của sự bức xạ tách khỏi một miếng vá và đến bất kỳ một miếng vá khác. Hiểu được Form Factors là trung tâm để hiểu về phương pháp radiosity

Phương pháp radiosity sản sinh một giải pháp nhìn độc lập, như tính toán cường độ những radiosity thì chỉ ảnh hưởng bởi cấu trúc hình học và vị trí của nguồn sáng.

Một cảnh từ việc thực thi, chú ý sự đổ màu trên chiếc hộp



Hình 8.13 phương pháp radiosity

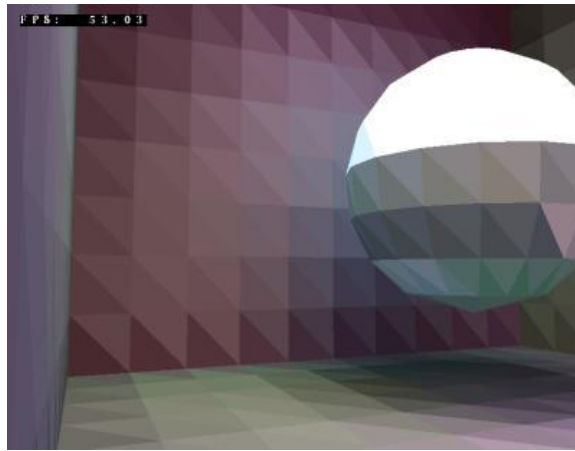
8.3.2.3. Một số giả thiết về những miếng vá (Some assumptions about Patches)

Những miếng vá là những hình chữ nhật, được giả thiết để có cùng radiosity trên mọi vùng của chúng. Đồng thời, những miếng vá được giả thiết là một bề mặt Lambertian khuếch tán hoàn hảo (Lambertian= mặt phẳng hoàn hảo), không chỉ có vậy các bề mặt phía dưới (subsurface) rải rác của ánh sáng đặt vào vị trí (khuếch tán hoàn hảo = phản chiếu radiosity đến ngang bằng trong mọi phương).

Những miếng vá được xây dựng dựa trên hình học của khung cảnh. Sự chính xác của giải pháp (**influcenced**) ảnh hưởng phần lớn bởi vật chất (granularity) của miếng vá, nhiều miếng vá mang lại (yield) một giải pháp tốt hơn, nhưng cũng tăng thêm thời gian tính toán.

Giải pháp cho sự chia nhỏ thích ứng của miếng vá cũng tồn tại. Những giải thuật này chia nhỏ miếng vá vào trong những miếng vá nhỏ hơn trong một khu vực mà sự thay đổi quyết định của radiosity, ví dụ trong những ranh giới bóng đổ. Cái này được so sánh là giải pháp tốt hơn với ít công việc thực hiện hơn, so sánh tới sự bố trí vật chất của tất cả khung cảnh cao hơn. Sự chia nhỏ sẽ không được mô tả ở đây.

Sự thực thi của chúng ta sử dụng những hình tam giác như những miếng vá, dù sự định nghĩa của chúng là hình chữ nhật:



Hình 8.14 Sử dụng hình tam giác như hình miếng vá

Radiosity của một miếng vá trong toàn bộ số lượng năng lượng phát ra từ miếng vá, mà cho bằng nhau của tổng số bức xạ nhận được và bức xạ phát ra.

Nhân tố mẫu(Form Factors)

Nhân tố mẫu F_{ij} của 2 vùng (miếng vá) A_i và A_j là phần nhỏ của sự bức xạ phát ra khỏi A_i và đạt đến A_j :

$$F_{ij} = \frac{\text{Radiative energy reaching } A_j \text{ from } A_i}{\text{Total radiative energy leaving } A_i \text{ in all directions}}$$

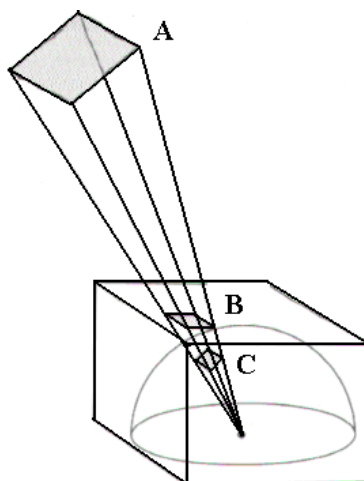
Nhân tố mẫu được tính toán từ sự tương tác bức xạ giữa những bề mặt rất nhỏ, mang lại toàn bộ phương trình:

$$F_{ij} = \frac{1}{A_i} \int_{A_i} \int_{A_j} \frac{\cos \phi_i \cos \phi_j}{\pi r^2} dA_j dA_i$$

Một điều nữa khá khó để tính toán phân tích cho hình học chung, bởi vậy cách tiếp cận khác để giải quyết phương trình ở trên là cần thiết. Một giải pháp sẽ sử dụng một hemicube.

Sử dụng hemicube trong xác định nhân tố mẫu, nhân tố mẫu hemicube là một hiệu quả nhưng giải pháp áng chừng để hình thành sự xác định nhân tố mẫu.

Sự biến hộ của hemicube thì dễ thấy từ bên dưới bức tranh.

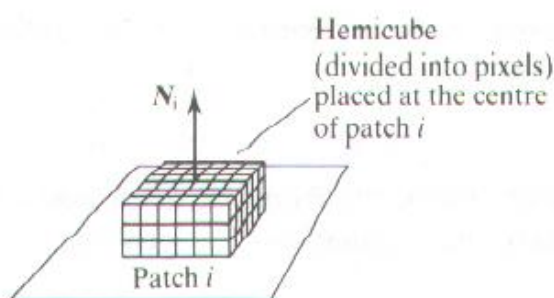


Hình 8.15 Miếng vá và sử dụng nhân tố mẫu hemicube

Miếng vá A và B có cùng khu vực khi chiếu tới bán cầu (cùng chiếu tới khu vực C).

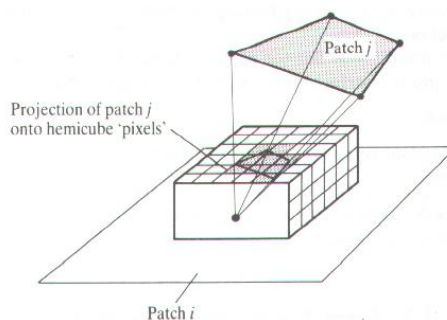
Nguyên do thứ 2 ở trên là sự hợp nhất qua một bán cầu. Việc tìm vùng trên một hình cầu là điều khó hơn so với chỉ đơn giản là tìm vùng của một hình tứ giác, từ đây là sử dụng một hemicube.

Hemicube được đặt đúng tâm của miếng vá, hướng đỉnh của hemicube trực giao với véc-tơ trực giao của miếng vá, có thể nhìn ở bên dưới.



Hình 8.16 Hemicube đặt ở tâm miếng vá mặt thì chia nhỏ

Năm mặt của hemicube được chia thành những mảnh nhỏ, giới hạn “pixels”. Để dò tìm nhân tố mẫu giữa 2 miếng vá P_i và P_j , trước tiên tất cả miếng vá được chiếu lên trên hemicube.

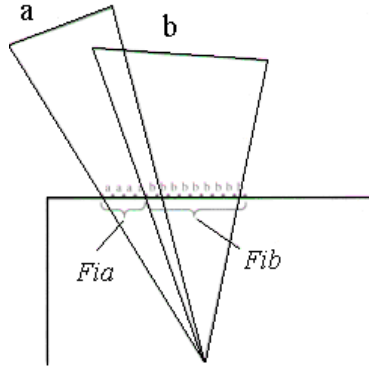


Hình 8.17 mô tả dò tìm nhân tố mẫu trong hemicube

Chương 8: Ánh sáng

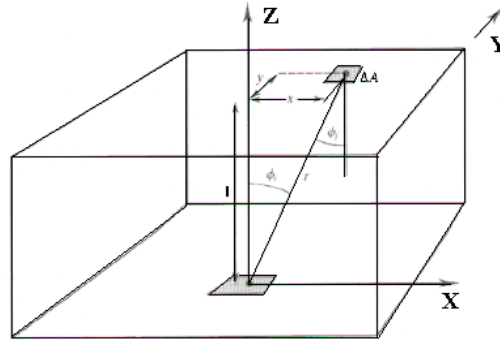
Sau đó nhân tố mẫu được tính toán bằng cách cộng những vùng của tất cả các pixels của miếng vá P_j được chạm tới. Nhân tố mẫu F_{ij} thì bằng vùng này.

Việc chiếu cũng giải quyết vấn đề với việc xen vào giữa miếng vá (giả thiết sử dụng một bộ đệm Z):



Hình 8.18 tính toán nhân tố mẫu

Vùng của mỗi điểm được gọi là “nhân tố mẫu Denta” được tính toán theo cách:



Hình 8.19 tính nhân tố mẫu Denta

Cho đỉnh của hemicube:

$$\Delta F_q = \frac{1}{\pi(x^2 + y^2 + 1)^2} \Delta A$$

Cho các mặt bên:

$$\Delta F_q = \frac{z}{\pi(y^2 + z^2 + 1)^2} \Delta A$$

Việc tính toán nhân tố mẫu được thực hiện cho mỗi miếng vá lần lượt. Chú ý rằng việc tính toán những nhân tố mẫu chỉ phụ thuộc vào hình học / những miếng vá, như vậy thay đổi trong màu sắc của ánh sáng nguồn và vị trí (tới những miếng vá khác) sẽ không yêu cầu 1 sự tính toán mới của những nhân tố mẫu (nhưng một giải pháp mới của một ma trận radiosity sẽ được tính toán, xem ở bên dưới).

Ghi chú thực hiện: một cách dễ để kiểm tra tính toán những nhân tố mẫu, mà tổng của tất cả nhân tố mẫu ở tại một miếng vá phải cân bằng với :

$$\sum_{k=1}^n F_{ik} = 1 \quad \text{for } i = 1 \text{ to } n$$

8.3.2.4. Thuật toán Radiosity (The Radiosity algorithm)

Tóm lại, radiosity B_i của miếng vá P_i là tổng của những radiosity phát ra và radiosity nhận được từ những miếng vá khác:

$$B_i dA_i = E_i dA_i + \rho_i \int_j B_j dA_j F_{dA_j dA_i}$$

$$B_i A_i = E_i A_i + \rho_i \sum_{j=1}^n B_j F_{ji} A_j$$

Từ khi :

$$F_{ij} A_i = F_{ji} A_j \iff F_{ij} = F_{ji} \frac{A_j}{A_i}$$

Đã cho ta những mối quan hệ radiosity cơ bản:

$$B_i = E_i + \rho_i \sum_{j=1}^n B_j F_{ij}$$

M có thể được biểu thị trong một ma trận sau:

$$\begin{bmatrix} 1 - \rho_1 F_{11} & -\rho_1 F_{12} & \dots & -\rho_1 F_{1n} \\ -\rho_2 F_{21} & 1 - \rho_2 F_{22} & \dots & -\rho_2 F_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ -\rho_n F_{n1} & -\rho_n F_{n2} & \dots & 1 - \rho_n F_{nn} \end{bmatrix} \begin{bmatrix} B_1 \\ B_2 \\ \vdots \\ B_n \end{bmatrix} = \begin{bmatrix} E_1 \\ E_2 \\ \vdots \\ E_n \end{bmatrix}$$

Giải pháp của ma trận phải được thực hiện cho mỗi bước sóng – trong trường hợp RGB nghĩa là giá trị màu đỏ, xanh lá cây và xanh lục. Giải pháp đạt đến được bởi việc giải quyết ma trận lặp đi lặp lại cho đến khi sự bức xạ tối đa luân phiên bởi 2 miếng vá ở bên dưới là một ngưỡng nhất định. Từ P là ít hơn một (mà có nghĩa là miếng vá hút một ít bức xạ), toàn bộ lượng bức xạ đang giảm bớt lặp đi lặp lại, bảo đảm sự kết thúc việc lặp đi lặp lại.

8.3.3. Photon Mapping

Trong phần này chúng ta sẽ miêu tả cách tiếp cận photon mapping. Phương pháp này có vài lợi thế so với kỹ thuật radiosity, trong khi nó không bị giảm mất chất lượng từ lưới giả (artifacts) và điều khiển cả khuếch tán – phản xạ khuếch tán và phản chiếu – phản xạ khuếch tán. Hơn nữa photon mapping cung cấp một giải pháp dễ dàng và tao nhã (elegant) để trả lại tự quang.

8.3.3.1. Giải thuật (The Algorithm)

Có 2 bước hay sự chuyển tiếp trong giải thuật:

- Xây dựng photon mapping sử dụng vạch dấu
- Tái sử dụng lại những photon mapping này

Bước đầu tiên: vạch dấu Photon (First Pass: The Photon Tracing)

Trong giai đoạn này, photon được bắn ra từ nguồn sáng sử dụng kỹ thuật raytracing chuẩn. Khi 1 photon chạm tới 1 đối tượng nó có thể cũng được phản chiếu, truyền hoặc hút. Việc gì sẽ xảy ra được quyết định bởi xác suất và tham số của bề mặt vật chất. Kỹ

thuật đã được sử dụng để quyết định này được gọi là *Russian Roulette*. Ý tưởng cơ bản là không phải tất cả photon đều cần phải được cho phép tiếp tục (survive) khi chạm 1 vật chất, một khi kết quả này trong 1 số mũ của số photon được lưu trữ, và có lẽ phép đệ quy vô hạn. Thay vào đó là phản chiếu photon khác, vận chuyển hoặc hút. Cách này theo lý thuyết xác suất thì tất cả các photon sẽ dần dần được hút.

Những photon được lưu giữ trong bản đồ photon thì chỉ được lưu trữ khi chúng được phản chiếu. Việc lưu giữ những photon trên sự phản xạ phản chiếu không cho bất kỳ thông tin hữu ích nào, trong khi những điều đó được trả lại khá tốt sử dụng kỹ thuật raytracing chuẩn. Một photon có thể được lưu giữ vài lần dọc theo đường dẫn của nó, và mỗi lần 1 photon được lưu vị trí, năng lượng photon đi vào và điều khiển vào được cất giữ. Cấu trúc dữ liệu cho việc cất giữ photon sẽ trông như sau:

```
typedef struct Photon {
    Vector pos;
    Vector power;
    Vector dir;
    short flag;
};
```

Jensen gợi ý một cấu trúc dữ liệu gọn hơn nơi anh ta đóng gói năng lượng và phương hướng vào và ít byte hơn. Trong sự thực hiện, chúng ta được chọn để tập trung vào sự rõ ràng của mã thay vì kích cỡ và tốc độ. Tuy nhiên với 1 số lượng rất lớn photon chúng ta gặp vài vấn đề hiệu năng bởi việc tải bộ nhớ rất nặng. Cấu trúc của photon sử dụng 38 byte ngược lại với 20 bytes mà Jensen gợi ý trong tài liệu của anh ấy. Biên cò được sử dụng trong KD – sự thực hiện cây.

Chuẩn bị Photon mapping cho sự chuyển trở lại (Preparing the photonmap for the rendering pass). Trong thời gian trả lại, chúng ta cần định vị những photon đóng tới 1 điểm tùy ý trong không gian 3D nhiều thời gian, vài cấu trúc dữ liệu cho phép tìm nhanh hơn ở những photon liền kề, và cũng điều khiển tốt những những phân phối không đồng dạng. Về điều này, Jensen gợi ý 1 cây kd cân bằng. Điều này cho thời gian tìm kiếm tốt hơn và cân bằng của cây kd có thể thực hiện trong thời gian $O(n \log(n))$ nơi mà n là 1 số photon trong photon mapping. Sự cân bằng của cây kd được thực hiện bởi việc tìm kiếm điểm giữa của trục lớn nhất và chọn đó như là một đường chia (splitplane). Mã giải để xây dựng một cây kd cân bằng:

```
balance(photons P) {
    find the bounding box of P
    choose the largest axis A as splitplane
    find the median of P in axis A
    divide the pointsets into S1 and S2 according to median
    balance(S1)
    balance(S2)
}
```

Như được tìm thấy trong những hình ảnh chúng ta thử các kiểu khác nhau của nguồn sáng, chỉ với điểm nguồn sáng xuyên qua. Tuy nhiên một đặc tính mạnh mẽ khác tìm được bởi việc sử dụng photon mapping, có thể về cơ bản dù mẫu nào hoặc hình dạng nào của nguồn sáng mà ta muốn. Từ thứ duy nhất cần thay đổi là phương hướng và nguồn

gốc thực tế của lightrays. Chú ý làm sao để 2 ảnh dưới đây sử dụng những mô hình phân phối khác nhau, và hiệu ứng đó có trên kết quả cuối cùng. Đầu tiên một cái được bắn tại tất cả các phương, nơi mà ngay lập tức đưa nhiều phương sáng theo hướng của phòng. Sau đó khi photon mapping được làm cho trực quan sử dụng một sự chiếu sáng đánh giá những sự khác nhau này trở nên thậm chí còn sáng hơn nữa.

Bước 2: trả lại bản đồ photon (Second Pass: Rendering The Photon Map)

Bây giờ với sự xây dựng photon mapping và khả năng nhanh chóng xem những photon trong đó, chúng ta sẵn sàng để bắt đầu trả lại khung cảnh thực tế.

8.3.3.2. Đánh giá chiếu sáng (The Radiance Estimate)

Chúng ta nhìn thấy photon mapping như sự trình bày của dòng đến. Để tính toán sự chiếu sáng cho một điểm x đã cho trong khung cảnh, ta cần sự toàn vẹn thông tin. Để làm điều này ta sử dụng biểu thức cho sự phản chiếu:

$$L_r(x, \vec{\omega}) = \int_{\Omega_x} f_r(x, \vec{\omega}', \vec{\omega}) L_i(x, \vec{\omega}') |\vec{n}_x \cdot \vec{\omega}'| d\omega'_i$$

L_r là sự phản chiếu tại x trong phương w . Ω_x là hình cầu của phương đến, f_r là BRDF. Ta có thể xấp xỉ được dòng đến sử dụng photon mapping của ta bằng cách định vị n photon gần nhất đến x . Khi giả thiết rằng mỗi photon p cắt ngang bề mặt tại x . $\Delta\Phi_p$ là năng lượng của photon p . Chúng ta có đánh giá sau:

$$L_r(x, \vec{\omega}) \approx \sum_{p=1}^n f_r(x, \vec{\omega}_p, \vec{\omega}) \frac{\Delta\Phi_p(x, \vec{\omega}_p)}{\Delta A}$$

Trong thực tế ta định nghĩa 1 bán kính tìm kiếm r và nhìn vào khu lân cận vòng quanh x . Trong khu lân cận này ta sử dụng n photon gần nhất để đánh giá sự chiếu sáng. Khi giả thiết rằng bề mặt xung quanh x là mặt phẳng cục bộ ta có thể tính toán diện tích này dựa trên việc chiếu hình cầu lên trên bề mặt và sử dụng vùng của những vòng tròn kết quả để đánh giá vùng:

$$\Delta A = \pi r^2$$

Như vậy việc có sự chiếu sáng sau sự đánh giá xấp xỉ:

$$L_r(x, \vec{\omega}) \approx \frac{1}{\pi r^2} \sum_{p=1}^N f_r(x, \vec{\omega}_p, \vec{\omega}) \Delta\Phi_p(x, \vec{\omega}_p)$$

Đánh giá này dựa trên nhiều giả thiết và sự chính xác phụ thuộc nhiều vào số những photon sử dụng và bán kính của sự tìm kiếm. Hơn nữa có thể bao gồm những photon sai trong góc của một hộp cho ví dụ. Sự chính xác có thể giảm bớt bởi việc sử dụng một đĩa thay vì một hình cầu. Từ việc sử dụng một hình cầu sẽ hiển nhiên có lợi thế tính toán hơn (ví dụ tính toán việc chiếu một vùng và khoảng cách dự kiến là một hàm dễ) ta đã không theo đuổi cách tiếp cận này trong việc thực hiện và những hiện tượng giả cạnh nhất định bởi vậy có thể cần phải chú ý.

8.3.3.3. Lọc (Filtering)

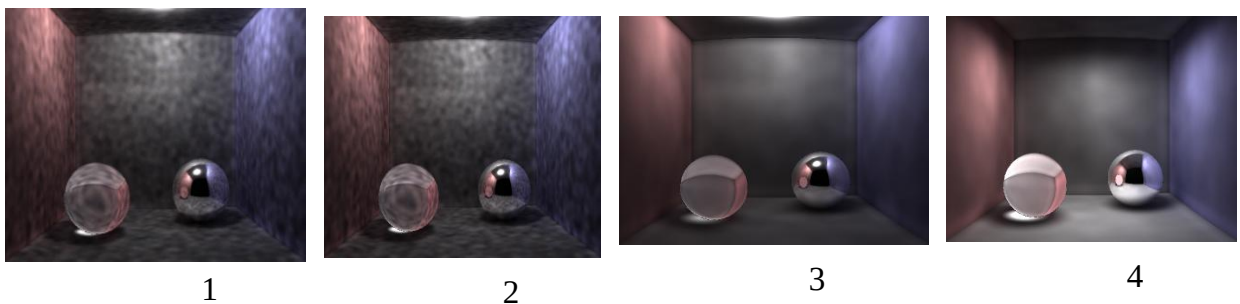
Để xúc tiến việc giảm bớt những photon rò rỉ vào trong những vùng và sự chiếu sáng blurry, việc lọc có thể sử dụng để giảm bớt những hiện tượng giả. Ý tưởng của việc lọc là làm những photon tới gần điểm x quan trọng hơn là ra khỏi. Ta đã thử lọc hình nón và lọc gaussian để làm giảm nhiễu và kém chính xác. Nhưng từ đó chúng ta đã bị giới hạn kiểm tra trên những tụ quang mà không được chú ý.

a. Định vị photon (Locating the Photons)

Việc định vị những photon được làm bằng cách tìm xuyên qua cây KD và định vị n điểm gần nhất tới x. Ta thực hiện một giải thuật tìm kiếm những photon láng giềng chung. Nó bắt đầu tại nút gốc của cây KD và sau đó thêm photon vào một danh sách nếu chúng ở trong một tầm nhất định. Cho n photon gần nhất trong danh sách được sắp xếp, sao cho photon xa nhất được xóa và thêm một photon tới gần nếu danh sách đã chứa n photon. Để đạt được điều này chúng ta sử dụng một hàng đợi có quyền ưu tiên để giữ phần thông tin tìm kiếm photon.

Cho giải thuật tìm kiếm này chúng ta cần một bán kính tìm kiếm ban đầu. Trong quá trình thực hiện bán kính này được thao tác bằng tay cho mỗi khung cảnh riêng biệt. Một điều rất quan trọng là chọn được bán kính tốt, nếu nó quá lớn thì việc tìm kiếm sẽ rất chậm, và nếu nó nhỏ quá thì không đủ số photon tìm thấy, nên cần một mức độ điều chỉnh. Ta cũng cố để thực hiện việc xác định bán kính tìm kiếm, khởi động với một bán kính nhỏ và tăng dần nó nếu số photon tìm thấy vẫn chưa đủ. Chúng ta không có bất cứ sự cải tiến nào do điều này, bởi việc tìm kiếm hai hoặc ba lần là quá lớn.

Dưới đây ta có thể nhìn thấy một vật lai (hybrid) của góc hộp trả lại với 20000 photon, nhưng với bán kính tìm kiếm là 5 trên 2 lần đầu và 15 trên 2 lần cuối. Ta cũng có thể nhìn thấy sự khác nhau sử dụng phân phối xác suất khác nhau của những photon mô tả trong phần đầu. Cột đầu tiên là phân phối bình thường, cột thứ 2 là phân phối xuống phía dưới GAUSS.



Hình 8.20 Giải thuật tìm kiếm dùng định vị photon

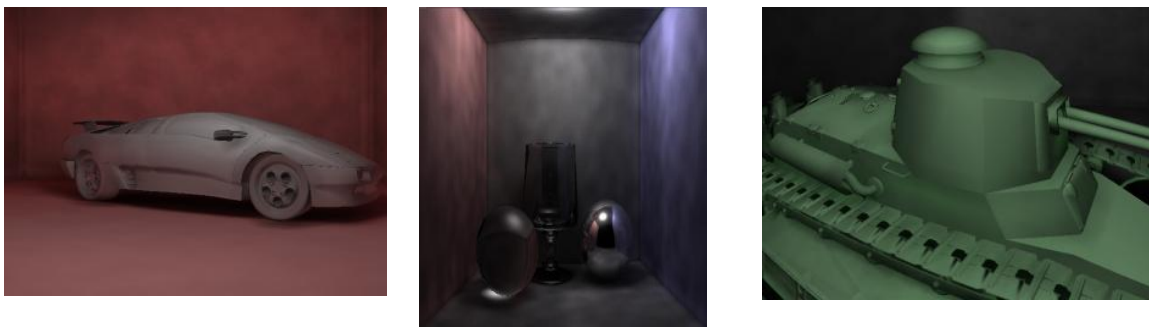
b. Tô trát (Rendering)

Một lần ánh xạ photon đã được phát sinh theo vài cách để trực quan hơn. Photon mapping là một cái nhìn độc lập, nên nó có thể sử dụng từ góc nhìn nào đó trong khung cảnh. Một cách làm trực quan photon mapping rất nhanh là tính toán giá trị chiếu cho mỗi đỉnh trong khung cảnh và sau đó sử dụng sự chiếu sáng đỉnh được hỗ trợ bởi đa số phần cứng 3D trả lại cảnh cho bất kỳ góc nhìn nào. Một cách khác, hãy trực tiếp tạo nên sự trực

quan cho ánh xạ. Mà đơn giản cho mỗi pixel theo vết của một tia xuyên qua khung cảnh và tính toán đánh giá sự chiếu sáng đến bề mặt điểm. Đây là cách tiếp cận mà chúng ta sử dụng. Trong tài liệu Jensen gợi ý trả lại hình ảnh cuối cùng sử dụng phân tán raytracing, nơi mà sự chiếu sáng được đánh giá trung bình một số đánh giá mẫu. Mỗi mẫu gồm vết của một tia từ mắt xuyên qua điểm tới khung cảnh. Sự chiếu sáng trở lại bởi mỗi tia, cân bằng với sự chiếu sáng đang rời khỏi theo hướng tia sáng để cho điểm giao nhau tại bề mặt bị cắt ngang đầu tiên bởi tia sáng.

Công thức quá trình chiếu sáng có thể chia ra thành từng phần vào trong 4 phần sử dụng kiến thức của ta về BRDF. Định hướng chiếu sáng, phản xạ bóng loáng và lấp lánh, tụ quang (caustics) và sự phản xạ đa khuếch tán. Việc làm trực quan hơn photon mapping trực tiếp sử dụng raytracing mẫu mang lại kết quả tốt, nhưng yêu cầu nhiều photon để đạt được kết quả tốt. Đặc biệt tụ quang thì khó để làm đúng mức nếu không có một photon mapping chuyên dụng nào để điều khiển tụ quang. Trừ phi toàn bộ những kết quả không tốt như một trong số đó. Đây phần lớn là cách tiếp cận thiếu một raytracing phân tán thu nhất. Tuy nhiên việc sử dụng một cách tiếp cận trực quan hóa trực tiếp làm cho nó dễ chuyển từ photon mapping vào trong đồ họa thời gian thực, nơi mà photon mapping có thể sử dụng để trả lại kết cấu cho bản đồ cũng được gọi là lightmap, như vậy việc làm cho nó có thể cho thời gian thực sử dụng với việc sử dụng một bước tiền xử lý. Để có thêm chi tiết hãy xem Radiosity Using Photon Maps

Dưới đây là một vài bức ảnh được tô trát:



Hình 8.21 các bức ảnh được tô trát

8.4. SỰ SO SÁNH GIỮA CÁC KỸ THUẬT (COMPARISON OF TECHNIQUES)

Trong phần này ta sẽ đánh giá 3 thuật giải và chú ý đến năm khía cạnh sau:

- Tốc độ - lượng thời gian (bao gồm cả quá trình tiền xử lý) yêu cầu để trả lại một hình ảnh.
- Chất lượng ảnh – cách tạo ra hình ảnh, tùy chọn theo yêu cầu – xử lý tham số được yêu cầu để đạt được kết quả mong muốn.
- Sự hạn chế hình học – kỹ thuật có thể điều khiển những cảnh chuyên biệt hoặc có hạn chế. Có thể sẽ tốt hơn ở một vài khung cảnh so với những cảnh khác.
- Sự thực hiện khó khăn – mất nhiều công sức để thực hiện giải thuật.

8.4.1. Raytracing

Raytracing là một giải thuật khá nhanh. Thời gian trả lại thường nằm trong đơn vị giây thậm chí cho những khung cảnh khó với một sự thực hiện cẩn thận.

Những hình ảnh sản sinh thì ấn tượng nhưng lại rất xa thực tế và nó thường được nhìn thoáng qua rằng hình ảnh đó được tạo ra bởi một raytracer. Từ tốt nhất để mô tả hình ảnh “siêu thực”. Những phản xạ xuất hiện với độ sắc không giảm và hình bóng thì rất sắc. Raytracing bởi vậy cho kết quả rất thực tế trên khung cảnh nơi mà những đặc tính này thì rất nổi bật như là khung cảnh với nhiều kính, gương và kim loại bóng. Mặt khác khung cảnh với đối tượng “khuếch tán” giống như thảm và nguồn sáng bề mặt hoặc cảnh ngoài trời không thể được trả lại chân thực với một raytracer cơ bản. Kết luận là sự tương tác khuếch tán ánh sáng thì nổi bật trong thực tế nhưng nó chưa được sử dụng đúng mức vào trong một raytracer.

Có rất ít các yêu cầu của khách hàng và không có sự hạn chế về hình học. Hơn nữa việc thực hiện giải thuật thì khá dễ.

8.4.2. Radiosity

Radiosity thì khá chậm so với raytracing nhưng từ giải thuật không tính đến sự tương tác phản chiếu với giải pháp trả lại cảnh quan độc lập. Giải pháp có thể được sử dụng bởi một bộ phần cứng tăng tốc trả lại tiêu chuẩn, ví dụ: toàn bộ radiosity tính toán có thể là tốn kém với bước tiến xử lý sau đó sự tương tác tỷ lệ khung hình (framerates) là có thể.

Chất lượng hình ảnh của những radiosity phát sinh có thể là rất tốt cho khung cảnh, mà không có sự tương tác bóng lách nào nhưng những khung cảnh như vậy thì không dễ tìm thấy trong thực tế (nhưng dễ hơn để tìm khung cảnh chỉ với sự tương tác bóng lách cho cái mà raytracing tốt nhất). Từ khi sự tương tác bóng lách được hoàn toàn bỏ đi bởi thuật giải, khung cảnh mà chứa đựng điều này thì không thể trả lại khung cảnh hiện thực.

Vài yêu cầu của khách hàng về một khung cảnh là cần thiết để đạt được những kết quả mong muốn. Đặc biệt kích cỡ của miếng vá và năng lượng của ánh sáng có thể khó đánh giá sớm được.

Giải thuật sử dụng miếng vá hình chữ nhật xấp xỉ hình học khi tính toán giải pháp và từ sự phức tạp của số miếng vá hình vuông đó cũng không quá nhiều. Điều này có nghĩa là muốn uốn cong những bề mặt như một hình cầu toán học có thể chỉ xấp xỉ tới độ nào đó hoặc sự tính toán sẽ rất lâu. Sự phức tạp của khung cảnh không thể quá cao cho cùng một lý do.

Sự thực hiện giải thuật có thể sử dụng lượng thời gian khá lớn chủ yếu vì ở đó có một số bước quan trọng.

8.4.3. Photon mapping

Photon mapping có thể nhanh hơn raytracing nếu chỉ có vài photon được sử dụng. Tuy nhiên trong khi cả chất lượng hình ảnh và thời gian trả lại tăng lên theo số lượng photon được sử dụng có lẽ nó sẽ có phần chậm hơn raytracer khi sử dụng trong thực tế.

Từ khi giải pháp photon mapping thường được sử dụng để xây dựng phía trên của 1 raytracer bình thường thì nó có thể sản sinh tất cả các hình ảnh đó như một raytracer bình

thường. Hiệu năng của phương pháp photon mapping là nó có khả năng mô hình hóa rất nhiều hiện tượng-vật kỳ lạ (phenomeanas) trong thế giới thực mà đây là điều không thể đối với một mình raytracing (như là màu máu tươi, tụ quang, hình bóng mềm ...). Hình ảnh sẽ rất tốt và hiện thực khi có đủ số photon được sử dụng.

Xuất hiện vấn đề làm sao sử dụng để cho một kết quả tốt yêu cầu rất nhiều sự ngắt quãng, và như vậy nó không dễ để trả lại hình ảnh hoàn hảo ở lần thử đầu tiên. Tức là photon mapping không đặt những sự hạn chế về khung cảnh, photon có thể chứa bất kỳ loại hình học nào miễn là điểm giao nhau có thể được tính toán và sự thực hiện của phương pháp khá tiên tiến mới lạ. Phần khó nhất là ngắt quãng nó vào trong việc sản sinh một kết quả có sức thuyết phục.

Tóm tắt:

Ánh sáng là phần rất quan trọng trong đồ họa máy tính. Trong chương này đi sâu vào xét các giải thuật đồ họa ánh sáng, cụ thể là các giải thuật chiếu sáng toàn cục: Raytracing, Radiosity và Photo mapping.

Mỗi một thuật toán đều xét đặc điểm, trình bày các bước và các công thức trong giải thuật, cuối cùng đánh giá ưu nhược điểm của từng giải thuật.

Kết thúc chương là so sánh ba giải thuật được đưa ra.

Chương 9: Thư viện đồ họa OpenGL

9.1. Khái niệm về thư viện OpenGL

9.1.1. Giới thiệu OpenGL

OpenGL (Open Graphics Library) là phần mềm giao diện với các phần cứng đồ họa được phát triển bởi Silicon Graphic Inc. (SGI). OpenGL còn được hiểu như là một hệ giao tiếp lập trình ứng dụng (application program interface- API) bao gồm khoảng 250 câu lệnh được hỗ trợ bởi nhiều ngôn ngữ như C, C++, Java...cho phép người lập trình sử dụng tạo ra các đối tượng đồ họa. OpenGL được thiết kế không phụ thuộc vào nền tảng phần cứng cũng như hệ điều hành máy tính (independence of hardware platform and operating system). Với OpenGL ta sẽ tạo ra các mô hình từ các đối tượng hình học cơ bản đó là điểm (point), đường (line) và đa giác (polygon). Cú pháp lệnh của OpenGL: Các câu lệnh của OpenGL đều sử dụng tiền tố `gl` và các từ tiếp theo được bắt đầu bằng kí tự hoa, ví dụ `glClearColor()`. Các hằng được định nghĩa bằng tiền tố `GL_` tiếp theo là các từ viết hoa được ngăn cách bằng kí tự gạch dưới, ví dụ `GL_COLOR_BUFFER_BIT`.

Các thư viện liên quan của OpenGL: Mặc dù OpenGL là một công cụ mạnh nhưng các đối tượng vẽ đều là các đối tượng hình học cơ bản. Để đơn giản hóa một số công việc, chúng ta được trang bị thêm một số thư viện cho phép sử dụng các thủ tục vẽ ở mức cao hơn:

- OpenGL Utility Library (GLU): Bao gồm một số thủ tục thiết lập ma trận xác định hướng nhìn (viewing orientation), ma trận các phép chiếu (projection), và biểu diễn các mặt trong không gian 3 chiều (rendering surfaces)

- OpenGL Utility Toolkit (GLUT): Là một bộ công cụ được viết bởi Mark Kilgard bao gồm các thủ tục giúp cho đơn giản hóa việc xây dựng các đối tượng hình học. Các thủ tục của GLUT được bắt đầu bằng tiền tố `glut`.

9.1.2. Cài đặt OpenGL trong Visual C++

Nếu chưa cài đặt Visual Studio trên máy tính chúng ta thực hiện 2 bước sau:

- Tải tập tin cài đặt Visual Studio C++ từ:

<http://www.microsoft.com/express/Downloads/>

- Thực thi tập tin cài đặt và làm theo hướng dẫn để cài đặt Visual Studio C++.
- Cài đặt thư viện GLUT:

Tải tập tin "glut-3.7.6-bin-32and64.zip" từ:

<http://www.mediafire.com/?12ncmty5g6mgr7t>

Giải nén tập tin glut-3.7.6-bin-32and64.zip ra thư mục glut-3.7.6-bin

Trong thư mục glut-3.7.6-bin:

- Với phiên bản Windows 32bit (viết ứng dụng 32 bit)

Sao chép tập tin `glut.h` và `glut.def` vào thư mục `C:\Program Files\Microsoft SDKs\Windows\v7.0A\Include\gl`

Sao chép tập tin glut32.lib vào thư mục C:\Program Files\Microsoft SDKs\Windows\v7.0A\Lib

Sao chép tập tin glut32.dll vào thư mục C:\windows\system32\

- Với phiên bản Windows 64bit (viết ứng dụng 32 bit và 64 bit)

Sao chép tập tin glut.h và glut.def vào thư mục C:\Program Files(x86)\Microsoft SDKs\Windows\v7.0A\Include\gl

Sao chép glut32.lib và glut64.lib vào thư mục C:\Program Files(x86)\Microsoft SDKs\Windows\v7.0A\Lib

Sao chép glut32.lib và glut64.lib vào thư mục C:\Program Files(x86)\Microsoft SDKs\Windows\v7.0A\Lib\x64

Sao chép tập tin glut32.dll và glut64.dll vào thư mục C:\windows\SysWOW64

Sao chép tập tin glut32.dll và glut64.dll vào thư mục C:\windows\system32

- Cài đặt GLEW

Tải tập tin "Windows 32-bit and 64-bit Binaries" từ

<http://glew.sourceforge.net/>

Giải nén tập tin glew-1.10.0-win32.zip vào thư mục có tên glew

- Với phiên bản Windows 32bit

Sao chép tất cả tập tin trong thư mục include/GL vào thư mục C:\Program Files\Microsoft SDKs\Windows\v7.0A\Include\gl

Sao chép tập tin glew32.lib trong thư mục lib/Release/Win32/ vào thư mục C:\Program Files\Microsoft SDKs\Windows\v7.0A\Lib

Sao chép tập tin glew32.dll trong thư mục bin/Release/Win32/ vào thư mục C:\windows\system32

- Với phiên bản Windows 64bit o Sao chép tất cả tập tin trong thư mục include/GL vào thư mục C:\Program Files(x86)\Microsoft SDKs\Windows\v7.0A\Include\gl

Sao chép tập tin glew32.lib trong thư mục lib/Release/x64/ vào thư mục C:\Program Files(x86)\Microsoft SDKs\Windows\v7.0A\Lib

Sao chép tập tin glew32.dll trong thư mục bin/Release/x64/ vào thư mục C:\windows\SysWOW64

9.2. Vẽ hình

9.2.1. Cấu trúc lệnh OpenGL và các kiểu dữ liệu trong OpenGL

OpenGL sử dụng tiền tố gl và tiếp theo đó là những từ được viết hoa ở chữ cái đầu để tạo nên tên của một lệnh. Ví dụ : glColor2f() - Trong đó gl–Tiền tố Color – hàm cơ bản chỉ màu sắc 2–Số lượng (hai chiều). Tức có hai tham số trong glColor2f(x, y) f – Hậu tố chỉ kiểu dữ liệu và (x, y có kiểu float).

Bảng 9.1. Các tiền tố OpenGL

Thư viện	Bộ hạt nhân	Utility	Auxiliary	WGL	Win32 API
Tiền tố	Gl	Glu	Aux	Wgl	Không có tiền tố đặc biệt

Bảng 9.2. Các hậu tố OpenGL

Hậu tố	Kiểu dữ liệu của đối số
B	Glbyte
B	GLdouble hay Glclampd
F	GLfloat hay Glclampf
I	GLint hay Glsizei
S	Glshort
Ub	GLubyte hay Glboolean
Ui	GLuint, GLnum hay Glbitfield
Us	Glushort
Bv	GLbyte hay vector
Dv	GLdouble hay Glclampd dạng vector
Fv	GLfloat hay Glclampf dạng vector
Iv	GLint hay Glsizei dạng vector
Sv	Glshort dạng vector
Ubv	GLubyte hay Glboolean dạng vector
Uiv	GLuint, GLenum hay Glbitfield dạng vector
Usv	Glushort dạng vector

OpenGL định nghĩa kiểu dữ liệu để dễ dàng thao tác trong chương trình.

Bảng 9.3. Các kiểu dữ liệu trong OpenGL

Kiểu dữ liệu trong OpenGL	Kiểu dữ liệu tương ứng trong C	Ý nghĩa
Glbyte	Signed char	Số nguyên 8 bit
Glshort	Short	Số nguyên 16 bit
GLint, Glsizei	Long	Số nguyên 32 bit
GLfloat, Glclampd	Float	Dấu chấm động 32 bit
GLdouble, Glclampf	Double	Dấu chấm động 64 bit
GLubyte, Glboolean	Unsigned char	Char không dấu 8 bit
Glushort	Unsigned short	Số nguyên không dấu 16 bit
GLuint, GLenum, Glbitfield	Unsigned long	Số nguyên không dấu 32 bit

Glvoid	Void	Hàm không đối số hoặc không trả về giá trị
HGLRC	HGDI OBJ	Handle của đối tượng GDI

Ví dụ khai báo biến trong OpenGL :

```
GLint x;
```

```
GLfloat y;
```

9.2.2. Chương trình OpenGL tối thiểu

1. Mở chương trình Visual Studio C++.
2. Nhấn vào mục Create new project trong Visual Studio
3. Lựa chọn Win32 Console Application, nhập tên của project và nhấn nút OK
4. Trong cửa sổ wizard hiện ra sau đó, nhấn vào mục Application Settings và chọn mục Empty Project, sau đó nhấn nút Finish.
5. Lựa chọn Add New Item từ Project menu, sau đó chọn loại tập tin cpp, nhập tên tập tin và nhấn OK.
6. Nội dung trong tập tin mã nguồn vừa thêm như phần dưới
7. Nhấn chuột phải vào Project và chọn Properties
8. Ở cửa sổ vừa hiện ra, ở trên cùng, đổi Configuration thành All Configurations.
9. Ở phía bên cột bên trái, chọn Linker Input
10. Khi đó ở phía bên phải sẽ hiện ra danh mục, trong đó có mục Additonal Dependencies, thêm ";glut32.lib" vào sau mục này, nếu có sử dụng glew, cũng thêm cả ";glew32.lib" vào sau đó.
11. Nhấn nút OK và chọn Build Solution từ Build menu.
12. Thực thi chương trình bằng cách nhấn tổ hợp phím Ctrl+F5.

Nếu cài đặt đúng chính xác tất cả các bước, chương trình khi thực thi sẽ có kết quả như sau

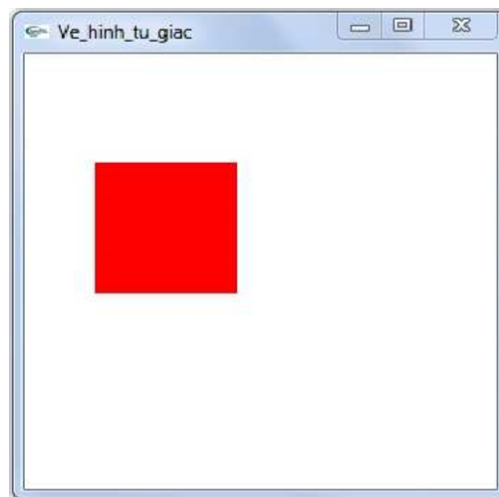
Chương trình OpenGL tối thiểu

Đoạn mã sau sử dụng các hàm OpenGL để vẽ một tứ giác màu đỏ

```
//Khai báo thư viện đồ họa glut.h
#include <gl/glut.h>
void ve_hinh_tu_giac(){
    //Thiết lập màu nền là màu trắng
    glClearColor(1.0f, 1.0f, 1.0f, 0.0f);
    //Xóa bộ đệm màu
    glClear(GL_COLOR_BUFFER_BIT);
    //Định nghĩa màu vẽ là màu đỏ
    glColor(1.0f, 0.0f, 0.0f)
    //Định nghĩa hình vẽ là hình tứ giác
```

```
glBegin(GL_QUADS);  
//Định nghĩa tọa độ các đỉnh tứ giác  
    glVertex2f(-0.7f, -0.1f);  
    glVertex2f(-0.1f, -0.1f);  
    glVertex2f(-0.1f, 0.5f);  
    glVertex2f(-0.7f, 0.5f);  
//Kết thúc vẽ tứ giác  
glEnd()  
//Bảo đảm các lệnh trên được thực hiện  
glFlush()  
}  
  
int main(int argc, char** argv){  
    glutInit(&argc, argv);  
    //Tạo cửa sổ với tên ve_hinh_tu_giac  
    glutCreateWindow("Ve_hinh_tu_giac");  
    //Thực hiện hàm ve_hinh_tu_giac()  
    glutDisplayFunc(ve_hinh_tu_giac);  
    glutMainLoop();  
    return 0;  
}
```

Kết quả xuất ra màn hình như sau:



Hình 9.1. Ví dụ một chương trình OpenGL

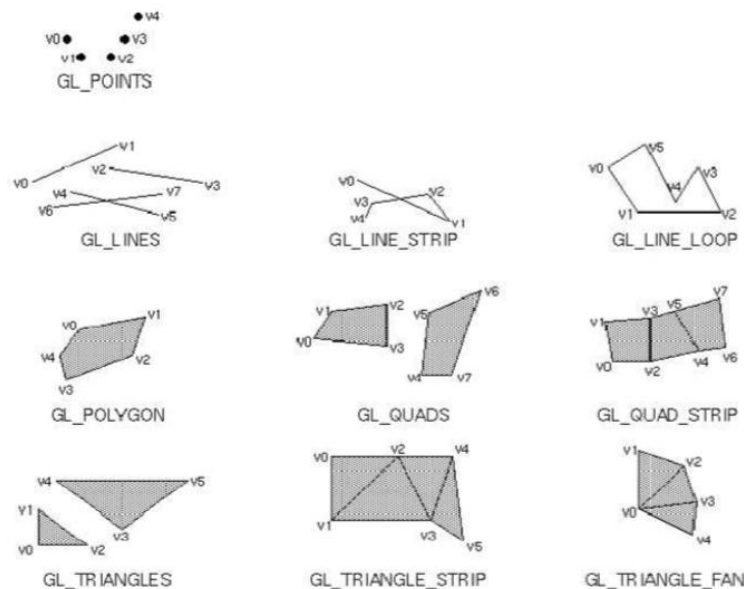
9.2.3. Vẽ hình

OpenGL định nghĩa vẽ một hình như sau:

```
glBegin(tham số);  
    .....  
glEnd();
```

Bảng 9.4. Giá trị các tham số định nghĩa hình vẽ của glBegin(tham số)

GL_POINTS	Các điểm
GL_LINES	Đoạn thẳng
GL_POLYGON	Đa giác lồi
GL_TRIANGLES	Tam giác
GL_QUADS	Tứ giác
GL_LINES_STRIP	Đường gấp khúc không khép kín
GL_LINE_LOOP	Đường gấp khúc khép kín
GL_TRIANGLE_STRIP	Một dải các tam giác liên kết với nhau
GL_TRIANGLE_FAN	Một dải các tam giác liên kết theo hình quạt
GL_QUAD_STRIP	Một dải các tứ giác liên kết với nhau

**Hình 9.2. Các đối tượng hình học cơ bản**

Dựa vào các hàm vẽ hình cơ bản mà OpenGL cung cấp chúng ta có thể vẽ được hầu hết các hình phức tạp. Để vẽ được hình chúng ta phải sử dụng các điểm vẽ, để chỉ định 1 điểm ta dùng lệnh sau:

```
glVertex{234}{sifd}[v] (tọa độ điểm);
```

Trong đó:

-{234} : chỉ định số chiều của không gian.

-{sifd} : chỉ định kiểu dữ liệu của tọa độ(Glshort, Glint hoặc Glsizei, Gfloat hoặc Glclampf, Gldouble hoặc Glclampd).

-[v] : nếu tọa độ được truyền vào từ một mảng cho trước

Ví dụ:

```
//Tọa độ thuộc kiểu Glshort trong không gian 2 chiều
glVertex2s(2, 3);
```

```
//Tọa độ kiểu Gldouble trong không gian 3 chiều  
glVertex3d(0.0, 0.0, 1.2);
```

Vẽ điểm

Vẽ cùng lúc 3 điểm trên hệ tọa độ 2D

```
glBegin(GL_POINTS);  
    glColor3f(1.0, 0.0, 0.0);  
    glVertex2f(0.5, -0.5);  
    glVertex2f(0.5, 0);  
    glVertex2f(0.5, 0.5);  
glEnd();
```

Điểm thể hiện như hình trên là quá nhỏ và rất khó quan sát do đó OpenGL cung cấp lệnh `glPointSize()`, cho phép vẽ điểm với bất kỳ kích cỡ nào. Trước hết chúng ta phải thiết lập ma trận đơn vị `glLoadIdentity()` và định nghĩa cửa sổ hiển thị.

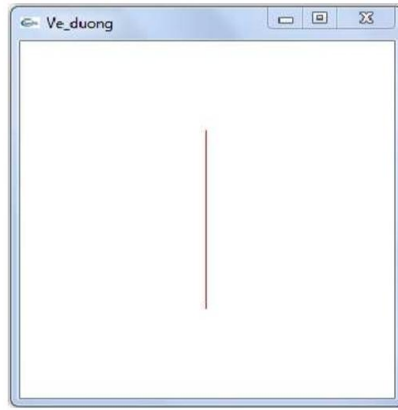
```
glPointSize(5.0);  
// 5 pixel dot  
glLoadIdentity();  
//Thiết lập ma trận tọa độ là ma trận đơn vị  
gluOrtho2D(0.0, (Gldouble)400, 0.0, (Gldouble)400);  
//Kích cỡ của sổ hiển thị x,y thuộc (0,400)  
glBegin(GL_POINTS);  
    glColor3f(1.0f, 0.0f, 0.0f);  
    glVertex2f(50.0, 50.0);  
    glVertex2f(50.0, 200.0);  
    glVertex2f(50.0, 300.0);  
glEnd();
```

Vẽ đường

Một đường được xác định bằng điểm đầu và điểm cuối

Vẽ cùng lúc 3 điểm trên hệ tọa độ 2D

```
glBegin(GL_POINTS);  
    glColor3f(1.0, 0.0, 0.0);  
    glVertex2f(0.5, -0.5);  
    glVertex2f(0.5, 0);  
    glVertex2f(0.5, 0.5);  
glEnd();
```

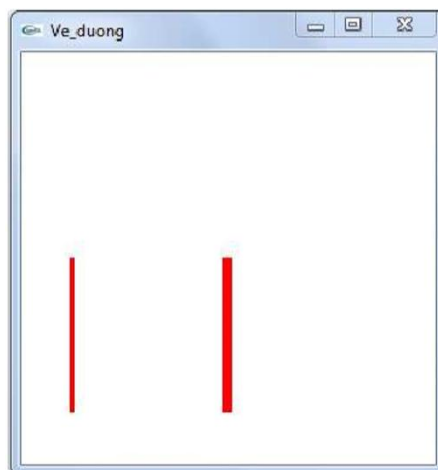


Hình 9.3. Vẽ đường

Để xác định kích thước đường chúng ta có thể sử dụng lệnh `glLineWidth(Glfloat);`

// Độ rộng đường 4pixel và phải định nghĩa bên ngoài

```
glBegin() glLineWidth(4);
glBegin(GL_LINES);
    glColor3f(1.0, 0.0, 0.0);
    glVertex2f(50.0, 50.0);
    glVertex2f(50.0, 200.0);
glEnd();
glLineWidth(20);
glBegin(GL_LINES);
    glVertex2f(200.0, 50.0);
    glVertex2f(200.0, 200.0);
glEnd();
```

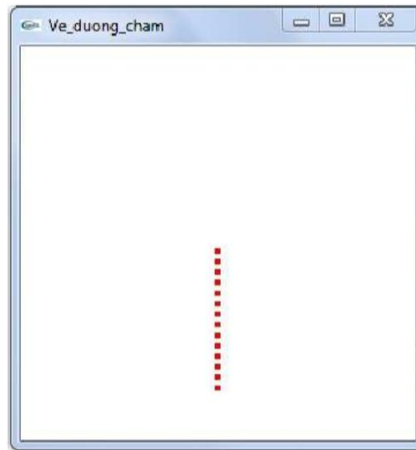


Hình 9.4. Vẽ đường với `glLineWidth()`

Vẽ đường chấm bằng cách sử dụng lệnh `glLineStipple()`. Chúng ta cần định nghĩa màu chấm bằng cách tạo ra một giá trị nhị phân, với 1 tương ứng với chấm, 0 tương ứng với không chấm. Để vẽ đường chấm chúng ta cần kích hoạt nó bằng lệnh `glEnable(GL_LINE_STIPPLE);`

//`glLineStipple()` cần định nghĩa bên ngoài

```
glBegin() glLineWidth(5);
glEnable(GL_LINE_STIPPLE);
/*Hai đối số của hàm gồm 1 giá trị Glint biểu thị hệ số lặp
mẫu, và một giá trị GLushort chứa mẫu chấm. Ví dụ với hệ số lặp
bằng 2, mẫu chấm 01 thì đường chấm được vẽ là 0011*/
glLineStipple(1,0x0F0F);
//0F0F hệ nhị phân : 0000111100001111
glBegin(GL_LINES);
glColor3f(1.0, 0.0, 0.0);
glVertex2f(200.0, 50.0);
glVertex2f(200.0, 200.0);
glEnd();
glFlush();
```



Hình 9.5. Vẽ đường với glLineStipple()

Vẽ đa giác lồi

Đa giác là hình được tạo bởi các đường nối giữa một tập hợp các vertex. Một đa giác có ít nhất 3 cạnh (tương ứng ít nhất 3 vertex phân biệt). Đa giác đơn giản nhất là tam giác. OpenGL có thể vẽ đa giác như các điểm, các đường ngoài, hay đối tượng đặc biệt. Một đa giác có 2 mặt trước và sau do đó có thể vẽ theo hai cách riêng và có thể quay một đa giác để xem mặt bên kia.

Hàm `glPolygonMode()` được sử dụng trong vẽ đa giác:
`glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);`

Hai đối số của hàm này là các giá trị GLenum. Đối số đầu tiên có thể là hằng `GL_FRONT`, `GL_BACK`, hay `GL_FRONT_AND_BACK`, cho biết mặt đa giác muốn thiết lập là chế độ vẽ mặt trước hay sau hoặc cả 2. Đối số thứ 2 cho biết chế độ vẽ cho mặt đa giác đã chọn. Các chế độ vẽ có thể là `GL_POINT` (Chỉ vẽ điểm tại các vertex của đa giác), `GL_LINE` (Vẽ đa giác với các cạnh đa giác khung lưới), `GL_FILL` (đa giác đặc). Trong đó `GL_FILL` là chế độ mặc định.

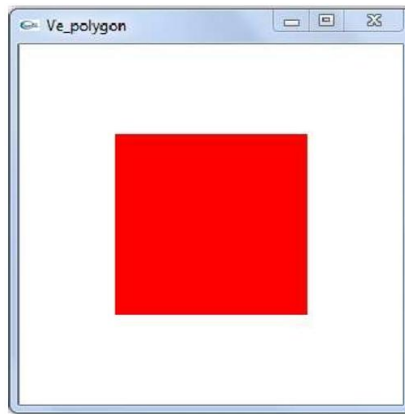
Để cho OpenGL biết mặt nào của đa giác là trước, mặt nào là sau, ta dùng thứ tự khi định nghĩa các vertex tạo nên đa giác. Một đa giác được xem là đối diện mặt trước, khi

các vertex của nó được định nghĩa ngược chiều kim đồng hồ. Tuy nhiên có thể thay đổi bằng cách gọi : `glFontFace(GL_CW);`

Lời gọi này cho OpenGL biết đa giác đối diện mặt trước chọn định nghĩa theo chiều nào. Đối số đơn của hằng là `GL_CW` khi chọn chiều kim đồng hồ, và `GL_CCW` khi chọn chiều ngược chiều kim đồng hồ (mặc định).

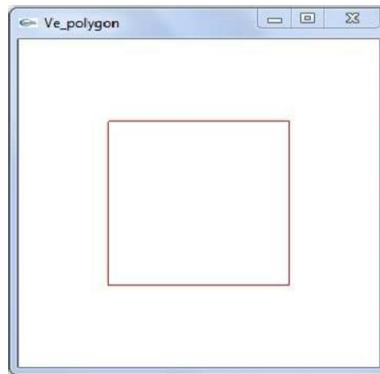
Đoạn mã thực hiện vẽ 1 đa giác:

```
glBegin(GL_POLYGON);
    glColor3f(1.0, 0.0, 0.0);
    glVertex2f(-0.5f, 0.5f);
    glVertex2f(-0.5f, -0.5f);
    glVertex2f(0.5f, -0.5f);
    glVertex2f(0.5f, 0.5f);
glEnd();
```



Hình 9.6. Vẽ polygon

Nếu dùng lệnh : `glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);`

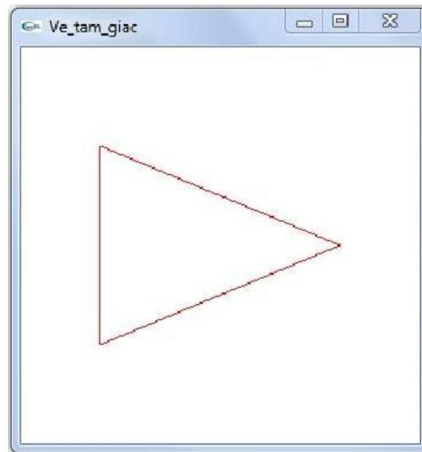


Hình 9.7. Vẽ polygon với chế độ GL_LINE

Vẽ tam giác

```
glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
glBegin(GL_TRIANGLES);
    glColor3f(1.0, 0.0, 0.0);
    glVertex2f(-0.6, 0.5);
    glVertex2f(-0.6, -0.5);
```

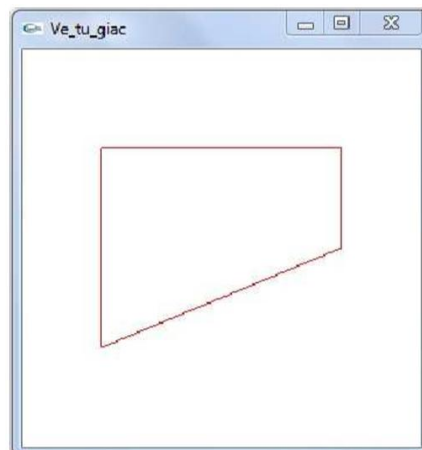
```
glVertex2f(0.6, 0);  
glEnd();
```



Hình 9.8. Vẽ tam giác

Vẽ tứ giác

```
glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);  
glBegin(GL_QUADS);  
    glColor3f(1.0, 0.0, 0.0);  
    glVertex2f(-0.6, 0.5);  
    glVertex2f(-0.6, -0.5);  
    glVertex2f(0.6, 0);  
    glVertex2f(0.6, 0.5);  
glEnd();
```

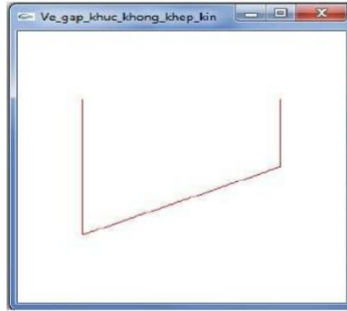


Hình 9.9. Vẽ tứ giác

Vẽ đường gấp khúc không khép kín

```
glBegin(GL_LINE_STRIP);  
    glColor3f(1.0, 0.0, 0.0);  
    glVertex2f(-0.6, 0.5);  
    glVertex2f(-0.6, -0.5);  
    glVertex2f(0.6, 0);  
    glVertex2f(0.6, 0.5);
```

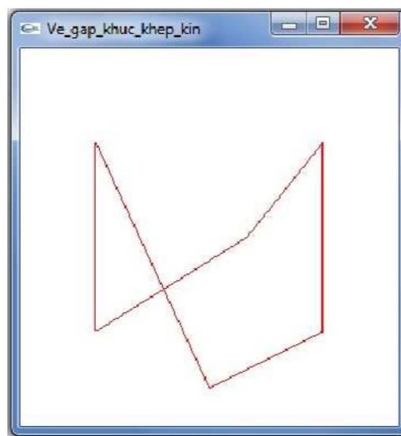
`glEnd();`



Hình 9.10. Vẽ đường gấp khúc không khép kín

Vẽ đường gấp khúc khép kín

```
glBegin(GL_LINE_LOOP);
    glColor3f(1.0, 0.0, 0.0);
    glVertex2f(-0.6, 0.5);
    glVertex2f(-0.6, -0.5);
    glVertex2f(0.2, 0);
    glVertex2f(0.6, 0.5);
    glVertex2f(0.6, -0.5);
    glVertex2f(0, -0.8);
glEnd();
```

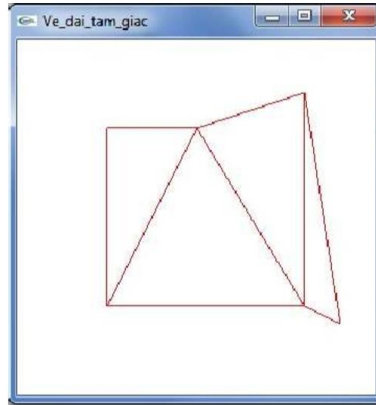


Hình 9.11. Vẽ đường gấp khúc khép kín

Vẽ dải tam giác

```
glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
glBegin(GL_TRIANGLE_STRIP);
    glColor3f(1.0, 0.0, 0.0);
    glVertex2f(-0.5, 0.5);
    glVertex2f(-0.5, -0.5);
    glVertex2f(0, 0.5);
    glVertex2f(0.6, -0.5);
    glVertex2f(0.6, 0.7);
    glVertex2f(0.8, -0.6);
```

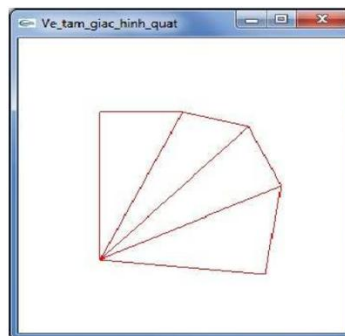
`glEnd();`



Hình 9.12. Vẽ dải tam giác liên kết

Vẽ các tam giác liên kết theo hình quạt

```
glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
glBegin(GL_TRIANGLE_FAN);
    glColor3f(1.0, 0.0, 0.0);
    glVertex2f(-0.5, -0.5);
    glVertex2f(-0.5, 0.5);
    glVertex2f(0, 0.5);
    glVertex2f(0.4, 0.4);
    glVertex2f(0.6, 0);
    glVertex2f(0.5, -0.6);
glEnd();
```

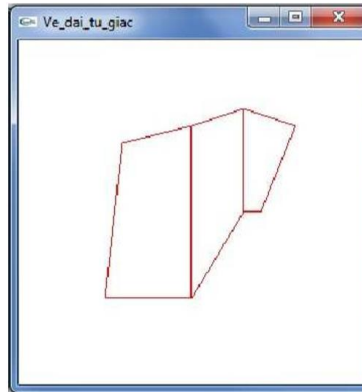


Hình 9.13. Vẽ dải tam giác liên kết theo hình quạt

Vẽ 1 dải tứ giác liên kết với nhau

```
glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
glBegin(GL_QUAD_STRIP);
    glColor3f(1.0, 0.0, 0.0);
    glVertex2f(-0.5, -0.5);
    glVertex2f(-0.4, 0.4);
    glVertex2f(0, -0.5);
    glVertex2f(0, 0.5);
    glVertex2f(0.3, 0);
```

```
glVertex2f(0.3, 0.6);
glVertex2f(0.4, 0);
glVertex2f(0.6, 0.5);
glEnd();
```



Hình 9.14. Vẽ dải tứ giác liên kết

9.2.4. Màu sắc

Chế độ màu RGBA

OpenGL hỗ trợ 2 chế độ màu : RGBA và Color – Index. Trong chế độ màu RGBA, RGB lần lượt thể hiện màu Red, Green, Blue. Còn thành phần A(alpha) không thực sự ảnh hưởng trực tiếp lên màu pixel, người ta có thể dùng thành phần A để xác định độ trong suốt hay thông số cần quan tâm nào đó.

Để thiết lập màu vẽ hiện hành trong chế độ RGBA, chúng ta có thể sử dụng các hàm sau:

```
void glColor3{b s I f d ub us ui} (TYPEr, TYPEg, TYPEb);
void glColor3{b s I f d ub us ui} (TYPEr, TYPEg, TYPEb,
TYPEa);
void glColor3{b s I f d ub us ui}v (const TYPE*v);
void glColor3{b s I f d ub us ui}v(const TYPE*v);
```

Trong đó, nếu các tham số là số thực thì bảng màu tương ứng sẽ nằm trong đoạn [0,1], ngược lại sẽ được chuyển đổi như ở bảng sau :

Bảng 9.5. Chuyển đổi các tham số trong bảng màu

Suffix	Data Type	Minimum Value	Min Value Maps to	Maximum Value	Max Value Maps to
b	1-byte integer	-128	-1.0	127	1.0
s	2-byte integer	-32768	-1.0	32767	1.0
i	4-byte integer	-2147483648	-1.0	2147483647	1.0
ub	Unsigned 1 - byte integer	0	0.0	255	1.0
us	unsigned 2 - byte integer	0	0.0	65535	1.0

ui	unsigned 4 - byte integer	0	0.0	4294967295	1.0
----	---------------------------	---	-----	------------	-----

Thiết lập mô hình shading

Một đoạn thẳng có thể được tô bởi 1 màu đồng nhất (chế độ flat) hay bởi nhiều màu sắc khác nhau (chế độ smooth). Để thiết lập chế độ shading phù hợp, chúng ta có thể sử dụng hàm :

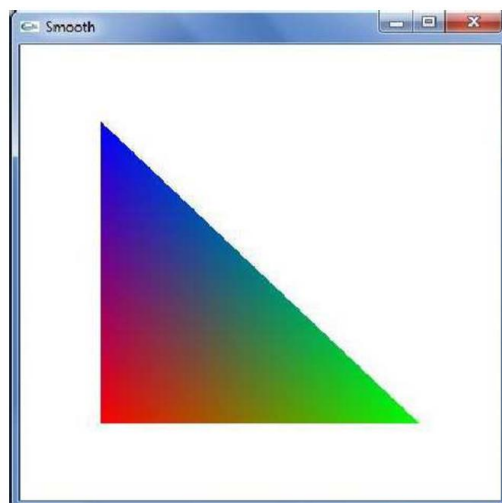
```
void glShadeModel (GLenum mode);
```

Trong đó mode là chế độ mong muốn, nó có thể nhận một trong hai giá trị GL_SMOOTH hoặc GL_FLAT. □

Chế độ SMOOTH

Xét đoạn mã sau để thấy rõ hơn màu vẽ mà được tô bởi chế độ SMOOTH

```
glClearColor (1.0, 1.0, 1.0, 0.0);
glClear (GL_COLOR_BUFFER_BIT);
glShadeModel (GL_SMOOTH);
glBegin (GL_TRIANGLES);
    //Đỉnh thứ nhất màu đỏ
    glColor3f (1.0, 0.0, 0.0);
    glVertex2f (5.0, 5.0);
    //Đỉnh thứ 2 màu xanh lá cây
    glColor3f (0.0, 1.0, 0.0);
    glVertex2f (25.0, 5.0);
    //Đỉnh thứ 3 màu xanh da trời
    glColor3f (0.0, 0.0, 1.0);
    glVertex2f (5.0, 25.0);
glEnd();
```

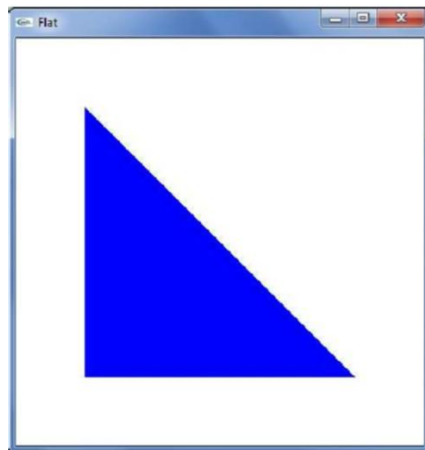


Hình 9.15. Vẽ hình với chế độ SMOOTH

Chế độ FLAT

Chế độ FLAT tô hình đang xét bằng một màu đồng nhất. Khi đó OpenGL sẽ lấy màu của một đỉnh làm màu tô cho toàn bộ hình. Vẫn đoạn mã trên nhưng với chế độ FLAT ta sẽ thấy kết quả như sau :

```
glClearColor (1.0, 1.0, 1.0, 0.0);
glClear (GL_COLOR_BUFFER_BIT);
glShadeModel (GL_FLAT); glBegin (GL_TRIANGLES);
//Đỉnh thứ nhất màu đỏ glColor3f (1.0, 0.0, 0.0);
glVertex2f (5.0, 5.0);
//Đỉnh thứ 2 màu xanh lá cây
glColor3f (0.0, 1.0, 0.0);
glVertex2f (25.0, 5.0);
//Đỉnh thứ 3 màu xanh da trời
glColor3f (0.0, 0.0, 1.0);
glVertex2f (5.0, 25.0);
glEnd();
```



Hình 9.16. Vẽ hình với chế độ màu FLAT

Cách lấy màu của đỉnh như sau :

Đối với một đoạn thẳng, điểm được lấy màu là điểm cuối của đoạn thẳng □

Đối với đa giác, điểm được chọn lấy màu theo quy tắc như bảng sau.

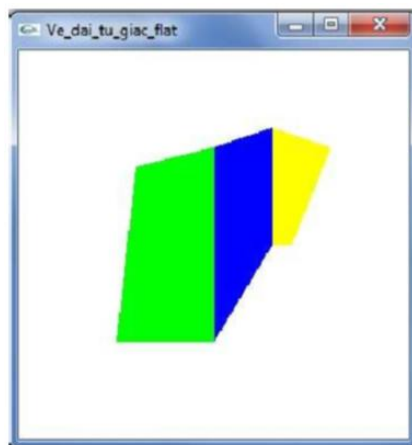
Bảng 9.6. Quy tắc lấy màu vẽ cho hình

Loại đa giác	Đỉnh được chọn để lấy màu cho đa giác thứ i
Đa giác đơn	1
Dải tam giác liên kết	i+2
Dải tam giác liên kết theo hình quạt	i+2
Tam giác độc lập	3i
Dải tứ giác liên kết	2i+2
Tứ giác độc lập	4i

Màu vẽ của hình tam giác (một tam giác độc lập) trên chính là màu vẽ “BLUE” được định nghĩa ở đỉnh thứ 3.

Một ví dụ khác cho dải tứ giác liên kết.

```
glClearColor (1.0, 1.0, 1.0, 0.0);
glClear (GL_COLOR_BUFFER_BIT);
glShadeModel(GL_FLAT);
glBegin(GL_QUAD_STRIP);
glColor3f(1.0, 0.0, 0.0);
// Vertex 1 : Đỏ
glVertex2f(-0.5, -0.5);
glVertex2f(-0.4, 0.4);
glColor3f(0.0, 1.0, 0.0);
// Vertex 3 : Xanh lá
glVertex2f(0, -0.5);
glVertex2f(0, 0.5);
glColor3f(0.0, 0.0, 1.0);
// Vertex 5 : Xanh da trời
glVertex2f(0.3, 0);
glVertex2f(0.3, 0.6);
glColor3f(1.0, 1.0, 0.0);
// Vertex 7 : Vàng
glVertex2f(0.4, 0);
glVertex2f(0.6, 0.5);
glEnd();
```



Hình 9.17. Lấy màu vẽ cho dải tứ giác liên kết.

9.3. Các phép biến đổi 3D

9.3.1. Quá trình chuyển đổi tọa từ không gian 3D đến pixel trên màn hình

Mục đích của đồ họa máy tính đó là tạo ra hình ảnh hai chiều (2D) trên màn hình máy tính của các vật thể trong không gian ba chiều (3D). Quá trình chuyển đổi tọa độ từ không gian 3D thành các điểm (pixel) xuất hiện trên màn hình được mô tả như sau:

Bước 1 : Áp dụng các phép biến đổi được biểu diễn bởi các phép nhân ma trận bao gồm phép biến đổi mô hình (Modeling), phép biến đổi điểm nhìn (Viewing) và phép chiếu

(Projection). Nhìn chung ta kết hợp một số phép biến đổi khi đó đối tượng cần vẽ trong không gian 3D được cắt theo một không gian thực được gọi là viewing volume. Viewing volume xác định cách thức mà vật thể được chiếu lên màn hình là phép chiếu phối cảnh (perspective projection) hay phép chiếu trực giao (orthographic projection) và phần nào của đối tượng được hiển thị.

Bước 2 : Viewing volume được chiếu lên một mặt phẳng chiếu hình chữ nhật được gọi là khung nhìn (rectangular window).

Bước 3: Ánh xạ hình ảnh trong khung nhìn thành các điểm ảnh trên một cửa sổ của màn hình máy tính mà ta gọi đó là cổng nhìn (viewport).

9.3.2. Thao tác trên ModelView

Trước khi thực hiện thao tác trên ModelView, chúng ta cần gọi hàm : `glMatrixMode(GL_MODELVIEW);`

Để lựa chọn ma trận cần thao tác. Để sử dụng ma trận vào việc biến hình, trước tiên cần khởi tạo nó bằng ma trận đơn vị. Chúng ta cần gọi hàm :

```
glLoadIdentity();
```

Một số hàm biến đổi

OpenGL hỗ trợ sẵn các hàm biến đổi cơ bản như sau: □

Tịnh tiến :

```
glTranslate{fd}(TYPEx, TYPEy, TYPEz);
```

Ý nghĩa : tịnh tiến mô hình theo vector tịnh tiến (x,y,z). □

Phép quay :

```
glRotate{fd}(TYPE angle, TYPEx, TYPEy, TYPEz);
```

Ý nghĩa : Quay mô hình một góc angle ngược chiều kim đồng hồ xung quanh tia nối từ gốc tọa độ đến điểm (x, y, z). □

Phép tỉ lệ :

```
glScale{fd}(TYPEx, TYPEy, TYPEz);
```

Ý nghĩa : Biến đổi tỉ lệ mô hình với hệ số tỉ lệ tương ứng với ba trục ox, oy, oz lần lượt là x, y, z.

Thiết lập view

Giống như chụp hình, thiết lập view là thiết lập vị trí cũng như góc, hướng của camera. Thư viện GLUT có một hàm giúp thiết lập view một cách nhanh chóng :

```
gluLookAt(Gldouble eyex, Gldouble eyey, Gldouble eyez,
Gldouble centerx, Gldouble centery, Gldouble centerz, Gldouble
upx, Gldouble upy, Gldouble upz).
```

Trong đó :□

(eyex, eyey, eyez) là vị trí đặt của view. □

(centerx, centery, centerz) là điểm nằm trên đường thẳng xuất phát từ tâm view hướng ra ngoài.

(upx, upy, upz) là vector chỉ hướng lên trên của view.

9.3.3. Thao tác trên Projection

Trước khi thực hiện thao tác chiếu, chúng ta cần gọi hai hàm:

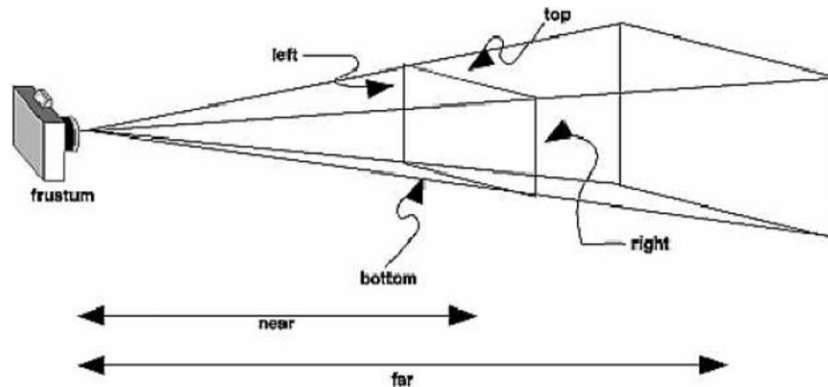
```
glMatrixMode (GL_PROJECTION);  
glLoadIdentity();
```

Phép chiếu phối cảnh (Perspective Projection)

Đặc điểm của phép chiếu này là đối tượng càng lùi xa càng nhỏ. Để thiết lập phép chiếu này, OpenGL có hàm:

```
glFrustum(Gldouble left, Gldouble right, Gldouble bottom,  
Gldouble top, Gldouble near, Gldouble far)
```

Viewing volume được định nghĩa là 1 hình chóp cắt, đỉnh của hình chóp tại gốc tọa độ. Đáy lớn của viewing volume nằm trên mặt phẳng $z = -\text{far}$ và đáy nhỏ của viewing volume (khung nhìn) nằm trên mặt phẳng $z = -\text{near}$ là mặt phẳng mà vật thể sẽ được chiếu lên đó. Các giá trị near và far là các số dương.

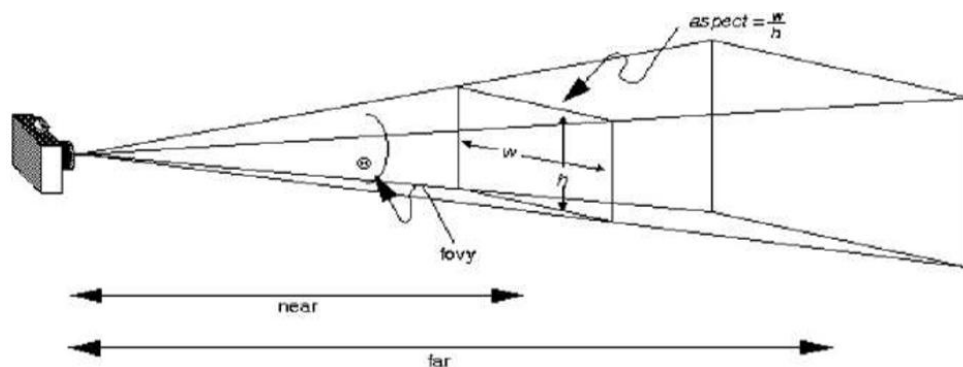


Hình 9.18. Viewing volume được định nghĩa bởi glFrustum()

Ngoài ra chúng ta có thể sử dụng hàm :

```
gluPerspective(Gldouble fovy, Gldouble aspect, Gldouble near,  
Gldouble far).
```

Tác dụng của lệnh giống như hàm glFrustum() nhưng các tham số mang một ý nghĩa khác. Tham số fovy xác định góc tại đỉnh của hình chóp dọc theo mặt phẳng yz, tham số aspect chính là tỉ số giữa bề rộng và chiều cao đáy nhỏ của hình nón cắt, hai tham số near và far giống như hàm glFrustum().



Hình 9.19. Viewing volume được định nghĩa bởi gluPerspective()

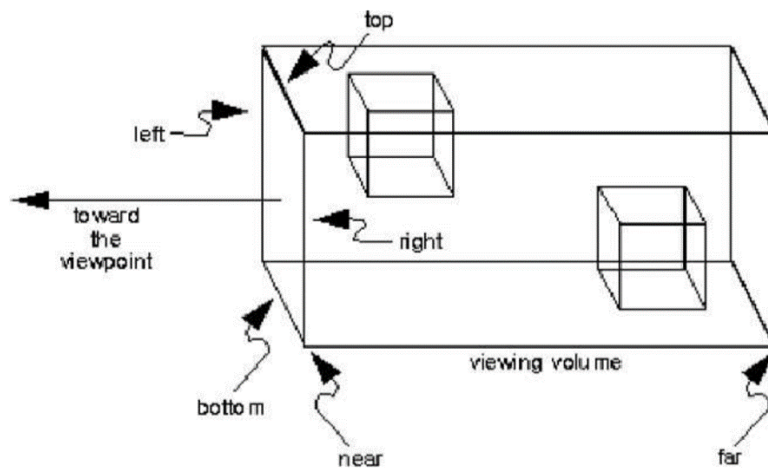
Phép chiếu trực giao

Với phép chiếu trực giao, viewing volume được định nghĩa là một hình hộp chữ nhật. Vật thể nằm trong viewing volume được chiếu trực giao lên khung nhìn do đó trong phép chiếu trực giao khoảng cách từ camera đến vật thể không ảnh hưởng đến độ lớn của ảnh. Phép chiếu trực giao thường được dùng nhiều trong các ứng dụng CAD/CAM như các trình ứng dụng tạo các bản vẽ kỹ thuật lĩnh vực thiết kế.

Để thực hiện phép chiếu trực giao chúng ta có thể sử dụng hàm :

```
glOrtho(Gldouble left, Gldouble right, Gldouble bottom,
Gldouble top, Gldouble near, Gldouble far);
```

Đối tượng nằm trong viewing volume được chiếu trực giao lên đáy của hình hộp chữ nhật nằm trên mặt phẳng $z = -near$, phương của phép chiếu song song với trục oz. Ảnh thu được không bị co lại và không có chiều sâu.



Hình 9.20. Viewing volume được định nghĩa bởi glOrtho()

Sử dụng hàm :

```
gluOrtho2D(Gldouble left, Gldouble right, Gldouble bottom,
Gldouble top);
```

Lệnh có ý nghĩa giống như lệnh glOrtho() ngoại trừ khoảng giá z mặc định của vật thể nằm trong viewing volume mặc định là $[-1, 1]$.

9.3.4. Phép biến đổi cổng nhìn (viewport transformation)

Cổng nhìn là một miền hình chữ nhật nằm phía trong một cửa sổ (window) đang mở trên màn hình cho phép hiển thị hình ảnh. Hình ảnh của vật thể sau khi được chiếu lên khung nhìn sẽ được ánh xạ lên cổng nhìn. Ta có thể định nghĩa nhiều cổng nhìn để hiển thị nhiều khung cảnh khác nhau trên cửa sổ. Định nghĩa cổng nhìn ta dùng hàm sau :

```
glViewport(Glint x, Glint y, Glsizei width, Glsizei height);
```

Trong đó (x, y) xác định tọa độ góc trái phía dưới của cổng nhìn, tọa độ này là tọa độ trong cửa sổ đang mở. Góc trái phía dưới của cửa sổ có tọa độ (0, 0). Hai tham số width và height là chiều rộng và cao của khung nhìn. Ở chế độ mặc định giá trị tham số của khung nhìn là (0, 0, winWidth, winHeight) trong đó winWidth và winHeight là kích thước của cửa sổ hiển thị khung nhìn.

9.4. Ánh sáng

9.4.1. Các loại nguồn sáng

Ánh sáng cung cấp màu sắc và tạo ánh sáng cho đối tượng 3D làm hình ảnh nhận được trở lên thực hơn. OpenGL có 4 loại nguồn sáng : môi trường, khuếch tán, phản chiếu, và nguồn sáng.

Ánh sáng môi trường là ánh sáng đến từ mọi hướng cùng lúc

Ánh sáng khuếch tán là ánh sáng đến từ một hướng chiếu vào bề mặt đối tượng, làm cho bề mặt đối tượng trở nên sáng chói hơn, sau đó ánh sáng bị khuếch tán đi mọi hướng.

Ánh sáng phản chiếu là ánh sáng dõm phản chói, thường là màu trắng, trên các bề mặt có tính phản chiếu cao.

Cuối cùng, nguồn phát là nguồn ánh sáng phát ra từ đối tượng như bóng đèn chẳng hạn.

OpenGL cho phép 8 nguồn sáng khác nhau trong một chương trình, mỗi nguồn sáng có nhiều thuộc tính để kiểm soát các ánh sáng tác động đến một cảnh. Chúng ta cần xác định các thành phần phần trăm đỏ, xanh dương, xanh lục trong các thuộc tính ánh sáng môi trường, khuếch tán, và phản chiếu. Đồng thời xác định các tọa độ x,y,z, và w của nguồn sáng. Công việc này được thực hiện bằng các mảng chứa các giá trị yêu cầu.

9.4.2. Thiết lập các mảng giá trị ánh sáng

Các mảng sau chứa các giá trị dương định nghĩa một nguồn sáng :

```
GLfloat ambientLight0[] = {0.3f, 0.3f, 0.3f, 0.3f};  
GLfloat diffuseLight0[] = {0.5f, 0.5f, 0.5f, 1.0f};  
GLfloat specularLight0[] = {0.0f, 0.0f, 0.0f, 1.0f};  
GLfloat positionLight0[] = {0.0f, 0.0f, 1.0f, 0.0f};
```

Trong mảng `ambientLight0[]`, lượng màu được thiết lập cho ánh sáng môi trường gồm 30% đỏ, 30% xanh dương, 30% xanh lục. (Giá trị 1.0 là các thành phần alpha có thể bỏ qua). Mảng `diffuseLight0[]` chứa lượng màu cho ánh sáng khuếch tán, gồm 50% đỏ, 50% xanh dương, 50% xanh lục. Do cả thành phần màu trong các mảng trên bằng nhau, chúng định nghĩa ánh sáng môi trường và ánh sáng khuếch tán là ánh sáng trắng với các độ sáng riêng. Thay đổi tỉ lệ phần trăm các thành phần màu sẽ mang lại ánh sáng màu. Mảng `specularLight0[]` chứa các giá trị của ánh sáng trắng tối nhất (hay không có ánh sáng nào). Mảng `positionLight0[]` chứa vị trí nguồn sáng, xác định bởi các tọa độ x, y, z, w. Trong đó w chỉ chứa giá trị 0 và 1. Nếu $w = 0$, ta có nguồn sáng theo hướng, là nguồn sáng ở khoảng cách vô hạn so với các cảnh, tức là các tia sáng đến đối tượng là song song với nhau. Mặt trời là ví dụ cho nguồn sáng vô hướng. Nếu $w = 1$, ta có nguồn sáng theo vị trí, là nguồn sáng có vị trí gần cảnh, nên các tia sáng chiếu vào đối tượng theo các góc khác nhau. Đèn là một ví dụ cho nguồn sáng theo vị trí.

9.4.3. Chuyển mảng cho OpenGL

Các giá trị định nghĩa nguồn sáng được chuyển cho OpenGL bằng cách gọi hàm `glLight()` như sau:

```
glLightfv(GL_LIGHT0, GL_AMBIENT, ambientLight0);
```

```
glLightfv(GL_LIGHT0, GL_DIFFUSE, diffuseLight0);
glLightfv(GL_LIGHT0, GL_SPECULAR, specularLight0);
glLightfv(GL_LIGHT0, GL_POSITION, positionLight0);
```

Ba đối số của `glLightfv()` gồm nguồn sáng để định nghĩa, hằng số thể hiện tính chất ánh sáng mà ta muốn thay đổi, và địa chỉ mảng chứa các giá trị dùng thay đổi tính chất ánh sáng. Đối với các thuộc tính ánh sáng môi trường, khuếch tán, và phản chiếu, mảng chứa các giá trị RGBA. Đối với địa chỉ nguồn sáng mảng chứa các tọa độ x, y, z, w của nguồn sáng.

Một chương trình OpenGL có thể có 8 nguồn sáng, đối số đầu tiên là hằng số từ `GL_LIGHT0` đến `GL_LIGHT7`. Đối số thứ hai là các hằng số như bảng sau:

Bảng 9.7. Đối số thứ hai của nguồn sáng

Hằng số	Ý nghĩa
<code>GL_AMBIENT</code>	Xác định ánh sáng môi trường
<code>GL_CONSTANT_ATTENUATION</code>	Xác định lượng giảm ánh sáng theo 1 hệ số
<code>GL_DIFFUSE</code>	Xác định ánh sáng khuếch tán
<code>GL_LINEAR_ATTENUATION</code>	Xác định lượng giảm ánh sáng trên cơ sở khoảng cách từ nguồn sáng đến đối tượng
<code>GL_POSITION</code>	Xác định vị trí nguồn sáng
<code>GL_QUADRATIC_ATTENUATION</code>	Xác định lượng giảm ánh sáng theo bình phương khoảng cách từ nguồn sáng đến đối tượng
<code>GL_SPECULAR</code>	Xác định ánh sáng phản chiếu
<code>GL_SPOT_CUTOFF</code>	Xác định góc trải của đèn chiếu
<code>GL_SPOT_DIRECTION</code>	Xác định hướng đèn chiếu
<code>GL_SPOT_EXPONENT</code>	Xác định cường độ đèn chiếu, có tính đến góc của ánh sáng

9.4.4. Kích hoạt nguồn sáng

Khi đã có định nghĩa nguồn sáng, ta phải kích hoạt chúng bằng `glEnable()`:

```
glEnable(GL_LIGHTING);
glEnable(GL_LIGHT0);
```

Lời gọi đầu kích hoạt sự chiếu sáng. Lời gọi thứ hai bật nguồn sáng 0. Vị trí nguồn sáng bị ảnh hưởng bởi phép biến hình đối tượng như mọi đối tượng khác. Việc biến hình xảy ra khi hàm `glLight()` được gọi để xác định vị trí hay hướng của nguồn sáng. Do đó phải gọi `glMatrixMode(GL_MODELVIEW)` và `glLoadIdentity()` trước khi gọi `glLight()`. Nếu muốn biến đổi vị trí và hướng nguồn sáng theo các đối tượng khác trong cảnh, phải thực hiện các phép tịnh tiến, co giãn, và quay trước khi gọi `glLight()`.

9.4.5. Định nghĩa tính chất vật liệu.

Tính chất vật liệu ảnh hưởng lớn đến cảnh. Ví dụ hình khối màu trắng trong ánh sáng đỏ sẽ có màu đỏ chứ không phải màu trắng. Trong khi một khối đỏ trong ánh sáng xanh lục sẽ có màu đen. Ngoài ra các vật liệu có độ sáng khác nhau sẽ phản xạ ánh sáng khác nhau.

Màu và ánh sáng

Màu của đối tượng trong cảnh phụ thuộc vào màu của ánh sáng mà nó phản xạ. Ví dụ, nếu đối tượng phản xạ ánh sáng màu xanh lục, thì nó sẽ có màu xanh lục trong ánh sáng trắng. Do các thành phần màu đỏ và xanh dương của ánh sáng bị hấp thụ chỉ còn lại thành phần xanh lục đến được mắt người.

Thông thường cả ánh sáng môi trường và ánh sáng khuếch tán phản xạ từ đối tượng theo cùng một cách. Tức là nếu đối tượng phản xạ ánh sáng xanh lục, thì nó phản xạ màu xanh lục đối với ánh sáng môi trường lẫn ánh sáng khuếch tán. Trong khi ánh sáng phản chiếu hầu như luôn luôn cùng màu với ánh sáng chiếu chiếu vào đối tượng. Ví dụ xét một khối vuông được thể hiện trong ánh sáng vàng (màu hợp nhất của màu đỏ và xanh dương) Nếu muốn khối vuông có màu đỏ, vật liệu của nó phải phản xạ màu đỏ của ánh sáng môi trường và ánh sáng khuếch tán. Để làm khối vuông trở nên sáng hơn, thì phải thiết lập cho nó tính phản xạ màu vàng đối với ánh sáng phản chiếu.

OpenGL cho phép định nghĩa các tính chất của đối tượng để xác định loại ánh sáng mà đối tượng phản xạ, và như vậy gián tiếp xác định màu cuối cùng của đối tượng.

Thiết lập các mảng giá trị vật liệu

Bước đầu tiên để định nghĩa các tính chất của vật liệu là thiết lập các giá trị phản xạ ánh sáng môi trường khuếch tán và ánh sáng phản chiếu

```
GLfloat materialAmbient[] = {0.0, 0.7, 0.0, 1.0};
GLfloat materialSpecular[] = {1.0, 1.0, 1.0, 1.0};
```

Mỗi mảng này chứa các giá trị RGBA cho 1 kiểu phản xạ ánh sáng riêng biệt, đó là các giá trị RGBA tương ứng với màu sắc ánh sáng dội ra từ vật liệu đối tượng. Mảng materialAmbient[] định nghĩa vật liệu phản xạ 70% màu xanh lục trong ánh sáng môi trường khuếch tán mà nó nhận được, nhưng không phản xạ màu đỏ và xanh dương. Mảng materialSpecular[] định nghĩa vật liệu phản xạ ánh sáng trắng, gồm 100% các thành phần đỏ, xanh dương, xanh lục.

Chuyển mảng vật liệu cho OpenGL

Để chuyển mảng vật liệu cho OpenGL, ta dùng các con trỏ đến đối tượng mảng làm đối số trong lời gọi glMaterialv():

```
glMaterialfv(GL_FRONT, GL_AMBIENT_AND_DIFFUSE, materialAmbient);
glMaterialfv(GL_FRONT, GL_SPECULAR, materialSpecular);
```

Các đối số đầu tiên là GL_FRONT, GL_BACK, hay GL_FRONT_AND_BACK, tương ứng với sự lựa chọn mặt trước, mặt sau, hay cả mặt trước lẫn sau của đa giác tạo nên đối tượng. Đối với đối tượng đặc như hình khối, GL_FRONT thường được chọn vì mặt sau sẽ không bao giờ thấy được. Nhưng nếu hình khối mất một mặt thì

GL_FRONT_AND_BACK được chọn, vì ở một cảnh nào đó có thể nhìn thấy cả hai mặt sau và trước của đa giác. Đối thứ hai có thể là một trong các hằng số trong bảng sau:

Bảng 9.8. Đối số thứ hai để chuyển mảng vật liệu cho OpenGL

Hằng số	Ý nghĩa
GL_AMBIENT	Ánh sáng môi trường bị ánh xạ
GL_AMBIENT_AND_DIFFUSE	Ánh sáng môi trường và khúc tán bị ánh xạ
GL_COLOR_INDEXES	Xác định chỉ số màu cho ánh sáng
GL_DIFFUSE	Ánh sáng khúc tán bị phản xạ
GL_EMISSION	Xác định nguồn sáng phát
GL_SHININESS	Xác định độ bóng vật liệu
GL_SPECULAR	Ánh sáng phản chiếu bị phản xạ

Trong các hằng ở bảng trên, có hằng GL_COLOR_INDEXES xác định các chỉ số màu cho ánh sáng. Nhưng ở đây, ta chỉ sử dụng OpenGL ở chế độ RGBA.

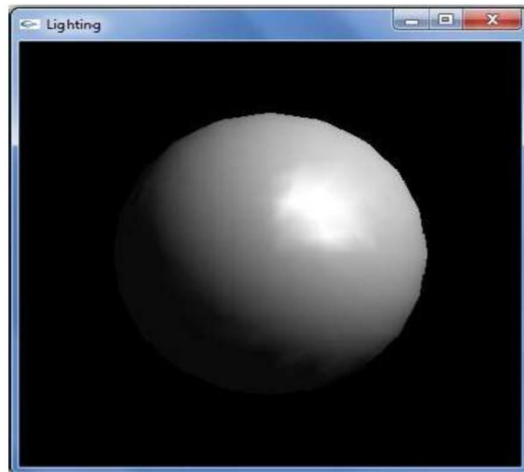
Nếu đối tượng phản xạ ánh sáng phản chiếu, ta phải xác định độ bóng vật liệu, hằng số quyết định độ phản xạ ánh sáng phản chiếu. Lời gọi hàm glMaterialfv() lúc đó như sau: glMaterialfv(GL_FRONT, GL_SHININESS, 60);

Các đối số hàm glMaterialfv() gồm một hằng xác định bề mặt đa giác cần định nghĩa vật liệu, hằng GL_SHININESS (chỉ dùng trong trường hợp này), và một giá trị trong khoảng từ 0 – 128 định nghĩa độ bóng.

Cần phân biệt rõ sự khác nhau giữa nguồn sáng phát và ánh sáng do phản xạ, khi cần định nghĩa các nguồn sáng ta quyết định lượng các ánh sáng môi trường, khúc tán và phản chiếu trong cảnh. Khi định nghĩa tính chất vật liệu, ta quyết định lượng ánh sáng do đối tượng phản xạ các ánh sáng môi trường, khúc tán và phản chiếu.

Một đoạn mã ví dụ :

```
GLfloat mat_specular[] = { 1.0, 1.0, 1.0, 1.0 };
GLfloat mat_shininess[] = { 50.0 };
GLfloat light_position[] = { 1.0, 1.0, 1.0, 0.0 };
glClearColor (0.0, 0.0, 0.0, 0.0);
glShadeModel (GL_SMOOTH);
glMaterialfv(GL_FRONT, GL_SPECULAR, mat_specular);
glMaterialfv(GL_FRONT, GL_SHININESS, mat_shininess);
glLightfv(GL_LIGHT0, GL_POSITION, light_position);
glEnable(GL_LIGHTING); glEnable(GL_LIGHT0);
glEnable(GL_DEPTH_TEST);
glClear (GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
glutSolidSphere (1.0, 20, 16);
glFlush ();
```



Hình 9.21. Một ví dụ Lighting

9.5. Tạo cảnh 3D

9.5.1. Sử dụng các phép biến hình OpenGL để tạo cảnh 3D

Có hai phương pháp tương đương về cách mà phép biến hình đối tượng OpenGL tác động lên cảnh 3D như thế nào : theo gốc tọa độ cục bộ và gốc tọa độ thế giới hay theo hệ tọa độ cố định.

Khi tổ hợp các đối tượng nhỏ vào đối tượng lớn thì các phép biến hình theo các gốc tọa độ cục bộ và thế giới sẽ đơn giản hơn. Gốc tọa độ thế giới luôn là gốc tọa độ ba chiều. Đó là điều hoàn toàn hợp lý vì hệ đề các 3D là thế giới chứa các cảnh được vẽ. Như vậy khi định nghĩa các vị trí trong hệ tọa độ đề các 3D, các tọa độ được dùng là các tọa độ thế giới.

Trong khi đó gốc tọa độ cục bộ được đặt tại tâm đối tượng và di chuyển cùng đối tượng. Với hệ thống này có thể tương đương việc xây dựng các đối tượng 3D là thay vì di chuyển toàn bộ thế giới trước khi đặt các phần của đối tượng vào vị trí, thì ta chỉ đặt chúng vào vị trí trong thế giới tĩnh.

Giả sử khởi tạo ma trận modelview theo ma trận đơn vị như sau:

```
glLoadIdentity();
```

Tại đây nếu ta gọi `glutSolidCube()`, OpenGL sẽ vẽ một khối vuông 3D có tâm trùng với tâm đề các 3D. Nếu ta gọi hai hay ba hay nhiều hơn nữa các lệnh `glutSolidCube` thì tâm của chúng vẫn trùng với tâm đề các 3D. Để có thể thấy các khối vuông thì các đối tượng này cần phải được đặt ở các vị trí khác nhau để tạo nên hình có ý nghĩa. Một cách để thực hiện điều này là phép tịnh tiến, tức di chuyển đối tượng với gốc tọa độ cục bộ của nó đến vị trí mới.

Phép tịnh tiến

Sử dụng lệnh `glTranslatef` để gọi phép tịnh tiến vào ma trận modelview như sau:

```
glTranslatef(x, y, z);
```

Ví dụ gọi `glTranslatef(1.0f, 0.0f, 0.0f)` sau đó gọi `glutSolidCube` để vẽ khối vuông thứ hai thì khối vuông đó sẽ xuất hiện ở vị trí cách khối vuông thứ nhất 1 đơn vị về phía

bên phải, do ma trận modelview chứa phép tịnh tiến 1 đơn vị trên trục X. Điều này giống như di chuyển gốc tọa độ cục bộ của khối vuông 1 đơn vị về phía bên phải. Nếu tiếp tục thực hiện phép tịnh tiến như thế để vẽ khối vuông thứ hai thì khối vuông này sẽ cách gốc tọa độ thế giới 2 đơn vị. Tương tự nếu ta tịnh tiến theo trục y hay z.

Phép quay Phép quay thay đổi hướng các trục cục bộ của đối tượng.

9.5.2. Sử dụng các Stack ma trận

Khi đã thiết lập một ma trận modelview làm ma trận nền, dùng nó để thực hiện các phép biến hình, rồi trả lại ma trận nền, theo trạng thái ban đầu của nó. Ví dụ ta muốn thiết lập điểm nhìn lúc bắt đầu chương trình, sau đó không quan tâm đến nó nữa. Nhưng mọi phép biến hình được thực hiện sau khi thiết lập điểm nhìn sẽ thay đổi ma trận modelview, mà ma trận này chứa phép biến đổi điểm nhìn. Có nghĩa là phải khởi tạo lại điểm nhìn ở lần thay đổi cảnh kế tiếp.

Nhưng OpenGL có cách để lưu trữ và hoàn trả nội dung 1 ma trận, đó là các stack ma trận. Stack ma trận thì tương tự như stack mà máy tính sử dụng, nhưng chứa các giá trị cho các ma trận liên quan, thay vì các kiểu dữ liệu dùng cho stack máy tính. Mỗi ma trận biến hình có stack ma trận riêng. Để lưu nội dung ma trận hiện hành, chỉ đơn giản thực hiện lời gọi hàm sau :

```
glPushMatrix();
```

Sau khi đẩy ma trận hiện hành vào phía trên stack ma trận, ta có thể thực hiện mọi phép biến hình mà không lo ma trận gốc bị biến đổi. Khi muốn trở lại ma trận gốc, lời gọi sau sẽ được thực hiện:

```
glPopMatrix();
```

Cả hai hàm trên đều không có đối số. Chúng luôn luôn tác động đến ma trận hiện được chọn. Ví dụ các dòng sau lưu nội dung ma trận modelview:

```
glMatrixMode(GL_MODELVIEW);
```

```
glPushMatrix();
```

Còn các dòng sau lưu nội dung ma trận chiếu:

```
glMatrixMode(GL_PROJECTION);
```

```
glPushMatrix();
```

Sử dụng các thao tác stack ma trận trên , ta dễ dàng thiết lập một ma trận modelview cơ sở để định nghĩa một điểm nhìn, và trả nó về khi cần thiết. Khi bắt đầu chương trình: `glMatrixMode(GL_PROJECTION);`

```
glLoadIdentity();
```

```
glFrustum(-1, 1, -1, 2, 9);
```

```
glViewport(0, 0, cx, cy);
```

```
glMatrixMode(GL_MODELVIEW);
```

```
glLoadIdentity();
```

```
glTranslatef(0, 0, -5);
```

Bốn dòng đầu tiên thiết lập một không gian quan sát và viewport. Trong khi ba dòng sau định nghĩa điểm nhìn bằng cách di chuyển “mắt” 5 đơn vị theo chiều âm trục Z. (Thực ra là di chuyển ra xa mắt gốc tọa độ cục bộ của bất kỳ đối tượng nào được vẽ).

Để vẽ cảnh mà không làm thay đổi nội dung ma trận modelview (là ma trận chứa phép biến hình thiết lập điểm nhìn) thì ta phải lưu nó trước khi thực hiện bất kỳ phép biến đổi nào: `glMatrixMode(GL_MODELVIEW);`

```
glPushMatrix();
glTranslatef(1,0,0);
glRotatef(90.0, 0, 1, 0);
glutSolidCube(0.5);
glTranslatef(0.0,0.0,1.0);
glutSolidCube(0.5);
glPopMatrix();
```

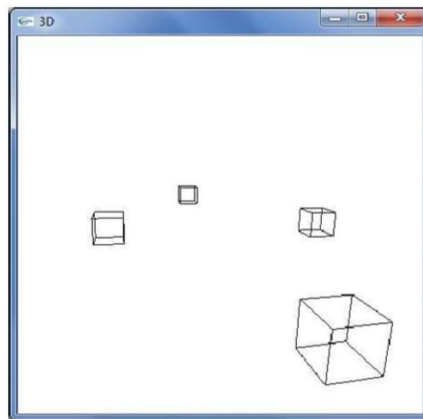
Đoạn mã trên lưu ma trận modelview, thực hiện các phép biến hình cần thiết để vẽ đối tượng, sau đó trả lại ma trận modelview, xóa mọi phép biến hình đã thực hiện lời gọi `glPushMatrix()`. Ở lần thực hiện biến hình tiếp theo, phép biến hình sẽ không ảnh hưởng bởi các phép biến hình trước. Như vậy có thể thấy stack ma trận có giá trị lớn khi tạo ảo giác chuyển động OpenGL, vì khi đó các phép biến hình 1 tương tự nhau sẽ được lặp đi lặp lại với các giá trị khác nhau.

Mỗi loại ma trận có stack ma trận riêng. Chẳng hạn ma trận modelview có stack riêng và ma trận chiếu có stack riêng. Khi gọi `glPushMatrix()`, thực chất là bản sao của ma trận hiện tại được đẩy vào đỉnh stack ma trận của riêng nó, và ta có hai ma trận đơn vị trên đỉnh stack. Khi gọi `glPopMatrix()`, OpenGL loại bỏ bản sao, lúc này có thể đã bị thay đổi, và để lại ma trận gốc trên đỉnh stack.

Ví dụ:

```
glViewport(0,0,(GLsizei)400,(GLsizei)400) ;
glMatrixMode(GL_PROJECTION) ;
glLoadIdentity() ;
gluPerspective(60.0,(GLfloat)400/(GLfloat)400,1.0,30.0) ;
glMatrixMode(GL_MODELVIEW) ;
glLoadIdentity() ;
glTranslatef(0.0,0.0,-6.6) ;
glClearColor(1.0, 1.0, 1.0, 1.0);
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT) ;
glPushMatrix();
glRotatef(20.0,1.0,1.0,0.0);
glColor3f(0.0,0.0,0.0) ;
glPushMatrix() ;
glTranslatef(-2.0, 0.0, 0.0) ;
glutWireCube(0.5) ;
glPopMatrix() ;
glPushMatrix() ;
glTranslatef(0.0, 0.0, 4.0) ;
```

```
glutWireCube(0.5) ;
glPopMatrix() ;
glPushMatrix() ;
glTranslatef(2.0, 0.0, 0.0) ;
glutWireCube(0.5) ;
glPopMatrix() ;
glPushMatrix() ;
glTranslatef(0.0, 0.0, 4) ;
glutWireCube(0.5) ;
glPopMatrix() ;
glPopMatrix() ;
glutSwapBuffers();
```



Hình 9.22. Tạo cảnh 3D với glTranslatef, glRotatef, glPushMatrix, glPopMatrix

9.5.4. Ảnh và các hiệu ứng chỉ ảnh

OpenGL hỗ trợ hai loại đối tượng đồ họa

Loại đầu tiên là bitmap, là một mask đơn sắc dùng để tạo cá thể đơn giản như các ký tự trong một font. Loại thứ hai là ảnh, bitmap đầy màu sắc, và có thể chứa đựng một hình tượng hay một bức tranh nhiều chi tiết.

Thiết lập việc gán cấu trúc Chúng ta cần gọi hàm glTexParameteri() để thiết lập các thông số khác nhau như:

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
```

Hàm glTexParameteri() có 4 phiên bản. Các đối số của hàm gồm hằng xác định kiểu cấu trúc, thông số cần thiết lập, và giá trị thiết lập cho thông số. Đối số đầu tiên là GL_TEXTURE_1D cho GL_TEXTURE_1D cho cấu trúc một chiều, hay GL_TEXTURE_2D cho cấu trúc hai chiều. GL_TEXTURE_2D, là ảnh hình chữ nhật, thường được sử dụng.

Đối số thứ hai cho OpenGL biết cách quản lý texture mapping dưới các điều kiện nào đó. Ví dụ với GL_TEXTURE_WRAP_S và GL_TEXTURE_WRAP_T, OpenGL dùng ảnh đơn hoặc đa cấu trúc, tương ứng với cấu trúc theo phương ngang hay thẳng

đứng. Các thông số này phải thiết lập với GL_CLAMP (cho ảnh đơn) hay với GL_REPEAT (cho mẫu lặp lại). Khi dùng ảnh đơn cho đa giác, phải sử dụng GL_CLAMP. GL_TEXTURE_MAG_FILTER, GL_TEXTURE_MIN_FILTER xác định cách phóng to hay thu nhỏ cấu trúc khi diện tích nơi đến lớn hơn hoặc nhỏ hơn cấu trúc. Việc sử dụng GL_NEAREST với các thông số này cho phép texture mapping này cho phép texture mapping nhanh và nhận được kết quả sắc nét nhất. GL_NEAREST thì chậm hơn do phải tính toán màu sắc cho 1 pixel trên cơ sở trung bình của ba bốn pixel.

Sau khi thiết lập các thông số cấu trúc, phải thiết lập môi trường cấu trúc, tức là xác định hàm cấu trúc nào mà OpenGL dùng tính toán các giá trị màu cho bề mặt cấu trúc. Việc này được thực hiện với lời gọi hàm glTexEnv():

```
glTexEnv(GL_TEXTURE_ENVI, GL_TEXTURE_ENVI_MODE, GLDECAL);
```

Hai đối số đầu của hàm lần lượt là GL_TEXTURE_ENVI và GL_TEXTURE_ENVI_MODE, hay GL_MODULATE. Thông thường GL_DECAL được sử dụng. Khi đó hàm thay thế các pixel nơi đến bằng cấu trúc. Hai hằng còn lại kết hợp nơi đến và các giá trị cấu trúc theo các cách khác nhau.

Bước cuối cùng là khi cùng là khởi động texture mapping:

```
glEnable(GL_TEXTURE_2D);
```

Nếu cấu trúc 1 chiều thì sẽ là GL_TEXTURE_1D.

9.5.5. Định nghĩa các vertex và cấu trúc của chúng

Để thiết lập texture mapping có rất nhiều việc phải làm, nhưng hầu hết đã được hoàn tất ở trên. Các việc cộng lại là trao cấu trúc cho OpenGL, định nghĩa các vertex đa giác và nêu quan hệ giữa chúng với các tọa độ cấu trúc. Đoạn mã sau thể hiện điều đó:

```
glTexImage2D(GL_TEXTURE_2D,
             0,
             3,
             Width,
             Height,
             0,
             GL_COLOR_INDEX,
             GL_UNSIGNED_BYTE,
             pTextureBits);
glBegin(GL_POLYGON);
glTexCoord(0.0, 1.0);
glVertex3f(-1.0, 1.0, 0.0);
glTexCoord(0.0, 0.0);
glVertex3f(-1.0, -1.0, 0.0);
glTexCoord(1.0, ..0);
glVertex3f(1.0, -1.0, 0.0);
glTexCoord(1.0, 1.0);
glVertex3f(1.0, 1.0, 0.0);
glEnd();
glFlush();
```

Trong đoạn mã trên, lời gọi `glTexImage2D` cho OpenGL biết nơi chứa cấu trúc cần sử dụng. Đối số đầu tiên phải là `GL_TEXTURE_2D`. Đối số thứ hai là cho biết chỉ có 1 mức phân giải ảnh đơn cấu trúc, trừ khi dùng với ảnh đa đa cấu trúc, còn lại giá trị 0 dùng cho đối số này. Đối số thứ ba cho biết thành phần màu trong cấu trúc. Trong trường hợp dùng ảnh RGB, giá trị này là 3. Đối số thứ tư và năm là chiều rộng và chiều cao cấu trúc. Đối số thứ sáu xác định chiều rộng đường viền cấu trúc, là 0 hay 1, thường là 0. Sau khi trao cấu trúc cho OpenGL, ta phải định nghĩa các vertex của đa giác cùng lúc cho biết quan hệ giữa chúng với các tọa độ cấu trúc bằng lời gọi hàm `glTexCoord2f()`. Hai đối số này là các tọa độ S và T của cấu trúc (S và T tương tự như X và Y).

9.6. Cài đặt game đơn giản

9.6.1. Phát triển game 2D đơn giản

Về cửa sổ đơn giản

```
glutInit(&argc, argv);
glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
glutInitWindowSize(795, 500);
glutInitWindowPosition(0, 0);
glutCreateWindow("Osama Hosam's simple OpenGL game");
```

Giả sử chúng ta biểu diễn quả bóng là hình chữ nhật, đầu tiên chúng ta định nghĩa cấu trúc quả bóng đặt tên `RECTA`. Chúng ta khai báo các đối tượng ball (bóng), wall (tường) và bat (tấm ván).

```
struct RECTA { float left, top, right, bottom; };
RECTA ball={100,100,120,120};
RECTA wall;
RECTA bat ={0,490,40,500};
```

Chúng ta dùng hàm `DrawRectangle` để vẽ các hình chữ nhật

```
void DrawRectangle(RECTA rect){
    glBegin(GL_QUADS);
        glVertex2f(rect.left,rect.bottom);
        //Left - Bottom
        glVertex2f(rect.right,rect.bottom);
        glVertex2f(rect.right,rect.top);
        glVertex2f(rect.left,rect.top);
    glEnd();
}
```

Chúng ta vẽ hình chữ nhật sau đó quay theo chiều kim đồng hồ trong hàm sau `glBegin(GL_QUADS)` và `glEnd()`. Chúng ta cài đặt hàm viết chữ `drawText`, trong game này viết ra điểm cho các người dùng đạt điểm cao.

```
void drawText(char*string,int x,int y){
    char *c;
    glPushMatrix();
    glTranslatef(x, y,0);
    glScalef(0.1,-0.1,1);
```

```
for (c=string; *c != '\\0'; c++){
    glutStrokeCharacter(GLUT_STROKE_ROMAN , *c);
}
glPopMatrix();
}
```

Hàm viết xâu char* tại tọa độ (x,y). Muốn tịnh tiến với hàm glTranslate, muốn thay đổi kích thước với hàm tỷ lệ glScale.

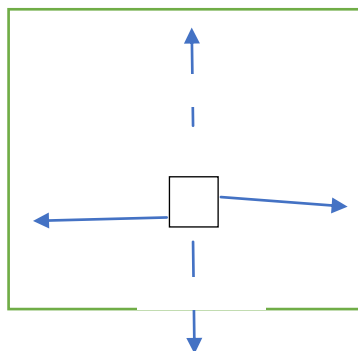
Di chuyển quả bóng và tấm ván:

Đường đi của bóng được điều chỉnh qua vị trí. Tốc độ của quả bóng được định nghĩa trong hàm Timer và độ dài của bước dịch chuyển bóng.

```
static float Xspeed=1,Yspeed=1;
static float delta=1;
void Timer(int v){
    ball.left+=Xspeed;
    ball.right+=Xspeed;
    ball.top+=Yspeed;
    ball.bottom+=Yspeed;
    glutTimerFunc(1,Timer,1);
}
```

Chúng ta định nghĩa 2 biến toàn cục Xspeed, Yspeed và độ chuyển dịch delta. Giá trị của delta sẽ thay đổi +1/-1 khi có đụng độ.với tường. Khi bóng chạm tường phải thì Xspeed=-delta (làm cho quả bóng quay ngược lại). Và ngược lại khi bóng chạm tường trái thì Xspeed=delta. Tương tự Yspeed liên quan đến tường trên và tường dưới. Giá trị của Xspeed và Yspeed được thay đổi theo code:

```
if(Test_Ball_Wall(ball,wall)== FROM_RIGHT)
    Xspeed=-delta;
if(Test_Ball_Wall(ball,wall)== FROM_LEFT)
    Xspeed=delta;
if(Test_Ball_Wall(ball,wall)== FROM_TOP)
    Yspeed=delta;
if(Test_Ball_Wall(ball,wall)== FROM_BOTTOM){
    Yspeed=-delta;
    pcResult +=1;
```



Di chuyển của tấm ván được thực hiện do di chuyển chuột. Khi chuột di chuyển chỉ thay đổi giá trị tọa độ x của tấm ván.

```
static mouse_x=0;
void MouseMotion(int x,int y){
    mouse_x=x;
}
```

Chúng ta có thể dừng game khi ấn phím ESC

```
void keyboard(unsigned char key, int x, int y){
    switch (key){
        case 27:
            exit(0);
            break;
    }
}
```

Phát hiện va chạm bóng:

Khi bóng chạm tường nó sẽ bị bật ngược lại. Để phát hiện va chạm giữa bóng và tường (ví dụ chúng ta xét tường phải) bằng cách so sánh tọa độ phải của tường và tọa độ phải của bóng. Sẽ xảy ra va chạm khi 2 tọa độ này bằng nhau hoặc tọa độ phải của bóng lớn hơn. Tương tự ta phát hiện đụng độ theo chiều đứng bằng cách so sánh tọa độ trên của tấm ván với tọa độ dưới của quả bóng, đụng độ xảy ra khi 2 tọa độ trùng nhau hoặc tọa độ dưới của bóng lớn hơn tọa độ x của thuộc tấm ván.

```
int Test_Ball_Wall(RECTA ball , RECTA wall) {
    if(ball.right >=wall.right)
        return FROM_RIGHT;
    if(ball.left <=wall.left)
        return FROM_LEFT;
    if(ball.top <=wall.top)
        return FROM_TOP;
    if(ball.bottom >=wall.bottom)
        return FROM_BOTTOM;
    else
        return 0 ;
}

bool Test_Ball_Player(RECTA ball,RECTA player){
    if(ball.bottom >= player.top && ball.left>= player.left &&
    ball.right <=player.right ){
        playerResult++;
        return true;
    }
    return false;
}
```

Hợp lại chúng ta viết hàm Render trong đó biến toàn cục playerResult để lưu trữ điểm của người chơi.

```
void Render() {
    glClear(GL_COLOR_BUFFER_BIT );
    glLoadIdentity();
    sprintf(string,"PC : %d ",pcResult);
    drawText(string,10,80);
    sprintf(string,"Player : %d ",playerResult);
    drawText(string,10,120);
    wall.left=wall.top=0;
    wall.right=WINDOW_WIDTH;
    wall.bottom=WINDOW_HEIGHT;
    DrawRectangle(ball);
    if(Test_Ball_Wall(ball,wall)== FROM_RIGHT)
        Xspeed=-delta;
    if(Test_Ball_Wall(ball,wall)== FROM_LEFT)
        Xspeed=delta;
    if(Test_Ball_Wall(ball,wall)== FROM_TOP)
        Yspeed=delta;
    if(Test_Ball_Wall(ball,wall)== FROM_BOTTOM) {
        Yspeed=-delta;
        pcResult +=1;
    }
    DrawRectangle(bat);
    bat.left=mouse_x-20;
    bat.right=mouse_x+40;
    if(Test_Ball_Player(ball,player_1)==true)
        Yspeed=-delta;
    glutSwapBuffers();
}
```

9.6.2. Ví dụ game quả bóng

Giới thiệu: Một quả bóng di chuyển bắt đầu từ giữa và đi lên từ trái sang phải. Nếu nó va chạm với tường thay đổi hướng nhưng tốc độ vẫn giữ nguyên. Nó có khả năng di chuyển theo hướng ngang.

Cài đặt game

```
#include<stdio.h>
#include<GL/glut.h>
#include<math.h>
#define pi 3.142857
// Khai báo các biến toàn cục
// c và d lưu trữ số lần ấn phím 'b' và 'n'
// left và right chỉ ra vị trí trái nhất và phải nhất của
// thanh ván di chuyển hình chữ nhật
int c = 0, d = 0, left = 0, right = 0;
int m = 0, j = 1, flag1 = 0, l = 1, flag2 = 0, n = 0, score =
0, count = 1;
//hàm khởi tạo
```



```

void myInit (void){
// Đặt nền màu trắng (3 tham số đều là 1.0)
glClearColor(1.0, 1.0, 1.0, 0.0);
//Đặt màu vẽ là màu đỏ (không gian màu RGB)
glColor3f(1.0f, 0.0f, 0.0f);
// Đặt độ rộng cho nét vẽ là 1
glPointSize(1.0);
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
gluOrtho2D(-620.0, 620.0, -340.0, 340.0);
}
// Chức năng phím : nó nhận hoạt động khi phím được ấn
void keyboard(unsigned char key, int x, int y){
left = -200 + 200 * (d - c);
right = 200 + 200 * (d - c);
// Nếu chỉ số trái nhất của cửa sổ
// Thanh ván hình chữ nhật không di chuyển sang trái được nữa
khi ấn phím b
// Khi này nó chỉ sang phải nên ấn phím n
if (left == -600){
// '110' là giá trị trong bảng Ascii của ký tự 'n'
// và biến d tăng khi n được ấn
if (key == 110)
d++;
}
// Nếu chỉ số phải nhất của cửa sổ
// thì thanh ván HCN không thể di chuyển sang phải được nữa
khi ấn phím n
// nó chỉ di chuyển sang trái nên ấn phím b
else
if(right == 600){
// '98' là mã trong bảng Ascii của ký tự 'b'
// và biến c tăng khi b được ấn
if (key == 98)
c++;
}
// Khi thanh ván HCN ở giữa nó có thể di chuyển về cả 2 phía
// Hướng nào là phụ thuộc ấn phím nào
else{
if (key == 98)
c++;
if (key == 110)
d++;
}
}
glutPostRedisplay();
}

```

```
void myDisplay(void){
// x và y Là 1 điểm trên đường tròn
int x, y, k;
// vòng lặp 'for' là cho di chuyển quả bóng
for (k = 0; k <= 400; k += 5){
glClear(GL_COLOR_BUFFER_BIT);
glBegin(GL_LINE_STRIP);
float i = 0;
//thay đổi m là di chuyển theo hướng thẳng đứng
//thay đổi n là di chuyển theo hướng ngang
m = m + 6;
n = n + 4;
//vẽ tâm của đường tròn ở (0, 12) lặp tới 2*pi (360 độ)
while (i <= 2 * pi){
y = 12 + 20 * cos(i);
x = 20 * sin(i);
i = i + 0.1;
// flag1=0 chuyển động của quả bóng theo hướng đi lên,
//=1 theo hướng đi xuống
if(m == 288 && flag1 == 0){
j = -1;
m = -288;
flag1 = 1;
score++;
}
if (m == 288 && flag1 == 1){
j = 1;
m = -288;
flag1 = 0;
}
// flag2=0 chuyển động của quả bóng hướng sang phải
//=1 hướng sang trái
if (n == 580 && flag2 == 0){
l = -1;
n = -580;
flag2 = 1;
}
if (n == 580 && flag2 == 1){
l = 1;
n = -580;
flag2 = 0;
}
//phương trình cho hướng chuyển động của bóng
glVertex2i((x - l * n), (y - j * m));
}
glEnd();
}
```

```
// 4 điểm vẽ hình chữ nhật bao ngoài xác định của sỏ
glBegin(GL_LINE_LOOP);
glVertex2i(-600, -320);
glVertex2i(-600, 320);
glVertex2i(600, 320);
glVertex2i(600, -320);
glEnd();

// 4 điểm vẽ hình chữ nhật nhỏ hơn bắt bóng
glBegin(GL_LINE_LOOP);
left = -200 + 200 * (d - c);
right = 200 + 200 * (d - c);
glVertex2i(left, -315);
glVertex2i(left, -295);
glVertex2i(right, -295);
glVertex2i(right, -315);
glEnd();

// điều kiện kiểm tra bóng khi rơi thuộc của sỏ hay ra ngoài
if ((j * m) == 276){
    if ((left > ((-1 * l * n) + 20)) || (right < (-1 * l * n) -
20)){
        printf("Game Over !!!\nYour Score is : \t%d\n", score);
        exit(0);
    }
}

glutSwapBuffers();
}
}

// Hàm main
int main (int argc, char** argv){
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGBA);
    // khai báo kích thước của sỏ
    glutInitWindowSize(1100, 600);
    // khi báo vị trí trái trên của cửa sỏ (0, 0)
    glutInitWindowPosition(0, 0);
    // tên cho window
    glutCreateWindow("Game");
    // các phím chức năng
    glutKeyboardFunc(keyboard);
    myInit();
    glutDisplayFunc(myDisplay);
    glutMainLoop();
}
```

9.7. Bài toán game

9.7.1. Giới thiệu bài toán game

Hiện nay giao thông đang là vấn đề nhức nhối của xã hội. Lượng người tham gia giao thông quá lớn trong khi cơ sở hạ tầng không đáp ứng được. Hơn nữa ý thức của nhiều người tham gia giao thông còn hạn chế, trong đó có những nguyên nhân chủ quan và khách quan, chủ quan có thể do hiểu luật nhưng không chịu chấp hành, nguyên nhân khách quan có thể do không nhớ luật và cách thực hiện luật khi gặp các biển hiệu giao thông. Phần xây dựng ứng dụng sẽ xây dựng một game 3D về đua xe ô tô theo luật nhằm góp phần giúp cho người chơi có thể hiểu hơn về luật giao thông, những gì mà họ cần phải làm theo trên đường đi.

9.7.2. Khái quát về ứng dụng

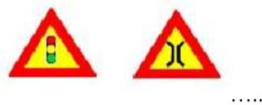
Đây là một game tính điểm.

Người chơi sẽ điều khiển một chiếc xe ô tô đi trên đường đua cùng với các xe khác, trên đường đi sẽ có các biển báo hiệu giao thông như :

Các biển báo cấm:



Các biển báo chỉ dẫn:



Để tăng phần hấp dẫn và mang tính giải trí trên đường đi sẽ có những đối tượng có thể ăn được khi cho xe đi qua và được cộng điểm, thêm những chướng ngại vật cản trở trên đường. Trò chơi chú trọng tính an toàn và tính chấp hành các biển hiệu giao thông của các xe đua tham gia trên đường, nên những xe chấp hành đúng luật sẽ được thưởng nhiều điểm cho mình.

9.7.3. Phân tích bài toán game

Các thành phần cơ bản trong trò chơi

Vòng lặp trò chơi: Quản lý vòng đời, trạng thái của trò chơi.

Logic: Phần xử lý của trò chơi, xử lý thuật toán, trí thông minh nhân tạo, tương tác với người chơi.

Đồ họa: Phần thể hiện của trò chơi, hình ảnh của trò chơi được quyết định qua đồ họa.

Âm thanh của trò chơi: Là phần không thể thiếu, nó giúp trò chơi trở nên sinh động hơn.

Cấu trúc của trò chơi

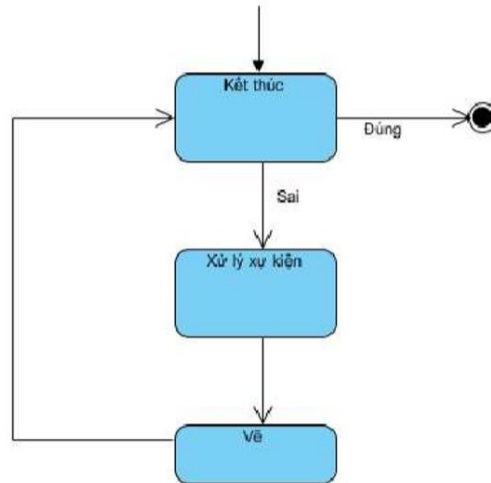
Vòng lặp trò chơi

Vòng lặp trò chơi chính là trung tâm của trò chơi. Trò chơi khác hẳn so với các ứng dụng hướng sự kiện khác, ngay cả khi không có sự kiện xuất nhập, trò chơi vẫn hoạt động và đáp ứng liên tục để tạo ra các hiệu ứng hoạt động của các nhân vật, đối tượng trong trò chơi.

Công việc chính của vòng lặp trò chơi:

Cập nhật, tính toán trạng thái hiện tại (Xử lý các sự kiện đầu vào: như các sự kiện bàn phím, chuột tác động lên đối tượng).

Vẽ các nhân vật, đối tượng, giao diện màn hình.



Hình 9.23. Biểu diễn vòng lặp trò chơi

Ví dụ: Trong trò chơi này, trạng thái kết thúc khi xe của người chơi về tới đích và nếu chưa đạt trạng thái này nó sẽ tiếp tục nhận các sự kiện và xử lý chúng rồi vẽ lại trạng thái, vị trí của chiếc xe.

Cấu trúc trò chơi.

Một trò chơi có thể bao gồm nhiều giao diện khác nhau.

Ví dụ: Màn hình load game, bảng chọn chính, mục giới thiệu trò chơi, mục hướng dẫn, mục điểm cao... Để tích hợp điều này vào vòng lặp chính, chúng ta sẽ cần có một biến trạng thái lưu lại giao diện trò chơi, rồi theo đó xử lý chúng. Dựa vào biến trạng thái chúng ta có thể lưu lại giao diện hiện tại và xử lý trạng thái tương ứng, tương tự như hàm cập nhật hoặc hàm vẽ. Hoặc chúng ta cũng có thể dựa vào biến trạng thái để loại bỏ một giao diện như giao diện Load game chẳng hạn, vì khi đã vào game thì chúng ta sẽ không cần nhớ đến nó nữa. Trong trò chơi này, sẽ có các giao diện sau:

Giao diện load game: Thực hiện load game trong một khoảng thời gian trước khi vào game

Giao diện cho menu chính: Hiển thị các mục để người chơi có thể truy xuất các mục khác trong trò chơi.

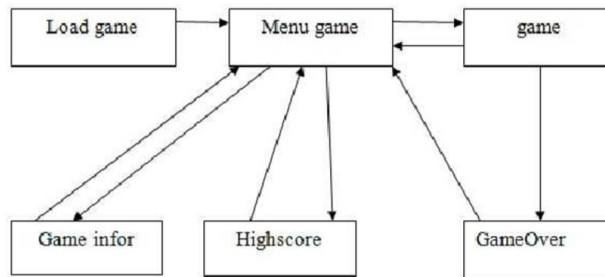
Giao diện nhập thông tin người chơi: Nhập và lưu trữ thông tin của người chơi. Để dùng cho việc lưu và tính điểm.

Game: Giao diện chính, nơi mà người chơi chơi và ghi điểm. Sau khi người chơi chọn Newgame thì giao diện này sẽ xuất hiện.

Highscore: Ghi lại điểm của các người chơi và hiển thị theo thứ tự giảm dần của điểm.

Infor: Giới thiệu trò chơi, hướng dẫn cách chơi và tính điểm.

GameOver: Kết thúc trò chơi khi người chơi không thể về đích trong top 3. Hiện thông báo gameover ngay trên nền game. Chúng ta có thể biểu diễn đơn giản các các giao diện trò chơi như sau:



Hình 9.24. Giao diện của trò chơi

9.8. Thiết kế và cài đặt game

9.8.1. Nội dung và các chức năng chính của trò chơi

Nội dung

Trò chơi mô phỏng một chiếc xe ô tô đi trên đường với những biển hiệu giao thông cần tuân thủ. Trên đường đi đối tượng ô tô không những phải chú ý đến các biển hiệu giao thông mà còn phải chú ý đến những đồ vật có thể ăn được ở trên đường đi và những chướng ngại vật sẽ cản trở trên đường.

Các chức năng chính của trò chơi

Newgame :

Để có thể bắt đầu chơi người chơi cần chọn vào chức năng newgame để chương trình khởi tạo một game mới và người chơi chính thức bắt đầu chơi, người chơi có thể chọn “Enter” để chọn newgame và bắt đầu trò chơi.

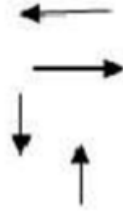
Thông tin người chơi: Sau khi chọn Newgame 1 giao diện mới hiện ra bắt người chơi phải nhập tên của mình vào và nhấn “ Enter ” để có thể bắt đầu chơi.

Pause : Khi người chơi vào trò chơi vì một lý do nào đó muốn dừng trò chơi lại (pause game) người chơi sẽ chọn phím “space –phím cách”.

Full Screen : Màn hình sẽ được mở rộng - full screen giúp người chơi có màn hình quan sát tốt hơn, người chơi bấm phím “F12” để mở rộng màn hình, và ngược lại khi muốn thu nhỏ nó lại. Dịch chuyển đối tượng xe do mình điều khiển Chức năng cho phép người chơi dịch chuyển xe theo ý muốn:

- Sang trái
- Sang phải : •
- Giảm tốc : •
- Tăng tốc :

Như hình dưới :



Hình 9.24. Các phím di chuyển

- Âm thanh khi chơi game
- Âm thanh nền game và theo sự kiện.
- Hiển thị minimap :

Khi người chơi muốn biết mình đang ở đâu, vị trí so với các xe khác như thế nào thì minimap thực sự là một công cụ hữu ích, người chơi có thể bấm phím “M” để bật tắt minimap.

Tính điểm :

- Với chức năng tính điểm: mỗi đồ vật mà người mà xe ăn được sẽ được cộng 5 điểm sau khi ăn đồ vật này sẽ biến mất (Đối tượng đồ vật này quay), ngược lại sẽ bị trừ 5 điểm khi va phải các chướng ngại vật và chúng không biến mất sau va chạm (Đối tượng này đứng yên).
- Đặc biệt đây là một game chú trọng đến tính chấp hành luật giao thông lên trong quá trình tham gia trò chơi người chơi sẽ nhận được phần điểm lớn khi chấp hành đúng luật
 - 30 điểm cho mỗi lần chấp hành đúng. Thế nào là chấp hành đúng. Ví dụ khi gặp biển



Thì người chơi không được bấm còi trên đoạn đường bắt đầu từ khi có biển báo này đến khi gặp 1 biển báo khác.

- Trừ 30 điểm cho hành vi không chấp hành.
- Cũng mang tính đua tranh trên đường người chơi không những phải chấp hành luật, ăn thật nhiều đồ vật mà còn phải đảm bảo tốc độ để xe của mình có thể về đích ở vị trí cao nhất.
 - 500 điểm khi về nhất.
 - 400 điểm khi về nhì.
 - 300 điểm khi về ba.
 - 200 điểm cho các thứ tự còn lại.
- Độ khó của trò chơi : Khi người chơi đã lọt được vào top 3 trong lần đua đầu tiên thì sẽ chuyển sang 1 đường đua mới với các chướng ngại mới, và quy tắc tính điểm cũng tương tự như trên nhưng các chướng ngại xuất hiện nhiều hơn.

• Kết thúc :

- Khi người chơi không thể về đích trong top 3, hiển thị giao diện GameOver
- Ngược lại người chơi sẽ được chuyển sang mức độ tiếp theo.

9.8.2. Đồ họa trong game

Các đối tượng trong game được vẽ bằng thư viện đồ họa OpenGL. Sau đây là các đối tượng đồ họa chính, vì là vẽ trong không gian 3D.

Vẽ một nền game: Tại đây chúng ta sẽ vẽ đường, lề đường. Sử dụng một bản text lưu ma trận các giá trị 0, 1, 2, 4, 5. Ví dụ cụ thể như sau :

0	1	5	5	5	5	5	5	5	1	0
0	1	0	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	0	1	0
0	2	0	0	0	0	0	0	0	2	0
0	1	0	0	0	3	0	0	0	1	0
0	1	0	0	0	0	0	0	0	1	0
0	1	3	0	0	0	0	4	0	1	0
0	2	0	0	0	0	0	0	0	2	0
0	1	0	0	3	0	0	0	0	1	0
0	1	0	0	0	0	0	0	0	1	0
0	1	0	4	0	0	0	3	0	1	0
0	2	0	0	0	4	0	0	0	2	0
0	1	0	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	0	1	0

Với các giá trị 0 nằm được bao quanh bởi các giá trị 1,2 chúng ta sẽ vẽ nền đường và xe của người chơi chỉ có thể di chuyển trên các giá trị này. Còn những giá trị 0 bên ngoài có thể được sử dụng làm khoảng không gian để vẽ các đối tượng khác cho game sinh động hơn, chúng được ngăn cách bởi các giá trị 1 để bảo đảm xe không thể đi vào.

- Với các giá trị 1 chúng ta sẽ vẽ lề đường.
- Với các giá trị 2 chúng ta sẽ vẽ 1 biển hiệu, các biển hiệu này nằm trên khoảng không gian vẽ lề đường.
- Với các giá trị 3 chúng ta sẽ vẽ một đối tượng có thể ăn được để tăng điểm đối tượng này nằm trên mặt đường.
- Với các giá trị 4 chúng ta sẽ vẽ các đối tượng vật cản cũng nằm trên mặt đường.
- Với các giá trị 5 chúng ta sẽ vẽ đích về.

Vẽ car

Trên nền đường các xe sẽ được vẽ bằng tập các vertex và ban đầu xe được khởi tạo tại hàng các giá trị 0 cuối cùng.

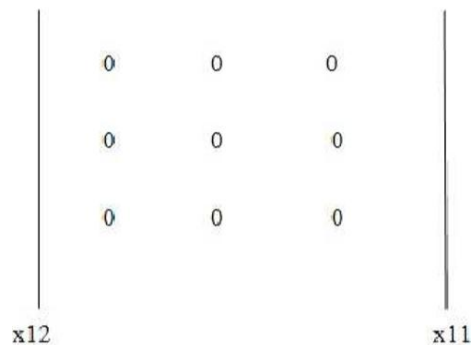
Về các đối tượng vật cản ăn được và không ăn được.

Minimap Một map nhỏ ánh xạ theo tỉ lệ với ma trận map ở trên thu nhỏ ở góc dưới màn hình nó hiển thị map cho xe và vị trí của các xe khác. Minimap được vẽ trên màn hình 2D Oxy.

9.8.3. Va chạm và xử lý va chạm

Va chạm xảy ra khi xe của người chơi va vào các đối tượng cản trở trên đường đi như các chướng ngại vật, lề đường, va chạm với các xe khác hoặc xuyên va chạm với những chướng ngại có thể ăn được để tăng điểm.

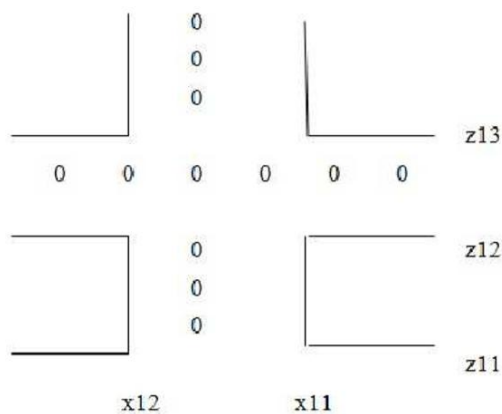
Như trên đã định nghĩa một ma trận đồ họa, thực chất mỗi giá trị 0,1 tương ứng với các miền giá trị để xác định danh giới giữa mặt đường và lề đường. Khoảng các giá trị 0 sẽ tương ứng với giới hạn các tọa độ nhất định, ví dụ như giới hạn của một đoạn thẳng AB, nó được giới hạn bởi 2 điểm A(xa, ya), B(xb, yb). Tương tự như vậy ở đây các giá trị 0 cũng có bề rộng được giới hạn bởi miền mà các giá trị 1 đã phân ra:



Hình 9.25. ma trận map cho đường và lề đường

Ở đây x11 và x12 chính là khoảng giá trị theo trục x sẽ giới hạn mặt đường và mặt đường chỉ có thể vẽ được khi nó nằm trong khoảng x11–x12. Do đó xe trong game này cũng vậy, khi mà nó được vẽ trên nền đường tức nó cũng sẽ bị giới hạn bởi x11–x12 do đó khi mà muốn dịch chuyển ngang theo trục x thì sẽ bị giới hạn bởi x11–x12, đó chính là cách xử lý để xe không thể ra được khỏi mặt đường.

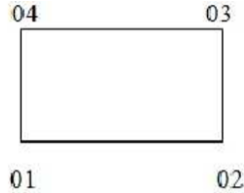
Nếu chúng ta có một map như sau:



Hình 9.26. ma trận map cho đường và lề đường

Nền đường chính là các giá trị 0 mà xe có thể đi, tuy nhiên giới hạn theo x11–x12 sẽ bị phá bỏ khi đến z21–z22, vì lúc này xe có thể đi trên đường giới hạn bởi z12–z13 và

không còn phụ thuộc vào giới hạn x nữa, nhưng cách xử lý rất đơn giản, khi đó chúng ta sẽ xoay toàn bộ hình 90° theo trục y để chuyển trục z về trục x và ngược lại, hoặc chuyển sự giới hạn về trục z khi miền giá trị giới hạn x không còn nữa. Mỗi giá trị 0, hay 1 được xác định bằng một diện tích nhất định, ví dụ giá trị 0 đầu tiên từ dưới lên, bên trái được xác định bằng 4 tọa độ không gian 01(x_1, y_1, z_1), 02(x_2, y_1, z_1), 03(x_2, y_3, z_3), 04(x_1, y_3, z_3).



Đây chính là cơ sở để có thể tính toán chính xác tọa độ nhờ cách quy định phạm vi các giá trị chúng ta có thể xử lý được các va chạm.

Với các va chạm chương ngại vật nhờ có quy định giá trị tương ứng với nó là 4 và các giá trị ăn được là 3, chúng ta sẽ cho phép xe đi qua các miền được xác định bởi các giá trị 3 và không cho qua các miền giá trị xác định bởi giá trị 4 như kiểu một miền giá trị giới hạn.

9.8.4. Âm thanh và hình ảnh sử dụng trong game

Âm thanh

Các tập tin âm thanh được sử dụng trong game là các file âm thanh dạng “.wav”. Tập tin wave là một dạng tập tin dùng để lưu trữ dữ liệu âm thanh thô (dạng sóng) và nó là một trong những định dạng phổ biến nhất của hệ điều hành windows. Các tập tin wave thuộc chuẩn RIF (Resource Interchange File Format—dạng tập tin tài nguyên có thể trao đổi). Có hai kiểu cấu trúc tệp tin wave là cấu trúc dạng nén và dạng chuẩn, chúng ta xét dạng chuẩn của nó:

Cấu trúc của tệp tin wave chuẩn: 3 khối

- Khối mô tả RIFF : xác định dạng RIFF và có kích thước 12 byte. Trong đó 4 byte đầu tiên chứa chuỗi “RIFF”, 4 byte cuối chứa chuỗi “WAVE”.
- Khối thuộc tính “fmt” : xác định các thuộc tính của dữ liệu âm thanh và có kích thước 24 byte, trong đó 4 byte đầu tiên chứa chuỗi “fmt”.
- Khối dữ liệu “data” : 4 byte đầu chứa chuỗi “data”.

Các loại âm thanh dùng trong game:

- Âm thanh nền game : Gồm các âm thanh nền cho phần load game, menu game, trong game, gameover.
- Âm thanh sự kiện : Ví dụ như khi người chơi điều khiển xe tăng ga thì âm thanh của động cơ xe sẽ to lên, âm thanh này có nguồn phát đi cùng với vị trí của xe...
- Âm thanh còi xe : Khi người chơi bấm còi xe chẳng hạn.
- Âm thanh va chạm hay ăn các đối tượng trên đường.

Hình ảnh sử dụng trong game

Các hình ảnh sử dụng trong game là tệp tin “.bmp”

Cấu trúc file .bmp : Header (54 byte) + Bảng màu (nếu có) + Data (màu từng điểm ảnh).

Header gồm 2 phần:

- BitmapHeader : chứa ký tự “BM”, kích thước file.. □
- BitmapInfor : Chứa các thông tin file như chiều rộng, cao, số bit cho 1 pixel...

Trong game các hình ảnh này sẽ được sử dụng để vẽ các giao diện load game, menugame...trong game thì được dùng nhiều trong vẽ xe, vẽ nền đường, vẽ các biển báo...

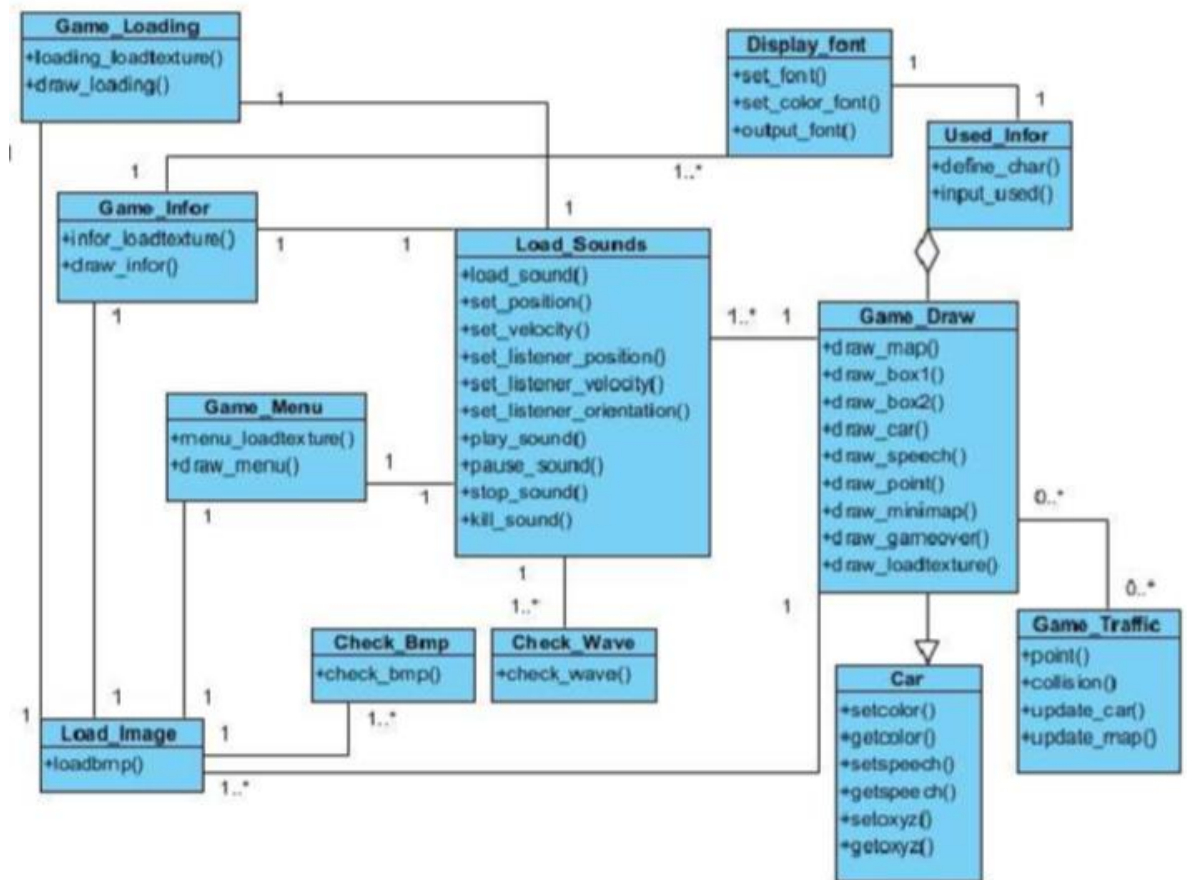
9.8.5. Xây dựng các class

- Class Check_Wave:
 - check_wave(): Kiểm tra đầu vào file âm thanh, có phải là file có định dạng RIFF và là kiểu file WAVE hay không.
- Class Load_Sounds:
 - load_sound(): Load sound âm thanh.
 - set_position(): Thiết lập vị trí nguồn phát âm.
 - set_velocity(): Thiết lập tốc độ nguồn phát âm.
 - set_listener_position() : Thiết lập vị trí listener.
 - set_listener_velocity() : Thiết lập tốc độ listener.
 - set_listener_orientation() : Thiết lập định hướng listener.
 - play_sound() : Play sound.
 - pause_sound() : Pause sound.
 - stop_sound() : Stop sound.
 - kill_sound() : Xóa bộ nhớ mà khi phát âm thanh nó đã chiếm, giải phóng tài nguyên.
- Class Check_Bmp:
 - check_bmp() : Kiểm tra hình ảnh đưa vào có phải là file .bmp hay không.
- Class Load_Image:
 - loadbmp() : Load image.
- Class Game_Loading :
 - loading_loadtexture() : Load image và convert texture.
 - draw_loading() : Vẽ màn hình load game.
- Class Display_Font : Định nghĩa cách in ký tự trên màn hình OpenGL dùng cho việc người dùng đánh tên vào trước khi bắt đầu chơi, hiển thị thông tin của game.
 - set_font() : Định nghĩa font chữ sẽ thể hiện trên màn hình game infor...

- `set_color_font()` : Thiết lập màu chữ hiển thị font.
- `output_font()`: Hiển thị các ký tự lên màn hình.
- Class `Game_HighScore`:
 - `highscore_loadtexture` : Ảnh nền highscore.
 - `highscore()` : Hiển thị thông tin về điểm cho người chơi.
- Class `Game_Infor` : Giới thiệu game và hướng dẫn chơi.
 - `infor_loadtexture()` : Load ảnh nền.
 - `draw_infor()` : Hiển thị các thông tin lên màn hình.
- Class `Used_Infor` : Cho phép người chơi nhập tên của mình, lưu tên của người chơi trước khi bắt đầu chơi.
 - `define_char()` : Định nghĩa các ký tự bàn phím để có thể đánh vào tên người chơi.
 - `input_used()` : Nhập tên người chơi.
- Class `Game_Menu`: Xây dựng giao diện menu game.
 - `menu_loadtexture()` : Load ảnh nền.
 - `draw_menu()` : Vẽ màn hình menu.
- Class `Game_Draw` : Định nghĩa các hàm vẽ chính trong game.
 - `draw_loadtexture()`;
 - `draw_map()` : Vẽ nền game : mặt đường, lề...
 - `draw_box_1()`: Vẽ các đối tượng vật cản.
 - `draw_box_2()` : Vẽ các đối tượng ăn được để tính điểm.
 - `draw_car()` : Vẽ xe.
 - `draw_speech()` : Hiển thị và cập nhật tốc độ.
 - `draw_point()` : Hiển thị điểm và cập nhật.
 - `draw_minimap()` : Hiển thị minimap.
 - `draw_gameover()`;
- Class `Game_Traffic` : Thực hiện tính toán, cập nhật các thông số dữ liệu
 - `point()` : Tính điểm.
 - `collision()` : Xử lý, tính toán khi có va chạm.
 - `update_car()` : Cập nhật tọa độ xe.
 - `update_map()` : Cập nhật minimap.
- Class `Car` : Thiết lập màu sắc, speech, tọa độ xe.
 - `setColor()`.
 - `getColor()`.
 - `setspeech()`.
 - `getspeech()`.

- setoxyz().
- getoxyz().

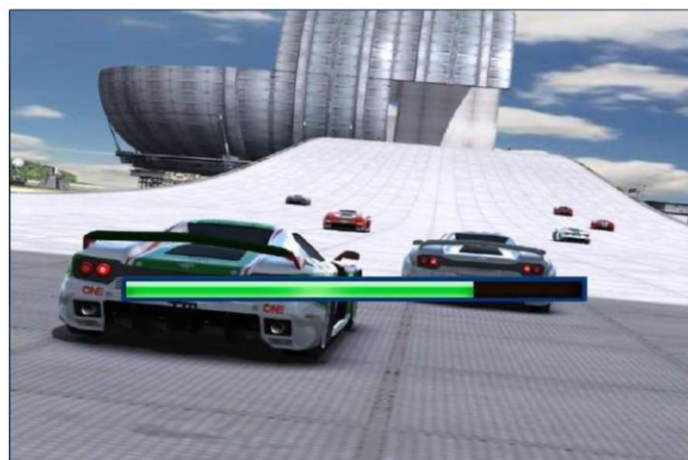
9.8.6. Quan hệ giữa các lớp



Hình 31. Quan hệ giữa các lớp

9.8.7. Một số giao diện cho chương trình game

Giao diện load game



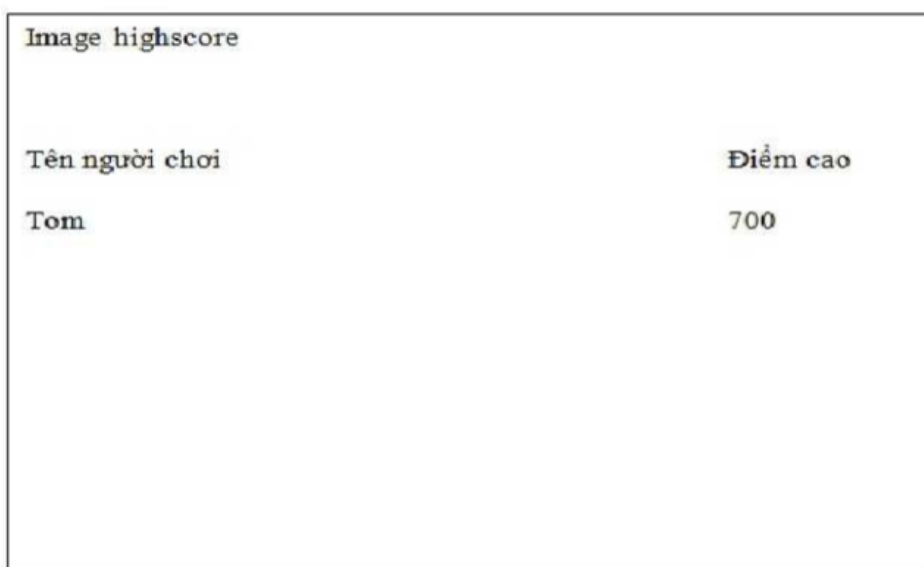
Hình 9.27. Load game

Giao diện menu game

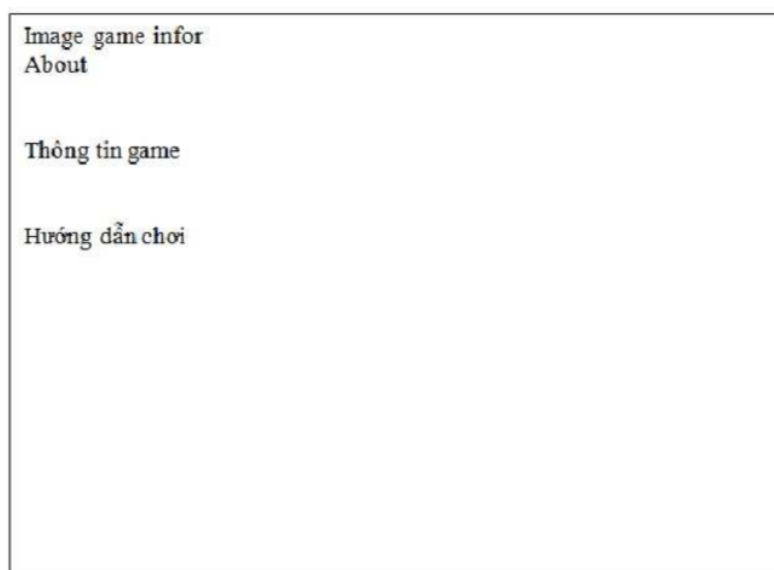


Hình 9.28. Menu game

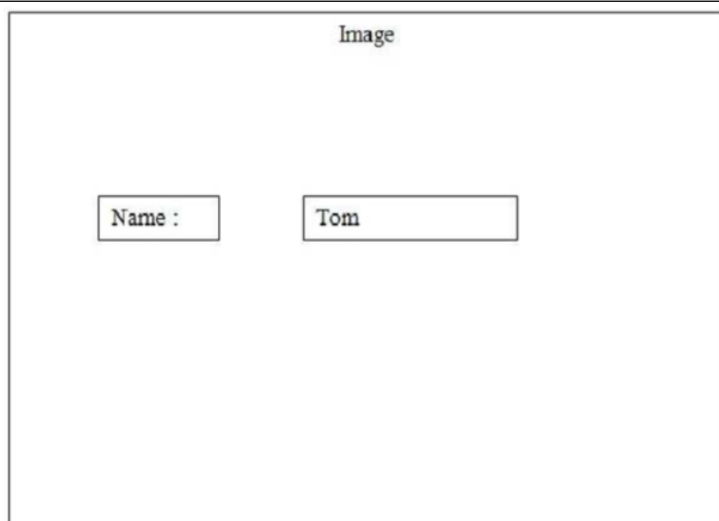
Mô phỏng các giao diện khác:



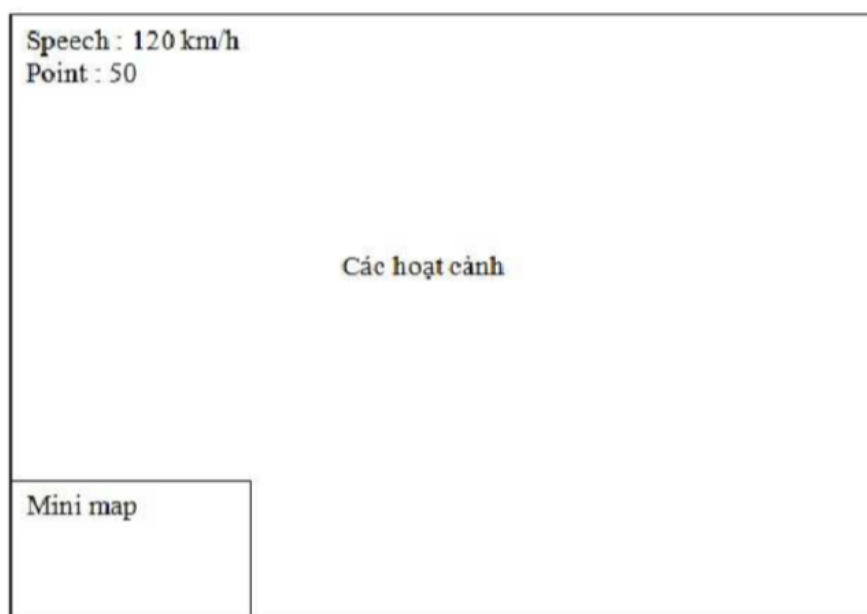
Hình 9.29. Giao diện Highscore



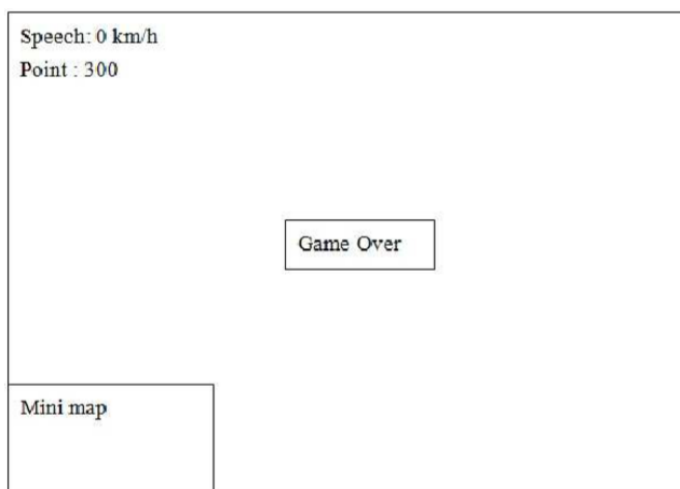
Hình 9.30. Giao diện Game infor



Hình 9.31. Giao diện nhập thông tin người chơi



Hình 9.32. Giao diện game



Hình 9.33. Game Over.

Tóm tắt chương

Với thư viện đồ họa OpenGL chúng ta có thể vẽ các hình như điểm, đa giác, tam giác... Thêm nữa có thể phối hợp các hình này để vẽ các kiểu hình phức tạp hơn. Không những thế OpenGL còn cung cấp đầy đủ các yếu tố như màu sắc, ánh sáng... từ đó chúng ta có thể thao tác và làm việc hiệu quả với các ứng dụng đồ họa để đem lại những hình ảnh sinh động, đa màu sắc trong thế giới ba chiều. Sự hỗ trợ mạnh mẽ này khiến OpenGL trở thành một sự lựa chọn tốt cho những sản phẩm cần đến sự góp mặt của đồ họa. Thậm chí thư viện đồ họa này còn chạy trên các ứng dụng web, thiết bị thông minh như điện thoại...

PHỤ LỤC

Hướng dẫn sử dụng thư viện đồ hoạ trong C/C++ hay BC

1. Yêu cầu

Phải có tập tin điều khiển màn hình EGAVGA.BGI (thông thường tệp này thường nằm trong thư mục \BC\BGI hay TC\BGI khi cài đặt).

Nếu chúng ta có sử dụng tới font chữ thì cần phải có thêm các file (*.CHR) như: GOTH.CHR (chữ Gothic), LITT.CHR (chữ Small Font), SANS.CHR (chữ Sans Serif), TRIP.CHR (chữ cao gấp 3).

Để dùng được thư viện các hàm đồ hoạ cần có dòng lệnh:

#include <graphics.h> và đặt mục chọn Graphics library là ON ([x] trong menu Options/Linker/Libraries.

Khi cần tham khảo cú pháp, cách sử dụng của bất kỳ một hàm đồ hoạ nào, đưa con trỏ về tên hàm trong chương trình sau đó nhấn tổ hợp phím CTRL+F1. Muốn tham khảo danh sách toàn bộ các hàm của thư viện đồ hoạ nhấn tổ hợp phím CTRL+F1 ngay tại dòng chữ <graphics.h>.

2. Khởi tạo và đóng chế độ đồ hoạ

Độ phân giải của màn hình được đo bằng số điểm theo chiều ngang nhân với số điểm theo chiều dọc của màn hình đồ hoạ. Toạ độ gốc của màn hình đồ hoạ (0,0) là điểm nằm tại góc trên cùng phía bên trái. Mỗi kiểu đồ hoạ dùng một hệ toạ độ riêng. Hệ toạ độ cho màn hình VGA là 640x480.

Khởi động đồ hoạ với màn hình ngằm định:

```
#include <graphics.h>
void main(void) {
    int    gdriver, gmode, errorcode;
    gdriver = DETECT; //ngằm định
    initgraph(&gdriver, &gmode, "C:\\TC\\BGI"); //tìm mode màn hình
    trong thư mục BGI
    errorcode = graphresult();
    if (errorcode != grOk) {
        printf("\n Không khởi tạo được");
        getch(); exit(1);
    }
    .....// Các thao tác đồ hoạ tiếp theo
    closegraph();
}
```

Ví dụ: viết chương trình hai đường thẳng cắt nhau

```
#include <conio.h>
#include <graphics.h>
#include <math.h>
```

```
// hệ số đổi từ độ sang radian
#define RADS 0.017453293
void giaodiem(double x1, double y1, double x2, double y2,
double a1, double b1, double a2, double b2) {
double dx, dy, da, db, x, y, t, tich;
dx = x2 - x1;
dy = y2 - y1;
da = a2 - a1;
db = b2 - b1;
tich = db * dx - da * dy;
if (tich != 0) {
t = ((a1 - x1) * dy - (b1 - y1) * dx) / tich;
if (t >= 0.0 && t <= 1.0) {
x = t * (a2 - a1) + a1;
y = t * (b2 - b1) + b1;
line(x1, y1, x2, y2);
line(a1, b1, a2, b2);
setfillstyle(SOLID_FILL, RED);
fillellipse(x, y, 3, 3);
}
}
}
void main() {
int gr_drive = DETECT, gr_mode;
double x1, y1, x2, y2, a1, b1, a2, b2;
printf("\nNhap vao toa do doan thang thu nhat: ");
scanf("%lf%lf%lf%lf", &x1, &y1, &x2, &y2);
printf("\nNhap vao toa do doan thang thu hai: ");
scanf("%lf%lf%lf%lf", &a1, &b1, &a2, &b2);
initgraph(&gr_drive, &gr_mode, "");
giaodiem(x1, y1, x2, y2, a1, b1, a2, b2);
getch();
closegraph(); //đóng chế độ đồ hoạ
}
```

3. Các hàm cơ bản

3.1. Bảng màu của màn hình đồ hoạ.

Setbkcolor(int color)	thiết lập màu nền cho màn hình đồ hoạ
setcolor(int color)	đặt màu vẽ cho nét vẽ hiện tại
getmaxcolor()	lấy số màu cao nhất được sử dụng trong chế độ đồ hoạ hiện tại
setallpalette(struct palettetype far *palette)	sẽ làm thay đổi toàn bộ màu trong bảng màu palette
setpalette(int colox, int colory)	sẽ làm thay đổi màu thứ colox thành màu colory

getpalette(struct palettetype far *palette)	sẽ lấy lại giá trị của palette đang dùng
textheight("W")	lấy chiều cao của dòng văn bản
outtextxy(int x, int y, char * msg)	Chỉ thị đưa xâu kí tự msg ra màn hình đồ hoạ tại vị trí (x,y)

3.2. Điểm

Đơn giản nhất chính là điểm, mọi đối tượng khác đều được xây dựng nên từ điểm.

int getmaxx()	Số chấm điểm lớn nhất theo chiều ngang trong chế độ đồ hoạ hiện tại
int getmaxy()	Số chấm điểm lớn nhất theo chiều dọc trong chế độ đồ hoạ hiện tại
putpixel(x, y, color)	Đề vẽ một điểm sáng lên màn hình tại điểm (x, y) có màu color
getpixel(x,y)	nhận biết màu hiện tại của điểm (x, y)

3.3. Đường

Các hàm cho đường:

moveto(int x, int y)	di chuyển vị trí con trỏ hiện tại của màn hình đồ hoạ tới toạ độ (x,y)
getx(), gety()	lấy toạ độ của con trỏ hiện tại theo chiều ngang và chiều dọc màn hình đồ hoạ
lineto(int x, int y)	vẽ một đường thẳng từ vị trí con trỏ hiện tại tới vị trí có toạ độ (x, y) trên màn hình đồ hoạ
line(int x1, int y1, int x2, int y2)	Vẽ một đường thẳng từ toạ độ (x1,y1) đến toạ độ (x2, y2) trên màn hình đồ hoạ. Đường thẳng mới được vẽ không phụ thuộc vào vị trí hiện thời của con trỏ màn hình
linereel(int dx, int dy)	Vẽ một đoạn thẳng từ vị trí con trỏ hiện tại tới vị trí (x +dx, y+dy). Sau khi vẽ con trỏ chuyển tới vị trí mới (x+dx, y+dy)

3.4. Hình chữ nhật

rectangle(int x1, int y1, int x2, int y2) vẽ hình chữ nhật có toạ độ phía góc trên bên trái là (x1, y1) và góc dưới bên phải có toạ độ (x2,y2).

bar(int x1, int y1, int x2, int y2) vẽ hình chữ nhật có tô màu phía trong. Hai chỉ thị rectangle và bar khác nhau ở chỗ rectangle chỉ tạo nên một hình chữ nhật với đường viền bao quanh.

bar3d(int x1,int y1,int x2,int y2,int depth, int top) vẽ khối hộp chữ nhật, mặt ngoài của nó là hình chữ nhật xác định bởi các toạ độ (x1,y1,x2,y2) hình chữ nhật này được tô màu, depth là chiều sâu của khối 3 chiều, top nhận giá trị 1 hoặc 0 để khối 3 chiều có nắp hay không có nắp.

3.5. Hình tròn

Tất cả các hàm dưới đây, góc tính theo độ và giá trị từ 0⁰ đến 360⁰.

arc(int x, int y, int gd,int gc,int r) vẽ cung tròn có (x,y) toạ độ tâm tròn, r bán kính, gd góc đầu, gc góc cuối.

circle(int x, int y, int r) là hàm vẽ hình tròn có tâm tại điểm (x,y) với bán kính r.

ellipse(int x, int y, int gd, int gc, int xr, int yr) vẽ 1 hình ellipse có (x,y) tọa độ tâm cung, gd góc đầu, gc góc cuối, xr là bán trục ngang, yr là bán trục đứng.

pieslice(int x, int y, int gd, int gc, int r) vẽ và tô màu hình quạt có (x,y) là tọa độ tâm quạt, gd là góc đầu, gc là góc cuối, r là bán kính.

3.6. Đa giác

drawpoly(int numpoints, int far *polypoints) sẽ vẽ nên một đường gấp khúc bất kỳ với numpoints là số điểm mà đường gấp khúc đi qua, polypoints (mảng) tọa độ điểm (x1,y1,x2,y2,...). Khi điểm cuối (xn,yn) trùng với điểm đầu (x1,y1) thì được một đa giác.

fillpoly(int numpoints, int *polypoints) sẽ tô màu đa giác bằng màu hiện thời.

setfillstyle(int pattern, int color) dùng để xác định mẫu tô cho đa giác, trong đó mẫu tô là các hằng số nguyên được định nghĩa như sau:

Tên hằng mẫu	Giá trị	Mô tả
EMPTY_FILL	0	Tô bằng màu nền
SOLID_FILL	1	Tô bằng nét liền
LINE_FILL	2	Tô -----
LTSLASH_FILL	3	Tô ///
SLASH_FILL	4	Tô //// in đậm
BKSLASH_FILL	5	Tô \\\\ in đậm
LTBKSLASH_FILL	6	Tô \\
HATCH_FILL	7	Tô đường gạch bóng nhật
XHATCH_FILL	8	Tô đường gạch bóng chữ thập
INTERLEAVE_FILL	9	Tô đường đứt quãng
WIDE_DOT_FILL	10	Tô bằng dấu chấm thưa
CLOSE_DOT_FILL	11	Tô bằng dấu chấm dày

3.7. Văn bản

outtext(char far *textstring) sẽ hiển thị nội dung xâu textstring tại vị trí hiện thời của màn hình đồ họa.

outtextxy(int x, int y, char far *textstring) hiển thị nội dung xâu textstring tại tọa độ (x, y) trong màn hình đồ họa.

settextstyle(int font, int direction, int charsize) dùng để xác lập kiểu chữ với các font chữ khác nhau.

Trong đó int font được xác lập thông qua các hằng sau:

TÊN FONT	Giá trị	Ý nghĩa
DEFAULT_FONT	0	Font 8x8 bit-mapped

TRIPLEX_FONT	1	Stroked triplex font
SMALL_FONT	2	Stroked small font
SANS_SERIF_FONT	3	Stroked sans-serif font
GOTHIC_FONT	4	Stroked gothic font

int direction được xác định nếu `HORIZ_DIR = 0` là nằm ngang từ trái qua phải, `VERT_DIR = 1` là thẳng đứng từ dưới lên trên.

int charsize nhận giá trị từ 1 đến 10 là hệ số phóng đại chữ.

`settextjustification( int hoz, int vert)` xác định vị trí dòng văn bản được đưa ra màn hình đồ hoạ bởi `outtext()` và `outtextxy()`.

Trong đó int hoz có thể nhận một trong các hằng sau:

`LEFT_TEXT = 0` văn bản xuất hiện phía bên trái con trỏ màn hình đồ hoạ

`CENTER_TEXT = 1` văn bản xuất hiện ở giữa với tâm là con trỏ màn hình đồ hoạ

`RIGHT_TEXT = 2` văn bản xuất hiện phía bên phải con trỏ màn hình đồ hoạ

Còn int vert là tham số có thể nhận các giá trị sau:

`BOOTTOM_TEXT = 0` văn bản xuất hiện ở phía trên con trỏ

`CENTER_TEXT = 1` văn bản xuất hiện ở quanh con trỏ

`TOP_TEXT = 2` văn bản xuất hiện ở phía dưới con trỏ

`textheight(char *s)` trả về chiều cao (theo pixel) của chuỗi do s trở tới. Với 8x8 bit map Font và hệ số khuếch đại chữ là 1 thì `textheight("H")=8`

`textwidth(char *s)` trả về chiều dài của chuỗi tính theo pixel.

3.8. Cửa sổ (viewport)

viewport là 1 vùng hình chữ nhật trên màn hình đồ hoạ giống như window trong textmode.

`setviewport(int x1, int y1, int x2, int y2, int clip)` trong đó (x1,y1,x2,y2) là góc trái trên và góc phải dưới thoả mãn điều kiện như sau:

$0 \leq x1 \leq x2$ và $0 \leq y1 \leq y2$

Tham số clip: clip=1 không cho phép vẽ ra ngoài viewport

clip=0 cho phép vẽ ra ngoài viewport

`getviewsettings(struct viewporttype *vp)` nhận viewport hiện hành,

```
struct viewporttype {
    int left, top, right, bottom;
    int clip;
}
```

`clearviewport(void)` xoá viewport

`cleardevice(void)` xoá mọi thứ trên màn hình và đưa con trỏ về toạ độ (0,0) của màn hình

Chú ý: nhờ sử dụng viewport ta có thể viết các chương trình đồ hoạ có trục toạ độ, bằng cách thiết lập viewport với `clip=0` (cho phép vẽ ra ngoài giới hạn)

3.9. Tạo hình ảnh chuyển động

Nguyên sơ mới mà chúng ta sẽ xây dựng là tạo hình ảnh chuyển động trên màn hình đồ hoạ. Về nguyên tắc, để có thể tạo nên những hình ảnh chuyển động chúng ta cần có một hình mẫu, sau đó lưu hình mẫu trên màn hình đồ hoạ lại bằng chỉ thị `getimage(int x1, int y1, int x2, int y2, void far *bitmap)`; trong đó `bitmap` là miền nhớ dùng để lưu hình ảnh của hình chữ nhật có toạ độ (x1,y1) và (x2, y2) trên màn hình đồ hoạ.

Chỉ thị `getimagesize(int x1, int y1, int x2, int y2)` dùng để xác định kích thước bộ nhớ dùng để cất hình ảnh giới hạn trong hình chữ nhật có toạ độ (x1, y1), (x2,y2).

Chỉ thị `putimage(int x, int y, void far * bitmap, int copymode)` dùng để khôi phục lại hình ảnh đã được cất giữ bằng `getimage()`. Chỉ thị `putimage()` kết hợp với chỉ thị làm trễ (`delay()`) sao cho số các hình ảnh hiển thị trên màn hình đồ hoạ khoảng 24 hình/giây chúng ta sẽ nhận được một hình ảnh chuyển động.

Ví dụ 1: Tạo hình ảnh chuyển động là một mũi tên.

```
#include <graphics.h>
#include <stdlib.h>
#include <stdio.h>
#include <conio.h>
#define ARROW_SIZE 10
void draw_arrow(int x, int y);
int main(void) {
    /* request autodetection */
    int gdriver = DETECT, gmode, errorcode;
    void *arrow;
    int x, y, maxx;
    unsigned int size;
    /* initialize graphics and local variables */
    initgraph(&gdriver, &gmode, "\\TC\\BGI");
    /* read result of initialization */
    errorcode = graphresult();
    if (errorcode != grOk) { /* an error occurred */
        printf("Graphics          error:          %s\n",
grapherrormsg(errorcode));
        printf("Press any key to halt:");
        getch();
        exit(1); /* terminate with an error code */
    }
    maxx = getmaxx();  x = 0;
    y = getmaxy() / 2;
    /* draw the image to be grabbed */
```

```
draw_arrow(x, y);
    /* calculate the size of the image */
size = imagesize(x, y-ARROW_SIZE, x+(4*ARROW_SIZE),
y+ARROW_SIZE);
    /* allocate memory to hold the image */
arrow = malloc(size);
    /* grab the image */
getimage(x, y-ARROW_SIZE, x+(4*ARROW_SIZE), y+ARROW_SIZE,
arrow);
    /* repeat until a key is pressed */
while (!kbhit()){
    /* erase old image */
    putimage(x, y-ARROW_SIZE, arrow, XOR_PUT);
    x += ARROW_SIZE;
    if (x >= maxx)
        x = 0;
    /* plot new image */
    putimage(x, y-ARROW_SIZE, arrow, XOR_PUT);
}
    /* clean up */
free(arrow);
closegraph();
return 0;
}

void draw_arrow(int x, int y) {
    /* draw an arrow on the screen */
    moveto(x, y);
    linerel(4*ARROW_SIZE, 0);
    linerel(-2*ARROW_SIZE, -1*ARROW_SIZE);
    linerel(0, 2*ARROW_SIZE);
    linerel(2*ARROW_SIZE, -1*ARROW_SIZE);
}
```

Các code chương trình ví dụ cho bài tập lập trình

Bài 1: quay đổi tượng

```
#include<stdio.h>
#include<dos.h>
#include<conio.h>
#include<graphics.h>
#include<stdlib.h>
#include<math.h>
#include<alloc.h>
#define RADS 0.017453293
struct point{
    int x,y,z;
}
```

```
    a[4]={
    {110,180,80},
    {10,200,80},
    {120,50,80},
    {110,180,250}
    };
int noi[4][4];
void Bresenham_Line(int x1,int y1, int x2, int y2, int c){
    if(x1 == x2){
        int i;
        if(y1 < y2)
            for(i = y1; i <= y2;i ++){
                putpixel(x1,i,c);          delay(10);
            }
        else
            for(i = y2; i<= y1; i ++){
                putpixel(x1,i,c);          delay(10);
            }
    }
    if(y1 == y2){
        int i;
        if(x1 < x2)
            for(i = x1; i <= x2; i ++){
                putpixel(i,y1,c);delay(10);
            }
        else
            for(i = x2; i <= x1; i ++){
                putpixel(i,y1,c);          delay(10);
            }
    }
    if(x1 < x2){
        if(y1 < y2) {
            if((y2 - y1)/(x2-x1) < 1){
                int i;
                int Dx = x2 - x1;
                int Dy = y2 - y1;
                int p = 2*Dy - Dx;
                putpixel(x1,y1,c);
                for(i = x1; i < x2; i ++){
                    if(p < 0)
                        p += 2*Dy;
                    else{
                        p += 2*(Dy - Dx);
                        y1 ++;
                    }
                }
                x1 ++;
            }
        }
    }
}
```

```

        putpixel(x1,y1,c);          delay(10);
    }
}
else{
    int i;
    int Dx = x2 - x1;
    int Dy = y2 - y1;
    int p = 2*Dx - Dy;
    putpixel(x1,y1,c);
    for(i = y1; i < y2; i ++){
        if(p < 0)
            p += 2*Dx;
        else{
            p += 2*(Dx - Dy);
            x1 ++;
        }
        y1 ++;
        putpixel(x1,y1,c);          delay(10);
    }
}
}
if(y1 > y2)
{
    if((y2 - y1)/(x2 - x1) > -1 ){
        int i;
        int Dx = x2 - x1;
        int Dy = y2 - y1;
        int p = 2*Dy + Dx;
        putpixel(x1,y1,c);
        for(i = x1; i <= x2; i ++){
            if(p > 0)
                p += 2*Dy;
            else{
                p += 2*(Dy + Dx);
                y1-- ;
            }
            x1 ++;
            putpixel(x1,y1,c);      delay(10);
        }
    }
}
else {
    int i;
    int Dx = x2 - x1;
    int Dy = y2 - y1;
    int p = 2*Dx + Dy;
    putpixel(x1,y1,c);
    for(i = y1; i >= y2; i --){

```

```
        if(p < 0)
            p += 2*Dx;
        else{
            p += 2*(Dx + Dy);
            x1++;
        }
        y1--;
        putpixel(x1,y1,c);        delay(10);
    }
}
}
}
if(x1 > x2){
    if(y1 < y2)
        {   if((y2-y1)/(x2-x1) > -1){
            int i;
            int Dx = x2 - x1;
            int Dy = y2 - y1;
            int p = -2*Dy - Dx;
            putpixel(x1,y1,c);
            for(i = x1; i > x2; i --){
                if(p < 0){
                    p=p - 2*Dy - 2*Dx;
                    y1 ++;
                }
                else
                    p =p - 2*Dy;
                x1 --;
                putpixel(x1,y1,c);delay(10);
            }
        }
        else {
            int i;
            int Dx = x2 - x1;
            int Dy = y2 - y1;
            int p = -2*Dx - Dy;
            putpixel(x1,y1,c);
            for(i = y1; i < y2; i ++){
                if(p < 0)
                    p -= 2*Dx;
                else{
                    p -= 2*(Dx + Dy);
                    x1 --;
                }
                y1 ++;
                putpixel(x1,y1,c);    delay(10);
            }
        }
    }
}
```

```
        }
    }
}
if(y1 > y2){
    if((y2-y1)/(x2-x1) < 1) {
        int i;
        int Dx = x2 - x1;
        int Dy = y2 - y1;
        int p = 2*Dy - Dx;
        putpixel(x1,y1,c);
        for(i = x1; i > x2; i --){
            if(p > 0){
                p =p - 2*Dy +2*Dx ;
                y1--;
            }
            else
                p -= 2*Dy;

            x1 --;
            putpixel(x1,y1,c);                delay(10);
        }
    }
    else {
        int i;
        int Dx = x1 - x2;
        int Dy = y1 - y2;
        int p = 2*Dx + Dy;
        putpixel(x1,y1,c);
        for(i = y1; i > y2; i --){
            if(p < 0) {
                p =p - 2*Dx + 2*Dy ;
                x1 --;
            }
            else
                p =p - 2*Dx ;
            y1 --;
            putpixel(x1,y1,c);                delay(10);
        }
    }
}

}
}
void Dothi(){
```

```
Bresenham_Line(getmaxx()/2,getmaxy()/2,getmaxx()-
10,getmaxy()/2,15);
Bresenham_Line(getmaxx()/2,10,getmaxx()/2,getmaxy()/2,15);
Bresenham_Line(getmaxx()/2,getmaxy()/2,getmaxx()/2-
170,getmaxy()/2+170,15);
Bresenham_Line(getmaxx()/2-170,getmaxy()/2+170,getmaxx()/2-
170,getmaxy()/2+166,15);
Bresenham_Line(getmaxx()/2-170,getmaxy()/2+170,getmaxx()/2-
163,getmaxy()/2+167,15);
settextjustify(CENTER_TEXT,CENTER_TEXT);
outtextxy(getmaxx()-9,getmaxy()/2+1,"");
outtextxy(getmaxx()-9,getmaxy()/2+10,"y");
outtextxy(getmaxx()/2,10,"-");
outtextxy(getmaxx()/2 - 10,10,"z");
outtextxy(getmaxx()/2-155,getmaxy()/2+170,"x");
}
point Diem3d(int x , int y , int z){
point p;
if(x>=0&&y>=0&&z>=0) {
p.x = int(getmaxx()/2+y - x*cos(RADS*45));
p.y = int(getmaxy()/2-z + x*cos(RADS*45));
}
if(y>=0&&x<0&&z>=0) {
p.x = int(y+getmaxx()/2-x*cos(RADS*45));
p.y = int(getmaxy()/2-z+x*cos(RADS*45));
}
if(x>=0&&y<0&&z>=0) {
p.x = int(getmaxx()/2+y-x*cos(RADS*45));
p.y = int(getmaxy()/2-(z-x*cos(RADS*45)));
}
if(x>=0&&y>=0&&z<0) {
p.x = int(getmaxx()/2+y-x*cos(RADS*45));
p.y = getmaxy()/2-z+x*cos(RADS*45);
}
if(y>=0&&x<0&&z<0) {
p.x = int(getmaxx()/2+y-x*cos(RADS*45));
p.y = int(getmaxy()/2+(-z-x*cos(RADS*45)));
}
if(x>=0&&y<0&&z<0) {
p.x = int(getmaxx()/2+y-x*cos(RADS*45));
p.y = int(getmaxy()/2-z+x*cos(RADS*45));
}
if(z>=0&&y<0&&x<0) {
p.x = int(getmaxx()/2-(-y+x*cos(RADS*45)));
p.y = int(getmaxy()/2-(z-x*cos(RADS*45)));
}
```

```
    if(x<0&&y<0&&z<0) {
        p.x = int(getmaxx()/2+y-x*cos(RADS*45));
        p.y = int(getmaxy()/2-z+x*cos(RADS*45));
    }
    return(p);
}
point hinhchieu(int &x, int &y , int &z , int chieu){
    point p;
    if(chieu==1){
        p.x =int( getmaxx()/2+y - x*cos(RADS*45));
        p.y =int( getmaxy()/2 + x*cos(RADS*45));
    }
    if(chieu==2){
        p.x =int ( getmaxx()/2+y-x*cos(RADS*45)-y);
        p.y =int ( getmaxy()/2-z+x*cos(RADS*45));
    }
    if(chieu == 3){
        p.x =int( getmaxx()/2+y-x*cos(RADS*45)+x*sin(RADS*45));
        p.y =int( getmaxy()/2-z);
    }
    return(p);
}
point quay(int &x, int &y, int &z, int goc , int chieu){
    point p;
    if(chieu==1){
        p.x = x*cos(RADS*goc) + z*sin(RADS*goc);
        p.z = -x*sin(RADS*goc) + z * cos(RADS*goc);
        p.y = y;
    }
    if(chieu==2){
        p.y = y*cos(RADS*goc) - z*sin(RADS*goc);
        p.z = y*sin(RADS*goc) + z * cos(RADS*goc);
        p.x =x;
    }
    if(chieu==3) {
        p.x = x*cos(RADS*goc)-y*sin(RADS*goc);
        p.y = x*sin(RADS*goc)+y*cos(RADS*goc);
        p.z = z;
    }
    return p;
}
void chieumat(point k[],int t, int m ){
    int i,j=0;
    int n[4][4];
    for(i = 0 ; i<4 ; i++)
        for(j = 0 ; j<4 ; j++) {
```

```
n[i][j]=1;
n[j][i]=1;
}
for(i = 0 ; i< 4 ; i++) {
    k[i] = hinhchieu(k[i].x,k[i].y,k[i].z,t);
}
for( i = 0 ; i< 4 ; i++)
for( j = 0 ; j< 4 ; j++){
    if(i!=j&& n[i][j]==1&& n[j][i]==1){
        Bresenham_Line(k[i].x,k[i].y,k[j].x,k[j].y,m);
        n[i][j]=0;
        n[j][i]=0;
    }
}
}
void main(){
    clrscr();
    point b[4],c[4],l[4];
    int j=0;
    int goc[2]={45,35};
    int driver = DETECT, mode;
    initgraph(&driver,&mode,"c:\\tc\\bgi");
    Dothi();
    for(int i = 0; i < 4; i ++){
        b[i] = Diem3d(a[i].x,a[i].y,a[i].z);
        for( i = 1; i < 4; i ++ ){
            Bresenham_Line(b[j].x,b[j].y,b[i].x,b[i].y,10);
        }
        Bresenham_Line(b[1].x,b[1].y,b[2].x,b[2].y,10);
        Bresenham_Line(b[1].x,b[1].y,b[3].x,b[3].y,10);
        Bresenham_Line(b[2].x,b[2].y,b[3].x,b[3].y,10);
        setttextjustify(LEFT_TEXT,RIGHT_TEXT);
        printf("\nQuay quanh trục : oy goc 45");
        printf("\nQuay quanh trục : ox goc 35.5");
        for( i = 0 ; i< 4 ; i++){
            c[i] = quay(a[i].x , a[i].y , a[i].z , goc[1] ,1);
        }
        for(i = 0 ; i< 4 ; i++) {
            c[i] = quay(c[i].x,c[i].y,c[i].z,goc[2],2);
        }
        for(i = 0; i< 4; i ++){
            l[i] = c[i];
        }
        for( i = 0; i < 4; i ++){
            c[i] = Diem3d(c[i].x,c[i].y,c[i].z);
            outtextxy(10,110,"Chon mat chieu: ");
            outtextxy(10,120,"xoy: Bam so 1.");
        }
    }
```

```
outtextxy(10,130,"xoz: Bam so 2.");
outtextxy(10,140,"yoz: Bam so 3.");
outtextxy(10,150,"Bam so 4 de ket thuc");
char m;
do{
    m = getch();
    for(i = 0;i < 4; i ++){
        c[i] = l[i];
        switch (m){
            case '1' :{
                chieumat(c,1,5);
                break;
            }
            case '2' :{
                chieumat(c,2,7);
                break;
            }
            case '3' : {
                chieumat(c,3,9) ;
                break;
            }
        }
    }while( m !='4');
    closegraph();
}
```

Bài 2: xén tỉa

```
#include <conio.h>
#include <graphics.h>
#include <math.h>
#include <stdlib.h>
#define ROUND(a) ((double) (a+0.5))
#define TRUE 1
#define FALSE 0
int cliptest(double p, double q, double *u0, double *u1){
    double r;
    int retVal = TRUE;
    if (p < 0.0){
        r = q / p;
        if (r > *u1)
            retVal = FALSE;
        else
            if (r > *u0)
                *u0 = r;
    }
    else
        if ( p > 0.0){
```

```
        r = q / p;
        if (r < *u0)
            retVal = FALSE;
        else
            if (r < *u1)
                *u1 = r;
    }
    else{
        if (q < 0.0)
            retVal = FALSE;
    }
    return (retVal);
}

void clipline(double x1, double y1, double x2, double y2,
              double xmin, double ymin, double xmax, double ymax){
    double u0 = 0.0, u1 = 1.0, dx, dy;
    double oldx1, oldy1, oldx2, oldy2;
    oldx1 = x1;
    oldy1 = y1;
    oldx2 = x2;
    oldy2 = y2;
    dx = x2 - x1;
    if (cliptest(-dx, x1 - xmin, &u0, &u1))
        if (cliptest(dx, xmax-x1, &u0, &u1)){
            dy = y2 - y1;
            if (cliptest(-dy, y1 - ymin, &u0, &u1))
                if (cliptest(dy, ymax - y1, &u0, &u1)){
                    if (u1 < 1.0){
                        x2 = x1 + u2 * dx;
                        y2 = y1 + u2 * dy;
                    }
                    if (u0 > 0.0){
                        x1 += u0 * dx;
                        y1 += u0 * dy;
                    }
                }
            setcolor(RED);
            line(oldx1, oldy1, x1, y1);
            line(x2, y2, oldx2, oldy2);
            setcolor(YELLOW);
            line(x1, y1, x2, y2);
        }
    }
}

void main(){
    int gr_drive = DETECT, gr_mode;
    char c;
```



```
double x1, y1, x2, y2;
initgraph(&gr_drive, &gr_mode, "");
rectangle(100,50, getmaxx()-100, getmaxy()-50);
randomize();
outtextxy(100, getmaxy()-10, "Nhan phim bat ky de sinh
duong khac; ESC de thoat");
do {
    x1 = random(getmaxx() / 2);
    y1 = random(getmaxy());
    x2 = getmaxx()/2 + random(getmaxx() / 2);
    y2 = random(getmaxy());
    clipline(x1, y1, x2, y2, 100, 50, getmaxx()-100,
getmaxy()-50);
    c = getch();
} while (c != 27);
closegraph();
}
```

Bài 3: Phép chiếu

```
#include<stdio.h>
#include<dos.h>
#include<conio.h>
#include<graphics.h>
#include<stdlib.h>
#include<math.h>
#include<alloc.h>
#define RADS 0.017453293
struct point{
    int x,y,z;
}

a[8]={
    {30,100,50},
    {30,230,50},
    {30,230,90},
    {30,100,90},
    {120,100,50},
    {120,230,50},
    {120,230,90},
    {120,100,90}

}

int n = 3,*d;
int noi[20][20];
void Dothi(){
    line (getmaxx()/2,getmaxy()/2,getmaxx()-10,getmaxy()/2);
    line (getmaxx()/2,10,getmaxx()/2,getmaxy()/2);
```

```

line          (getmaxx() / 2, getmaxy() / 2, getmaxx() / 2 -
170, getmaxy() / 2 + 170);
line          (getmaxx() / 2 - 170, getmaxy() / 2 + 170, getmaxx() / 2 -
170, getmaxy() / 2 + 166);
line          (getmaxx() / 2 - 170, getmaxy() / 2 + 170, getmaxx() / 2 -
163, getmaxy() / 2 + 167);
    setttextjustify(CENTER_TEXT, CENTER_TEXT);
    outtextxy(getmaxx() - 9, getmaxy() / 2 + 1, "0");
    outtextxy(getmaxx() - 9, getmaxy() / 2 + 10, "y");
    outtextxy(getmaxx() / 2, 10, "-");
    outtextxy(getmaxx() / 2 - 10, 10, "z");
    outtextxy(getmaxx() / 2 - 155, getmaxy() / 2 + 170, "x");
}
point Diem3d(int &x, int &y, int &z){
    point p;
    p.x = int(getmaxx() / 2 + y - x * cos(RADS * 45));
    p.y = int(getmaxy() / 2 - z + x * cos(RADS * 45));
    return p;
}
point chance(int &x, int &y, int &z){
    point p;
    if(x >= 0 && y >= 0 && z >= 0){
        p.x = getmaxx() / 2 + y - x * cos(RADS * 45);
        p.y = getmaxy() / 2 - z + x * cos(RADS * 45);
    }
    if(y >= 0 && x < 0 && z >= 0){
        p.x = y + getmaxx() / 2 - x * cos(RADS * 45);
        p.y = getmaxy() / 2 + z - x * cos(RADS * 45);
    }
    if(x >= 0 && y < 0 && z >= 0){
        p.x = getmaxx() / 2 + y - x * cos(RADS * 45);
        p.y = getmaxy() / 2 - (z - x * cos(RADS * 45));
    }
    if(x >= 0 && y >= 0 && z < 0){
        p.x
=
getmaxx() / 2 + (sqrt(pow(x, 2) + pow(y, 2))) * cos(RADS * 45);
        p.y
=
getmaxy() / 2 + (sqrt(pow(x, 2) + pow(y, 2))) * cos(RADS * 45) - z;
    }
    if(y >= 0 && x < 0 && z < 0){
        p.x = getmaxx() / 2 + y - x * cos(RADS * 45);
        p.y = getmaxy() / 2 + (-z - x * cos(RADS * 45));
    }
    return p;
}
point hinhchieu(int &x, int &y, int &z, int chieu){

```

```
point p;
if(chieu==1){
    p.x =int( getmaxx()/2+y - x*cos(RADS*45));
    p.y =int( getmaxy()/2+x*cos(RADS*45));
}
if(chieu==2){
    p.x =int ( getmaxx()/2+y-x*cos(RADS*45)-y);
    p.y =int ( getmaxy()/2-z+x*cos(RADS*45));
}
if(chieu == 3){
    p.x =int( getmaxx()/2+y-x*cos(RADS*45)+x*sin(RADS*45));
    p.y =int( getmaxy()/2-z);
}
return(p);
}
point quay(int &x, int &y, int &z, int goc , int chieu){
point p;
if(chieu==1){
    p.x = x*cos(RADS*goc) + z*sin(RADS*goc);
    p.z = -x*sin(RADS*goc) + z * cos(RADS*goc);
    p.y = y;
}
if(chieu==2){
    p.y = y*cos(RADS*goc) - z*sin(RADS*goc);
    p.z = y*sin(RADS*goc) + z * cos(RADS*goc);
    p.x =x;
}
if(chieu==3){
    p.x = x*cos(RADS*goc)-y*sin(RADS*goc);
    p.y = x*sin(RADS*goc)+y*cos(RADS*goc);
    p.z = z;
}
return p;
}
void chieumat(point k[],int t, int &m ){
int i,j;
for(i = 0 ; i< 8 ; i++)
k[i] = hinhchieu(k[i].x,k[i].y,k[i].z,t);
for( i = 0 ; i< 8 ; i++)
for( j = 0 ; j< 8 ; j++){
    if(noi[i][j]==1&&noi[j][i]==1){

        Bresenham_Line(k[i].x,k[i].y,k[j].x,k[j].y,m);
    }
}
noi[0][3]=1,noi[3][0]=1;
```

```
noi[4][7]=1,noi[7][4]=1;
noi[3][7]=1,noi[7][3]=1;
}
void main(){
clrscr();
d = &n;
point b[8],g[3][8],c[8],l[8];
int t, goc ;
int driver = DETECT, mode;
initgraph(&driver,&mode,"c:\\tc\\bgi");
Dothi();
for(int i = 0; i < 8; i ++){
b[i] = Diem3d(a[i].x,a[i].y,a[i].z);
for( i = 0; i < 3; i ++ ){
line(b[i].x,b[i].y,b[i + 4].x,b[i + 4].y);
line(b[i].x,b[i].y,b[i+1].x,b[i+1].y);
line(b[i+4].x,b[i+4].y,b[i+5].x,b[i+5].y);
noi[i][i+4]=1 , noi[i+4][i] =1;
noi[i][i+1] = 1, noi[i+1][i]=1;
noi[i+4][i+5]=1, noi[i+5][i+4]=1;
}
noi[0][3]=1,noi[3][0]=1;
noi[4][7]=1,noi[7][4]=1;
noi[3][7]=1,noi[7][3]=1;
line(b[0].x,b[0].y,b[3].x,b[3].y);
line(b[4].x,b[4].y,b[7].x,b[7].y);
line(b[3].x,b[3].y,b[7].x,b[7].y);
settextjustify(LEFT_TEXT,RIGHT_TEXT);
printf("\nQuay quanh trục :");
printf("\ny.Bam so 1");
printf("\nx.Bam so 2.");
printf("\nz.Bam so 3.");
scanf("%d",&t);
printf("\nQuay goc bao nhieu do:");
scanf("%d",&goc);
for( i = 0 ; i<= 7 ; i++){
c[i] = quay(a[i].x , a[i].y , a[i].z , goc ,t);
l[i] = c[i];
}
for( i = 0; i < 8; i ++){
c[i] = Diem3d(c[i].x,c[i].y,c[i].z);
for( i = 0; i < 3; i ++ ){
line(c[i].x,c[i].y,c[i + 4].x,c[i + 4].y);
line(c[i].x,c[i].y,c[i+1].x,c[i+1].y);
line(c[i+4].x,c[i+4].y,c[i+5].x,c[i+5].y);
}
}
```

```
line(c[0].x,c[0].y,c[3].x,c[3].y);
line(c[4].x,c[4].y,c[7].x,c[7].y);
line(c[3].x,c[3].y,c[7].x,c[7].y);
outtextxy(10,410,"Chon mat chieu: ");
outtextxy(10,420,"xoy: Bam so 1.");
outtextxy(10,430,"xoz: Bam so 2.");
outtextxy(10,440,"yoz: Bam so 3.");
outtextxy(10,450,"Bam so 4 de ket thuc");
char m;
do{
    m = getch();
    for(i = 0;i < 8; i ++){
        c[i]= l[i];
        switch (m){
            case '1' :{
                chieumat(c,1,*(d));
                break;
            }
            case '2' :{
                chieumat(c,2,*(d));
                break;
            }
            case '3' :
            {
                chieumat(c,3,*(d)) ;
                break;
            }
        }
    }
}while( m !='4');
    closegraph();
}
```

TÀI LIỆU THAM KHẢO

- [1] James D.Foley, Andrie van Dam, Steven K.Feiner, Jonhn F. Hughes, Computer Graphics Principles and Practice, Addison Wesley, 1994.
- [2] Hoàng Kiếm, Dương Anh Đức, Lê Đình Duy, Vũ Hải Quân. Giáo trình cơ sở Đồ hoạ Máy tính, NXB Giáo dục, 2000.
- [3] Lê Tấn Hùng, Huỳnh Quyết Thắng. Kỹ thuật đồ hoạ máy tính, NXB khoa học và kỹ thuật, 2002.
- [4] Steven Harrington, Computer Graphics A Programming Approach, McGraw Hill International Edition, 1987.
- [5] Gerald Farin, Curves and Surfaces for Computer Aided Geometric Design A Practical Guide, Academic Press Inc, 1990.
- [6] [Watt92] ACM Press, A. Watt and M. Watt. Advanced Animation and Rendering Techniques. Addison Wesley Longman Limited, Edinburgh Gate, England, 1992.
- [7] [CG92] Springer, M.de Berg, M. van Kreveld, M. Overmars, O. Schwarzkopf. Computational Geometry. Springer-Verlag 1997, Germany.
- [8] [Jensen2000] Henrik Wann Jensen, Niels Jørgen Christensen. A Practical Guide to Global Illumination using Photon Maps. Siggraph 2000 Course 8.
- [9] OpenGL Programming Guide (Red-book) <http://www.glprogramming.com/red/>
- [10] OpenGL Reference Manual (Blue-book) <http://www.glprogramming.com/blue/>